

台灣自編大學教科書

@prof

# 機器人學導論

機構、感知、控制與具身人工智慧

## 第 四 篇

### 感知、定位與自主移動

第 22 章 機器人視覺與感知

第 23 章 行動機器人運動學

第 24 章 狀態估測與感測器融合

第 25 章 同步定位與建圖 ( SLAM )

第 26 章 路徑規劃與導航

第 27 章 無人機與飛行機器人

第 28 章 腿式機器人與運動控制

編著：李世淵 ( 機器人叫獸 )

助教：李天宇、李宇晴

2026 年 9 月

## 第四篇 篇章導讀

第三篇教你機器人該用多大的力氣動起來，這一篇要教你它如何知道自己在哪裡、又該往哪裡去。

前三篇的所有技術，都建立在一個前提上：機器人知道自己在哪裡、也知道目標在哪裡。固定基座的手臂靠編碼器就知道姿態，工件的位置則由治具保證。但一旦機器人要移動、要面對未知的環境，這個前提就崩解了——而第四篇的七章，就是要重建它。

**第 22 章 (視覺與感知)** 建立由影像走向三維世界的完整鏈條。相機把三維壓成二維，丟掉的深度永遠救不回來；本章要處理的，正是如何用校正、雙視角、深度感測與先驗知識，把那個維度補回來。

**第 23 章 (行動機器人運動學)** 處理輪型機器人的運動。它的核心概念是**非完整約束**——輪子不能側滑，這使得「當下能走的方向」被限制了，但「長期能到的地方」卻沒有。理解這個看似矛盾的性質，就理解了為何路邊停車要來回好幾次。

**第 24 章 (狀態估測)** 是本篇的數學核心。里程計短期準但會漂移、外部感測不漂移但更新慢——該相信哪一個？答案是**依可信度加權**，而這正是 Kalman 濾波器的全部祕密。本章也會揭曉第 6 章那條互補濾波公式的真正來源。

**第 25 章 (SLAM)** 解開機器人學最著名的雞生蛋難題：要建圖必須先定位、要定位又必須先有地圖。解法出乎意料地優雅——不要分開解，一起解。而全章最戲劇性的時刻，是迴路閉合時整張地圖被瞬間扭正。

**第 26 章 (路徑規劃)** 決定該往哪裡走。它的核心抽象是**位形空間**——把複雜形狀的碰撞問題，轉化為「一個點是否落在禁區」。而在畫不出來的六維空間裡，取樣式方法給出了「不建構、只查詢」的突破。

**第 27、28 章 (飛行與行走)** 則是全書技術的綜合考驗。無人機六個自由度卻只有四顆馬達、且失控即墜落；腿式機器人自由度更高，且每走一步接觸狀態就要切換一次。兩者都必須靠**分層架構與簡化模型**才能駕馭。

### 這一篇要回答的七個問題

- 相機看到零件在第 320 個像素，機器手臂該把夾爪移到哪裡？

- 兩輪小車為什麼不能往正右方平移？停車為什麼要來回好幾次？
- 兩個定位系統給出不同答案，該相信哪一個？還是各取一半？
- 要建圖得先定位、要定位得先有地圖——這個循環怎麼打破？
- 六維空間長什麼樣？機器人如何在一個看不見的空間裡找路？
- 六個自由度只有四顆馬達，無人機憑什麼飛得這麼穩？
- 為什麼說「走路的本質，是一連串被打斷的跌倒」？

### 三條貫穿全篇的線索

**線索一：量化不確定性，而非忽略它。** 第三篇的世界是確定的——給定關節角度，末端位置唯一確定。本篇的世界則充滿雜訊：每一個感測讀值、每一次辨識、每一步落腳都可能出錯。因應之道不是假裝它不存在，而是用協方差、誤差橢圓與捕捉點裕度**把它算出來**。

**線索二：依可信度融合，而非二選一。** 這是第 24 章的核心，也是整篇的主軸。里程計與視覺、光達與 IMU、模型與學習——正確的作法從來不是切換或覆蓋，而是依各自的不確定性加權。這個原則在 SLAM 的迴路閉合、無人機的姿態估測、腿式機器人的狀態融合中反覆出現。

**線索三：用簡化模型處理複雜系統。** 第 26 章把機器人簡化為位形空間中的一個點、第 27 章用小角度近似把姿態與位置解耦、第 28 章更把三十自由度的人形機器人化為一根倒立擺。這些抽象使原本不可解的問題變得可解——而底層仍用完整模型執行，兩者透過分層架構銜接。

### 讀完這一篇之後

你將能夠：校正相機並由影像推算三維位置、分析各種輪型的運動約束、實作 Kalman 與粒子濾波器融合多重感測、建立 SLAM 系統並處理迴路閉合、以 A\* 與 RRT 規劃路徑並設計分層導航、以及理解無人機與腿式機器人這兩類欠驅動系統的控制架構。

到這裡，機器人已經具備了完整的**物理能力**：它能看見、能移動、能定位、能導航、能飛行也能行走。但它仍然缺少一件關鍵的東西——**理解與決策**。目前為止的所有能力，都建立在「人類明確地告訴它該做什麼」之上。

第五篇「智慧機器人與具身人工智慧」將處理這個轉變。屆時你會發現，前四篇建立的所有能力，都將成為第五篇的執行基礎——一個大型模型可以理解「幫我把桌上的杯子拿到廚房」，但要真正做到，需要視覺辨識、抓取、導航與控制。**智慧與身體，缺一不可。**

## 本篇章節配置

| 章次     | 章名             | 核心內容                                   | 叫獸開講                               |
|--------|----------------|----------------------------------------|------------------------------------|
| 第 22 章 | 機器人視覺與感知       | 針孔模型、相機校正、深度恢復、特徵匹配、手眼校正、視覺伺服          | 「相機看到零件在第 320 個像素，手臂該移到哪裡？」——相機會說謊 |
| 第 23 章 | 行動機器人運動學       | 輪型與約束、瞬時旋轉中心、差速與阿克曼、非完整性、里程計           | 「兩輪小車為什麼不能往正右方平移？」——瞬時受限，全域自由      |
| 第 24 章 | 狀態估測與感測器融合     | 貝氏濾波、協方差、Kalman 與 EKF/UKF、粒子濾波、互補濾波    | 「兩個定位系統都不完全對，該相信哪一個？」——依可信度加權      |
| 第 25 章 | 同步定位與建圖 (SLAM) | 雞生蛋問題、EKF-SLAM、FastSLAM、佔據柵格、圖最佳化、迴路閉合 | 「要建圖得先定位，要定位得先有地圖——怎麼辦？」——一起解      |
| 第 26 章 | 路徑規劃與導航        | 位形空間、A* 與啟發函數、PRM 與 RRT、勢場與 DWA、分層導航   | 「六維空間長什麼樣？」——既然畫不出來，就用取樣去探索        |
| 第 27 章 | 無人機與飛行機器人      | 分配矩陣、欠驅動、微分平坦、串級控制、姿態估測、分級降級           | 「六個自由度只有四顆馬達，憑什麼飛得穩？」——時間尺度分離      |
| 第 28 章 | 腿式機器人與運動控制     | 混合動態、ZMP、線性倒立擺、捕捉點、步態、全身控制、強化學習        | 「走路的本質，是一連串被打斷的跌倒。」——30 自由度化為一根倒立擺 |

## 第四篇 感知、定位與自主移動

# 第 22 章

## 機器人視覺與感知

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為一張筆直鐵軌延伸至地平線的照片，兩條平行軌道在遠方交會成一點，旁註「相機會說謊：三維壓成二維，深度永遠救不回來」。中央為針孔相機模型的幾何示意——物體、投影中心、成像平面與相似三角形，並以一條由像素射出的虛線延伸成空間中的射線，標註「影像上的一點  $\Leftrightarrow$  空間中的一條射線」。右半為一支機械手臂末端裝著相機正對工件，畫面上疊印四個座標系（像素、相機、末端、基座）與串接它們的變換箭頭。標題文字：從像素走向三維世界。

## 叫獸開講 「相機看到零件在第 320 個像素，手臂該移到哪裡？」

叫獸在投影幕上打出一張照片：一條筆直的鐵軌延伸到遠方，兩條軌道在地平線處交會成一個點。

「請問，」他指著螢幕，「這兩條鐵軌，在真實世界裡是不是平行的？」全班點頭。「那照片上呢？」「相交了。」

「所以第一件事：**相機會說謊**。」叫獸在白板上寫下這五個字，「它把三維的世界壓成二維的影像，過程中丟掉了一整個維度——**深度**。而丟掉的資訊，永遠救不回來。」

他換了一張投影片：一張照片上有一顆看起來很大的橘子。「這顆橘子有多大？」小翰說：「大概拳頭大吧。」叫獸按下一鍵，鏡頭拉遠——那其實是一顆離鏡頭很近的小彈珠。「你剛才判斷錯了。為什麼？因為你潛意識假設了它的距離。**單張照片無法區分「大而遠」與「小而近」**，這叫做**尺度歧義**。」

小妤舉手：「那用兩台相機不就好了？像人的兩隻眼睛。」

「很好，這確實是一條路。但我要先問一個更基礎的問題。」叫獸拿出一台工業相機與一支雷射筆，「假設現在我用相機看到一個零件在畫面的第 320 個像素、第 240 個像素的位置。請問——**機器手臂該把夾爪移到哪裡？**」

教室安靜了。小翰試著說：「畫面中央？」「畫面中央是相機的中央，不是機器人的中央。」叫獸搖頭，「像素座標是相機的語言，機器人的關節聽不懂。中間至少缺了三件事：**相機的內部參數**（畫面上一個像素對應多少角度）、**相機與機器人的相對位置**（相機裝在哪裡、朝哪個方向）、以及**深度**（那個零件離相機多遠）。少了任何一項，這個轉換就做不出來。」

他在白板上畫了一條鏈：**像素 → 相機座標 → 機器人座標 → 關節角度**。「這條鏈的後半段，我們在第 8 章與第 10 章已經學會了。今天要補上的是前半段——**如何從影像走向三維世界**。」

最後他補了一句：「還有一件事你們必須有心理準備。第二篇與第三篇的世界很乾淨：給定關節角度，末端位置是唯一確定的。但從今天開始，**每一個數字都帶著不確定性**。光線會變、陰影會動、玻璃會反光、物體會被遮住。感知不是精確的計算，而是在**充滿雜訊的證據中做出最合理的推測**。這個心態的轉換，比任何一條公式都重要。」

## 本章為什麼重要

第三篇的所有技術都建立在一個前提上：**機器人知道自己在哪裡、知道目標在哪裡**。固定基座的手臂靠編碼器就知道姿態，工件位置則由治具保證。但一旦要面對未知的環境，這個前提就崩解了。

視覺是彌補這個缺口最重要的工具。它提供的資訊密度遠超其他感測器——一張影像包含數十萬個像素，能同時告訴你物體的位置、形狀、顏色、紋理、文字甚至狀態。這是光達或超音波做不到的。

但代價是：**這些資訊需要大量的處理才能使用**（第 6.7.1 節）。光達直接給你距離，相機給你的是一堆數字。本章的任務，就是建立由「像素」走向「三維世界」的完整鏈條：相機模型告訴我們投影如何發生、校正讓我們知道相機的真實參數、深度感測補回丟失的維度、手眼校正把相機座標接回機器人座標，而視覺伺服則讓機器人能直接依影像回授來運動。

本章也是第四篇的起點。第 23 ~ 28 章的移動機器人、定位、SLAM 與自主導航，全部依賴感知作為輸入。理解視覺的能力**與極限**，是後續所有章节的基礎。

## 學習目標

- 能說明針孔相機模型，並寫出投影方程。
- 能區分相機的內部參數與外部參數，並說明各自的物理意義。
- 能說明鏡頭畸變的成因與校正方法。
- 能敘述相機校正的流程並說明棋盤格標定板的作用。
- 能說明由影像回推三維位置所缺少的資訊，並列舉三種補足方式。
- 能比較常見的影像特徵並說明特徵匹配的流程。
- 能說明手眼校正的兩種構型與其求解問題的形式。
- 能區分眼在手上與眼在手外的優缺點與適用場合。
- 能說明視覺伺服的兩種架構及其差異。
- 能推導影像雅可比並說明其與第 11 章雅可比的串接關係。
- 能說明深度學習在物體偵測與位姿估計中的角色與限制。

- 能分析視覺系統常見的失效情境並提出因應方式。

## 先備知識

第 8 章的座標系與齊次變換（本章大量使用，尤其是轉換樹的概念）、第 6.7 節的相機與深度感測器、第 11 章的雅可比（視覺伺服需要串接影像雅可比與機器人雅可比）、第 10 章的逆運動學。

數學方面需要：齊次座標、矩陣運算、以及基本的最小平方法概念。本章的推導不深，但座標系的追蹤極為重要——建議隨時畫圖標記每個量屬於哪個座標系，這是避免混淆的關鍵。

## 核心名詞中英對照

表 22-1 本章核心名詞

| 中文     | 英文                   | 說明               |
|--------|----------------------|------------------|
| 針孔相機模型 | Pinhole Camera Model | 以單一投影中心描述成像的理想模型 |
| 焦距     | Focal Length         | 投影中心到成像平面的距離     |
| 主點     | Principal Point      | 光軸與成像平面的交點       |
| 內部參數   | Intrinsic Parameters | 描述相機自身成像特性的參數    |
| 外部參數   | Extrinsic Parameters | 描述相機在世界中位姿的參數    |
| 鏡頭畸變   | Lens Distortion      | 鏡頭造成的影像幾何失真      |
| 相機校正   | Camera Calibration   | 求出內外部參數的程序       |
| 尺度歧義   | Scale Ambiguity      | 單張影像無法區分大而遠與小而近  |
| 對極幾何   | Epipolar Geometry    | 描述雙視角之間幾何關係的理論   |
| 視差     | Disparity            | 同一點在左右影像中的位置差    |
| 特徵點    | Feature Point        | 影像中易於重複偵測與匹配的位置  |
| 描述子    | Descriptor           | 描述特徵點鄰域外觀的向量     |
| 手眼校正   | Hand-eye Calibration | 求相機與機器人之間變換的程序   |
| 眼在手上   | Eye-in-hand          | 相機安裝於機器人末端的構型    |
| 眼在手外   | Eye-to-hand          | 相機固定於環境中的構型      |
| 視覺伺服   | Visual Servoing      | 以視覺回授直接驅動機器人運動   |
| 影像雅可比  | Image Jacobian       | 相機運動與影像特徵變化的關係矩陣 |
| 位姿估計   | Pose Estimation      | 由影像推算物體的位置與姿態    |



## 22.1 針孔相機模型

### 22.1.1 最簡單也最有用的模型

相機成像的物理過程極為複雜（涉及透鏡、光圈、感光元件），但工程上有一個極簡卻極有效的近似：**針孔相機模型**。

想像一個不透光的盒子，前方有一個極小的孔。外界的光線只能穿過這個孔投射到後方的成像平面上。由於孔極小，物體上每一點只有一條光線能通過——因此成像是清晰的，且遵守簡單的相似三角形關係。

設相機座標系的原點在針孔（稱為**投影中心**）、Z 軸沿光軸指向前方。空間中一點  $P = (X, Y, Z)$  投影到成像平面上的位置為：

$$x = f \cdot X/Z, \quad y = f \cdot Y/Z$$

*投影方程。f 為焦距（投影中心到成像平面的距離）。這是本章最基本的一條式子。*

### 22.1.2 除以 Z：一切問題的根源

上式中的 **除以 Z** 是整個機器人視覺的核心困難所在。它帶來三個直接的後果：

**後果一：深度資訊被摧毀。**同一個  $(x, y)$  可以對應無限多個  $(X, Y, Z)$ ——只要  $X/Z$  與  $Y/Z$  的比值相同即可。這在幾何上意味著：**影像上的一個點，對應到空間中的一整條射線**。

**後果二：尺度歧義。**這正是開頭故事中「橘子還是彈珠」的問題。物體變大兩倍、距離也變遠兩倍，影像完全相同。

**後果三：非線性。**除法使投影成為非線性運算，這使得許多線性代數的工具無法直接套用——這也是齊次座標（第 8.6 節）在視覺中如此重要的原因，它能把除法「藏」到最後一步。

$$\text{影像上的一點} \iff \text{空間中的一條射線}$$

*這個對應關係值得牢記。它解釋了為何要恢復三維，必須引入額外的資訊（第 22.4 節）。*

### 22.1.3 由米制到像素

上述的  $(x, y)$  單位是長度（如公釐），但影像處理的單位是**像素**。兩者的轉換需要三個參數：

$$u = f_x \cdot X/Z + c_x$$

$$v = f_y \cdot Y/Z + c_y$$

像素座標。 $f_x$ 、 $f_y$  為以像素為單位的焦距， $c_x$ 、 $c_y$  為主點（光軸與成像平面的交點，通常接近影像中心）。

為何  $f_x$  與  $f_y$  可能不同？因為感光元件的像素未必是正方形。實務上多數相機兩者接近相等，但校正時仍分開估計。

寫成矩陣形式（使用齊次座標）：

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

中間那個  $3 \times 3$  矩陣稱為內部參數矩陣  $K$ 。 $s$  為尺度因子（實際上就是  $Z$ ），這正是齊次座標把除法延後的技巧。

## 22.2 內部參數、外部參數與畸變

### 22.2.1 內外部參數的分工

完整的成像過程包含兩個步驟，各由一組參數描述。

**外部參數（Extrinsic）**：描述相機在世界中的位姿，即世界座標系到相機座標系的變換。它就是第 8 章的齊次變換矩陣：

$${}^cP = {}^cR_W \cdot {}^Wp + {}^c t_W \quad \text{即} \quad {}^cT_W = [R \mid t]$$

外部參數。共 6 個自由度（3 平移 + 3 旋轉）。它描述的是「相機在哪裡」，與相機本身無關。

**內部參數（Intrinsic）**：描述相機自身的成像特性，即由相機座標系到像素座標的投影。就是 22.1.3 節的矩陣  $K$ ，共 4 個參數（ $f_x$ 、 $f_y$ 、 $c_x$ 、 $c_y$ ）。它描述的是「相機長什麼樣」，與相機在哪裡無關。

兩者串接就是完整的投影：

$$s \cdot \tilde{p} = K \cdot [R \mid t] \cdot \tilde{P}$$

完整的相機投影方程。 $\tilde{P}$  為世界座標的齊次表示， $\tilde{p}$  為像素座標的齊次表示。 $K \cdot [R \mid t]$  合稱為投影矩陣。

#### ◎ 為何要區分內外部參數

這個區分有極重要的工程意義。內部參數在相機出廠後就固定了（除非變焦或更換鏡頭），因此只需校正一次，之後可以永久使用。外部參數則隨相機的安裝位置而變，每次移動相機都必須重新求取。實務上這意味著：內部參數校正可以在實驗室從容進行；而外部參數（尤其是相機與機器人的關係）則必須在現場做手眼校正（22.6 節），且機器人一旦被碰撞移位就要重做。

## 22.2.2 鏡頭畸變

針孔模型假設光線直線傳播，但真實的鏡頭會使光線彎折，造成**畸變**。最主要的兩類是：

**徑向畸變 ( Radial Distortion )**：由鏡片的球面形狀造成，使影像呈桶形 ( barrel，中央凸出 ) 或枕形 ( pincushion，邊緣拉伸 )。廣角鏡頭的桶形畸變尤其明顯。

$$x_d = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \quad r^2 = x^2 + y^2$$

*徑向畸變模型。k<sub>1</sub>、k<sub>2</sub>、k<sub>3</sub> 為畸變係數。注意畸變量與 r 有關——離影像中心越遠，畸變越嚴重。*

**切向畸變 ( Tangential Distortion )**：由鏡頭與感光元件未完全平行造成，通常較小。以 p<sub>1</sub>、p<sub>2</sub> 兩個係數描述。

因此完整的相機模型共有 **4 個內部參數 + 5 個畸變係數 = 9 個參數** ( 外加每張影像的 6 個外部參數 )。

## 22.2.3 畸變對機器人的影響

畸變的實務影響常被低估。一個典型的廣角鏡頭在影像邊緣的畸變可達數十個像素——若不校正，直接把像素座標換算成空間位置，誤差可能達到數公分。

因此標準的處理流程是：**先校正畸變、再做任何幾何計算**。多數視覺函式庫提供 undistort 函式，能把畸變影像轉換成符合針孔模型的理想影像。

一個實務要點：**畸變校正會裁掉影像邊緣** ( 因為校正後的有效區域是不規則的 )。若視野本來就緊繃，校正後可能不夠用——這在選鏡頭時就應納入考量。

# 22.3 相機校正

## 22.3.1 校正在做什麼

相機校正的目標，是求出 22.2 節所述的內部參數與畸變係數。它的基本原理是：

### ◎ 校正的基本邏輯

拍攝一個幾何尺寸已知的物體 ( 標定板 )，我們就同時知道了「空間中的點在哪裡」與「它成像在哪個像素」。有了足夠多的這種對應關係，就能反推出相機的參數——這是一個**最小平方問題**：找出一組參數，使得「用該參數預測的像素位置」與「實際觀測到的像素位置」差距最小。

最常用的標定板是**棋盤格**，原因有三：角點的位置可以用亞像素精度精確偵測 ( 黑白交界處

的梯度極大)、格子的尺寸精確已知、且平面的假設簡化了數學。

### 22.3.2 標準流程

Zhang 於 2000 年提出的方法是目前的標準，流程為：

1. **準備標定板**：列印或訂製一張棋盤格，量測其格子的精確尺寸。板子必須**夠平**——貼在玻璃或壓克力板上是常見作法。
2. **拍攝多張影像**：從不同的角度與距離拍攝標定板，建議 15 ~ 30 張。**關鍵是姿態要多樣**——若所有照片都是正對著板子拍的，某些參數會無法分離。
3. **偵測角點**：以程式自動找出每張影像中所有的棋盤格角點，並以亞像素精度精修。
4. **初始估計**：由平面單應性 (homography) 的性質，線性求解出參數的初始值。
5. **非線性最佳化**：以重投影誤差為目標函數，用 Levenberg-Marquardt 等方法精修所有參數 (含畸變係數)。
6. **驗證**：檢查重投影誤差——把已知的三維點用求得的參數投影回影像，與實際偵測的角點比較。

### 22.3.3 如何判斷校正好不好

重投影誤差 (reprojection error) 是最重要的品質指標：

$$\text{重投影誤差} = \text{平均} || \text{預測的像素位置} - \text{實際偵測的像素位置} ||$$

以像素為單位。這個數字直接反映模型與現實的吻合程度。

經驗判準是：

表 22-2 重投影誤差的判讀

| 重投影誤差        | 品質  | 說明                      |
|--------------|-----|-------------------------|
| < 0.3 像素     | 優良  | 可用於精密應用 (如視覺量測)         |
| 0.3 ~ 0.5 像素 | 良好  | 一般機器人應用足夠               |
| 0.5 ~ 1.0 像素 | 可接受 | 精度要求不高時可用               |
| > 1.0 像素     | 不佳  | 應重新校正，檢查板子是否平整、角點偵測是否正確 |

三個常見的校正失敗原因：一、**標定板不夠平** (列印在紙上拿在手中拍是最常見的錯誤)；二、**拍攝姿態不夠多樣** (都從同一角度拍，參數無法分離)；三、**影像模糊或曝光不當** (角點偵

測不準)。

## 22.4 由影像回到三維

### 22.4.1 缺少的那一個維度

回到 22.1.2 節的核心問題：影像上的一點對應空間中的一條射線。要確定該點在射線上的哪個位置，必須引入額外的資訊。共有三類方法。

### 22.4.2 方法一：第二個視角

若同一個點能被兩個位置不同的相機看到，就有兩條射線，它們的交點即為該點的三維位置。這稱為三角測量 (triangulation)。

第 6.7.2 節已介紹過立體視覺的深度公式：

$$Z = (f \cdot B) / d$$

$B$  為基線 (兩相機的間距)、 $d$  為視差 (同一點在兩影像中的像素位置差)。

這個公式有一個關鍵的性質：深度誤差與距離的平方成正比。

$$\Delta Z \approx (Z^2 / (f \cdot B)) \cdot \Delta d$$

微分上式所得。這意味著遠處的深度估計會急遽劣化——這是立體視覺的根本限制。

雙視角之間的幾何關係由對極幾何 (epipolar geometry) 描述。它的核心結論是：左影像中的一個點，在右影像中的對應點必定落在一條特定的直線上 (稱為對極線)。這把二維的搜尋降為一維，是立體匹配演算法的基礎。

### 22.4.3 方法二：主動測距

與其被動地推算深度，不如主動地量測它。這就是第 6.7.2 節的結構光與 ToF 深度相機——它們直接輸出每個像素的距離。

優點是不需要紋理 (立體視覺在白牆上會失效)、計算量小。缺點如第 6.7.2 節所述：結構光只能室內使用、ToF 在牆角有多重反射誤差、且兩者對透明與反光物體都會失效。

### 22.4.4 方法三：先驗知識

第三條路是利用已知的資訊。若我們已經知道物體的尺寸或形狀，就能由影像中的大小反推距離。

這在工業上極為常見。例如：已知工件的 CAD 模型，由影像中的輪廓即可解出其六自由度位姿（這稱為 **PnP 問題**，Perspective-n-Point）。或者更簡單的作法——使用**視覺標記**（如 AprilTag、ArUco），它們的尺寸與圖案都是已知的，因此單張影像就能算出完整的位姿。

### ◎ 視覺標記：工程上的務實選擇

在研究論文中，用視覺標記常被認為是「作弊」——因為它繞過了物體辨識的困難。但在工程實務上，這往往是最正確的決定。一張成本幾元的 AprilTag，能提供公釐級的位姿估計、毫秒級的計算、以及極高的可靠度。若你的應用允許在物體或治具上貼標記（例如自家產線的載具、AGV 的充電站、機器人的校正基準），**就應該優先採用**。把辨識的困難交給演算法，只在無法貼標記時才是必要的。這與第 20 章「用一個倒角解決插銷入孔」是同一種工程思維。

表 22-3 三種恢復深度方法的比較

| 方法         | 所需條件     | 優點         | 限制          | 典型應用      |
|------------|----------|------------|-------------|-----------|
| 立體視覺       | 兩個視角、有紋理 | 被動、可戶外     | 無紋理失效、遠距精度差 | 戶外導航、自駕車  |
| 結構光 / ToF  | 主動投射     | 不需紋理、計算量小  | 室內限制、怕透明反光  | 室內操作、桌面抓取 |
| 先驗知識 (PnP) | 已知模型或標記  | 單張影像即可、精度高 | 需事先建模或貼標記   | 工業定位、標記導航 |

## 22.5 影像特徵與匹配

### 22.5.1 為何需要特徵

一張影像有數十萬個像素，但其中絕大多數是「不特別」的——一片白牆上的任一點看起來都一樣。若要在兩張影像之間找對應關係（立體視覺、運動估計、物體辨識都需要），逐像素比對既慢又不可靠。

**特徵點**（feature point）的思想是：只挑出那些「容易被重複找到、且容易區分」的位置。好的特徵點應具備三個性質：

- **可重複性**：從不同角度、不同光線下拍攝，同一個實體位置都能被偵測到。
- **獨特性**：其鄰域的外觀足夠特別，不會與其他位置混淆。
- **局部性**：只依賴小範圍的鄰域，因此部分遮蔽時仍然有效。

哪些位置符合這些條件？答案是**角點**——在兩個方向上灰階都有劇烈變化的位置。相對地，平坦區域（各方向都無變化）與邊緣（只有一個方向有變化）都不適合：平坦區無法定位、邊緣則有「沿邊滑動」的歧義。

## 22.5.2 偵測與描述兩個步驟

特徵的處理分為兩個獨立的步驟。

**步驟一：偵測 (Detection)**。找出影像中的特徵點位置。經典的方法是 Harris 角點偵測——計算每個像素鄰域的梯度結構矩陣，若其兩個特徵值都大，代表兩個方向都有變化，即為角點。

**步驟二：描述 (Description)**。為每個特徵點計算一個**描述子 (descriptor)**——一個能刻畫其鄰域外觀的向量。之後比對兩張影像的特徵時，就是比對這些向量的相似度。

好的描述子需要具備**不變性**：即使物體旋轉、縮放、或光線改變，同一個特徵點的描述子應該保持相似。這是描述子設計的核心挑戰。

表 22-4 常見特徵演算法的比較

| 演算法    | 特點              | 旋轉不變 | 尺度不變 | 速度    | 適用場合        |
|--------|-----------------|------|------|-------|-------------|
| Harris | 經典角點偵測，無描述子     | 否    | 否    | 極快    | 簡單追蹤、教學     |
| SIFT   | 尺度空間極值，描述子強健    | 是    | 是    | 慢     | 高精度匹配、離線處理  |
| SURF   | SIFT 的加速版       | 是    | 是    | 中     | 折衷方案        |
| ORB    | FAST 偵測 + 二元描述子 | 是    | 部分   | 極快    | 即時 SLAM、嵌入式 |
| 深度學習特徵 | 由網路學習偵測與描述      | 是    | 是    | 需 GPU | 困難場景、大視角變化  |

**實務上的選擇**：即時系統（如第 25 章的視覺 SLAM）幾乎都用 **ORB**——它的二元描述子可以用位元運算比對，速度比 SIFT 快數十倍，精度損失有限。離線的高精度應用則仍偏好 SIFT。

## 22.5.3 匹配與外點剔除

有了描述子，匹配的作法是為每個特徵找出另一張影像中描述子最相似的那一個。但這必然會產生**錯誤匹配 (outlier)**——相似的紋理、重複的圖案、或被遮蔽的區域都會造成誤配。

因此匹配之後必須做**外點剔除**。標準的作法是 **RANSAC**（隨機取樣一致性）：

1. 隨機挑選最少數目的匹配點（例如求基礎矩陣需要 8 對）。



2. 由這些點計算出一個幾何模型（如兩視角之間的變換）。
3. 檢查所有匹配點是否符合這個模型，統計「內點」的數目。
4. 重複多次，保留內點最多的那個模型，並剔除其餘的外點。

RANSAC 的核心思想值得一記：與其試圖用所有資料（包含錯誤的）擬合出一個模型，不如假設「有一小群正確的資料」並找出它們。這使它對高比例的外點（甚至超過 50%）仍然有效，是感知系統中最重要穩健技術之一。

## 22.6 手眼校正

### 22.6.1 問題的定義

回到開頭故事的核心問題：相機看到零件在像素 (320, 240)，機器人該把夾爪移到哪裡？

即使已經完成相機校正（知道內部參數）並取得了深度（知道零件在相機座標系中的三維位置），仍然缺少最後一塊拼圖：相機座標系與機器人座標系之間的變換。求出這個變換的程序，稱為手眼校正（hand-eye calibration）。

### 22.6.2 兩種構型

依相機的安裝位置，有兩種構型，其求解的未知量不同。

**眼在手上（Eye-in-hand）**：相機固定在機器人末端，隨手臂一起移動。要求的是「相機相對於末端法蘭」的變換  ${}^T_C$ ——這是一個固定值（只要相機不鬆脫）。

**眼在手外（Eye-to-hand）**：相機固定在環境中（如架在工作台上方），不隨機器人移動。要求的是「相機相對於機器人基座」的變換  ${}^B_T$ ——同樣是固定值。

表 22-5 兩種手眼構型的比較

| 面向   | 眼在手上             | 眼在手外             |
|------|------------------|------------------|
| 相機位置 | 末端法蘭             | 固定於環境            |
| 待求變換 | ${}^T_C$ （法蘭到相機） | ${}^B_T$ （基座到相機） |
| 視野   | 隨手臂移動，可主動改變      | 固定               |
| 解析度  | 可靠近物體取得高解析度      | 受限於固定距離          |
| 遮蔽問題 | 手臂本身可能擋住視線       | 手臂常擋住工件          |
| 末端負載 | 增加（相機重量與線材）      | 無影響              |



| 面向   | 眼在手上             | 眼在手外       |
|------|------------------|------------|
| 典型應用 | 精密裝配、視覺伺服、探索未知場景 | 產線取放、大範圍監看 |

### 22.6.3 $AX = XB$ 問題

手眼校正在數學上歸結為一個經典的問題形式。作法是：讓機器人移動到多個不同的姿態，每個姿態都拍攝同一個固定的標定板。

在兩個姿態之間，可以得到兩個「相對運動」：機器人末端的運動  $A$ （由正向運動學算出，第 9 章）與相機的運動  $B$ （由標定板的兩次觀測算出）。這兩者透過待求的變換  $X$  相連：

$$A \cdot X = X \cdot B$$

*手眼校正常式。A 為末端的相對運動、B 為相機的相對運動、X 為待求的手眼變換。*

這條式子的直觀意義是：「先移動再換算」與「先換算再移動」必須得到相同的結果——這是變換的一致性要求。

求解需要至少兩組（實務上建議 10 組以上）不同的相對運動，且這些運動必須包含足夠的旋轉多樣性——若所有運動都繞同一個軸旋轉， $X$  的某些分量無法被確定。這與相機校正需要多樣的拍攝姿態是同樣的道理。

### 22.6.4 實務要點

**要點一：運動要有足夠的旋轉。**純平移的運動組合無法求解旋轉部分。建議規劃的姿態包含繞不同軸的顯著旋轉（至少 30 度以上）。

**要點二：驗證方法。**求出  $X$  後，應以獨立的資料驗證：讓機器人移動到一個未參與校正的姿態，用  $X$  預測標定板在影像中的位置，與實際觀測比較。誤差應在數個像素以內。

**要點三：什麼時候要重做。**相機被碰撞、鏡頭被更換或調焦、機器人被重新安裝、末端加裝了新的工具——這些情況都必須重新校正。實務上建議定期驗證（如每月一次），以及在精度異常時優先懷疑手眼參數。

**圖 22-1 兩種手眼構型與座標系關係**

建議配圖：左圖為眼在手上——機械手臂末端裝有相機，畫出基座座標系  $\{B\}$ 、法蘭座標系  $\{F\}$ 、相機座標系  $\{C\}$  與工件座標系  $\{O\}$ ，並以箭頭標出四個變換： ${}^B T_F$ （正向運動學，已知）、 ${}^F T_C$ （待求的手眼變換）、 ${}^O T_O$ （相機量測，已知）。右圖為眼在手外——相機架在工作台上方，同樣標出四個座標系，待求的是  ${}^B T_C$ 。兩圖下方

各附一個閉環方程，說明變換鏈必須自治。

## 22.7 視覺伺服

### 22.7.1 兩種架構

前面的討論都是「看一次、算一次、動一次」的開迴路作法：拍照、計算目標位置、規劃軌跡、執行。這個流程有一個弱點——執行過程中若有誤差（機器人定位誤差、物體移動），沒有機會修正。

視覺伺服（visual servoing）則把視覺放進控制迴路中：持續拍攝、持續比對目前影像與目標影像的差異、持續調整運動。這是一個閉迴路的作法，能自動補償各種誤差。

依「誤差在哪個空間中定義」，有兩種架構。

位置式視覺伺服（PBVS, Position-Based）：先由影像估計出物體的三維位姿，再在笛卡兒空間中計算誤差並控制。

$$\text{誤差 } e = x_{\text{current}} - x_{\text{desired}} \quad (\text{三維位姿的差})$$

PBVS。誤差有明確的物理意義（公釐、度），軌跡在空間中是直線，直觀易懂。

影像式視覺伺服（IBVS, Image-Based）：直接在影像空間中定義誤差——比較特徵點目前的像素位置與期望的像素位置。

$$\text{誤差 } e = s_{\text{current}} - s_{\text{desired}} \quad (\text{像素座標的差})$$

IBVS。不需要估計三維位姿，因此對相機校正誤差與深度估計誤差較不敏感。

表 22-6 PBVS 與 IBVS 的比較

| 面向     | 位置式 PBVS   | 影像式 IBVS             |
|--------|------------|----------------------|
| 誤差定義空間 | 笛卡兒空間（三維）  | 影像空間（像素）             |
| 需要位姿估計 | 是          | 否                    |
| 對校正誤差  | 敏感         | 較不敏感                 |
| 空間軌跡   | 直線，可預測     | 可能彎曲甚至繞遠路            |
| 影像軌跡   | 特徵可能離開視野   | 特徵保持在畫面內             |
| 大旋轉時   | 表現良好       | 可能退離（camera retreat） |
| 適用場合   | 已知模型、大範圍運動 | 未知模型、精密對位            |

兩者的取捨很有意思：**PBVS** 保證空間軌跡漂亮但可能讓特徵跑出畫面；**IBVS** 保證特徵留在畫面但空間軌跡可能很怪。實務上也有混合式（2.5D 視覺伺服）試圖兼顧兩者。

### 22.7.2 影像雅可比

IBVS 的核心是回答一個問題：**相機移動時，影像上的特徵點會怎麼移動？** 這個關係由**影像雅可比**（image Jacobian，又稱互動矩陣）描述：

$$\dot{s} = L(s, Z) \cdot v_{\text{camera}}$$

$s$  為特徵的像素座標、 $v_{\text{camera}}$  為相機的六維速度（三平移三旋轉）、 $L$  為影像雅可比。

對單一個特徵點  $(u, v)$ ，其影像雅可比為（此處以歸一化座標表示）：

$$L = \begin{bmatrix} -f/Z & 0 & u/Z & uv/f & -(f+u^2/f) & v \\ 0 & -f/Z & v/Z & f+v^2/f & -uv/f & -u \end{bmatrix}$$

單點的影像雅可比（ $2 \times 6$ ）。注意它**含有深度  $Z$** ——這是 IBVS 仍需要深度估計的原因，但好消息是它對  $Z$  的誤差相當不敏感。

由於一個點只提供 2 個方程式，而相機有 6 個自由度，因此**至少需要 3 個特徵點**（提供 6 個方程式）才能完整控制。實務上常用 4 個以上以增加穩健性。

### 22.7.3 控制律與雅可比的串接

有了影像雅可比，控制律的設計就與第 11、17 章完全一致——這是本書工具重複使用的又一個例子。

$$v_{\text{camera}} = -\lambda \cdot L^+ \cdot e$$

IBVS 的控制律。 $L^+$  為影像雅可比的偽逆（第 10.7.2 節）、 $\lambda$  為增益。這使誤差以指數衰減。

最後，把相機速度轉換為關節速度，需要**串接兩個雅可比**：

$$\dot{q} = J_{\text{robot}}^+ \cdot {}^B V_C \cdot L^+ \cdot (-\lambda e)$$

完整的鏈條：影像誤差  $\rightarrow$  相機速度  $\rightarrow$  末端速度  $\rightarrow$  關節速度。 ${}^B V_C$  是相機速度到末端速度的伴隨變換（第 11.7 節）。

這條式子清楚展現了本書各章工具的串接：**影像雅可比（本章）+ 伴隨變換（第 11 章）+ 機器人雅可比的偽逆（第 10、11 章）**。理解了每一塊，整條鏈就自然成立。

#### ◎ 視覺伺服的取樣率問題

視覺伺服有一個先天的困難：**相機的更新率遠低於控制迴路**。典型的相機是 30 Hz，而伺服迴路需

要 1 kHz——相差 33 倍。這意味著在兩幀影像之間，控制器有 33 個週期「沒有新資訊」。實務上的作法有三：一、以編碼器的高頻資訊做內插（在兩幀之間用運動學預測）；二、使用高幀率相機（可達 200 Hz 以上，但成本高）；三、降低視覺迴路的增益，使其反應速度與更新率匹配。這正是第 6.9.1 節「延遲比更新率更致命」在視覺上的體現。

## 22.8 深度學習在機器人視覺中的角色

### 22.8.1 三類任務

近十年，深度學習徹底改變了機器人視覺的能力邊界。它主要處理三類任務。

**任務一：物體偵測與分割。**在影像中找出物體的位置（邊界框）與類別，或逐像素標出物體的輪廓（語意分割、實例分割）。這是傳統方法極難處理的問題——因為「什麼是一個杯子」無法用幾何規則描述。

**任務二：六自由度位姿估計。**不只知道物體在哪裡，還要知道它的完整姿態。這是抓取（第 21 章）與裝配所必需的。傳統方法需要 CAD 模型與特徵匹配；學習式方法則能直接由影像回歸出位姿，且能處理無紋理或對稱的物體。

**任務三：深度與結構估計。**由單張影像預測深度圖（單目深度估計），或由多張影像重建三維結構。單目深度估計在物理上是不可能的（22.1.2 節的尺度歧義），但網路能從**先驗知識**（物體的典型大小、透視線索、紋理梯度）做出合理的推測——這正是人類單眼也能大致判斷距離的原理。

### 22.8.2 與傳統方法的分工

深度學習並未取代傳統的幾何方法，兩者的分工相當清楚：

表 22-7 傳統幾何方法與深度學習的分工

| 面向   | 傳統幾何方法          | 深度學習             |
|------|-----------------|------------------|
| 擅長   | 精確的幾何計算、多視角關係   | 語意理解、辨識、處理未知物體   |
| 需要   | 校正參數、幾何模型       | 大量標註資料           |
| 精度   | 高（可達亞像素）        | 中（受訓練資料限制）       |
| 泛化   | 只要幾何成立就有效       | 受限於訓練分布          |
| 可解釋  | 高               | 低                |
| 典型應用 | 校正、三角測量、SLAM 後端 | 偵測、分割、位姿估計、抓取點預測 |

**實務上的標準架構是混合式**：用深度學習做「理解」（這是什麼、大概在哪裡），用幾何方法做「精算」（精確的位置與姿態）。例如視覺抓取系統：先用網路偵測出物體並給出粗略的抓取點，再用點雲配準（幾何方法）精修位姿。

這與第 21.9.5 節的結論完全一致，也再次呼應第 17 章的原則——**不同的工具處理各自擅長的部分**。

### 22.8.3 資料與泛化的現實

學習式方法的最大挑戰是**資料**。標註一張分割影像可能需要數分鐘，而訓練通常需要數萬張。因應之道有三：

**方式一：合成資料**。在模擬器中渲染大量場景並自動產生標註。優點是免費且無限；缺點是與真實影像有落差（視覺的 **Sim-to-Real Gap**——渲染的光影、材質、雜訊都不夠真實）。因應之道同樣是**領域隨機化**：隨機改變光照、材質、背景與相機參數。

**方式二：預訓練與微調**。使用在大規模通用資料集上預訓練的模型，再用少量的任務專屬資料微調。這是目前最實用的作法。

**方式三：自我監督**。讓機器人自己收集資料——例如抓取後由「有沒有拿起來」自動產生標籤（第 21.9.3 節）。

**必須誠實面對的限制**：無論用什麼方法，網路在**訓練分布之外**的表現都會下降。而機器人的部署環境往往與訓練環境不同（不同的光線、不同的背景、沒見過的物體）。因此安全關鍵的系統絕不能只依賴網路的輸出，必須有幾何驗證或其他感測器的交叉檢查。

## 22.9 視覺系統的失效與因應

### 22.9.1 七種常見的失效情境

視覺系統在實驗室裡常常表現優異，一到現場就問題百出。以下七種情境涵蓋了絕大多數的失敗原因。

**情境一：光線變化**。這是最常見也最頑固的問題。日光隨時間與天氣改變、室內燈光可能閃爍（與相機快門產生頻閃）、機器人自身可能在工件上投下陰影。**因應**：使用主動光源（環形燈、

同軸光)並遮蔽環境光；固定曝光參數而非自動曝光；選用高動態範圍的相機。

**情境二：反光與鏡面。**金屬工件的高光會使影像局部飽和，深度相機在鏡面上完全失效。**因應：**使用偏光片濾除鏡面反射；改用漫射光源；或改用非光學的感測方式（如力覺探測）。

**情境三：透明物體。**玻璃、透明塑膠對深度相機幾乎是「不存在」的（第 6.7 節）。這是目前仍未完全解決的難題。**因應：**使用背景對比（在物體後方放特殊背景）；使用偏光或熱像；或改用觸覺確認。

**情境四：遮蔽。**物體被其他物體或機器人自身擋住。**因應：**多視角配置；主動視覺（移動相機以取得更好的視角）；或利用先驗模型推測被遮蔽的部分。

**情境五：無紋理表面。**白牆、單色平面上，立體視覺找不到對應點。**因應：**改用主動測距；或投射隨機圖案（這正是結構光的原理）。

**情境六：運動模糊。**相機或物體移動時，曝光期間的位移造成影像模糊。**因應：**縮短曝光時間（需要更強的光源）；使用全域快門（第 6.7.1 節）；或降低運動速度。

**情境七：校正漂移。**相機被碰撞、溫度變化使機構變形、鏡頭鬆動。**因應：**定期驗證重投影誤差；設置固定的校正基準（如工作台上的 AprilTag）供系統自我檢查。

## 22.9.2 系統設計的三個原則

面對這些失效，有三個貫穿性的設計原則。

**原則一：控制環境比改進演算法有效。**與其寫更聰明的程式去對抗變化的光線，不如加一個遮光罩與固定光源。工業視覺的成功，八成來自打光的設計而非演算法。

**原則二：不要只依賴單一感測。**視覺失效時應有備援——力覺可以確認接觸、光達可以偵測障礙、碰撞板是最後防線。這是第 6.12 節「互補而非重複」原則的延續。

**原則三：設計失效的偵測機制。**比「不失效」更重要的是「知道自己失效了」。例如：檢查偵測到的特徵點數目是否足夠、重投影誤差是否異常、深度圖的有效像素比例是否過低。一旦偵測到異常，系統應停止並求援，而非用錯誤的資料繼續動作。

### ◎ 工業視覺的第一課：打光

初學者常把 90% 的時間花在演算法上、10% 花在打光；資深工程師的比例恰好相反。原因是：**好的打光能讓困難的問題變簡單，而壞的打光會讓簡單的問題變不可能。**常見的打光技巧包括——背

光（拍輪廓，對量測外形極有效）、**同軸光**（拍平整反光表面，如晶圓）、**環形光**（通用照明）、**低角度掠射光**（凸顯表面刮痕與凹凸）、**藍光或紅外**（濾除環境光干擾）。面對一個視覺問題時，第一個該問的是「能不能換個打光方式讓目標特徵變得明顯？」而不是「該用什麼演算法？」

## 22.10 感知的心態轉換

### 22.10.1 由確定到機率

開頭故事中叫獸提到的「心態轉換」，值得在此展開。

第二篇與第三篇的世界是**確定性**的：給定關節角度，末端位置由運動學唯一確定；給定力矩，運動由動力學唯一確定。誤差來自模型的不完美，但問題本身有唯一的答案。

感知的世界則不然。同一個場景，換個光線就得到不同的影像；同一張影像，可能對應多種三維解釋（22.1.2 節的射線歧義）；同一個特徵，可能被錯誤匹配。**感知不是計算一個答案，而是在不完整、有雜訊的證據中做出最合理的推測。**

這個轉換帶來三個方法論上的改變：

- **輸出應包含不確定性。**不只回報「物體在  $(x, y, z)$ 」，還應回報「這個估計的信賴度」。這使後續的決策能適當地加權。
- **應主動處理外點。**感知資料必定含有錯誤（誤匹配、雜訊、失效的像素），演算法必須設計成對此穩健——RANSAC（22.5.3 節）正是這種思維的代表。
- **應整合多個來源。**單一感測的估計不可靠，融合多個互補的來源才能得到穩健的結果——這正是第 24 章的主題。

### 22.10.2 為第 24 章鋪路

這個心態轉換，直接引出了第 24 章的核心工具：**機率狀態估測**。

屆時我們會學到：如何用機率分布表示「我大概在這裡，但不確定有多大」、如何在移動時正確地累積不確定性、以及如何在取得新的觀測時最佳地更新估計（Kalman 濾波器與粒子濾波器）。

而本章的視覺，正是那些「觀測」的主要來源。第 25 章的 SLAM 則會把兩者結合——一邊用視覺觀測環境、一邊估計自己的位置、同時建立地圖。



## 22.11 由看見到理解

### 22.11.1 本章的成果

本章建立了由影像走向三維世界的完整鏈條。你現在能夠：用針孔模型描述成像過程、區分內外部參數並執行相機校正、理解深度為何丟失以及三種恢復的方式、運用特徵匹配與 RANSAC 建立影像間的對應、執行手眼校正把相機接回機器人座標、設計視覺伺服控制律、以及判斷深度學習與傳統幾何方法各自的適用場合。

更重要的是那個心態的轉換：**感知不是精確的計算，而是在雜訊中的合理推測**。這將貫穿整個第四篇。

### 22.11.2 接下來：讓機器人動起來

本章讓機器人「看見」了環境，但它仍然固定在原地。第四篇接下來的六章，要讓它真正走出去。

**第 23 章 ( 行動機器人運動學 )**：輪式機器人如何運動？差速輪、麥克納姆輪、阿克曼轉向各有什麼特性？為何有些機器人「無法直接橫 move」（非完整約束）？

**第 24 章 ( 狀態估測 )**：在充滿雜訊的世界中，如何最佳地估計自己的位置？Kalman 濾波器如何融合里程計與感測器？

**第 25 章 ( SLAM )**：如何一邊移動、一邊建立地圖、同時定位自己？這個「雞生蛋、蛋生雞」的問題如何求解？

**第 26 章 ( 路徑規劃 )**：如何找出一條避開障礙的路徑？A\*、RRT 等演算法如何運作？

**第 27、28 章**：無人機與腿式機器人這兩種特殊的移動形態。

換句話說：第 22 章讓機器人**看見**世界，第 23 ~ 28 章讓它**在世界中行動**。

## 完整計算例題

### 例題 22-1 相機投影與視野計算

題目：某工業相機的感光元件為 1/2.5 吋（有效尺寸  $5.70 \times 4.28 \text{ mm}$ ）、解析度  $1920 \times 1440$  像素，搭配焦距 8 mm 的鏡頭。（a）求以像素為單位的焦距  $f_x$  與  $f_y$ 。（b）若相機距離



工作平面 500 mm，求視野的實際尺寸。(c) 求每個像素對應工作平面上的實際尺寸(解析度)。(d) 若要量測 0.5 mm 的特徵，判斷此配置是否足夠。

解(a)：像素尺寸為感光元件尺寸除以像素數：

$$\text{像素寬} = 5.70 / 1920 = 0.002969 \text{ mm/pixel}$$

$$\text{像素高} = 4.28 / 1440 = 0.002972 \text{ mm/pixel}$$

兩者幾乎相同，代表像素接近正方形。

$$f_x = 8 / 0.002969 = 2694 \text{ pixels}$$

$$f_y = 8 / 0.002972 = 2692 \text{ pixels}$$

以像素為單位的焦距。這正是內部參數矩陣  $K$  中的值。

解(b)：由投影方程的相似三角形關係，視野(FOV)的實際尺寸為：

$$\text{視野寬} = (\text{感光元件寬} / \text{焦距}) \times \text{距離} = (5.70 / 8) \times 500 = 356.3 \text{ mm}$$

$$\text{視野高} = (4.28 / 8) \times 500 = 267.5 \text{ mm}$$

工作距離 500 mm 處的視野約  $356 \times 268 \text{ mm}$ 。

解(c)：每像素對應的實際尺寸：

$$\text{解析度} = 356.3 / 1920 = 0.1856 \text{ mm/pixel}$$

約 0.19 公釐。

解(d)：要量測 0.5 mm 的特徵，該特徵在影像中只佔  $0.5 / 0.1856 \approx 2.7$  個像素。

判定：不足。一般的經驗法則是「待測特徵至少要橫跨 3~5 個像素才能可靠偵測，若要精確量測則需 10 個像素以上」。因此此配置勉強能「看到」該特徵，但無法可靠量測。

改善方案有三：一、縮短工作距離(如降到 250 mm，解析度提升一倍，但視野減半)；二、使用更長的焦距鏡頭(如 16 mm，同樣使解析度加倍)；三、使用更高解析度的相機。三者的取捨在於視野、成本與景深——這正是視覺系統選型的核心權衡。

## 例題 22-2 立體視覺的深度精度

題目：某立體相機的基線  $B = 120 \text{ mm}$ 、焦距  $f = 1200$  像素、視差量測精度  $\Delta d = \pm 0.25$  像素。(a) 求距離 1 m、3 m、5 m 處的視差值。(b) 求三個距離處的深度誤差。(c) 若要求深度誤差不超過  $\pm 10 \text{ mm}$ ，最遠可測到多遠？(d) 若把基線加大到 240 mm，重新計算(c)。

解(a)：由  $Z = fB/d$  反解  $d = fB/Z$ ：

$$Z = 1000 \text{ mm} : d = 1200 \times 120 / 1000 = 144.0 \text{ pixels}$$

$$Z = 3000 \text{ mm} : d = 1200 \times 120 / 3000 = 48.0 \text{ pixels}$$

$$Z = 5000 \text{ mm} : d = 1200 \times 120 / 5000 = 28.8 \text{ pixels}$$

距離越遠，視差越小——這是精度劣化的根源。

解 (b)：由 22.4.2 節的誤差公式  $\Delta Z = Z^2 \cdot \Delta d / (f \cdot B)$ ：

$$Z = 1000 : \Delta Z = 1000^2 \times 0.25 / (1200 \times 120) = 250000 / 144000 = 1.74 \text{ mm}$$

$$Z = 3000 : \Delta Z = 3000^2 \times 0.25 / 144000 = 15.6 \text{ mm}$$

$$Z = 5000 : \Delta Z = 5000^2 \times 0.25 / 144000 = 43.4 \text{ mm}$$

深度誤差與距離的平方成正比。距離增為 5 倍，誤差增為 25 倍。

解 (c)：令  $\Delta Z \leq 10 \text{ mm}$ ：

$$Z^2 \times 0.25 / 144000 \leq 10 \Rightarrow Z^2 \leq 5.76 \times 10^6 \Rightarrow Z \leq 2400 \text{ mm}$$

最遠約 2.4 公尺。

解 (d)：基線加倍為 240 mm 時，分母加倍：

$$Z^2 \times 0.25 / 288000 \leq 10 \Rightarrow Z^2 \leq 11.52 \times 10^6 \Rightarrow Z \leq 3394 \text{ mm}$$

最遠約 3.4 公尺，改善了 41% (即  $\sqrt{2}$  倍)。

**工程判讀：**基線加倍只讓有效距離增加  $\sqrt{2}$  倍，而非 2 倍——因為誤差與  $Z^2$  成正比。這說明了**加大基線的效益是遞減的**。此外加大基線還有兩個代價：**共同視野縮小**（近處的物體可能只有一台相機看得到）、以及**體積增大**。這正是自駕車的立體相機基線通常在 20 ~ 50 公分（而非更大）的原因。

### 例題 22-3 像素座標到機器人座標的完整轉換

題目：某眼在手外的系統中，相機固定在工作台上方。已知：內部參數  $f_x = f_y = 800 \text{ pixels}$ 、主點  $(c_x, c_y) = (640, 480)$ ；相機到機器人基座的變換  ${}^B T_C$  為——旋轉為繞 X 軸  $180^\circ$ （相機朝下）、平移為  $(300, 200, 900) \text{ mm}$ 。現在相機偵測到一個零件在像素  $(740, 380)$ ，且由深度相機測得該點的深度  $Z_C = 600 \text{ mm}$ 。（a）求該點在相機座標系中的三維位置。（b）求其在機器人基座座標系中的位置。（c）若深度量測有  $\pm 10 \text{ mm}$  的誤差，估算對基座座標的影響。

解 (a)：由 22.1.3 節的投影方程反解。已知像素座標與深度：

$$\begin{aligned} X_C &= (u - c_x) \cdot Z_C / f_x = (740 - 640) \times 600 / 800 = 100 \times 0.75 = 75.0 \text{ mm} \\ Y_C &= (v - c_y) \cdot Z_C / f_y = (380 - 480) \times 600 / 800 = -100 \times 0.75 = -75.0 \text{ mm} \end{aligned}$$

$$Z\_C = 600.0 \text{ mm}$$

相機座標系中的位置。注意  $Y$  為負值，因為該像素在主點的上方（影像座標  $v$  向下為正）。

解（b）：套用齊次變換  ${}^B P = {}^B R\_C \cdot {}^C P + {}^B t\_C$ 。繞  $X$  軸旋轉  $180^\circ$  的旋轉矩陣為：

$${}^B R\_C = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

繞  $X$  軸  $180^\circ$ ： $Y$  與  $Z$  皆變號。

$${}^B R\_C \cdot {}^C P = (75.0, -(-75.0), -600.0) = (75.0, 75.0, -600.0)$$

旋轉後的向量。

$${}^B P = (75.0 + 300, 75.0 + 200, -600.0 + 900) = (375.0, 275.0, 300.0) \text{ mm}$$

零件在機器人基座座標系中的位置。這就是可以直接送給逆運動學的目標。

合理性檢查： $Z = 300 \text{ mm}$  為正值，代表零件在基座上方  $300 \text{ mm}$  處——合理（相機在  $900 \text{ mm}$  高，零件在其下方  $600 \text{ mm}$  處）。

解（c）：深度誤差  $\Delta Z\_C = \pm 10 \text{ mm}$  會同時影響三個分量。由（a）的關係式：

$$\begin{aligned} \Delta X\_C &= (u - c_x)/f_x \times \Delta Z\_C = (100/800) \times 10 = \pm 1.25 \text{ mm} \\ \Delta Y\_C &= (v - c_y)/f_y \times \Delta Z\_C = (-100/800) \times 10 = \mp 1.25 \text{ mm} \\ \Delta Z\_C &= \pm 10 \text{ mm} \end{aligned}$$

橫向誤差遠小於深度誤差。

經旋轉後，在基座座標中的誤差為  $\Delta X\_B = \pm 1.25$ 、 $\Delta Y\_B = \pm 1.25$ 、 $\Delta Z\_B = \mp 10 \text{ mm}$ 。

**重要觀察：**深度誤差主要影響垂直方向（因為相機朝下）。這對抓取有直接的意義——垂直方向的  $10 \text{ mm}$  誤差可能導致夾爪撞到工件或抓空。因應之道有二：一、加入力控制或阻抗控制（第 20 章），讓垂直方向能順應；二、以「先移動到上方安全高度、再緩慢下降直到接觸」的策略執行。這正是實務上視覺抓取幾乎總是搭配力覺或柔順控制的原因。

## 程式範例：相機模型、校正與座標轉換

以下 Python 程式實作本章的核心計算：相機投影與反投影、視野與解析度計算、立體視覺深度精度分析、以及像素到機器人座標的完整轉換鏈。

```
import numpy as np

# ===== 相機模型 =====
class Camera:
```

```

"""針孔相機模型 (含畸變) """
def __init__(s, fx, fy, cx, cy, dist=None):
    s.K = np.array([[fx, 0, cx], [0, fy, cy], [0, 0, 1]], float)
    s.fx, s.fy, s.cx, s.cy = fx, fy, cx, cy
    s.dist = np.zeros(5) if dist is None else np.array(dist, float)

def project(s, P_cam):
    """三維點 (相機座標) → 像素座標 (22.1.3 節) """
    X, Y, Z = P_cam
    if Z <= 0:
        return None # 在相機後方
    u = s.fx * X/Z + s.cx
    v = s.fy * Y/Z + s.cy
    return np.array([u, v])

def backproject(s, pixel, Z):
    """像素座標 + 深度 → 三維點 (相機座標) """
    u, v = pixel
    X = (u - s.cx) * Z / s.fx
    Y = (v - s.cy) * Z / s.fy
    return np.array([X, Y, Z])

def fov(s, width_px, height_px, distance):
    """求指定距離處的視野尺寸與解析度"""
    w = width_px * distance / s.fx
    h = height_px * distance / s.fy
    return w, h, w/width_px

def apply_distortion(s, x, y):
    """套用徑向與切向畸變 (22.2.2 節) """
    k1, k2, p1, p2, k3 = s.dist
    r2 = x*x + y*y
    radial = 1 + k1*r2 + k2*r2**2 + k3*r2**3
    x_d = x*radial + 2*p1*x*y + p2*(r2 + 2*x*x)
    y_d = y*radial + p1*(r2 + 2*y*y) + 2*p2*x*y
    return x_d, y_d

# ===== 例題 22-1：視野與解析度 =====
print("=== 例題 22-1：相機配置分析 ===")
sensor_w, sensor_h = 5.70, 4.28 # mm
res_w, res_h = 1920, 1440
focal_mm = 8.0

px_w, px_h = sensor_w/res_w, sensor_h/res_h
fx, fy = focal_mm/px_w, focal_mm/px_h
cam = Camera(fx, fy, res_w/2, res_h/2)
print(f"像素尺寸: {px_w*1000:.3f} × {px_h*1000:.3f} μm")
print(f"焦距(像素): fx = {fx:.0f}, fy = {fy:.0f}")

for dist in (250, 500, 1000):
    w, h, res = cam.fov(res_w, res_h, dist)
    n_px = 0.5 / res # 0.5 mm 特徵佔幾個像素

```

```

verdict = ("X 不足" if n_px < 3 else
           "△ 勉強" if n_px < 10 else "✓ 充足")
print(f"距離 {dist:5d} mm: 視野 {w:6.1f} × {h:6.1f} mm "
      f"解析度 {res:.4f} mm/px 0.5mm 特徵佔 {n_px:4.1f} px {verdict}")

# ===== 例題 22-2：立體視覺深度精度 =====
print("\n=== 例題 22-2：立體視覺的深度精度 ===")
def stereo_error(Z, f, B, dd=0.25):
    d = f*B/Z
    dZ = Z**2 * dd / (f*B)
    return d, dZ

f_st, B_st = 1200.0, 120.0
print(f"基線 B = {B_st:.0f} mm, 焦距 f = {f_st:.0f} px, 視差精度 ±0.25 px")
print(f"{'距離(mm)':>10}{'視差(px)':>12}{'深度誤差(mm)':>16}{'相對誤差':>12}")
for Z in (500, 1000, 2000, 3000, 5000):
    d, dZ = stereo_error(Z, f_st, B_st)
    print(f"{Z:10d}{d:12.1f}{dZ:14.2f}{dZ/Z*100:11.2f}%")

# 最遠可測距離
for B in (120.0, 240.0):
    Z_max = np.sqrt(10.0 * f_st * B / 0.25)
    print(f"基線 {B:5.0f} mm → 誤差 ≤ 10 mm 的最遠距離 = {Z_max:.0f} mm")
print("→ 基線加倍，有效距離只增加 √2 倍（因誤差與 Z2 成正比）")

# ===== 例題 22-3：完整座標轉換鏈 =====
print("\n=== 例題 22-3：像素 → 相機 → 機器人基座 ===")
cam2 = Camera(800., 800., 640., 480.)

def make_T(R, t):
    T = np.eye(4); T[:3,:3] = R; T[:3,3] = t
    return T

# 相機朝下：繞 X 軸 180°
R_BC = np.array([[1,0,0],[0,-1,0],[0,0,-1]], float)
t_BC = np.array([300., 200., 900.])
T_BC = make_T(R_BC, t_BC)

pixel, depth = np.array([740., 380.]), 600.
P_cam = cam2.backproject(pixel, depth)
P_base = (T_BC @ np.append(P_cam, 1.0))[0:3]
print(f"像素座標:      ({pixel[0]:.0f}, {pixel[1]:.0f}), 深度 {depth:.0f} mm")
print(f"相機座標系:      ({P_cam[0]:7.2f}, {P_cam[1]:7.2f}, {P_cam[2]:7.2f}) mm")
print(f"機器人基座系:    ({P_base[0]:7.2f}, {P_base[1]:7.2f}, {P_base[2]:7.2f}) mm")

# 驗證：把結果投影回像素應得到原值
P_cam_check = (np.linalg.inv(T_BC) @ np.append(P_base, 1.0))[0:3]
print(f"反向驗證(投影回像素): {np.round(cam2.project(P_cam_check), 2)}")

```

```

# 深度誤差的影響
print("\n 深度誤差 ±10 mm 的影響:")
for dz in (-10., 0., 10.):
    Pc = cam2.backproject(pixel, depth + dz)
    Pb = (T_BC @ np.append(Pc, 1.0))[0:3]
    print(f" ΔZ={dz:+6.1f} mm → 基座座標 ({Pb[0]:7.2f}, {Pb[1]:7.2f}, {Pb[2]:7.2f})")
print("→ 深度誤差主要轉化為垂直方向的誤差 (相機朝下時)")

# ===== 畸變的影響 =====
print("\n=== 鏡頭畸變對邊緣的影響 (k1 = -0.25 桶形畸變) ===")
cam_d = Camera(800., 800., 640., 480., dist=[-0.25, 0.08, 0, 0, 0])
print(f"{'歸一化半徑':>12}{'畸變後':>12}{'像素偏移':>12}")
for r in (0.0, 0.2, 0.4, 0.6, 0.8):
    xd, yd = cam_d.apply_distortion(r, 0.0)
    shift = abs(xd - r) * 800
    print(f"{r:12.2f}{xd:12.4f}{shift:10.1f} px")
print("→ 離中心越遠畸變越嚴重，邊緣可達數十像素，必須校正")

# ===== 影像雅可比 (22.7.2 節) =====
print("\n=== 影像雅可比：相機運動 → 特徵點速度 ===")
def image_jacobian(u, v, Z, f=1.0):
    """單點的 2x6 影像雅可比 (歸一化座標)"""
    return np.array([
        [-f/Z, 0, u/Z, u*v/f, -(f + u*u/f), v],
        [0, -f/Z, v/Z, f + v*v/f, -u*v/f, -u]])

u_n, v_n, Z_n = 0.2, 0.15, 1.0
L = image_jacobian(u_n, v_n, Z_n)
print(f"特徵點(歸一化) = ({u_n}, {v_n}), 深度 = {Z_n} m")
print(f"影像雅可比 L (2x6) =\n{np.round(L, 3)}")

motions = [("相機前進 +Z", [0,0,0.1,0,0,0]),
           ("相機右移 +X", [0.1,0,0,0,0,0]),
           ("相機繞 Z 旋轉", [0,0,0,0,0,0.1])]
for name, v_cam in motions:
    s_dot = L @ np.array(v_cam, float)
    print(f" {name:14s} → 特徵速度 ({s_dot[0]:+7.4f}, {s_dot[1]:+7.4f})")
print("→ 前進使特徵向外擴散、右移使特徵向左、旋轉使特徵繞中心轉")

```

請試著修改：(一) 把例題 22-1 的焦距改為 16 mm，觀察視野與解析度如何改變，驗證「焦距加倍則解析度加倍、視野減半」。(二) 在立體視覺分析中把視差精度改為  $\pm 0.1$  像素 (次像素匹配)，觀察有效距離如何延伸。(三) 在座標轉換中故意把手眼變換的旋轉設錯 5 度，觀察對最終位置的影響——這說明了手眼校正精度的重要性。

## 實作：相機校正與視覺定位

本章實作是建立一套完整的視覺定位系統，從校正到座標轉換。所需器材：一台 USB 相機或工業相機、棋盤格標定板（可自行列印並貼在硬板上）、以及第 9 章已建立運動學模型的機械手臂。

- 製作標定板：**列印一張 9×6 內角點的棋盤格（格子邊長建議 20~30 mm），貼在玻璃或壓克力板上確保平整。用游標卡尺精確量測格子的實際邊長。
- 相機校正：**從至少 20 個不同的角度與距離拍攝標定板，用 OpenCV 的 `calibrateCamera` 求出內部參數與畸變係數。**記錄重投影誤差**，依表 22-2 判斷品質。若超過 0.5 像素，檢查板子平整度與拍攝多樣性後重做。
- 畸變校正驗證：**拍攝一張含有直線的場景（如門框、桌邊），比較校正前後的影像。用尺在螢幕上量測，驗證校正後的直線確實是直的。
- 視野與解析度量測：**把標定板放在已知距離處，由影像中格子的像素數反推解析度，與例題 22-1 的理論計算比較。
- 手眼校正：**把標定板固定在工作台上，讓機器人移動到 15 個以上姿態（確保包含繞不同軸的顯著旋轉），每個姿態記錄關節角度與拍攝影像。用 OpenCV 的 `calibrateHandEye` 求出手眼變換。
- 手眼驗證：**移動到一個未參與校正的姿態，用求得的變換預測標定板某個角點在影像中的位置，與實際偵測比較。誤差應在 5 像素以內。
- 端到端定位測試：**在工作台上放一個目標物（可貼 AprilTag），由影像計算其在機器人基座座標中的位置，命令機器人移動到該位置上方。用尺量測實際偏差。
- 誤差來源分析：**分別引入三種誤差——相機校正誤差、手眼校正誤差、深度量測誤差——各自估計其對最終定位精度的貢獻，找出主要瓶頸。
- （進階）視覺伺服：**實作簡單的 IBVS——以四個特徵點為目標，用影像雅可比計算相機速度，驅動機器人使特徵收斂到期望位置。觀察特徵在影像中的運動軌跡。

表 22-8 視覺系統驗證記錄表（第一列為範例）

| 項目         | 設定       | 理論值 | 實測值           | 誤差 | 備註   |
|------------|----------|-----|---------------|----|------|
| 相機校正       | 25 張標定影像 | —   | 重投影誤差 0.28 px | —  | 品質優良 |
| （自行填寫其餘項目） |          |     |               |    |      |



## 國中小也看得懂：照片會騙人

看看窗外的馬路，或找一張鐵軌的照片。兩條鐵軌明明是平行的，為什麼照片上看起來會相交在遠方？

因為相機（還有你的眼睛）做了一件很厲害、但也很可惜的事：把立體的世界壓成了一張平的圖片。壓平的時候，有一個資訊被丟掉了——那就是「有多遠」。

來做個實驗：閉上一隻眼睛，然後伸出兩根食指，試著讓它們在空中對準碰在一起。是不是有點困難？現在張開兩隻眼睛再試一次——立刻就對準了！

為什麼？因為兩隻眼睛看到的畫面稍微不一樣。左眼看到的東西比較偏右，右眼看到的比較偏左。而且——越近的東西，兩眼看到的差別越大。你的大腦就是用這個「差別」來判斷距離的。

機器人也用同樣的方法！有些機器人身上裝了兩台相機，就像兩隻眼睛，靠比較兩張照片的差別來算距離。這叫做「立體視覺」。

不過這裡有一個有趣的問題：越遠的東西，兩張照片的差別越小。所以機器人看近的東西很準，看遠的東西就很不準。想想看你自己——你能準確說出手上這本書離你多遠嗎？可以。那遠方那棟大樓呢？就只能猜了。機器人跟你一樣！

最後再告訴你一件事：相機看到的東西是用「第幾個點」（像素）來描述的，但機器人的手臂只聽得懂「往前幾公分、往右幾公分」。所以工程師必須做一件叫做「校正」的工作，教會機器人把「照片上的位置」翻譯成「真實世界的位置」。這就像幫兩個說不同語言的人當翻譯一樣！

## 叫獸提醒與常見錯誤

- 誤區一：以為單張影像就能得到三維位置。影像上的一點對應空間中的一整條射線，必須額外提供深度資訊。
- 誤區二：忽略鏡頭畸變。廣角鏡頭在影像邊緣的畸變可達數十像素，未校正就做幾何計算會有數公分的誤差。
- 誤區三：校正時標定板不夠平。列印在紙上拿在手中拍是最常見的錯誤，必須貼在硬板上。
- 誤區四：校正時拍攝姿態不夠多樣。都從同一角度拍攝會使某些參數無法分離，應涵蓋不同

角度與距離。

- 誤區五：不檢查重投影誤差就相信校正結果。這是唯一能量化校正品質的指標，應低於 0.5 像素。
- 誤區六：手眼校正時運動缺乏旋轉。純平移的運動組合無法求解旋轉部分，必須包含繞不同軸的顯著旋轉。
- 誤區七：相機被碰撞後不重做手眼校正。這是視覺系統精度突然劣化最常見的原因。
- 誤區八：忽略立體視覺的深度誤差隨距離平方增長。同樣的配置在 1 m 處誤差 2 mm、5 m 處可能就是 43 mm。
- 誤區九：混淆內部參數與外部參數。前者出廠後固定（校正一次即可），後者隨安裝位置而變（移動就要重做）。
- 誤區十：只依賴視覺而無備援。視覺對透明、反光、無紋理物體都會失效，必須有力覺或其他感測作為交叉檢查。
- 誤區十一：把時間花在演算法而非打光上。好的打光能讓困難的問題變簡單，這是工業視覺的第一課。
- 誤區十二：忽略視覺伺服的取樣率限制。相機典型 30 Hz 而伺服迴路需 1 kHz，兩者的落差必須妥善處理。

## 課堂討論

1. 開頭故事中「橘子還是彈珠」的例子說明了尺度歧義。請討論：人類單眼看世界時如何克服這個問題？機器人能否借用同樣的策略？
2. 22.4.4 節提到「使用視覺標記在研究上像作弊，在工程上卻往往是最正確的決定」。請討論：在哪些情況下應該堅持不用標記？判斷的準則是什麼？
3. PBVS 保證空間軌跡漂亮但特徵可能離開視野，IBVS 保證特徵留在畫面但空間軌跡可能很怪。請為三個情境各選一種並說明理由：（a）精密插件對位；（b）大範圍移動到目標上方；（c）追蹤移動中的物體。
4. 22.9.2 節主張「控制環境比改進演算法有效」。請討論這個原則的適用邊界——在什麼情況下必須改進演算法而不能只靠控制環境？
5. 本章的心態轉換是「由確定到機率」。請回顧第二、三篇，找出一個當時被視為確定、但其

實也含有不確定性的量，並討論若要處理其不確定性需要做什麼改變。

## 章末摘要

- 針孔相機模型的投影方程為  $x = fX/Z$ 、 $y = fY/Z$ ；其中的「除以  $Z$ 」摧毀了深度資訊，使影像上一點對應空間中一條射線。
- 內部參數 ( $f_x, f_y, c_x, c_y$ ) 描述相機自身特性、出廠後固定；外部參數 ( $R, t$ ) 描述相機在世界中的位姿、隨安裝而變。
- 鏡頭畸變分徑向與切向兩類，離影像中心越遠越嚴重；必須先校正畸變再做任何幾何計算。
- 相機校正以已知幾何的標定板求出參數；品質以重投影誤差判斷，應低於 0.5 像素。
- 恢復深度有三種方式：第二個視角（立體視覺）、主動測距（結構光 / ToF）、先驗知識（PnP 或視覺標記）。
- 立體視覺的深度誤差與距離的平方成正比；加大基線的效益遞減（只增加  $\sqrt{}$  倍）。
- 特徵處理分偵測與描述兩步；ORB 適合即時系統、SIFT 適合高精度離線處理。
- 匹配必定產生外點，須以 RANSAC 剔除——其思想是「假設有一小群正確資料並找出它們」。
- 手眼校正求相機與機器人之間的變換，歸結為  $AX = XB$  問題，需要包含足夠旋轉多樣性的運動。
- 眼在手上視野可主動改變、可靠近取得高解析度；眼在手外不增加末端負載、視野固定。
- PBVS 在笛卡兒空間定義誤差（軌跡直觀但特徵可能離開視野）；IBVS 在影像空間定義誤差（特徵留在畫面但空間軌跡可能彎曲）。
- 影像雅可比描述相機運動與特徵點速度的關係；控制律需串接影像雅可比、伴隨變換與機器人雅可比的偽逆。
- 深度學習擅長語意理解與處理未知物體，傳統幾何方法擅長精確計算；實務採混合式架構。
- 視覺的七種常見失效：光線變化、反光、透明、遮蔽、無紋理、運動模糊、校正漂移。
- 系統設計三原則：控制環境比改進演算法有效、不依賴單一感測、設計失效偵測機制。
- 感知的心態轉換：不是精確的計算，而是在不完整且有雜訊的證據中做出最合理的推測。

## 觀念題與計算題

## 觀念題

1. 寫出針孔相機的投影方程，並說明「除以  $Z$ 」帶來的三個後果。
2. 區分內部參數與外部參數，並說明兩者在工程實務上的不同意義。
3. 說明徑向畸變與切向畸變的成因，以及為何畸變在影像邊緣特別嚴重。
4. 敘述相機校正的六個步驟，並說明棋盤格為何是理想的標定板。
5. 說明重投影誤差的定義與判讀標準，並列舉三個校正失敗的常見原因。
6. 列舉恢復深度的三種方式，並比較其所需條件與限制。
7. 推導立體視覺的深度誤差公式，並說明加大基線為何效益遞減。
8. 說明好的特徵點應具備的三個性質，以及為何角點適合而邊緣不適合。
9. 說明 RANSAC 的四個步驟與其核心思想。
10. 寫出手眼校正方程  $AX = XB$ ，並解釋其直觀意義。
11. 比較眼在手上與眼在手外的優缺點，並各舉兩個適用場合。
12. 比較 PBVS 與 IBVS 的誤差定義空間、對校正誤差的敏感度與軌跡特性。
13. 說明影像雅可比的作用，以及為何 IBVS 至少需要三個特徵點。
14. 說明視覺伺服的完整控制鏈，並指出其中串接了哪三個矩陣。
15. 比較深度學習與傳統幾何方法的擅長領域，並說明混合式架構的作法。
16. 列舉視覺系統的七種常見失效情境，並各提出一種因應方式。

## 計算題

1. 某相機感光元件  $6.4 \times 4.8 \text{ mm}$ 、解析度  $1280 \times 960$ ，鏡頭焦距  $12 \text{ mm}$ 。(a) 求像素尺寸與以像素為單位的焦距。(b) 求工作距離  $400 \text{ mm}$  處的視野尺寸與解析度。(c) 若要可靠量測  $0.3 \text{ mm}$  的特徵（需佔 5 個像素），求所需的最大工作距離。(d) 若改用  $25 \text{ mm}$  鏡頭，重新計算 (c)。
2. 某立體相機基線  $150 \text{ mm}$ 、焦距  $1000$  像素、視差精度  $\pm 0.2$  像素。(a) 求  $2 \text{ m}$  與  $4 \text{ m}$  處的視差與深度誤差。(b) 求深度相對誤差不超過 1% 的最遠距離。(c) 若把視差精度提升到  $\pm 0.05$  像素（次像素匹配），重新計算 (b)。(d) 比較「提升視差精度」與「加大基線一倍」哪一種改善較多。
3. 某眼在手上的系統，相機內部參數  $f_x = f_y = 950$ 、主點  $(640, 512)$ 。手眼變換  $T_C$  為：無

旋轉、平移 (50, 0, 80) mm。目前正向運動學得法蘭位姿  ${}^B T_F$  為：繞 Z 軸旋轉  $90^\circ$ 、平移 (400, 100, 500) mm。相機偵測到目標在像素 (800, 400)、深度 350 mm。(a) 求目標在相機座標系中的位置。(b) 求其在法蘭座標系中的位置。(c) 求其在基座座標系中的位置。

4. 某 IBVS 系統以四個特徵點控制，影像雅可比為  $8 \times 6$  矩陣。特徵點的目前位置與期望位置的誤差為  $e$  (8 維向量)，增益  $\lambda = 0.5$ 。(a) 說明為何需要用偽逆而非反矩陣。(b) 若某時刻  $L$  的最小奇異值為 0.02，討論可能發生的問題。(c) 提出兩種因應方式 (提示：回顧第 10.7.3 節與第 12.5 節)。(d) 若只用兩個特徵點，判斷系統是否可控並說明理由。

## 動手做與專題延伸

### 本章實作：視覺工具模組

建立本章的視覺計算模組，這將是第 23 ~ 28 章的感知基礎。

- **階段一**：實作相機模型類別 (投影、反投影、畸變、視野計算)，並用例題 22-1 與 22-3 驗證。
- **階段二**：以 OpenCV 實作相機校正流程，並自動產生校正報告 (含重投影誤差與參數表)。
- **階段三**：實作立體視覺的深度計算與精度分析工具，能繪出「距離 vs 深度誤差」曲線。
- **階段四**：實作完整的座標轉換鏈 (像素  $\rightarrow$  相機  $\rightarrow$  手眼  $\rightarrow$  基座)，並整合第 9 章的正向運動學。
- **階段五**：實作影像雅可比與 IBVS 控制律，在模擬中驗證特徵收斂。

### 專題延伸

- **視覺系統選型計算器**：撰寫一個工具，輸入「待測特徵尺寸、所需視野、可用工作距離」，自動計算並建議合適的相機解析度與鏡頭焦距組合，並標示各方案的取舍 (視野 vs 解析度 vs 景深)。這是視覺工程師實務上最常做的計算，把它自動化能省下大量時間。
- **校正品質的敏感度研究**：系統性地探討校正品質的影響因素——分別改變標定影像的數目 (5、10、20、40 張)、姿態多樣性 (全部正對 vs 涵蓋各角度)、以及標定板的平整度 (紙張 vs 硬板)。記錄各情況下的重投影誤差與參數估計值，量化每個因素的貢獻。這個實驗能建立對校正的直覺，避免日後盲目重做。
- **進階：PBVS 與 IBVS 的軌跡比較**。在模擬中實作兩種視覺伺服，讓相機由同一個初始位姿收斂到同一個目標位姿。分別繪出：(一) 相機在三維空間中的運動軌跡；(二) 特徵點在

影像平面上的運動軌跡。重複測試多種初始姿態，特別包含「需要繞光軸旋轉 180 度」的情形——你會看到 IBVS 出現著名的「相機退離」現象（相機先後退再前進），而 PBVS 則會讓特徵跑出視野。這個實驗能讓你深刻理解兩者的取舍。

## 參考文獻與延伸閱讀

- Hartley, R. and Zisserman, A., Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press (多視角幾何的權威著作)。
- Szeliski, R., Computer Vision: Algorithms and Applications, 2nd ed., Springer (電腦視覺的完整教材，線上免費提供)。
- Corke, P., Robotics, Vision and Control, 3rd ed., Springer (機器人視覺的實作導向教材，含大量程式範例)。
- Zhang, Z., "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000 (目前標準相機校正方法的原始論文)。
- Tsai, R. Y. and Lenz, R. K., "A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration," IEEE Trans. Robotics and Automation, 1989 (手眼校正的經典方法)。
- Chaumette, F. and Hutchinson, S., "Visual Servo Control, Part I & II," IEEE Robotics & Automation Magazine, 2006-2007 (視覺伺服的權威教學論文)。
- Fischler, M. A. and Bolles, R. C., "Random Sample Consensus," Communications of the ACM, 1981 (RANSAC 的原始論文)。
- Rublee, E. et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV 2011 (ORB 特徵)。
- OpenCV 官方文件：相機校正、手眼校正、特徵偵測與匹配的完整 API 與教學。
- Olson, E., "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA 2011 (視覺標記系統)。

# 第 23 章

## 行動機器人運動學

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為路邊停車的俯視連續圖：一台車經由「前進打右、後退打左」的四個步驟，逐步挪進與車身平行的車位；每一步以箭頭標出當下的運動方向，並以紅色虛線標註「無法直接橫移」。右半為三種構型的瞬時旋轉中心對照——差速驅動（ICR 可落在兩輪共軸的任意位置，含原地旋轉）、阿克曼轉向（ICR 為前後輪軸線的交點，故有最小轉彎半徑）、麥克納姆全向平台（無側滑約束，可任意方向平移）。畫面下緣文字：瞬時受限，全域自由——當下不能做的事，繞個路就能做到。



## 叫獸開講 「兩輪小車為什麼不能往正右方平移？」

叫獸把一台自製的差速輪小車放在地上，然後對全班說：「請你們告訴我怎麼讓它往**正右方**移動——注意，是平移，車頭方向不能改變。」

小翰馬上說：「右輪往後轉、左輪往前轉。」叫獸照做，車子原地打轉了半圈——車頭轉了，但位置沒動。

小妤說：「那兩輪同速往前？」車子直直往前開——位置動了，但方向是前方不是右方。

學生們接連提出各種組合，車子不是原地轉、就是往前後走、要不就畫一個弧。二十分鐘過去，沒有人能讓它「正右方平移」。

「因為那是**做不到的**。」叫獸終於揭曉，「不是你們笨，是這台車在物理上就沒有這個能力。輪子只能滾，不能橫著滑——這叫做**非完整約束**。」

他在白板上寫下一個對比：「第 4 章我們數過自由度。這台車在平面上的位形有三個變數： $x$ 、 $y$ 、還有車頭角度  $\theta$ 。但它只有兩個馬達。」

小翰想了想：「那不就是欠驅動嗎？就像第 18 章的倒立擺。」

「很接近，但**性質完全不同**，而且這個差別非常微妙。」叫獸拿起小車，「倒立擺是『有些狀態你到不了』——桿子倒下去就爬不起來。但這台車不一樣：**停車場裡任何一個位置與角度，它都到得了**。你只是不能『直接』過去，必須來回打方向盤，像路邊停車那樣。」

他在白板上畫了兩個詞：**瞬時受限、全域自由**。「這就是非完整系統最迷人的地方——當下能走的方向被限制了，但長期能到的地方沒有被限制。這兩件事同時成立，聽起來矛盾，卻是真的。」

小妤問：「那為什麼不乾脆設計成能橫移的？」

「可以啊，那叫麥克納姆輪，掃地機器人展場上那種能橫著走的平台就是。」叫獸點頭，「但它的代價是：輪子結構複雜、容易卡髒東西、效率低、承載差、而且**里程計極不準**。工程上永遠是取捨——多數場合，兩個馬達加兩個輪子的簡單方案就夠了，剩下的用**規劃**來補。」

「所以今天要學兩件事。第一，**這些車子能怎麼動**——各種輪型的運動學模型。第二，**當下不能做的事，怎麼靠規劃繞過去**。學完之後，你會理解為什麼路邊停車需要來回三次，以及為

什麼機器人也一樣。」

## 本章為什麼重要

第二篇與第三篇處理的都是**固定基座**的機械手臂：基座不動，只有關節在轉。但機器人的另一大類——**移動機器人**——面對的是完全不同的問題。

差別的核心在於**約束的性質**。機械手臂的關節可以獨立控制，想讓末端往哪個方向動都可以（除非遇到第 12 章的奇異點）。但輪型機器人不同：**輪子只能沿其滾動方向前進，不能側滑**。這個看似微小的限制，帶來了機器人學中最深刻也最違反直覺的一類問題。

本章要建立三個層次的理解。**第一，運動學模型**：各種輪型構型（差速、阿克曼、全向）能產生什麼運動，以及如何由輪速推算車體運動。**第二，非完整約束的本質**：為何「當下受限」不等於「到不了」，以及這對控制與規劃的深遠影響。**第三，里程計與其極限**：如何由輪子的轉動推算位置，以及為何這個推算必定會漂移——這正是第 24、25 章要處理的問題。

本章也是第四篇的樞紐。第 24 章的狀態估測需要本章的運動模型作為預測方程、第 25 章的 SLAM 建立在里程計之上、第 26 章的路徑規劃必須考慮本章的運動約束。

## 學習目標

- 能說明輪子的四種基本類型及其約束特性。
- 能推導差速驅動機器人的正向與逆向運動學。
- 能說明瞬時旋轉中心的概念並用它分析各種輪型構型。
- 能定義非完整約束，並區分其與完整約束的差異。
- 能說明「瞬時受限但全域可達」的意義及其對規劃的影響。
- 能區分非完整約束與欠驅動，並說明兩者的關係。
- 能推導阿克曼轉向的運動學模型並說明其與差速驅動的差異。
- 能說明全向輪與麥克納姆輪的原理、優勢與代價。
- 能實作里程計的積分計算，並分析其誤差來源。
- 能說明里程計必定漂移的原因，並提出改善策略。
- 能為特定應用選擇合適的移動平台構型並說明理由。

## 先備知識

第 4 章的自由度與機構分析、第 8 章的座標系與旋轉矩陣（二維版本）、第 6.2 節的編碼器、第 11 章的雅可比概念（本章的運動學矩陣扮演類似角色）。

數學方面需要：二維旋轉、簡單的微分方程積分、以及矩陣的秩。本章的數學比第三篇簡單得多，但幾何直覺與物理意義更為關鍵——建議多用手邊的玩具車或椅子實際比劃。

## 核心名詞中英對照

表 23-1 本章核心名詞

| 中文     | 英文                                    | 說明                |
|--------|---------------------------------------|-------------------|
| 移動機器人  | Mobile Robot                          | 能在環境中移動的機器人       |
| 差速驅動   | Differential Drive                    | 以兩獨立驅動輪控制的構型      |
| 阿克曼轉向  | Ackermann Steering                    | 汽車式的前輪轉向構型        |
| 全向輪    | Omni Wheel                            | 可側向滑動的特殊輪子        |
| 麥克納姆輪  | Mecanum Wheel                         | 輪面有斜向滾子的全向輪       |
| 瞬時旋轉中心 | Instantaneous Center of Rotation, ICR | 剛體瞬時繞其轉動的點        |
| 完整約束   | Holonomic Constraint                  | 只限制位形、可積分的約束      |
| 非完整約束  | Nonholonomic Constraint               | 限制速度方向、不可積分的約束    |
| 位形空間   | Configuration Space                   | 描述機器人所有可能位形的空間    |
| 可達性    | Reachability / Controllability        | 能否到達位形空間中的任意點     |
| 李括號    | Lie Bracket                           | 衡量兩個運動方向組合能否產生新方向 |
| 里程計    | Odometry                              | 由輪子轉動推算位置的方法      |
| 航位推算   | Dead Reckoning                        | 由運動歷史推算當前位置       |
| 輪距     | Wheel Base / Track Width              | 兩驅動輪之間的距離         |
| 打滑     | Slip                                  | 輪子與地面之間的相對滑動      |
| 系統誤差   | Systematic Error                      | 可重複、可校正的誤差        |
| 隨機誤差   | Random Error                          | 不可預測、只能統計描述的誤差    |

## 23.1 輪子的類型與約束

### 23.1.1 四種基本輪型

移動機器人的能力，根本上取決於使用什麼輪子。常見的有四種。

**類型一：標準固定輪 (Fixed Wheel)**。最常見的輪子，只能繞輪軸滾動，轉向軸固定在車體上。它施加一個關鍵的約束：**輪子不能沿其軸向側滑**。差速驅動車的兩個驅動輪就是這一類。

**類型二：可轉向輪 (Steerable Wheel)**。在固定輪的基礎上，增加一個垂直的轉向軸，可以改變滾動方向。汽車的前輪、以及許多 AGV 的驅動輪屬於此類。

**類型三：腳輪 (Caster Wheel)**。轉向軸與輪心有一段偏移（像購物車的輪子）。它會自動對齊運動方向，因此**不施加任何約束**——純粹是支撐用途。

**類型四：全向輪 (Omni / Mecanum Wheel)**。輪面上安裝了許多可自由轉動的小滾子，使輪子在滾動的同時也能側向滑動。它**解除了不能側滑的約束**，代價見 23.6 節。

表 23-2 四種輪型的比較

| 輪型         | 自由度         | 是否施加約束   | 承載 | 里程計精度    | 典型應用         |
|------------|-------------|----------|----|----------|--------------|
| 標準固定輪      | 1 (滾動)      | 是 (不可側滑) | 高  | 高        | 差速車、AGV 驅動輪  |
| 可轉向輪       | 2 (滾動 + 轉向) | 是 (不可側滑) | 高  | 中高       | 汽車前輪、舵輪式 AGV |
| 腳輪         | 2 (被動)      | 否        | 中  | 不適用 (被動) | 支撐用、購物車      |
| 全向 / 麥克納姆輪 | 2 (滾動 + 側滑) | 否        | 低  | 低        | 全向平台、比賽機器人   |

### 23.1.2 「不能側滑」這個約束

標準輪的約束可以精確地寫出來。設輪子的位置為  $(x, y)$ 、輪面方向（滾動方向）與車體  $X$  軸夾角為  $\alpha$ ，則不可側滑的條件是：**輪子在其軸向的速度分量必須為零**。

$$-\dot{x} \sin(\theta + \alpha) + \dot{y} \cos(\theta + \alpha) = 0$$

不可側滑約束。 $\theta$  為車體的朝向角。這條式子說：速度向量必須垂直於輪軸，也就是必須沿著輪面方向。

這條方程式看似簡單，卻是本章所有討論的根源。它限制的是**速度**（含  $\dot{x}$ 、 $\dot{y}$ ），而非位置——這正是它與第 4 章那些機構約束的根本差異，也是 23.3 節「非完整」一詞的由來。

### 23.1.3 瞬時旋轉中心

有一個極為有用的幾何工具能統一分析各種輪型構型：**瞬時旋轉中心**（Instantaneous Center of Rotation, ICR）。

#### ◎ 瞬時旋轉中心

任何平面剛體的瞬時運動，都可以描述為「繞某一點的純旋轉」。這個點就是瞬時旋轉中心。若剛體在做純平移，則 ICR 位於無窮遠處。

ICR 的價值在於一個關鍵性質：**每一個標準輪的軸線，都必須通過 ICR**。原因很直觀——輪子只能沿輪面方向滾動，代表它的速度垂直於輪軸；而繞 ICR 旋轉的點，其速度也垂直於「該點到 ICR 的連線」。兩者要相符，輪軸就必須指向 ICR。

這給出了一個強大的分析工具：

- **差速驅動**：兩輪共軸，ICR 必在該共同軸線上（可以是軸線上任一點，因此可以原地旋轉）。
- **阿克曼轉向**：後輪共軸、前輪可轉向，ICR 為「後輪軸線」與「前輪軸線」的交點——這也是為何汽車無法原地旋轉（ICR 永遠在車體側方一定距離外）。
- **三輪或多輪皆固定**：若三個輪軸不交於一點，則系統被過度約束，只能直行或完全不能動。

圖 23-1 三種構型的瞬時旋轉中心

建議配圖：三格俯視圖。左格為差速驅動車——兩驅動輪共軸，虛線延伸的軸線上標出 ICR 可位於的任意位置（包含兩輪中點，即原地旋轉）。中格為阿克曼轉向車——後輪軸線與兩前輪軸線交會於一點（ICR），並標註內外前輪的轉角不同（阿克曼幾何）。右格為全向輪平台——三個或四個全向輪，標註「無側滑約束，ICR 可在平面上任意位置」。三格下方各標註對應的可行運動：原地旋轉、最小轉彎半徑、任意方向平移。

## 23.2 差速驅動運動學

### 23.2.1 最常見的構型

差速驅動（differential drive）是移動機器人最常見的構型：兩個同軸的獨立驅動輪，加上一至兩個腳輪支撐。掃地機器人、多數 AGV、教學小車都採用它。

它之所以普及，有四個原因：**機構極簡**（兩顆馬達、兩個輪子）、**能原地旋轉**（機動性佳，適合狹窄空間）、**運動學簡單**（易於控制與里程計）、**成本低**。

### 23.2.2 正向運動學

設兩輪的線速度分別為  $v_L$  (左輪) 與  $v_R$  (右輪)，輪距 (兩輪中心距) 為  $L$ 。車體中心的線速度與角速度為：

$$v = (v_R + v_L)/2 \quad \omega = (v_R - v_L)/L$$

*差速驅動的正向運動學。線速度是兩輪的平均、角速度正比於兩輪的差。這是本章最重要的一組公式。*

三種典型情形可以立刻讀出：

- $v_R = v_L$  :  $\omega = 0$ ，車子直行。
- $v_R = -v_L$  :  $v = 0$ ，車子原地旋轉 (ICR 在兩輪中點)。
- $v_L = 0$  : ICR 在左輪處，車子繞左輪畫圓。

接著把車體速度轉換到世界座標系。設車體位姿為  $(x, y, \theta)$ ：

$$\begin{aligned}\dot{x} &= v \cos \theta \\ \dot{y} &= v \sin \theta \\ \dot{\theta} &= \omega\end{aligned}$$

*車體的運動方程。這三條式子構成一個非線性系統，是第 24~26 章的共同基礎。*

### 23.2.3 逆向運動學

反過來，若要達成指定的  $(v, \omega)$ ，各輪應轉多快？由 23.2.2 節的兩式聯立求解：

$$v_R = v + \omega L/2 \quad v_L = v - \omega L/2$$

*差速驅動的逆向運動學。注意這個逆解永遠存在且唯一——與第 10 章手臂逆運動學的多解與無解形成鮮明對比。*

最後換算為馬達的角速度 (設輪半徑為  $r$ )：

$$\omega_R = v_R / r \quad \omega_L = v_L / r$$

*輪子的角速度。這才是真正送給馬達控制器的指令。*

### 23.2.4 轉彎半徑與速度限制

由  $v$  與  $\omega$  可求出瞬時的轉彎半徑：

$$R = v / \omega$$

*轉彎半徑。 $\omega = 0$  時  $R \rightarrow \infty$  (直行) ;  $v = 0$  時  $R = 0$  (原地旋轉)。*

實務上有一個重要的限制：**輪速不能超過馬達的最高速度**。這意味著  $v$  與  $\omega$  不能任意組合——高速行進時，可用的轉向能力會下降。

$$|v| + |\omega|L/2 \leq v_{\max}$$

速度限制條件。這在  $(v, \omega)$  平面上構成一個菱形的可行區域。

這個限制在軌跡追隨時必須考慮。若控制器算出的  $(v, \omega)$  超出可行區域，直接截斷會改變運動的方向；較好的作法是等比例縮小兩者，保持轉彎半徑不變而只降低速度。

## 23.3 非完整約束

### 23.3.1 兩種約束的根本差異

現在可以正式回答開頭故事的問題。約束分為兩大類，差別在於能否積分。

**完整約束 (Holonomic)**：只限制位形本身，可以寫成  $f(q) = 0$  的形式。例如第 4 章的閉環機構——連桿的長度限制了各關節角度的組合。它的效果是直接減少位形空間的維度。

**非完整約束 (Nonholonomic)**：限制的是速度，形式為  $f(q, \dot{q}) = 0$ ，且無法積分成只含  $q$  的形式。輪子的不可側滑就是典型例子。

完整約束： $f(q) = 0 \Rightarrow$  減少可達位形的維度

非完整約束： $f(q, \dot{q}) = 0$  (不可積分)  $\Rightarrow$  限制速度方向，但不減少可達位形

兩者的關鍵差異。這是本章最重要的概念。

### 23.3.2 為何「不可積分」如此關鍵

若不可側滑的約束能夠積分，就會得到一條形如  $g(x, y, \theta) = 0$  的方程——那代表車子被限制在一個曲面上，永遠到不了曲面外的位形。

但實際上它積分不出來。這意味著：約束只在每個瞬間限制速度的方向，卻沒有在位形空間中劃出任何禁區。車子當下不能側移，但透過一連串的前進、後退與轉向，它可以到達平面上的任何位置與任何朝向。

這正是路邊停車的數學解釋：你不能把車橫著推進車位，但可以透過「前進打右、後退打左」的組合逐步挪進去。每一步都遵守約束，組合起來卻達成了約束「禁止」的效果。

#### ◎ 李括號：為何組合能產生新方向

數學上，這個現象由李括號 (Lie bracket) 描述。設車子有兩個可控的運動方向：前進  $g_1$  與轉向  $g_2$ 。李括號  $[g_1, g_2]$  定義為「先做  $g_1$  再做  $g_2$ ，減去先做  $g_2$  再做  $g_1$ 」的淨效果。由於兩者不可交換（正如第 8 章的旋轉），這個差值不為零——而且它恰好指向側移的方向！Chow-Rashevskii



**定理**指出：若這些李括號能張滿整個位形空間，系統就是完全可控的。這給出了「瞬時受限但全域自由」的嚴格數學基礎，也解釋了為何「來回打方向盤」能產生側移。

### 23.3.3 自由度的三個數字

非完整系統有三個容易混淆的「自由度」數字，必須分清：

**表 23-3 差速驅動機器人的三個自由度**

| 名稱        | 數值 | 意義                           |
|-----------|----|------------------------------|
| 位形空間維度    | 3  | 描述位形需要 $(x, y, \theta)$ 三個變數 |
| 可控自由度（瞬時） | 2  | 當下只能控制 $v$ 與 $\omega$ 兩個量    |
| 可達自由度（全域） | 3  | 長期而言可到達任意 $(x, y, \theta)$   |

**第一與第三個數字相等，這是關鍵。**它代表系統雖然瞬時受限，卻沒有失去任何可達性。這也是非完整系統與欠驅動系統的分界（見下節）。

### 23.3.4 與欠驅動的區別

開頭故事中小翰的疑問值得深究：非完整與欠驅動有何不同？

兩者的共同點是「控制輸入數少於位形空間維度」。但**後果完全不同**：

**非完整系統**（如差速車）：可控自由度  $<$  位形維度，但**可達自由度 = 位形維度**。所有位形都到得了，只是路徑受限。

**欠驅動系統**（如倒立擺、無人機）：某些狀態根本無法穩定或到達。倒立擺的桿子若倒下超過某個角度就再也扶不起來（第 18 章的吸引域）。

因此兩者的**難處不同**：非完整系統的挑戰在**規劃**（如何找出一條合乎約束的路徑），欠驅動系統的挑戰在**控制**（如何穩定住不穩定的狀態）。這也預示了本章與第 26 章的關係——非完整約束會直接影響路徑規劃的方法。

## 23.4 阿克曼轉向

### 23.4.1 汽車式構型

阿克曼轉向（Ackermann steering）是汽車採用的構型：後輪固定驅動、前輪可轉向。自駕車、大型 AGV 與戶外移動平台多採此設計。

它的核心幾何要求是：**轉彎時，內外側前輪的轉角必須不同**。原因由 ICR 分析立刻可知——內外前輪到 ICR 的距離不同，因此其軸線指向 ICR 的角度也不同。若兩前輪同角度轉動（如簡單的平行連桿），輪胎必定有一個在打滑。

$$\tan \delta_{\text{inner}} = L/(R - W/2) \quad \tan \delta_{\text{outer}} = L/(R + W/2)$$

阿克曼幾何。 $L$  為軸距（前後輪距）、 $W$  為輪距（左右輪距）、 $R$  為轉彎半徑。內輪轉角恆大於外輪。

### 23.4.2 單車模型

為了簡化分析，常把左右輪合併為中間的一個輪子，稱為**單車模型**（bicycle model）。設前輪轉角為  $\delta$ 、後輪中心的速度為  $v$ ：

$$\begin{aligned}\dot{x} &= v \cos \theta \\ \dot{y} &= v \sin \theta \\ \dot{\theta} &= (v/L) \tan \delta\end{aligned}$$

單車模型的運動方程。這是自駕車與路徑追隨演算法最常用的模型。

與差速驅動比較，可以看出兩個關鍵差異：

**差異一：角速度與線速度耦合。** $\dot{\theta} = (v/L)\tan \delta$  中含有  $v$ ——車子不動就無法轉向。這與差速驅動能原地旋轉（ $v = 0$  而  $\omega \neq 0$ ）形成鮮明對比。

**差異二：最小轉彎半徑。**前輪轉角有機械極限  $\delta_{\text{max}}$ ，因此：

$$R_{\text{min}} = L / \tan \delta_{\text{max}}$$

最小轉彎半徑。這是阿克曼構型的根本限制，直接影響路徑規劃（第 26 章的 Dubins 路徑即由此而來）。

### 23.4.3 為何汽車採用這個構型

既然阿克曼的機動性明顯不如差速驅動（不能原地轉、有最小轉彎半徑），為何汽車都採用它？三個理由：

- **高速穩定性：**差速驅動在高速下容易因兩輪速度的微小差異而偏擺；阿克曼的前輪轉向提供了穩定的方向控制。
- **承載與效率：**後輪可用單一動力源加差速器驅動，機構簡單且能承受大扭力。
- **輪胎磨耗：**正確的阿克曼幾何使所有輪子都是純滾動，不會側滑，大幅延長輪胎壽命。差速驅動在轉彎時腳輪必定有側滑。

這是一個典型的工程取捨：**用機動性換取高速穩定性、承載能力與效率**。因此低速、室內、

需要靈活的場合用差速驅動；高速、戶外、大負載的場合用阿克曼。

**表 23-4 差速驅動與阿克曼轉向的比較**

| 面向     | 差速驅動                     | 阿克曼轉向                   |
|--------|--------------------------|-------------------------|
| 原地旋轉   | 可                        | 不可                      |
| 最小轉彎半徑 | 0                        | $L/\tan \delta_{\max}$  |
| 高速穩定性  | 較差                       | 佳                       |
| 機構複雜度  | 極簡                       | 中 ( 需轉向連桿與差速器 )         |
| 輪胎磨耗   | 較大 ( 腳輪側滑 )              | 小 ( 純滾動 )               |
| 運動學    | 解耦 ( $v$ 與 $\omega$ 獨立 ) | 耦合 ( $\theta$ 正比於 $v$ ) |
| 典型應用   | 掃地機器人、室內 AGV             | 汽車、自駕車、戶外平台             |

## 23.5 全向移動平台

### 23.5.1 解除約束的代價

既然非完整約束帶來這麼多麻煩，能不能乾脆消除它？答案是可以——用**全向輪**。

全向輪的原理是在輪面上安裝許多可自由轉動的小滾子。輪子在被驅動時沿滾動方向前進，同時可以沿滾子的方向自由滑動。這使「不可側滑」的約束消失了。

**麥克納姆輪 ( Mecanum wheel )** 是最常見的變體：滾子與輪軸成 45 度角。四個麥克納姆輪 ( 兩兩鏡像配置 ) 的組合，能產生平面上的完整三自由度運動——前後、左右、原地旋轉，且三者可任意組合。

### 23.5.2 麥克納姆輪的運動學

設四個輪子依序為左前、右前、左後、右後，輪半徑  $r$ ，車體半長  $L$ 、半寬  $W$ 。由車體速度  $(v_x, v_y, \omega)$  求各輪角速度：

$$\begin{aligned}\omega_{LF} &= (v_x - v_y - (L+W)\omega)/r \\ \omega_{RF} &= (v_x + v_y + (L+W)\omega)/r \\ \omega_{LR} &= (v_x + v_y - (L+W)\omega)/r \\ \omega_{RR} &= (v_x - v_y + (L+W)\omega)/r\end{aligned}$$

麥克納姆輪的逆運動學。注意四個式子中  $v_y$  與  $\omega$  的符號組合各不相同——這正是產生全向運動的關鍵。

這組公式有一個值得注意的結構：**四個輪子、三個自由度**。這代表系統是**冗餘驅動的** ( 第 5.4.2 節提過的 *overactuated* )。好處是可以分散負載；壞處是若四輪的速度不一致 ( 如某輪打

滑)，會產生內部的「打架」——輪子互相對抗，浪費能量並加速磨損。

### 23.5.3 全向平台的三大代價

**代價一：里程計精度極差。**滾子與地面的接觸面積小、且滾動方向不斷改變，打滑遠比標準輪嚴重。實務上麥克納姆平台的里程計誤差可達標準差速車的數倍——這使它幾乎必須依賴外部定位（第 25 章的 SLAM 或視覺標記）。

**代價二：承載與效率低。**滾子是點接觸或線接觸，承載能力遠不如整個輪面著地的標準輪。同時，斜向的滾子使部分驅動力被「浪費」在無用的方向上，效率典型只有標準輪的 70~80%。

**代價三：對地面敏感。**滾子的縫隙容易卡入砂石與毛髮；不平整的地面會使某些輪子懸空，瞬間失去該輪的貢獻，造成運動偏差。因此麥克納姆平台幾乎只用於平整的室內地面。

#### ◎ 為何工廠的 AGV 很少用麥克納姆輪

展場上的麥克納姆平台總是令人驚豔——能橫著走、能斜著走、能原地打轉。但走進真實的工廠，你會發現絕大多數的 AGV 仍是差速驅動或阿克曼構型。原因正是上述三個代價：工廠地面有油污與碎屑、需要長時間高負載運行、且對定位精度要求嚴格。全向能力是一個「好用但昂貴」的功能，只有在空間極度受限（如狹窄走道中的橫向靠泊）時才值得付出代價。這再次體現了工程上的取舍思維——多數場合，用規劃補足運動限制，比用機構解除限制更划算。

### 23.5.4 三輪全向構型

另一種常見的全向設計是三個全向輪呈 120 度均勻配置。它比麥克納姆簡單（三個輪子、三個自由度，剛好不冗餘）、運動學更對稱，常見於機器人足球賽（RoboCup）等競賽場合。

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} = \begin{bmatrix} 1 & -\sin(\theta_1) & \cos(\theta_1) & R \\ -1 & -\sin(\theta_2) & \cos(\theta_2) & R \\ r & -\sin(\theta_3) & \cos(\theta_3) & R \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

三輪全向平台的逆運動學。 $\theta$  為各輪的安裝角度（通常為  $0^\circ$ 、 $120^\circ$ 、 $240^\circ$ ）、 $R$  為輪子到中心的距離。

這個  $3 \times 3$  矩陣是可逆的（只要三輪不共線），因此正向運動學也有唯一解——這與差速驅動一樣，比機械手臂的逆運動學單純得多。

## 23.6 履帶與其他構型

### 23.6.1 履帶式（差速轉向）

履帶車（如挖土機、軍用機器人、救災機器人）在運動學上可視為「加寬的差速驅動」——

—左右履帶的速度差產生轉向。

但有一個關鍵差異：**履帶轉向必定伴隨大量的滑動**。因為履帶與地面是長條形接觸，轉彎時履帶的前後段必須相對地面滑移。這稱為**滑移轉向**（skid steering）。

後果是：

- **里程計極不準**：滑移量難以預測，且隨地面材質變化極大。實務上履帶車的航向誤差可達每公尺數度。
- **能耗高**：大量能量消耗在克服滑動摩擦上。
- **地面破壞**：轉向時會刮傷地面，這在室內或草坪上是問題。

但它的優勢也極為突出：**接地面積大、越野能力強、能爬樓梯與跨越障礙**。因此在戶外崎嶇地形、災難現場與軍事應用中不可取代。這類應用通常依賴 GNSS 或光達定位，而非里程計。

## 23.6.2 構型選擇的決策

整合前面各節，可以給出一個選型的決策準則：

表 23-5 移動平台構型的選型指引

| 需求           | 建議構型      | 理由               |
|--------------|-----------|------------------|
| 室內、狹窄空間、低速   | 差速驅動      | 能原地旋轉、機構簡單、里程計佳  |
| 高速、戶外、大負載    | 阿克曼轉向     | 高速穩定、承載佳、輪胎磨耗小   |
| 需要橫向靠泊、極窄走道  | 麥克納姆 / 全向 | 唯一能側移的方案，但需接受其代價 |
| 崎嶇地形、爬樓梯、越野  | 履帶        | 接地面積大、越野能力強      |
| 競賽、高機動性、平整場地 | 三輪全向      | 運動學對稱、機構較麥克納姆簡單  |
| 超大負載、需精確路徑   | 多舵輪（獨立轉向） | 各輪可獨立轉向，兼具全向與承載  |

## 23.7 里程計

### 23.7.1 由輪子推算位置

**里程計**（odometry）是最基本的定位方法：由輪子轉了多少，推算車子走了多遠、轉了多少角度。它屬於**航位推算**（dead reckoning）的一種——不依賴任何外部參照，純粹累積內部感測的資訊。

對差速驅動車，每個控制週期  $\Delta t$  內的計算為：

1. 讀取編碼器：得到左右輪的增量脈波數  $\Delta N_L$ 、 $\Delta N_R$ 。
2. 換算輪子位移： $\Delta s_L = 2\pi r \cdot \Delta N_L / (\text{PPR} \times 4 \times N_{\text{gear}})$ ，右輪同理（第 6.2.1 節的四倍頻與減速比）。
3. 計算車體位移與轉角： $\Delta s = (\Delta s_R + \Delta s_L) / 2$ ， $\Delta \theta = (\Delta s_R - \Delta s_L) / L$ 。
4. 更新位姿：積分到世界座標系。

### 23.7.2 積分方法的選擇

第四步的積分有兩種常見作法，精度差異值得注意。

方法一：直線近似（最簡單）。假設每個週期內車子沿直線移動：

$$\begin{aligned} x &\leftarrow x + \Delta s \cdot \cos \theta \\ y &\leftarrow y + \Delta s \cdot \sin \theta \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

直線近似。實作最簡單，但當  $\Delta \theta$  較大時誤差明顯。

方法二：中點近似（推薦）。用該週期的中間角度計算位移方向：

$$\begin{aligned} x &\leftarrow x + \Delta s \cdot \cos(\theta + \Delta \theta / 2) \\ y &\leftarrow y + \Delta s \cdot \sin(\theta + \Delta \theta / 2) \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

中點近似。只多一個除以 2，精度卻顯著提升——這是實務上的標準作法。

方法三：精確圓弧。若  $\Delta \theta \neq 0$ ，該週期的軌跡實際上是一段圓弧（半徑  $R = \Delta s / \Delta \theta$ ），可以精確積分：

$$\begin{aligned} x &\leftarrow x + R[\sin(\theta + \Delta \theta) - \sin \theta] \\ y &\leftarrow y - R[\cos(\theta + \Delta \theta) - \cos \theta] \\ \theta &\leftarrow \theta + \Delta \theta \end{aligned}$$

圓弧積分。理論上最精確，但當  $\Delta \theta \rightarrow 0$  時  $R \rightarrow \infty$ ，需特別處理數值問題。

**實務建議：**控制週期夠短（如 10 ~ 50 ms）時，中點近似的精度已足夠，且無數值問題。只有在低頻更新或高速旋轉時，才值得使用圓弧積分。

### 23.7.3 里程計的價值與定位

里程計有三個無可取代的優點：更新率極高（可達 kHz 級）、短期精度極佳、永不失效（不受環境影響，黑暗、無特徵、無 GNSS 都能運作）。

但它有一個致命缺陷：**誤差會累積且永不修正**。這正是第 6 章開頭故事中「步伐計數」的問題——走得越遠，錯得越多。

因此里程計在系統中的定位是：**提供高頻的短期預測，再由低頻但不漂移的外部感測（光達、視覺、GNSS）定期校正**。這正是第 24 章感測器融合與第 25 章 SLAM 的核心架構。

## 23.8 里程計的誤差分析

### 23.8.1 為何必定漂移

里程計的本質是**積分**——把每個微小的位移累加起來。而積分有一個數學上的必然性質：

#### ◎ 積分的宿命

任何量測都有誤差。當我們把一連串帶誤差的量測**累加**起來時，誤差也會被累加。而且關鍵在於：**沒有任何機制能讓誤差減小**——里程計不知道自己錯了，因為它沒有任何外部參照可以比對。這使得漂移不是「品質不好」的問題，而是**方法本身的宿命**。

更糟的是角度誤差的放大效應：**一個微小的航向誤差，會隨著行進距離線性放大成位置誤差**。

$$\text{位置誤差} \approx \text{行進距離} \times \text{航向誤差 (弧度)}$$

以 1 度 (0.0175 rad) 的航向誤差為例，走 10 公尺後橫向偏差就達 17.5 公分。

這解釋了一個實務現象：**里程計的位置誤差主要來自角度而非距離**。因此改善里程計最有效的方向，是提升航向的精度（例如融合陀螺儀，第 24 章）。

### 23.8.2 誤差的兩大類

Borenstein 的經典分類把里程計誤差分為兩類，處理方式完全不同。

**系統誤差 (Systematic)**：可重複、可預測的誤差，主要來自三個源頭：

- **輪徑不等**：兩輪的實際直徑有微小差異（製造公差、磨耗、胎壓）。這使機器人在指令直行時實際上會畫一個大弧。
- **輪距量測不準**：公式中的  $L$  若與實際不符，所有的角度計算都會有比例誤差。
- **輪徑量測不準**：使所有的距離計算都有比例誤差。

**隨機誤差 (Non-systematic)**：不可預測，主要來自：

- **打滑**：地面濕滑、加速過猛、碰撞。這是最嚴重也最難處理的一項。



- 地面不平：輪子跳動、瞬間懸空。
- 編碼器量化誤差：有限的解析度（第 6.2.1 節）。
- 離散取樣誤差：兩次取樣之間的運動被近似為直線或圓弧。

### 23.8.3 UMBmark 校正法

系統誤差是可以校正的，Borenstein 與 Feng 於 1996 年提出的 **UMBmark** 是標準方法。

其原理極為巧妙：讓機器人沿著一個正方形路徑行走，分別以順時針與逆時針各走數圈，然後量測回到起點時的偏差。

關鍵洞見是：兩種誤差在順逆時針下的表現不同。輪徑不等造成的偏差在順逆時針時方向相反（會互相抵消或加成），而輪距誤差造成的偏差則在兩者中同向。因此由兩組實驗的結果，可以分離出這兩個誤差源，並計算出對應的修正係數。

1. **準備**：在地面標出一個邊長 2~4 公尺的正方形，起點做精確標記。
2. **順時針實驗**：讓機器人沿正方形走五圈，記錄最終位置與起點的偏差（ $\Delta x, \Delta y$ ）。重複五次取平均。
3. **逆時針實驗**：同樣走五圈五次，記錄偏差。
4. **計算修正係數**：由兩組偏差的和與差，分別解出輪徑比修正係數  $E_d$  與輪距修正係數  $E_b$ 。
5. **驗證**：套用修正係數後重做實驗，偏差應顯著減小（通常可改善 3~10 倍）。

經過 UMBmark 校正後，典型的差速驅動機器人可以把系統誤差由數個百分點降到 0.5% 以下——這是一個投報率極高的程序，只需要一個下午與一條捲尺。

### 23.8.4 誤差的量化與模型

為了在第 24 章的感測器融合中使用里程計，必須先量化其誤差。常用的模型是把誤差視為與運動量成正比的隨機變數：

$$\sigma_{\Delta s^2} = k_s \cdot |\Delta s| \quad \sigma_{\Delta \theta^2} = k_\theta \cdot |\Delta \theta| + k_{s\theta} \cdot |\Delta s|$$

里程計的誤差模型。變異數正比於運動量——走得越多、轉得越多，不確定性越大。 $k$  值由實驗量測。

最後一項  $k_{s\theta} \cdot |\Delta s|$  特別值得注意：它代表**直行時也會累積航向誤差**（因為輪徑不等）。這一項往往是長距離漂移的主因。

表 23-6 里程計誤差來源與因應

| 誤差來源  | 類型 | 典型量級     | 因應方式              |
|-------|----|----------|-------------------|
| 輪徑不等  | 系統 | 0.5 ~ 3% | UMBmark 校正        |
| 輪距不準  | 系統 | 0.5 ~ 2% | UMBmark 校正        |
| 編碼器量化 | 隨機 | < 0.1%   | 提高解析度、裝在馬達端       |
| 打滑    | 隨機 | 0 ~ 數十 % | 限制加速度、偵測異常、融合其他感測 |
| 地面不平  | 隨機 | 1 ~ 5%   | 改善懸吊、融合 IMU       |
| 取樣離散化 | 隨機 | < 0.1%   | 提高更新率、用中點或圓弧積分    |

## 23.9 軌跡追隨控制

### 23.9.1 非完整系統的控制困難

有了運動學模型，下一步是控制：如何讓機器人走上指定的路徑？這裡有一個著名的理論障礙。

#### ◎ Brockett 條件

1983 年 Brockett 證明了一個令人意外的結果：非完整系統無法用任何連續的、非時變的狀態回授控制律，穩定到一個固定的位形點。換句話說——不存在一個形如  $u = f(x)$  的平滑控制器，能讓差速車由任意初始位姿收斂到指定的  $(x, y, \theta)$ 。

這個結論初看很違反直覺：明明第 23.3 節說過所有位形都到得了，為何不能用回授控制穩定過去？

直觀的理解是：問題出在「同時」。要讓  $x$ 、 $y$ 、 $\theta$  三個量同時收斂到零，但控制輸入只有兩個。當車子已經對準目標方向、位置也對了，卻只有  $\theta$  有誤差時——此時任何前進或後退都會破壞已經對好的位置。控制器陷入兩難。

因應之道有三：時變控制律（讓控制律隨時間變化，如加入振盪項）、不連續控制律（分階段切換，如「先轉向、再前進、最後調整角度」）、或放棄點穩定，改為軌跡追隨。實務上第三種最常用。

### 23.9.2 軌跡追隨與路徑追隨

實務上有兩種不同的任務，區別很重要：

**軌跡追隨 ( Trajectory Tracking )**：追隨一條帶時間的參考軌跡 ( $x_d(t)$ ,  $y_d(t)$ ,  $\theta_d(t)$ )。目標是「在第  $t$  秒到達第  $t$  秒該到的位置」。這與第 15 章的軌跡定義一致。

**路徑追隨 ( Path Following )**：只要求沿著幾何路徑行走，不規定何時到達某點。這在實務上更常用——因為速度可以依情況調整（遇到障礙就減速），而不會產生「落後於時間表」的累積誤差。

**關鍵差異**：軌跡追隨若因故落後，控制器會試圖加速趕上，可能導致危險；路徑追隨則只在乎「有沒有偏離路徑」，落後多少無所謂。因此戶外導航與自駕車幾乎都採用路徑追隨。

### 23.9.3 Pure Pursuit：最實用的路徑追隨法

**純追隨法 ( Pure Pursuit )** 是移動機器人最廣泛使用的路徑追隨演算法，因為它極為簡單且穩健。

它的思想模仿人類開車：**盯著前方一段距離的目標點，朝它轉向**。具體作法是：

1. 在參考路徑上，找出距離車子當前位置為  $L_d$  ( **前視距離**，look-ahead distance ) 的那個點，稱為目標點。
2. 計算通過車子當前位置、且以當前朝向為切線、並通過目標點的**圓弧**。
3. 沿此圓弧行進，即得所需的轉彎半徑與角速度。

由幾何關係可導出極簡的公式：

$$\kappa = 2 \cdot \sin(\alpha) / L_d \quad \Rightarrow \quad \omega = v \cdot \kappa = 2 \cdot v \cdot \sin(\alpha) / L_d$$

*Pure Pursuit 控制律*。 $\kappa$  為曲率、 $\alpha$  為「車頭方向」與「車子到目標點連線」的夾角。整個演算法只需要這一行。

**前視距離  $L_d$  是唯一的調參**，而它的影響極為直觀：

- $L_d$  太小：反應激烈、容易震盪（像新手駕駛只看車頭前方）。
- $L_d$  太大：轉彎時會「切內角」、路徑追隨不準（像看太遠而忽略眼前的彎道）。
- **實務作法**：讓  $L_d$  隨速度變化 ( $L_d = k \cdot v + L_{\min}$ )，高速時看遠、低速時看近——這正是人類駕駛的行為。

Pure Pursuit 的優點是簡單、穩健、無需調整複雜的增益；缺點是在高曲率路徑上會有穩態的內切誤差。改良版本（如 Stanley 控制器、模型預測控制）能改善此問題，但複雜度也顯著提

高。

圖 23-2 Pure Pursuit 的幾何原理

建議配圖：俯視圖。畫出一條彎曲的參考路徑（灰色），車子（含車頭方向箭頭）位於路徑旁略有偏離。以車子為圓心、 $L_d$  為半徑畫一個圓，與參考路徑交於一點——即目標點（紅色）。由車子當前位置出發、以車頭方向為切線、通過目標點畫一段圓弧（藍色虛線），標註其曲率  $\kappa$  與轉彎半徑  $R = 1/\kappa$ 。並標出夾角  $\alpha$ 。右側另附兩個小圖，比較  $L_d$  太小（軌跡震盪）與太大（轉彎切內角）的效果。

## 23.10 從運動學到動力學

### 23.10.1 何時運動學就夠了

本章的所有討論都停留在**運動學**層次——只談速度與幾何，沒有考慮質量、慣性與力。這個簡化在多數情況下是合理的，原因有三：

- **移動機器人通常低速運行**：室內 AGV 多在每秒 0.5 ~ 1.5 公尺，慣性效應相對於摩擦與驅動 force 並不顯著。
- **輪速可由內迴路精確控制**：馬達驅動器內部的速度迴路（第 5、16 章）已經處理了動力學，上層只需下達速度指令。
- **地面提供了大量阻尼**：滾動摩擦使系統自然穩定，不像機械手臂那樣容易振盪。

因此對多數室內移動機器人而言，**運動學模型加上速度控制的內迴路，已經足夠**。這與第三篇機械手臂必須處理完整動力學的情況形成鮮明對比。

### 23.10.2 何時必須考慮動力學

但在四種情況下，運動學模型會失效：

**情況一：高速運動**。速度超過每秒 2 ~ 3 公尺時，慣性使機器人無法瞬間改變速度。此時下達的速度指令與實際運動之間會有明顯的落差。

**情況二：急轉彎**。向心加速度  $a = v^2/R$  可能超過摩擦力所能提供的極限，導致側滑——此時不可側滑的約束被打破，運動學模型完全失效。這正是汽車過彎打滑的物理。

不打滑的條件： $v^2/R \leq \mu g$

以  $\mu = 0.7$ 、 $R = 2\text{ m}$  為例，最高過彎速度僅約  $3.7\text{ m/s}$ 。這是路徑規劃時的重要限制。

**情況三：重載或斜坡**。重力分量與大慣量使加減速能力受限，可能無法執行規劃的速度剖面。

**情況四：需要精確的力控制。**例如移動機器人推動一個重物，或在崎嶇地形上維持牽引力。

這些情況下，需要引入完整的動力學模型——而第 13、14 章建立的工具（慣性、摩擦、力矩限制）就再次派上用場。第 27 章的無人機與第 28 章的腿式機器人，更是完全無法用運動學近似處理。

## 23.11 由運動邁向定位

### 23.11.1 本章的成果

本章建立了移動機器人的運動基礎。你現在能夠：分析四種輪型的約束特性、用瞬時旋轉中心分析任何構型、推導差速驅動與阿克曼轉向的運動學、理解非完整約束「瞬時受限但全域自由」的本質、實作里程計並分析其誤差、執行 UMBmark 校正、以及用 Pure Pursuit 實現路徑追隨。

最重要的是那個核心洞見：**非完整約束限制的是「怎麼走」，而非「能到哪裡」**。這使移動機器人的挑戰從「控制」轉向「規劃」——這正是第 26 章的主題。

### 23.11.2 接下來：里程計不夠用

但本章也留下一個明確的缺口。23.8 節證明了里程計**必定漂移**——這不是工程品質的問題，而是「積分」這個方法的宿命。走十公尺可能偏差十幾公分，走一百公尺就可能偏差一公尺以上。對於需要長時間運行的機器人，這完全不可接受。

解方有兩個層次，正是接下來兩章的主題。

**第 24 章（狀態估測與感測器融合）**：里程計短期準但會漂移，光達或視覺不漂移但更新慢且有雜訊——**能不能把兩者的優點結合**？這一章將建立機率的框架，用 Kalman 濾波器與粒子濾波器回答這個問題。屆時你會發現，第 6.3.2 節那個「互補濾波」的一行公式，其實是 Kalman 濾波器的簡化版本。

**第 25 章（SLAM）**：更進一步的問題是——如果連地圖都沒有呢？機器人如何**一邊探索未知環境、一邊建立地圖、同時還要知道自己在哪裡**？這個看似雞生蛋的問題，是機器人學最著名的難題之一。

換句話說：第 23 章讓機器人**知道自己動了多少**，第 24、25 章讓它**知道自己真正在哪裡**。

## 完整計算例題

### 例題 23-1 差速驅動的正逆運動學

題目：某差速驅動機器人的輪半徑  $r = 0.05 \text{ m}$ 、輪距  $L = 0.30 \text{ m}$ 。(a) 若左輪以  $3.0 \text{ rad/s}$ 、右輪以  $5.0 \text{ rad/s}$  轉動，求車體的線速度、角速度與轉彎半徑。(b) 若要讓機器人以  $0.4 \text{ m/s}$  前進、同時以  $0.5 \text{ rad/s}$  左轉，求兩輪的角速度。(c) 若馬達的最高角速度為  $12 \text{ rad/s}$ ，求機器人的最高直行速度與最高原地旋轉角速度。(d) 若機器人以  $0.5 \text{ m/s}$  行進，求此時可用的最大角速度。

解 (a)：先由輪的角速度求線速度：

$$\begin{aligned} v_L &= r \cdot \omega_L = 0.05 \times 3.0 = 0.15 \text{ m/s} \\ v_R &= r \cdot \omega_R = 0.05 \times 5.0 = 0.25 \text{ m/s} \end{aligned}$$

兩輪的線速度。

$$\begin{aligned} v &= (v_R + v_L)/2 = (0.25 + 0.15)/2 = 0.20 \text{ m/s} \\ \omega &= (v_R - v_L)/L = (0.25 - 0.15)/0.30 = 0.333 \text{ rad/s} \end{aligned}$$

車體的線速度與角速度。 $\omega$  為正代表左轉（逆時針）。

$$R = v/\omega = 0.20/0.333 = 0.60 \text{ m}$$

轉彎半徑  $0.6$  公尺，圓心在車體左側。

解 (b)：由逆運動學公式：

$$\begin{aligned} v_R &= v + \omega L/2 = 0.40 + 0.5 \times 0.15 = 0.475 \text{ m/s} \\ v_L &= v - \omega L/2 = 0.40 - 0.5 \times 0.15 = 0.325 \text{ m/s} \end{aligned}$$

兩輪的線速度。

$$\omega_R = 0.475/0.05 = 9.5 \text{ rad/s} \quad \omega_L = 0.325/0.05 = 6.5 \text{ rad/s}$$

兩輪的角速度。

解 (c)：最高直行速度發生在兩輪同時全速：

$$v_{\max} = r \cdot \omega_{\text{motor}_{\max}} = 0.05 \times 12 = 0.60 \text{ m/s}$$

最高直行速度。

最高原地旋轉角速度發生在兩輪反向全速 ( $v_R = 0.6$ 、 $v_L = -0.6$ )：

$$\omega_{\max} = (0.6 - (-0.6))/0.30 = 1.2/0.30 = 4.0 \text{ rad/s} \approx 229^\circ/\text{s}$$

最高原地旋轉角速度。

解 (d)：由 23.2.4 節的速度限制條件：

$$|v| + |\omega|L/2 \leq v_{\max} \Rightarrow 0.5 + |\omega| \times 0.15 \leq 0.6 \Rightarrow |\omega| \leq 0.667 \text{ rad/s}$$

以 0.5 m/s 行進時，最大角速度僅 0.667 rad/s (對應最小轉彎半徑 0.75 m)。

**工程觀察：**速度越快，可用的轉向能力越差。這是所有差速驅動機器人的共同限制，路徑規劃時必須考慮——高速通過的路段必須是平緩的曲線。

### 例題 23-2 里程計的積分與誤差累積

題目：承上機器人，編碼器為 500 PPR (正交四倍頻)、裝在馬達端、減速比 30。控制週期 20 ms。(a) 求每個編碼器計數對應的輪面位移。(b) 某週期內左輪計數 +120、右輪計數 +150，求該週期的  $\Delta s$  與  $\Delta\theta$ 。(c) 若初始位姿為 (0, 0, 0)，用中點近似更新位姿。(d) 若兩輪的實際直徑相差 1%，估算直行 10 公尺後的橫向偏差。

解 (a)：輪端每轉的計數為：

$$\text{每輪轉計數} = 500 \times 4 \times 30 = 60000 \text{ counts/rev}$$

四倍頻後再乘減速比 (第 6.2.1 節的技巧)。

$$\begin{aligned} \text{每計數位移} &= 2\pi r / 60000 = 2\pi(0.05) / 60000 = 0.3142 / 60000 = 5.236 \times 10^{-6} \text{ m} \approx \\ &5.24 \mu\text{m} \end{aligned}$$

解析度極高。

解 (b)：

$$\begin{aligned} \Delta s_L &= 120 \times 5.236 \times 10^{-6} = 6.283 \times 10^{-4} \text{ m} = 0.6283 \text{ mm} \\ \Delta s_R &= 150 \times 5.236 \times 10^{-6} = 7.854 \times 10^{-4} \text{ m} = 0.7854 \text{ mm} \end{aligned}$$

兩輪的位移。

$$\begin{aligned} \Delta s &= (0.7854 + 0.6283) / 2 = 0.7069 \text{ mm} = 7.069 \times 10^{-4} \text{ m} \\ \Delta\theta &= (0.7854 - 0.6283) / 300 = 0.1571 / 300 = 5.236 \times 10^{-4} \text{ rad} = 0.030^\circ \end{aligned}$$

車體的位移與轉角。

驗算： $\Delta s / \Delta t = 7.069 \times 10^{-4} / 0.02 = 0.0353 \text{ m/s}$ ，這是一個合理的低速值。

解 (c)：用中點近似 (23.7.2 節)：

$$\begin{aligned} x &\leftarrow 0 + 7.069 \times 10^{-4} \times \cos(\theta + 5.236 \times 10^{-4} / 2) = 7.069 \times 10^{-4} \times \cos(2.618 \times 10^{-4}) \approx 7.069 \times 10^{-4} \text{ m} \\ y &\leftarrow 0 + 7.069 \times 10^{-4} \times \sin(2.618 \times 10^{-4}) \approx 1.851 \times 10^{-7} \text{ m} \\ \theta &\leftarrow 0 + 5.236 \times 10^{-4} = 5.236 \times 10^{-4} \text{ rad} \end{aligned}$$

更新後的位姿。因轉角極小， $x$  幾乎等於  $\Delta s$ 。



解 (d)：兩輪直徑相差 1%，代表在相同的編碼器計數下，兩輪的實際位移相差 1%。設平均輪徑對應的位移為  $s$ ，則：

$$\Delta s_R - \Delta s_L = 0.01 \times s \text{ (誤差)} \Rightarrow \text{累積轉角 } \Delta\theta_{\text{error}} = 0.01s/L$$

輪徑不等造成的虛假轉角。

直行 10 公尺 ( $s = 10 \text{ m}$ )：

$$\theta_{\text{error}} = 0.01 \times 10/0.30 = 0.333 \text{ rad} = 19.1^\circ$$

航向誤差高達 19 度！

這個誤差會使實際路徑彎曲。粗略估算橫向偏差（假設誤差均勻累積，取平均角度的一半）：

$$\text{橫向偏差} \approx 10 \times \sin(19.1^\circ/2) \approx 10 \times 0.166 = 1.66 \text{ m}$$

走 10 公尺，橫向偏差達 1.66 公尺。

這個結果令人震驚：僅僅 1% 的輪徑差異，就造成如此巨大的誤差。這正是 23.8.3 節 UMBmark 校正如此重要的原因——一個下午的校正工作，能把誤差降低數倍到十倍。

### 例題 23-3 阿克曼轉向與 Pure Pursuit

題目：某自駕小車採阿克曼構型，軸距  $L = 0.35 \text{ m}$ 、輪距  $W = 0.28 \text{ m}$ 、前輪最大轉角  $\delta_{\text{max}} = 30^\circ$ 。(a) 求最小轉彎半徑。(b) 當轉彎半徑為 2.0 m 時，求內外前輪的轉角。(c) 以 1.2 m/s 行駛、前輪轉  $20^\circ$  時，求車體的角速度。(d) 若採用 Pure Pursuit 且前視距離  $L_d = 1.0 \text{ m}$ ，目標點與車頭方向夾角  $\alpha = 25^\circ$ ，求所需的轉彎半徑與前輪轉角。

解 (a)：由 23.4.2 節的公式：

$$R_{\text{min}} = L/\tan \delta_{\text{max}} = 0.35/\tan(30^\circ) = 0.35/0.5774 = 0.606 \text{ m}$$

最小轉彎半徑約 0.61 公尺。注意這遠大於差速驅動的 0——這是阿克曼構型的根本限制。

解 (b)：由阿克曼幾何 (23.4.1 節)：

$$\begin{aligned} \tan \delta_{\text{inner}} &= L/(R - W/2) = 0.35/(2.0 - 0.14) = 0.35/1.86 = 0.1882 \Rightarrow \delta_{\text{inner}} = 10.66^\circ \\ \tan \delta_{\text{outer}} &= L/(R + W/2) = 0.35/(2.0 + 0.14) = 0.35/2.14 = 0.1636 \Rightarrow \delta_{\text{outer}} = 9.29^\circ \end{aligned}$$

內外輪轉角相差約 1.4 度。

工程意義：若採用簡單的平行連桿（兩輪同角度），必有一輪打滑。轉彎半徑越小，內外輪

的角度差越大，打滑越嚴重——這正是阿克曼連桿機構存在的理由。

解 (c)：由單車模型：

$$\dot{\theta} = (v/L) \cdot \tan \delta = (1.2/0.35) \times \tan(20^\circ) = 3.429 \times 0.3640 = 1.248 \text{ rad/s}$$

車體角速度約  $1.25 \text{ rad/s}$  ( $71.5^\circ/\text{s}$ )。

注意：若車子停止 ( $v = 0$ )，則  $\dot{\theta} = 0$ ——無論前輪轉多少角度都無法改變朝向。這與差速驅動能原地旋轉形成關鍵差異。

解 (d)：由 Pure Pursuit 公式：

$$\kappa = 2 \cdot \sin(\alpha) / L_d = 2 \times \sin(25^\circ) / 1.0 = 2 \times 0.4226 = 0.8452 \text{ /m}$$

$$R = 1/\kappa = 1.183 \text{ m}$$

所需的轉彎半徑。

由於  $R = 1.183 \text{ m} > R_{\min} = 0.606 \text{ m}$ ，此指令可執行。所需的前輪轉角為：

$$\delta = \arctan(L/R) = \arctan(0.35/1.183) = \arctan(0.2959) = 16.5^\circ$$

前輪應轉  $16.5$  度，在  $30$  度的極限內。

若  $\alpha$  更大會如何？例如  $\alpha = 60^\circ$  時， $\kappa = 2\sin(60^\circ)/1.0 = 1.732$ ， $R = 0.577 \text{ m} < R_{\min}$ ——超出車輛能力。此時控制器必須：一、增大前視距離  $L_d$ （看遠一點，使轉彎更平緩）；二、減速並倒車重新對準；或三、重新規劃路徑。這正是非完整約束對規劃的直接影響。

## 程式範例：運動學、里程計與路徑追隨

以下 Python 程式實作差速驅動與阿克曼的運動學、三種里程計積分方法的比較、輪徑誤差的影響模擬，以及 Pure Pursuit 路徑追隨。

```
import numpy as np

# ===== 差速驅動運動學 =====
class DiffDrive:
    def __init__(s, r=0.05, L=0.30, w_max=12.0):
        s.r, s.L, s.w_max = r, L, w_max

    def forward(s, wL, wR):
        """由輪角速度求車體速度 (23.2.2 節)"""
        vL, vR = s.r*wL, s.r*wR
        return (vR + vL)/2, (vR - vL)/s.L

    def inverse(s, v, w):
```

```

    """由車體速度求輪角速度 ( 23.2.3 節 ) """
    vR, vL = v + w*s.L/2, v - w*s.L/2
    return vL/s.r, vR/s.r

def limits(s):
    v_max = s.r*s.w_max
    w_max = 2*v_max/s.L
    return v_max, w_max

def max_omega_at(s, v):
    """給定線速度時的最大角速度 ( 23.2.4 節 ) """
    v_max, _ = s.limits()
    return max(0.0, 2*(v_max - abs(v))/s.L)

# ===== 例題 23-1 =====
print("=== 例題 23-1：差速驅動運動學 ===")
dd = DiffDrive()
v, w = dd.forward(3.0, 5.0)
print(f"(a)  $\omega_L=3.0$ ,  $\omega_R=5.0 \rightarrow v=\{v:.4f\}$  m/s,  $\omega=\{w:.4f\}$  rad/s, "
      f" $R=\{v/w:.3f\}$  m)")
wL, wR = dd.inverse(0.40, 0.5)
print(f"(b)  $v=0.40$ ,  $\omega=0.5 \rightarrow \omega_L=\{wL:.2f\}$ ,  $\omega_R=\{wR:.2f\}$  rad/s")
vm, wm = dd.limits()
print(f"(c) 最高直行  $v_{\max}=\{vm:.3f\}$  m/s, "
      f"最高原地旋轉  $\omega_{\max}=\{wm:.2f\}$  rad/s ( $\{np.rad2deg(wm):.0f\}^\circ/s$ )")
print(f"(d)  $v=0.5$  m/s 時最大  $\omega=\{dd.max\_omega\_at(0.5):.4f\}$  rad/s "
      f"(最小轉彎半徑  $\{0.5/dd.max\_omega\_at(0.5):.3f\}$  m)")

# ===== 三種里程計積分法 =====
print("\n=== 里程計積分方法的精度比較 ===")
def odom_step(x, y, th, ds, dth, method="midpoint"):
    if method == "straight":
        x += ds*np.cos(th); y += ds*np.sin(th)
    elif method == "midpoint":
        x += ds*np.cos(th + dth/2); y += ds*np.sin(th + dth/2)
    else:
        # exact arc
        if abs(dth) < 1e-9:
            x += ds*np.cos(th); y += ds*np.sin(th)
        else:
            R = ds/dth
            x += R*(np.sin(th + dth) - np.sin(th))
            y -= R*(np.cos(th + dth) - np.cos(th))
    return x, y, th + dth

# 測試：走一個半徑 1 m 的完整圓，比較回到起點的誤差
R_true, n_steps = 1.0, 200
ds = 2*np.pi*R_true/n_steps
dth = 2*np.pi/n_steps
print(f"測試：{n_steps} 步走完半徑 {R_true} m 的圓 ( 理論回到原點 ) ")
for m in ("straight", "midpoint", "arc"):
    x = y = th = 0.0
    for _ in range(n_steps):
        x, y, th = odom_step(x, y, th, ds, dth, m)

```

```

err = np.hypot(x, y)
print(f"  {m:10s}: 終點=({x:8.5f},{y:8.5f}) 誤差={err*1000:7.3f} mm")
print("  → 中點近似已足夠精確，圓弧積分最準但差異不大")

# ===== 例題 23-2：輪徑誤差的影響 =====
print("\n=== 例題 23-2：輪徑不等造成的漂移 ===")
def simulate_drift(dia_error_pct, distance=10.0, L=0.30, dt_ds=0.001):
    """模擬輪徑相差 dia_error_pct % 時直行 distance 公尺的漂移"""
    x = y = th = 0.0
    n = int(distance/dt_ds)
    for _ in range(n):
        # 指令直行，但右輪實際多走 error 比例
        dsL = dt_ds*(1 - dia_error_pct/200)
        dsR = dt_ds*(1 + dia_error_pct/200)
        ds, dth = (dsR + dsL)/2, (dsR - dsL)/L
        x, y, th = odom_step(x, y, th, ds, dth, "midpoint")
    return x, y, th

print(f"{'輪徑差':>8}{ '終點 X':>10}{ '終點 Y':>10}{ '航向誤差':>12}{ '橫向偏差':>12}")
for e in (0.1, 0.5, 1.0, 2.0):
    x, y, th = simulate_drift(e)
    print(f"{e:7.1f}% {x:10.3f} {y:10.3f} {np.rad2deg(th):10.2f}°"
          f" {y:11.3f} m")
print("  → 僅 1% 的輪徑差，走 10 m 就偏離超過 1.5 m")

# ===== 阿克曼轉向 (例題 23-3) =====
print("\n=== 例題 23-3：阿克曼轉向 ===")
class Ackermann:
    def __init__(s, L=0.35, W=0.28, d_max=np.deg2rad(30)):
        s.L, s.W, s.d_max = L, W, d_max
    def R_min(s):
        return s.L/np.tan(s.d_max)
    def steer_angles(s, R):
        di = np.arctan(s.L/(R - s.W/2))
        do = np.arctan(s.L/(R + s.W/2))
        return np.rad2deg(di), np.rad2deg(do)
    def yaw_rate(s, v, delta):
        return (v/s.L)*np.tan(delta)

ack = Ackermann()
print(f"(a) 最小轉彎半徑 R_min = {ack.R_min():.3f} m")
di, do = ack.steer_angles(2.0)
print(f"(b) R=2.0 m → 內輪 {di:.2f}°, 外輪 {do:.2f}° (差 {di-do:.2f}°)")
yr = ack.yaw_rate(1.2, np.deg2rad(20))
print(f"(c) v=1.2 m/s, δ=20° → θ̇={yr:.3f} rad/s ({np.rad2deg(yr):.1f}°/s)")

# ===== Pure Pursuit =====
print("\n=== Pure Pursuit 路徑追隨 ===")
def pure_pursuit(alpha_deg, Ld):
    """回傳曲率與轉彎半徑 (23.9.3 節)"""
    kappa = 2*np.sin(np.deg2rad(alpha_deg))/Ld
    R = np.inf if abs(kappa) < 1e-9 else 1/kappa
    return kappa, R

```

```

print(f"{'α':>6}{'L_d':>7}{'曲率 κ':>10}{'半徑 R':>10}{'前輪 δ':>10}{'可行?':>8}")
for alpha, Ld in ((25, 1.0), (25, 2.0), (60, 1.0), (60, 2.0), (10, 1.0)):
    k, R = pure_pursuit(alpha, Ld)
    delta = np.rad2deg(np.arctan(ack.L/R)) if np.isfinite(R) else 0.0
    ok = R >= ack.R_min()
    print(f"{alpha:5d}°{Ld:7.1f}{k:10.4f}{R:10.3f}"
          f"{'delta:9.2f'}{'✓' if ok else 'X 超出能力':>10}")
print(" → α 過大時需增大 L_d ( 看遠一點 ) 或減速重新對準")

# ===== Pure Pursuit 完整追隨模擬 =====
print("\n=== Pure Pursuit 追隨圓形路徑 ( 比較不同 L_d ) ===")
def follow_path(Ld, v=0.5, T=20.0, dt=0.01, R_path=2.0):
    """追隨一個半徑 R_path 的圓形路徑，回傳平均橫向誤差"""
    x, y, th = R_path, -0.3, 0.0          # 起點略偏離路徑
    errs = []
    for _ in range(int(T/dt)):
        # 找路徑上距離車子 Ld 的目標點 ( 圓形路徑可解析求解 )
        ang = np.arctan2(y, x)
        # 沿圓弧前進 Ld 對應的角度
        d_ang = Ld/R_path
        tx, ty = R_path*np.cos(ang + d_ang), R_path*np.sin(ang + d_ang)
        # 轉到車體座標
        dx, dy = tx - x, ty - y
        alpha = np.arctan2(dy, dx) - th
        kappa = 2*np.sin(alpha)/Ld
        w = v*kappa
        # 更新
        ds, dth = v*dt, w*dt
        x, y, th = odom_step(x, y, th, ds, dth, "midpoint")
        errs.append(abs(np.hypot(x, y) - R_path))
    return np.mean(errs[len(errs)//2:]), np.max(errs[len(errs)//2:])

print(f"{'L_d':>7}{'平均誤差':>14}{'最大誤差':>14} 行為")
for Ld in (0.2, 0.5, 1.0, 2.0, 4.0):
    mean_e, max_e = follow_path(Ld)
    behav = ("震盪" if Ld < 0.3 else "良好" if Ld <= 1.5
             else "切內角明顯")
    print(f"{Ld:7.1f}{mean_e*1000:12.1f} mm{max_e*1000:12.1f} mm {behav}")
print(" → L_d 太小則震盪、太大則內切；本例最佳約 0.5~1.0 m")

```

請試著修改：(一) 把輪徑誤差設為 0.1% (良好校正後的水準)，觀察 10 公尺的偏差降到多少。(二) 在 Pure Pursuit 中加入速度自適應的前視距離  $L_d = 0.5v + 0.3$ ，比較固定  $L_d$  的效果。(三) 把路徑改為一個有直線與急彎的組合，觀察哪一段的追隨誤差最大。

## 實作：里程計校正與路徑追隨

本章實作以實體移動機器人為主（可用附錄 F 的循跡車或任何差速驅動平台，需具備編碼器）。

1. **參數量測**：用游標卡尺量測輪徑（多量幾個方向取平均）、用鋼尺量測輪距（兩輪中心距）。記錄編碼器規格與減速比。
2. **里程計實作**：撰寫里程計程式，以中點近似積分。以固定週期（建議 20 ~ 50 ms）更新並輸出位姿。
3. **直線測試**：在地面拉一條直線，讓機器人指令直行 5 公尺。量測終點的實際位置與里程計輸出的差異，特別記錄**橫向偏差**與**航向偏差**。
4. **旋轉測試**：讓機器人原地旋轉 360 度（依里程計判定），量測實際的角度偏差。重複順逆時針各五次。
5. **UMBmark 校正**：依 23.8.3 節的流程，在地面標出 2×2 公尺的正方形，順逆時針各走五圈五次，記錄終點偏差。計算修正係數並套用。
6. **校正效果驗證**：重複步驟 3 與 4，量化校正前後的改善倍數。
7. **打滑實驗**：在光滑地面（如磁磚加水）與粗糙地面（如地毯）各重複直線測試，比較誤差的差異。這能直觀展示隨機誤差的影響。
8. **Pure Pursuit 實作**：規劃一條含直線與彎道的路徑（可用地面膠帶標示），實作 Pure Pursuit 控制器。
9. **前視距離調參**：測試  $L_d = 0.2$ 、 $0.5$ 、 $1.0$ 、 $2.0$  倍車長，記錄追隨誤差與行為（是否震盪、是否切內角）。找出最佳值。
10. **（進階）速度自適應**：實作  $L_d = k \cdot v + L_{\min}$ ，測試在不同速度下的表現，驗證是否優於固定  $L_d$ 。

表 23-7 里程計校正記錄表（前兩列為範例）

| 測試項目           | 校正前     | 校正後    | 改善倍數 | 備註      |
|----------------|---------|--------|------|---------|
| 直行 5 m 橫向偏差    | 18.3 cm | 2.1 cm | 8.7× | 主要為輪徑不等 |
| 原地旋轉 360° 角度誤差 | 12.5°   | 1.8°   | 6.9× | 主要為輪距誤差 |
| （自行填寫其餘測試）     |         |        |      |         |

## 國中小也看得懂：為什麼停車要來回好幾次

你有沒有看過爸媽路邊停車？車位明明就在旁邊，為什麼不能直接**橫著滑進去**，而要前進、後退、前進、後退好幾次？

答案很簡單：**輪子只能往前後滾，不能往旁邊滑**。這聽起來理所當然，但它帶來一個很有趣的結果——

**當下不能做的事，繞個路就能做到**。車子沒辦法直接橫移，可是只要「前進轉一點、後退轉一點」重複幾次，它就慢慢挪進車位了。你想去的地方**都去得了**，只是不能走直線過去。

機器人也一樣。掃地機器人只有兩個輪子，它不能橫著走，但它可以先原地轉個方向再往前——最後還是能走遍整個房間。

那有沒有能橫著走的輪子呢？有！叫做「麥克納姆輪」，輪子表面裝了很多斜斜的小滾輪，可以往任何方向移動，非常酷。但它有三個缺點：**容易卡髒東西、載不了重物、而且走起來不太準**。所以大部分的機器人還是用普通的輪子，寧可多轉幾次彎。

最後告訴你一件很重要的事：機器人怎麼知道自己走了多遠？它會**數輪子轉了幾圈**。但這個方法有個大問題——如果輪子打滑了，或者兩個輪子大小差了一點點，它就會算錯，而且**越走錯越多**。

有多嚴重呢？如果兩個輪子的大小只差了百分之一（大概比一根頭髮還細的差距），機器人走 10 公尺之後，會歪掉超過 1 公尺半！所以工程師必須很仔細地「校正」機器人，就像你考試前要對答案一樣。

## 叫獸提醒與常見錯誤

- 誤區一：以為非完整約束代表「到不了」。它只限制瞬時的運動方向，所有位形仍然可達——這與欠驅動系統有本質差異。
- 誤區二：混淆非完整與欠驅動。前者的挑戰在規劃、後者的挑戰在控制；差速車是非完整但完全可達，倒立擺則是真的有到不了的狀態。
- 誤區三：忽略速度與角速度的耦合限制。高速時可用的轉向能力會下降，超出時應等比例縮小而非直接截斷。



- 誤區四：以為阿克曼車能原地旋轉。 $\theta$  正比於  $v$ ，車子不動就無法改變朝向——這是與差速驅動最關鍵的差異。
- 誤區五：里程計用直線近似而非中點近似。多一個除以 2 的成本極低，精度卻顯著提升。
- 誤區六：忽略輪徑不等的影響。僅 1% 的差異，走 10 公尺就能造成 19 度的航向誤差與 1.6 公尺的偏差。
- 誤區七：跳過 UMBmark 校正。這是投報率最高的工程程序之一——一個下午能把系統誤差降低數倍到十倍。
- 誤區八：以為里程計的位置誤差主要來自距離。實際上主要來自角度——微小的航向誤差會隨距離線性放大。
- 誤區九：期待里程計能單獨長期使用。漂移是積分方法的宿命，必須與不漂移的外部感測融合（第 24 章）。
- 誤區十：為了全向能力而選麥克納姆輪，卻忽略其代價。里程計差、承載低、怕髒污，多數場合用規劃補足運動限制更划算。
- 誤區十一：Pure Pursuit 的前視距離設為固定值。應隨速度調整（高速看遠、低速看近），這才符合實際的駕駛行為。
- 誤區十二：忽略動力學的限制。急轉彎時  $v^2/R$  可能超過摩擦極限而側滑，此時運動學模型完全失效。

## 課堂討論

1. 開頭故事中，全班花二十分鐘也無法讓差速車「正右方平移」。請用瞬時旋轉中心的概念解釋為何這是不可能的，並說明為何「來回打方向盤」卻能達成同樣的效果。
2. 非完整約束「瞬時受限但全域自由」這個性質，對路徑規劃（第 26 章）會帶來什麼影響？請討論：規劃器需要額外考慮什麼？
3. 例題 23-2 顯示 1% 的輪徑差就造成 1.6 公尺的偏差。請討論還有哪些「微小的物理誤差被放大成巨大後果」的例子（可回顧第 9 章的運動學標定與第 12 章的奇異點）。
4. 麥克納姆輪解除了非完整約束，卻付出里程計、承載與可靠度的代價。請討論：在什麼情況下這個交易是划算的？並比較這與第 21 章「用機構解決問題 vs 用演算法解決問題」的思維。

5. Pure Pursuit 只有一個調參（前視距離），卻是工業上最常用的路徑追隨演算法。請討論「參數少」在工程實務上的價值，並與第 16 章 PID 調參的困難做對比。

## 章末摘要

- 輪子分四種：標準固定輪、可轉向輪（皆施加不可側滑約束）、腳輪（不施加約束）、全向輪（解除約束）。
- 瞬時旋轉中心（ICR）是分析各種構型的統一工具：每個標準輪的軸線都必須通過 ICR。
- 差速驅動的正向運動學： $v = (v_R + v_L)/2$ 、 $\omega = (v_R - v_L)/L$ ；逆解永遠存在且唯一。
- 差速驅動的速度限制： $|v| + |\omega|L/2 \leq v_{\max}$ ，因此高速時可用的轉向能力會下降。
- 完整約束限制位形（可積分、減少維度）；非完整約束限制速度方向（不可積分、不減少可達位形）。
- 非完整系統「瞬時受限但全域自由」：可控自由度  $2 < \text{位形維度 } 3$ ，但可達自由度仍為  $3$ 。
- 李括號解釋了為何運動的組合能產生新方向；Chow-Rashevskii 定理給出完全可控的條件。
- 非完整與欠驅動不同：前者所有位形都可達（挑戰在規劃），後者有些狀態根本到不了（挑戰在控制）。
- 阿克曼轉向的單車模型： $\theta = (v/L)\tan \delta$ ；角速度與線速度耦合，故不能原地旋轉，且有最小轉彎半徑  $L/\tan \delta_{\max}$ 。
- 全向輪解除了非完整約束，代價是里程計差、承載低、效率低、對地面敏感。
- 履帶車的滑移轉向必然伴隨大量滑動，里程計極不準但越野能力強。
- 里程計積分應使用中點近似（比直線近似精確、比圓弧積分簡單且無數值問題）。
- 里程計必定漂移，這是積分方法的宿命——沒有外部參照就無法修正誤差。
- 位置誤差主要來自航向誤差：位置誤差  $\approx \text{行進距離} \times \text{航向誤差}$ 。
- 誤差分系統誤差（輪徑不等、輪距不準，可用 UMBmark 校正）與隨機誤差（打滑、地面不平，只能統計描述）。
- Brockett 條件：非完整系統無法用連續非時變的狀態回授穩定到定點；實務上改用路徑追隨。
- Pure Pursuit 只需一個公式  $\kappa = 2\sin(\alpha)/L_d$  與一個調參（前視距離），是最實用的路徑追隨演算法。

## 觀念題與計算題

### 觀念題

1. 列舉四種輪型及其約束特性，並說明腳輪為何不施加約束。
2. 說明瞬時旋轉中心的定義，以及「標準輪的軸線必須通過 ICR」這個性質為何成立。
3. 推導差速驅動的正向與逆向運動學，並說明三種典型情形（直行、原地旋轉、繞單輪）。
4. 為何差速驅動的速度與角速度不能任意組合？超出限制時應如何處理？
5. 區分完整約束與非完整約束，並說明「可積分性」為何是關鍵。
6. 解釋「瞬時受限但全域自由」的意義，並以路邊停車為例說明。
7. 說明非完整約束與欠驅動的差異，並指出兩者的挑戰各在何處。
8. 寫出阿克曼轉向的單車模型，並說明它為何不能原地旋轉。
9. 為何阿克曼轉向需要內外前輪不同角度？若使用簡單的平行連桿會有什麼後果？
10. 說明全向輪的原理與三大代價，並討論其適用場合。
11. 比較里程計的三種積分方法，並說明實務上為何推薦中點近似。
12. 解釋里程計為何必定漂移，並說明為何位置誤差主要來自航向誤差。
13. 區分里程計的系統誤差與隨機誤差，並各列舉三個來源。
14. 說明 UMBmark 校正法的原理——為何順逆時針兩組實驗能分離兩種誤差源？
15. 敘述 Brockett 條件的內容，並說明實務上的三種因應方式。
16. 說明 Pure Pursuit 的原理與前視距離的影響，並解釋為何應隨速度調整。

### 計算題

1. 某差速驅動機器人輪半徑  $r = 0.075 \text{ m}$ 、輪距  $L = 0.42 \text{ m}$ 、馬達最高角速度  $15 \text{ rad/s}$ 。
  - (a) 若左輪  $4.0 \text{ rad/s}$ 、右輪  $7.0 \text{ rad/s}$ ，求  $v$ 、 $\omega$  與轉彎半徑。
  - (b) 若要以  $0.6 \text{ m/s}$  前進並以  $0.8 \text{ rad/s}$  右轉，求兩輪角速度並判斷是否超出馬達能力。
  - (c) 求最高直行速度與最高原地旋轉角速度。
  - (d) 求以  $0.8 \text{ m/s}$  行進時的最小轉彎半徑。
2. 某機器人編碼器 1000 PPR（正交四倍頻）、裝於馬達端、減速比 20、輪半徑  $0.06 \text{ m}$ 、輪距  $0.35 \text{ m}$ 。
  - (a) 求每個計數對應的輪面位移。
  - (b) 某週期左輪 +200 計數、右輪 +260 計數，求  $\Delta s$  與  $\Delta \theta$ 。
  - (c) 若初始位姿為  $(1.0, 2.0, 30^\circ)$ ，用中點近似更新。
  - (d) 若兩輪直徑相

差 0.8%，估算直行 20 公尺後的航向誤差。

3. 某阿克曼車軸距  $L = 2.7 \text{ m}$ 、輪距  $W = 1.6 \text{ m}$ 、前輪最大轉角  $35^\circ$ 。(a) 求最小轉彎半徑。(b) 轉彎半徑  $8 \text{ m}$  時，求內外前輪轉角及其差值。(c) 以  $15 \text{ m/s}$  行駛、前輪轉  $5^\circ$  時，求車體角速度與轉彎半徑。(d) 若路面摩擦係數  $\mu = 0.8$ ，判斷 (c) 的情形是否會側滑。
4. 某機器人採用 Pure Pursuit，前視距離  $L_d = 1.5 \text{ m}$ 。(a) 當目標點夾角  $\alpha = 30^\circ$  時，求曲率與轉彎半徑。(b) 若車輛為差速驅動且以  $1.0 \text{ m/s}$  行進，求所需的角速度。(c) 若改為阿克曼構型 ( $L = 0.5 \text{ m}$ )，求所需的前輪轉角。(d) 若  $\alpha$  增大到  $75^\circ$ ，重新計算 (a) 並討論可能的問題與因應方式。

## 動手做與專題延伸

### 本章實作：移動機器人運動學模組

建立移動機器人的程式庫，這將是第 24 ~ 26 章的共同基礎。

- 階段一：實作差速驅動的正逆運動學與速度限制檢查，用例題 23-1 驗證。
- 階段二：實作三種里程計積分方法，並用「走完整圓應回到原點」的測試比較精度。
- 階段三：實作誤差模擬——可設定輪徑誤差、輪距誤差與隨機打滑，觀察軌跡如何偏離。
- 階段四：實作阿克曼運動學與 Pure Pursuit 控制器，並在模擬中追隨各種路徑。
- 階段五：實作 UMBmark 校正的計算部分（由順逆時針的偏差資料反推修正係數）。

### 專題延伸

- **非完整約束的視覺化**：撰寫一個互動程式，讓使用者用滑鼠指定目標位姿，程式顯示差速車「無法直接過去」但「能透過一連串動作到達」。加入一個「可達性熱力圖」——顯示在固定時間內能到達的位形範圍，你會看到一個特殊的形狀（並非圓形），這直觀展示了非完整約束的效果。
- **里程計誤差的蒙地卡羅模擬**：對同一條路徑執行 1000 次模擬，每次加入隨機的輪徑誤差與打滑，記錄終點位置的分布。繪出散布圖與誤差橢圓——你會看到誤差主要沿著垂直於行進方向擴散（因為航向誤差的影響最大）。這張圖正是第 24 章 Kalman 濾波器中「協方差矩陣」的直觀來源。
- **進階：三種路徑追隨演算法的比較**。實作 Pure Pursuit、Stanley 控制器與一個簡單的 MPC，在同一組測試路徑（含直線、大彎、急彎、S 型）上比較。記錄橫向誤差、控制平順

度與計算時間。討論在什麼情況下該用哪一種——你會發現簡單的 Pure Pursuit 在多數情況下已經夠好，而這正是它成為工業標準的原因。

## 參考文獻與延伸閱讀

- Siegwart, R., Nourbakhsh, I. R. and Scaramuzza, D., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press ( 移動機器人的標準教材，第 3 章為運動學 ) 。
- Siciliano, B. et al., Robotics: Modelling, Planning and Control, Springer ( 第 11 章：移動機器人，含非完整約束的嚴謹處理 ) 。
- LaValle, S. M., Planning Algorithms, Cambridge University Press ( 線上公開 ) ( 第 13 章：微分約束下的規劃 ) 。
- Brockett, R. W., "Asymptotic Stability and Feedback Stabilization," in Differential Geometric Control Theory, 1983 ( Brockett 條件的原始論文 ) 。
- Borenstein, J. and Feng, L., "Measurement and Correction of Systematic Odometry Errors in Mobile Robots," IEEE Trans. Robotics and Automation, 1996 ( UMBmark 校正法 ) 。
- Coulter, R. C., "Implementation of the Pure Pursuit Path Tracking Algorithm," CMU Technical Report, 1992 ( Pure Pursuit 的經典技術報告 ) 。
- Snider, J. M., "Automatic Steering Methods for Autonomous Automobile Path Tracking," CMU Technical Report, 2009 ( 多種路徑追隨演算法的完整比較 ) 。
- Campion, G., Bastin, G. and D'Andrea-Novell, B., "Structural Properties and Classification of Kinematic and Dynamic Models of Wheeled Mobile Robots," IEEE Trans. Robotics, 1996 ( 輪型機器人的系統性分類 ) 。
- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press ( 第 5 章：運動模型與其機率描述，銜接第 24 章 ) 。
- ROS 2 導航堆疊 ( Nav2 ) 官方文件：里程計、路徑追隨控制器與移動平台整合的實務參考。

## 第 24 章

# 狀態估測與感測器融合

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為兩條位置估計曲線的對照：上圖是「直接覆蓋」的作法——曲線平順前進後每隔一段就猛地跳一下，轉彎處呈鋸齒狀，旁註「扔掉了資訊」；下圖是 Kalman 融合——曲線平滑且貼近真值，旁註「依可信度加權」。右半為誤差橢圓隨時間的演變序列：機器人沿路徑前進，橢圓逐步脹大並沿行進方向拉長（無量測時），經過地標時瞬間收縮成小圓。畫面下緣以兩個箭頭標註貝氏濾波的節奏——「預測：呼氣，不確定性增加」與「更新：吸氣，不確定性減少」。

## 叫獸開講 「兩個定位系統都不完全對，該相信哪一個？」

小妤的自製 AGV 有兩套定位系統。里程計每 20 毫秒更新一次，短時間內準得驚人；天花板上的視覺標記系統每秒只更新五次，但永遠不會累積誤差。

「兩個都有了，」她說，「所以我寫了一個切換邏輯：平常用里程計，每次看到標記就把位置覆蓋過去。」

叫獸請她示範。AGV 沿著走廊前進——螢幕上的位置平順地移動，然後每隔零點二秒，就猛地跳一下。跳動幅度有時三公分、有時七公分。到了轉彎處，車子甚至因為位置突變而急打方向盤，畫出一條鋸齒狀的軌跡。

「你看到問題了嗎？」叫獸問。小妤點頭：「跳來跳去。」

「更深一層的問題是——你在扔掉資訊。」叫獸在白板上畫了兩條線，「里程計說我在 A 點，視覺說我在 B 點，兩者差七公分。你選了 B，等於宣告『里程計完全錯了』。但里程計真的完全錯了嗎？」

「.....應該沒有那麼錯。」

「那視覺就完全對嗎？相機有雜訊、標記可能貼歪、光線會影響偵測。兩個都不完全對，也都不完全錯。」叫獸在 A 與 B 之間畫了一個點，「所以正確答案通常在中間某處。問題是——中間的哪裡？」

小翰試著說：「取平均？」

「接近了，但還不夠。」叫獸拿起兩支粗細不同的筆，「如果里程計剛校正過、只走了兩公尺，它的誤差可能只有一公分；而視覺標記在光線很暗的角落，誤差可能有十公分。這時候你還要各取一半嗎？」

教室安靜了幾秒。小妤說：「應該.....相信比較準的那個多一點。」

「這就是全部的祕密。」叫獸在白板上寫下一行字：依可信度加權平均。「而『可信度』要怎麼量化？答案是變異數——不確定性越小，權重越大。這個看似簡單的想法，在 1960 年被一位工程師寫成了嚴謹的數學，讓阿波羅太空船能夠導航到月球。它叫做 Kalman 濾波器。」

他停頓了一下，補上最後一句：「還有一件事。第 6 章我們用過一行公式做互補濾波——加



速度計配陀螺儀。當時我說『這是一個經驗法則』。今天你們會發現：那條式子其實是 Kalman 濾波器的簡化版，只是當年沒有告訴你們它從哪裡來。」

## 本章為什麼重要

第 23 章證明了一件殘酷的事：**里程計必定漂移**。這不是工程品質的問題，而是積分方法的宿命。但機器人不能因此就無法定位——它必須結合多種感測器，各取所長。

這個「結合」看似簡單，實際上是一個深刻的問題。不同感測器的更新率不同、精度不同、失效模式不同，而且**每一個讀值都帶著雜訊**。單純的切換或平均都會浪費資訊甚至造成危險（如開頭故事的跳動）。

本章要建立的，是一套**機率的思考框架**。核心的觀念轉換是：不再問「機器人在哪裡」，而是問「**機器人在各個位置的可能性各有多大**」。這個轉換使我們能夠：量化不確定性、依可信度融合多個來源、並在資訊不足時誠實地表達「我不確定」。

這套框架的具體工具是 Kalman 濾波器與粒子濾波器。它們不只用於定位——第 25 章的 SLAM、第 27 章的無人機姿態估計、第 28 章的腿式機器人狀態估測，全都建立在本章之上。可以說，**本章是第四篇的數學核心**。

## 學習目標

- 能說明機率式機器人學的核心觀念轉換及其必要性。
- 能定義信念分布並說明其與點估計的差異。
- 能敘述貝氏濾波的兩個步驟並說明各自的作用。
- 能說明高斯分布的性質及其為何適合表示不確定性。
- 能解釋協方差矩陣的幾何意義並繪製誤差橢圓。
- 能推導一維的資訊融合公式並說明權重的來源。
- 能敘述 Kalman 濾波器的五條方程式並說明各項的物理意義。
- 能說明 Kalman 增益的意義及其如何自動平衡預測與量測。
- 能說明擴展 Kalman 濾波器的線性化作法與其限制。
- 能說明粒子濾波器的原理並比較其與 Kalman 濾波器的適用場合。

- 能說明互補濾波與 Kalman 濾波器的關係。
- 能為特定應用選擇合適的估測方法並設定雜訊參數。

## 先備知識

第 23 章的運動模型（作為預測方程）、第 6 章的感測器特性與雜訊、第 22 章的視覺定位、第 18.2 節的狀態空間表示法。

數學方面需要：基本機率（條件機率、貝氏定理）、常態分布、矩陣運算。**不需要**測度論或嚴格的統計理論——本章會以幾何直覺與物理意義為主軸，公式的推導只在必要時進行。若對機率生疏，可先閱讀附錄 D.3。

## 核心名詞中英對照

表 24-1 本章核心名詞

| 中文            | 英文                           | 說明              |
|---------------|------------------------------|-----------------|
| 狀態估測          | State Estimation             | 由感測資料推斷系統內部狀態   |
| 信念            | Belief                       | 對狀態的機率分布表示      |
| 貝氏濾波          | Bayes Filter                 | 遞迴更新信念的通用框架     |
| 預測步驟          | Prediction Step              | 依運動模型推進信念       |
| 更新步驟          | Update / Correction Step     | 依量測修正信念         |
| 先驗            | Prior                        | 取得量測前的信念        |
| 後驗            | Posterior                    | 取得量測後的信念        |
| 似然            | Likelihood                   | 在給定狀態下觀測到該量測的機率 |
| 高斯分布          | Gaussian Distribution        | 以均值與變異數描述的常態分布  |
| 協方差矩陣         | Covariance Matrix            | 描述多維不確定性與相關性的矩陣 |
| 誤差橢圓          | Error Ellipse                | 協方差矩陣的幾何表示      |
| Kalman 濾波器    | Kalman Filter, KF            | 線性高斯系統的最佳估測器    |
| Kalman 增益     | Kalman Gain                  | 決定信任預測或量測的權重矩陣  |
| 擴展 Kalman 濾波器 | Extended Kalman Filter, EKF  | 以線性化處理非線性系統     |
| 無跡 Kalman 濾波器 | Unscented Kalman Filter, UKF | 以取樣點傳遞分布的方法     |
| 粒子濾波器         | Particle Filter, PF          | 以大量樣本表示任意分布     |
| 重取樣           | Resampling                   | 依權重重新抽取粒子的程序    |

| 中文   | 英文                   | 說明                |
|------|----------------------|-------------------|
| 粒子退化 | Particle Depletion   | 粒子多樣性喪失的現象        |
| 互補濾波 | Complementary Filter | 以高低通濾波融合兩種感測的簡化方法 |

## 24.1 機率的觀念轉換

### 24.1.1 從「在哪裡」到「可能在哪裡」

第二篇與第三篇的世界很乾淨：給定關節角度，末端位置是唯一確定的。但一旦進入感知的領域，這個確定性就消失了。

機率式機器人學 ( probabilistic robotics ) 的核心主張是：不要試圖回答「機器人在哪裡」這個問題，因為它沒有確定的答案；應該回答「機器人在各個位置的可能性各有多大」。

這個轉換帶來三個具體的好處：

- **能量化不確定性**：系統不只知道估計值，還知道這個估計有多可靠。這對決策極為重要——不確定時應該減速、應該主動去尋找地標。
- **能正確融合多個來源**：有了不確定性的量化，就能依可信度加權（開頭故事的核心）。
- **能誠實面對失敗**：當所有感測都不可靠時，機率分布會變得很寬——系統知道自己「迷路了」，而不是給出一個看似精確卻錯誤的數字。

### 24.1.2 信念分布

機器人對自身狀態的認知，用一個機率分布表示，稱為**信念** ( belief )：

$$\text{bel}(x_t) = p(x_t \mid z_{1:t} u_{1:t})$$

在給定所有歷史量測  $z$  與所有控制指令  $u$  的條件下，狀態  $x$  的機率分布。

信念分布的形狀本身就承載了豐富的資訊：

表 24-2 信念分布的形狀與其意義

| 分布形狀   | 意義       | 典型情境     |
|--------|----------|----------|
| 窄而高的單峰 | 位置確定     | 剛看到明確的地標 |
| 寬而矮的單峰 | 大致知道但不精確 | 長時間只靠里程計 |
| 多個峰    | 有數個可能位置  | 走廊上的對稱環境 |

| 分布形狀   | 意義           | 典型情境         |
|--------|--------------|--------------|
| 接近均勻分布 | 完全不知道在哪裡     | 剛開機、被綁架問題    |
| 沿某方向拉長 | 某方向確定、某方向不確定 | 沿長廊行進（縱向不確定） |

最後一列特別值得注意——它說明了為何需要**協方差矩陣**而非單一的誤差數字：不確定性在不同方向上可能截然不同。

### 24.1.3 三個經典問題

定位問題依難度分為三個層次，這個分類在文獻中極為常見：

**位置追蹤 ( Position Tracking )**：初始位置已知，只需持續追蹤。不確定性始終保持在局部，是最簡單的情形。Kalman 濾波器適用。

**全域定位 ( Global Localization )**：初始位置未知，機器人必須從頭找出自己在地圖中的哪裡。初期的信念可能遍布整個地圖，之後逐漸收斂。這需要能表示多峰分布的方法（粒子濾波器）。

**綁架機器人問題 ( Kidnapped Robot Problem )**：機器人在運行中被「瞬間搬到」另一個地方（實務上對應打滑嚴重、被人搬動、或定位失敗後的復原）。這是最困難的——系統必須先察覺自己錯了，再重新全域定位。

三者的難度遞增，而所需的方法也不同。這是後續選擇 Kalman 或粒子濾波的重要依據。

## 24.2 貝氏濾波

### 24.2.1 兩個步驟的循環

所有的遞迴估測方法，都建立在同一個框架上：**貝氏濾波**。它由兩個交替進行的步驟構成。

**步驟一：預測 ( Prediction )**。依運動模型，把上一時刻的信念往前推進：

$$\text{bel}^-(x_t) = \int p(x_t | x_{t-1}, u_t) \cdot \text{bel}(x_{t-1}) dx_{t-1}$$

預測步驟。積分是對所有可能的前一狀態求和。 $p(x_t | x_{t-1}, u_t)$  稱為**運動模型**，正是第 23 章建立的內容加上雜訊。

這一步的關鍵性質是：**不確定性必然增加**。因為運動本身有誤差（打滑、輪徑不準），每走一步就多累積一分不確定。這正是第 23.8 節「里程計必定漂移」在機率語言中的表述。

步驟二：更新 (Update / Correction) 。取得量測後，依貝氏定理修正信念：

$$\text{bel}(x_t) = \eta \cdot p(z_t | x_t) \cdot \text{bel}^-(x_t)$$

更新步驟。 $p(z_t | x_t)$  稱為量測模型 (似然) ， $\eta$  為正規化常數使總機率為 1 。

這一步的關鍵性質是：不確定性必然減少 (或至少不增加) 。因為量測提供了新的資訊。

#### ◎ 一呼一吸的節奏

貝氏濾波的兩個步驟，可以想像成呼吸：預測是「呼氣」——信念分布往外擴散，不確定性增加；更新是「吸氣」——信念被量測收攏，不確定性減少。整個估測過程就是這樣一呼一吸地循環。若長時間沒有量測 (例如走在沒有地標的長廊) ，分布會持續擴散；一旦看到地標，又會瞬間收窄。理解這個節奏，就理解了所有濾波器的本質。

### 24.2.2 馬可夫假設

上述公式隱含一個重要的假設：當前狀態包含了所有必要的資訊，過去的歷史可以丟棄。這稱為馬可夫假設 (Markov assumption) 。

$$p(x_t | x_{0:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t | x_{t-1}, u_t)$$

馬可夫性。只需要前一個狀態與當前的控制指令。

這個假設極為關鍵——它使遞迴計算成為可能。若必須記住所有歷史，計算量會隨時間無限成長；有了馬可夫性，我們只需維護一個當前的信念分布即可。

這與第 18.3 節 Bellman 最佳性原理的結構完全相同——那裡說「剩餘的最佳決策只取決於當前狀態」，這裡說「當前的信念只取決於前一個信念」。兩者都是把「歷史」壓縮進「狀態」的思想。

實務上馬可夫假設並非總是成立 (例如未建模的系統性偏差會使誤差相關) ，但它是一個極為有用的近似。

## 24.3 高斯分布與協方差

### 24.3.1 為何選擇高斯

貝氏濾波是一個通用框架，但要實作它，必須決定用什麼方式表示信念分布。最常用的選擇是高斯分布 (常態分布) ，原因有四：

- **只需兩個參數**：均值  $\mu$  (最可能的位置) 與變異數  $\sigma^2$  (不確定性的尺寸)。這使儲存與計算都極為高效。
- **中央極限定理**：許多獨立誤差源的疊加，趨近於高斯分布——這在物理上經常成立。
- **線性運算下封閉**：高斯分布經過線性變換後仍是高斯分布。這是 Kalman 濾波器能夠遞迴的數學基礎。
- **兩個高斯相乘仍是高斯**：這使貝氏更新步驟有封閉解。

第三與第四點特別重要——它們保證了**信念永遠可以用兩個參數表示**，不會隨時間變得越來越複雜。

### 24.3.2 協方差矩陣的幾何意義

多維情形下，不確定性由**協方差矩陣**  $\Sigma$  描述。以二維  $(x, y)$  為例：

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

協方差矩陣。對角元素是各方向的變異數，非對角元素是**相關性**。

它的幾何意義極為直觀：**協方差矩陣對應一個橢圓**（三維時為橢球）。

- **橢圓的主軸方向**： $\Sigma$  的特徵向量。這是不確定性的主要方向。
- **主軸的長度**：正比於特徵值的平方根。
- **非對角元素的意義**：若  $\sigma_{xy} \neq 0$ ，代表  $x$  與  $y$  的誤差是**相關**的——橢圓會傾斜。

這個橢圓稱為**誤差橢圓** (error ellipse)。習慣上會畫「 $1\sigma$  橢圓」(涵蓋約 39% 的機率，二維情形) 或「 $3\sigma$  橢圓」(涵蓋約 99%)。

第 23 章專題延伸提過的蒙地卡羅模擬，其散布圖的形狀正是這個橢圓——**沿垂直於行進方向拉長**，因為航向誤差的影響最大。這正是相關性的具體體現。

**圖 24-1 協方差矩陣與誤差橢圓**

建議配圖：四格對照圖。第一格： $\sigma_x = \sigma_y$ 、 $\sigma_{xy} = 0$ ——正圓，代表各方向不確定性相同。第二格： $\sigma_x > \sigma_y$ 、 $\sigma_{xy} = 0$ ——沿  $X$  軸拉長的橢圓，兩軸與座標軸對齊。第三格： $\sigma_{xy} > 0$ ——傾斜的橢圓，標註「 $x$  與  $y$  的誤差正相關」。第四格：機器人沿長廊行進的實際情形——橢圓沿行進方向拉長（縱向位置不確定），並標註特徵向量方向與特徵值的平方根。每格下方標註對應的協方差矩陣數值。

### 24.3.3 不確定性的傳播

當狀態經過一個變換時，其不確定性如何變化？對線性變換  $y = Ax$ ：

$$\mu_y = A \mu_x \quad \Sigma_y = A \Sigma_x A^T$$

不確定性的線性傳播。這是 Kalman 濾波器預測步驟的核心公式。

注意  $A \Sigma A^T$  這個形式——它與第 11 章的  $J M^{-1} J^T$ 、第 17 章的  $\Lambda$  矩陣有相同的結構。這不是巧合：兩者都在做「把一個空間中的二次型，映射到另一個空間」。

對非線性變換  $y = g(x)$ ，則需要線性化（用雅可比矩陣  $G = \partial g / \partial x$ ）：

$$\Sigma_y \approx G \Sigma_x G^T$$

非線性情形的近似傳播。這是 24.6 節擴展 Kalman 濾波器的基礎，也是其誤差的來源。

## 24.4 一維的資訊融合

### 24.4.1 最簡單的情形

在進入完整的 Kalman 濾波器之前，先看最簡單的問題：兩個獨立的量測，如何融合？

設有兩個對同一個量的估計：估計一為  $\mu_1$ ，變異數  $\sigma_1^2$ ；估計二為  $\mu_2$ ，變異數  $\sigma_2^2$ 。最佳的融合結果是：

$$\mu = (\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2) / (\sigma_1^2 + \sigma_2^2)$$

融合後的均值。注意權重的分配——變異數小的（較可信的）得到較大的權重。

$$1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2 \quad \text{即} \quad \sigma^2 = (\sigma_1^2 \sigma_2^2) / (\sigma_1^2 + \sigma_2^2)$$

融合後的變異數。倒數相加的形式極為優雅。

### 24.4.2 三個關鍵洞見

這兩條式子雖然簡單，卻包含了整個 Kalman 濾波器的精髓。

**洞見一：權重與變異數成反比。**這正是開頭故事的答案——不是各取一半，而是依可信度加權。若  $\sigma_1 = 1 \text{ cm}$ 、 $\sigma_2 = 10 \text{ cm}$ ，則估計一的權重是估計二的 100 倍（因為權重正比於  $1/\sigma^2$ ）。

**洞見二：融合後必定更確定。**由  $1/\sigma^2 = 1/\sigma_1^2 + 1/\sigma_2^2$  可知， $\sigma^2$  必定小於  $\sigma_1^2$  與  $\sigma_2^2$  中的任何一個。即使加入一個很差的量測，結果也不會變糟——這是資訊融合最重要的性質。

**洞見三：資訊是可加的。** $1/\sigma^2$  這個量稱為**資訊量**（information），也稱精確度（precision）。上式說：融合就是把資訊量相加。這個觀點導出了 Kalman 濾波器的另一種等價形式——**資訊濾波器**（information filter）。



### ◎ 為何「差的量測也不會讓結果變糟」

這個性質常令初學者驚訝。直觀的理解是：一個變異數很大的量測，其權重接近於零——它幾乎不影響均值，但仍然提供了一點點資訊，使總變異數略微下降。這意味著「多加一個感測器」在數學上永遠是好事。當然，前提是誤差模型正確——若某個感測器有未建模的偏差 (bias) 而非純粹的隨機雜訊，它就會系統性地拉偏結果。這是實務上感測器融合最常見的失敗原因，24.9 節會詳述。

#### 24.4.3 改寫成預測—修正的形式

把 24.4.1 節的公式重新整理，可以得到一個極有啟發性的形式。設  $\mu_1$  為「預測值」、 $\mu_2$  為「量測值」：

$$\mu = \mu_1 + K \cdot (\mu_2 - \mu_1), \quad \text{其中 } K = \sigma_1^2 / (\sigma_1^2 + \sigma_2^2)$$

預測—修正形式。K 就是 Kalman 增益的一維版本。

這個形式的解讀極為直觀：

- $(\mu_2 - \mu_1)$  是「創新」(innovation)——量測與預測的差距，代表「新的資訊」。
- K 決定要相信多少。K = 0 代表完全不理會量測 (預測極準)；K = 1 代表完全採信量測 (預測極差)。
- K 自動調節：預測的不確定性  $\sigma_1^2$  越大，K 越大 (越相信量測)；量測雜訊  $\sigma_2^2$  越大，K 越小 (越相信預測)。

變異數的更新也可以寫成對應的形式：

$$\sigma^2 = (1 - K) \cdot \sigma_1^2$$

更新後的變異數。K 越大，不確定性下降越多。

這四條式子 (預測值、增益、修正、變異數更新) 就是 Kalman 濾波器的全部。接下來一節要做的，只是把它們推廣到多維並加上運動模型。

## 24.5 Kalman 濾波器

### 24.5.1 系統的假設

Kalman 濾波器適用於滿足三個條件的系統：線性的運動模型、線性的量測模型、高斯的雜訊。形式為：

$$\text{運動模型: } \mathbf{x}_t = \mathbf{A} \mathbf{x}_{t-1} + \mathbf{B} \mathbf{u}_t + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$$

$$\text{量測模型: } \mathbf{z}_t = \mathbf{H} \mathbf{x}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathbf{N}(\mathbf{0}, \mathbf{R})$$

$A$  為狀態轉移矩陣、 $B$  為控制輸入矩陣、 $H$  為量測矩陣。 $Q$  為過程雜訊協方差、 $R$  為量測雜訊協方差。

兩個雜訊矩陣的意義極為重要，且是實務上最需要調整的參數：

- $Q$  (過程雜訊)：描述「運動模型有多不準」。輪子打滑、地面不平、模型簡化——這些都反映在  $Q$  中。 $Q$  越大，代表越不信任預測。
- $R$  (量測雜訊)：描述「感測器有多不準」。這通常可由感測器規格或實驗量測得到。 $R$  越大，代表越不信任量測。

### 24.5.2 五條方程式

Kalman 濾波器由五條方程式構成，分為兩組。

預測步驟 (兩條)：

$$\hat{\mathbf{x}}_t^- = \mathbf{A} \hat{\mathbf{x}}_{t-1} + \mathbf{B} \mathbf{u}_t$$

①狀態預測。依運動模型推進估計值。

$$\mathbf{P}_t^- = \mathbf{A} \mathbf{P}_{t-1} \mathbf{A}^T + \mathbf{Q}$$

②協方差預測。注意  $\mathbf{A} \mathbf{P} \mathbf{A}^T$  正是 24.3.3 節的不確定性傳播，再加上過程雜訊  $Q$ ——不確定性必然增加。

更新步驟 (三條)：

$$\mathbf{K}_t = \mathbf{P}_t^- \mathbf{H}^T (\mathbf{H} \mathbf{P}_t^- \mathbf{H}^T + \mathbf{R})^{-1}$$

③Kalman 增益。這是整個演算法的核心，24.5.3 節詳述。

$$\hat{\mathbf{x}}_t = \hat{\mathbf{x}}_t^- + \mathbf{K}_t (\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-)$$

④狀態更新。括號中的  $(\mathbf{z}_t - \mathbf{H} \hat{\mathbf{x}}_t^-)$  就是創新——量測與預測的差距。

$$\mathbf{P}_t = (\mathbf{I} - \mathbf{K}_t \mathbf{H}) \mathbf{P}_t^-$$

⑤協方差更新。 $K$  越大，不確定性下降越多——不確定性必然減少。

對照 24.4.3 節的一維版本，可以看出完全相同的結構：預測 → 算增益 → 依創新修正 → 更新不確定性。多維版本只是把純量換成矩陣。

### 24.5.3 Kalman 增益的意義

增益公式  $\mathbf{K} = \mathbf{P}^- \mathbf{H}^T (\mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R})^{-1}$  看似複雜，但其行為極為直觀。考慮一維的簡化情形 ( $H = 1$ )：

$$K = P^- / (P^- + R)$$

一維增益。分母是「預測不確定性 + 量測不確定性」，分子是「預測不確定性」。

三種極端情形立刻可讀出：

表 24-3 Kalman 增益的行為

| 情形                         | K 的值              | 行為       | 實務意義        |
|----------------------------|-------------------|----------|-------------|
| $P^- \gg R$ ( 預測很不準 )      | $K \rightarrow 1$ | 幾乎完全採信量測 | 剛開機、長時間無量測後 |
| $P^- \ll R$ ( 量測很不準 )      | $K \rightarrow 0$ | 幾乎忽略量測   | 感測器雜訊大、環境不佳 |
| $P^- \approx R$ ( 兩者相當 )   | $K \approx 0.5$   | 各取一半     | 正常運作狀態      |
| $R \rightarrow 0$ ( 量測完美 ) | $K \rightarrow 1$ | 完全採信量測   | 理想情形，實務不存在  |

**關鍵洞見：**K 是自動調節的。工程師不需要手動決定「該相信誰」——只要正確設定 Q 與 R，濾波器每一步都會依當下的不確定性自動計算出最佳的權重。這正是開頭故事中小好所缺少的東西。

另一個重要性質：P 的演化與量測值 z 完全無關——由方程②③⑤可見，它們只含 A、H、Q、R 與 P 自身。這意味著 P 與 K 的序列可以離線預先計算，這在計算資源受限的嵌入式系統上極為有用（稱為穩態 Kalman 濾波器）。

#### 24.5.4 為何說它是「最佳」的

Kalman 濾波器有一個強而有力的理論保證：

##### ◎ Kalman 濾波器的最佳性

在「線性系統 + 高斯雜訊」的假設下，Kalman 濾波器是最小均方誤差（MMSE）意義下的最佳估測器——沒有任何其他方法能得到更小的估計誤差。此外，即使雜訊不是高斯的，它仍是所有線性估測器中最佳的。

這個保證與第 18 章 LQR 的地位相同——兩者都是「在特定假設下的最佳解」，且數學結構驚人地相似：LQR 解 Riccati 方程求控制增益 K，Kalman 濾波器解 Riccati 方程求估測增益 K。這正是第 19.2.2 節提到的對偶性——估計與控制是同一件事的兩面。

兩者結合稱為 LQG（Linear Quadratic Gaussian）：用 Kalman 濾波器估計狀態、再用 LQR 計算控制。這是經典控制理論的巔峰成果。

## 24.6 非線性的處理：EKF 與 UKF

### 24.6.1 機器人為何是非線性的

Kalman 濾波器要求線性系統，但機器人幾乎都不是。以第 23 章的差速驅動車為例：

$$\begin{aligned}x_t &= x_{t-1} + \Delta s \cdot \cos(\theta_{t-1} + \Delta\theta/2) \\y_t &= y_{t-1} + \Delta s \cdot \sin(\theta_{t-1} + \Delta\theta/2) \\\theta_t &= \theta_{t-1} + \Delta\theta\end{aligned}$$

運動模型含三角函數，是非線性的。

量測模型同樣非線性。例如用光達量測到某地標的距離與角度：

$$\begin{aligned}r &= \sqrt{(x_L - x)^2 + (y_L - y)^2} \\\phi &= \text{atan2}(y_L - y, x_L - x) - \theta\end{aligned}$$

距離與方位的量測模型，同樣非線性。

因此必須擴展 Kalman 濾波器以處理非線性。有兩條主要路線。

### 24.6.2 擴展 Kalman 濾波器 (EKF)

EKF 的作法最直接：在當前估計值附近做一階泰勒展開，用雅可比矩陣取代  $A$  與  $H$ 。

$$\begin{aligned}G &= \partial g / \partial x \big|_{(\hat{x}_{t-1})} \quad (\text{運動模型的雅可比}) \\H &= \partial h / \partial x \big|_{(\hat{x}_t^-)} \quad (\text{量測模型的雅可比})\end{aligned}$$

EKF 的線性化。其餘四條方程式的形式與 KF 完全相同，只是把  $A$  換成  $G$ 。

注意狀態的預測與量測的預測仍使用非線性函數本身（而非線性化版本）——只有協方差的傳播使用雅可比。這個細節很重要：

$$\begin{aligned}\hat{x}_t^- &= g(\hat{x}_{t-1}, u_t) \quad (\text{用非線性函數}) \\P_t^- &= G P_{t-1} G^T + Q \quad (\text{用雅可比})\end{aligned}$$

EKF 的預測步驟。

EKF 的三個限制：

- **線性化誤差**：泰勒展開只在小範圍內準確。若不確定性很大（ $P$  很大），線性化的誤差就會顯著。
- **不再是最佳的**：EKF 只是一個近似，失去了 KF 的最佳性保證。
- **可能發散**：若線性化誤差累積，估計可能完全偏離真值且  $P$  卻顯示「很確定」——這是最危險的失效模式。

儘管有這些限制，EKF 仍是實務上最常用的方法，因為它計算效率高、實作成熟、且在多數

機器人應用中的非線性程度尚可接受。

### 24.6.3 無跡 Kalman 濾波器 (UKF)

UKF 採取完全不同的思路：與其線性化函數，不如取樣分布。

它的核心是無跡變換 (unscented transform)：

1. 由當前的均值與協方差，精心挑選出  $2n+1$  個代表點 (稱為 **sigma points**)，使它們的樣本均值與協方差恰好等於原分布。
2. 把每一個 sigma point 直接代入非線性函數 (不做任何近似)。
3. 由變換後的點，重新計算均值與協方差。

**UKF 的優勢**：不需要計算雅可比 (對複雜模型是很大的便利)、精度達二階 (EKF 只有一階)、且對強非線性系統更穩健。

**UKF 的代價**：計算量約為 EKF 的 2~3 倍。但由於 sigma points 的數目只有  $2n+1$  (線性成長)，對中等維度的系統仍完全可接受。

表 24-4 EKF 與 UKF 的比較

| 面向        | EKF          | UKF                 |
|-----------|--------------|---------------------|
| 處理非線性的方式  | 線性化 (泰勒展開)   | 取樣傳遞 (sigma points) |
| 需要雅可比     | 需要 (手推或數值微分) | 不需要                 |
| 近似精度      | 一階           | 二階                  |
| 計算量       | 低            | 中 (約 2~3 倍)         |
| 強非線性下的穩健性 | 較差，可能發散      | 較佳                  |
| 實作難度      | 中 (需推導雅可比)   | 低 (只需模型函數)          |
| 典型應用      | 多數機器人定位、SLAM | 姿態估計、強非線性系統         |

**實務建議**：若模型的雅可比容易推導且非線性不強，用 EKF；若模型複雜 (雅可比難推) 或非線性顯著 (如姿態估計含大角度旋轉)，UKF 往往是更好的選擇。

## 24.7 粒子濾波器

### 24.7.1 放棄高斯假設

EKF 與 UKF 都假設信念是高斯分布（單峰）。但 24.1.3 節提到的**全域定位**問題中，初始的信念可能遍布整個地圖、或有多個候選位置——這根本不是高斯分布。

**粒子濾波器**（particle filter）徹底放棄了參數化的表示，改用**大量的樣本**來表示任意形狀的分布。

$$\text{bel}(\mathbf{x}_t) \approx \{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}, i = 1, \dots, M$$

以  $M$  個帶權重的粒子表示信念。每個粒子是一個「可能的位置假設」，權重代表其可信度。

這個表示法的威力在於：它能表示任何形狀的分布——多峰、不對稱、甚至不連續。代價則是需要大量的粒子（典型 1000 ~ 100000 個），計算量遠高於 Kalman 家族。

### 24.7.2 三個步驟

粒子濾波器的每一次迭代包含三個步驟。

**步驟一：預測（採樣）**。對每一個粒子，依運動模型加上隨機雜訊，往前推進：

$$\mathbf{x}_t^{(i)} \sim p(\mathbf{x}_t | \mathbf{x}_{t-1}^{(i)}, \mathbf{u}_t)$$

每個粒子獨立地「走一步」，且各自加上不同的隨機擾動。這使粒子群逐漸散開——對應不確定性的增加。

**步驟二：加權**。依量測模型，計算每個粒子的可信度：

$$w_t^{(i)} = p(z_t | \mathbf{x}_t^{(i)})$$

與量測越吻合的粒子，權重越大。例如某粒子預測「前方 2 公尺有牆」，而光達確實量到 2 公尺，該粒子就得到高權重。

**步驟三：重取樣（Resampling）**。依權重重新抽取  $M$  個粒子（權重大的可能被抽中多次、權重小的可能消失）。

重取樣是粒子濾波器的關鍵機制——它使**計算資源自動集中到高可能性的區域**。若沒有重取樣，大量粒子會停留在不可能的位置，浪費計算量。

### 24.7.3 粒子退化與其對策

粒子濾波器有一個惡名昭彰的失效模式：**粒子退化**（particle depletion）。

#### ◎ 粒子退化

重取樣會複製高權重的粒子、淘汰低權重的粒子。經過多次迭代後，所有粒子可能都源自同一個祖先——**多樣性喪失，分布退化成一個點**。此時濾波器會過度自信，且一旦真實位置不在該點附近，

就再也無法恢復（因為沒有粒子在正確的地方）。

三個標準的對策：

- **只在必要時重取樣**：計算「有效粒子數」 $N_{\text{eff}} = 1/\Sigma(w^{(i)})^2$ ，只有當它低於門檻（如  $M/2$ ）時才重取樣。這是最重要的一項改善。
- **加入隨機粒子**：每次迭代注入少量（如 1~5%）隨機分布的粒子。這使系統能從綁架問題中恢復——若真實位置在某處，總會有粒子恰好落在附近。
- **低變異數重取樣**：使用系統性採樣（systematic resampling）而非簡單的多項式採樣，能顯著降低取樣造成的變異。

## 24.7.4 適用場合的比較

表 24-5 三種濾波器的比較

| 面向     | KF / EKF      | UKF              | 粒子濾波器          |
|--------|---------------|------------------|----------------|
| 分布表示   | 高斯（均值 + 協方差）  | 高斯（sigma points） | 任意（大量樣本）       |
| 能否多峰   | 否             | 否                | 是              |
| 處理非線性  | 線性化（一階）       | 取樣（二階）           | 完全不近似          |
| 計算量    | 低             | 中                | 高              |
| 維度可擴充性 | 佳（ $O(n^3)$ ） | 佳                | 差（維度詛咒）        |
| 全域定位   | 不適用           | 不適用              | 適用             |
| 綁架問題復原 | 不能            | 不能               | 可（需注入隨機粒子）     |
| 典型應用   | 位置追蹤、感測器融合    | 姿態估計             | 全域定位（MCL）、地圖匹配 |

**粒子濾波器最大的限制是維度詛咒**：所需的粒子數隨狀態維度指數成長。因此它適用於低維問題（如二維定位的 3 維狀態），但不適用於高維（如第 25 章 SLAM 中包含數百個地標的狀態向量）。這個限制決定了兩者的分工。

在移動機器人定位中，粒子濾波器的具體應用稱為**蒙地卡羅定位**（Monte Carlo Localization, MCL），是 ROS 2 導航堆疊中 AMCL 套件的核心演算法。

## 24.8 伏筆回收：互補濾波

### 24.8.1 第 6 章那條式子從哪裡來



第 6.3.2 節在討論 IMU 時，給出了一條融合加速度計與陀螺儀的公式：

$$\theta = \alpha \cdot (\theta_{\text{prev}} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{\text{accel}}$$

*互補濾波。當時只說它是一個「經驗法則」， $\alpha$  通常取 0.98。*

現在可以揭曉它的來源。把它與 24.4.3 節的 Kalman 更新式對照：

$$\text{Kalman: } \hat{x} = \hat{x}^- + K(z - \hat{x}^-) = (1 - K) \cdot \hat{x}^- + K \cdot z$$

$$\text{互補濾波: } \theta = \alpha \cdot \theta_{\text{gyro}} + (1 - \alpha) \cdot \theta_{\text{accel}}$$

*兩者結構完全相同！只要令  $\alpha = 1 - K$ ，兩條式子就一模一樣。*

因此：互補濾波就是「Kalman 增益固定不變」的 Kalman 濾波器。

## 24.8.2 為何叫「互補」

這個名稱來自頻域的觀點。兩個感測器的誤差特性恰好互補：

- **陀螺儀**：短期極準，但積分後會**漂移**（低頻誤差大）。因此對它用**高通濾波**——只取其快速變化的部分。
- **加速度計**：長期平均準確（重力方向不變），但受運動加速度干擾而**雜訊大**（高頻誤差大）。因此對它用**低通濾波**——只取其緩慢變化的部分。

兩個濾波器的頻率響應相加恰好為 1（互補），因此不會遺漏或重複計算任何頻段的資訊。這正是「互補」一詞的由來。

$$H_{\text{highpass}}(s) + H_{\text{lowpass}}(s) = 1$$

*互補的條件。這是頻域的觀點，與時域的加權平均是同一件事。*

## 24.8.3 什麼時候該用哪一個

既然互補濾波是簡化版，為何不總是使用完整的 Kalman 濾波器？答案是**成本與收益的權衡**。

表 24-6 互補濾波與 Kalman 濾波器的比較

| 面向        | 互補濾波              | Kalman 濾波器    |
|-----------|-------------------|---------------|
| 增益        | 固定（手動調 $\alpha$ ） | 自動計算（隨不確定性變化） |
| 計算量       | 極低（幾次乘加）          | 中（矩陣運算與求逆）    |
| 需要的模型     | 無                 | 運動模型、量測模型、Q、R |
| 能否處理變動的雜訊 | 不能                | 能             |
| 能否輸出不確定性  | 不能                | 能（協方差矩陣）      |

| 面向   | 互補濾波         | Kalman 濾波器     |
|------|--------------|----------------|
| 實作難度 | 極低（幾行程式）     | 中              |
| 典型應用 | 小型無人機、平衡車、教學 | 自駕車、專業無人機、SLAM |

**選擇的原則：**若感測器的雜訊特性**穩定**、且不需要不確定性的輸出，互補濾波往往就夠了——它簡單、可靠、且不會因為  $Q$ 、 $R$  設錯而發散。這正是為何許多商用飛控（尤其是入門級）採用它。

**但在三種情況下必須使用 Kalman：**一、雜訊特性會變化（如 GPS 訊號時好時壞）；二、需要不確定性的輸出以供決策（如「不確定時就減速」）；三、感測器數量多且更新率不同（Kalman 能自然處理，互補濾波則難以擴充）。

#### ◎ 教學上的一個建議

本書刻意在第 6 章先給出互補濾波、到第 24 章才揭曉其來源。這個安排是有意的：**先讓學生用一行公式做出能運作的東西，建立信心與直覺；再回頭理解其背後的理論。**這與第 2 章「即時學習」的策略一致。實務上也建議這個順序——先用互補濾波把系統跑起來，確認硬體與整體架構沒問題，再視需求升級為 Kalman。過早引入複雜的方法，往往會讓除錯變得極為困難，因為你分不清問題是出在硬體、模型還是濾波器參數。

## 24.9 實務上的陷阱

### 24.9.1 $Q$ 與 $R$ 該怎麼設

Kalman 濾波器的理論很優美，但實務上最大的困難是： **$Q$  與  $R$  該設多少？**這兩個矩陣沒有標準答案，卻直接決定濾波器的行為。

**$R$ （量測雜訊）相對容易：**讓感測器對著一個**靜止不動**的目標量測數千次，計算讀值的變異數即可。多數感測器的規格書也會提供標準差。

**$Q$ （過程雜訊）則困難得多，**因為它描述的是「模型有多不準」——這無法直接量測。三種實務作法：

- **由物理推算：**如第 23.8.4 節的里程計誤差模型，把變異數設為正比於運動量。這是最有依據的作法。
- **由實驗調整：**先設一個初值，觀察濾波器的行為並調整（見下節的判斷準則）。

- **自適應估計**：由創新序列的統計特性線上估計  $Q$  與  $R$ 。理論優雅但實作複雜，且可能不穩定。

## 24.9.2 如何判斷調得對不對

有一個極為實用的診斷工具：**檢查創新序列** (innovation，即  $z - H\hat{x}$ )。

理論上，若  $Q$  與  $R$  設定正確，創新序列應該具備兩個性質：

性質一：均值為零 (無系統性偏差)

性質二：其變異數應等於  $S = H P^- H^T + R$  (濾波器自己預測的值)

*創新的一致性檢驗。這是判斷濾波器健康與否的黃金標準。*

由此可以診斷三種問題：

表 24-7 濾波器的診斷

| 症狀             | 可能原因                  | 處理方式               |
|----------------|-----------------------|--------------------|
| 創新的實際變異數 > 預測值 | $Q$ 或 $R$ 設得太小 (過度自信) | 增大 $Q$ (更常見) 或 $R$ |
| 創新的實際變異數 < 預測值 | $Q$ 或 $R$ 設得太大 (過度保守) | 減小 $Q$ 或 $R$       |
| 創新的均值不為零       | 有未建模的系統性偏差            | 檢查感測器校正、模型是否遺漏項    |
| 估計值跟不上真實變化     | $Q$ 太小、過度信任模型         | 增大 $Q$             |
| 估計值雜訊大、抖動      | $R$ 太小、過度信任量測         | 增大 $R$             |
| 協方差 $P$ 持續增大   | 長時間無有效量測              | 檢查感測器是否失效          |

一個重要的實務原則：當不確定時，寧可把  $Q$  設大一點。 $Q$  太小會使濾波器過度自信、拒絕接受量測、最終發散且無法恢復； $Q$  太大則只是使估計稍微雜訊多一些，但系統仍然穩健。

## 24.9.3 三個常見的失效

**失效一：未建模的偏差 (bias)**。Kalman 濾波器假設雜訊的均值為零。若感測器有系統性偏差 (如陀螺儀的零漂、磁力計受鐵件干擾)，濾波器會被系統性地拉偏，且協方差仍顯示「很確定」——這是最危險的情形。

**因應之道**：把偏差本身加入狀態向量一起估計。例如狀態由 [位置, 速度] 擴充為 [位置, 速度, 感測器偏差]。這是實務上極為常用的技巧，稱為**狀態擴增** (state augmentation)。

**失效二：離群值 (outlier)**。單一個錯誤的量測 (如視覺誤匹配、光達打到玻璃) 可能嚴

重破壞估計。

**因應之道：馬氏距離門檻 ( Mahalanobis gating )**。計算創新的正規化距離，若超過門檻就拒絕該量測：

$$d^2 = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}, \quad \text{其中 } \mathbf{v} \text{ 為創新、} \mathbf{S} = \mathbf{H} \mathbf{P}^- \mathbf{H}^T + \mathbf{R}$$

馬氏距離的平方。若  $d^2$  超過卡方分布的門檻 ( 如 95% 對應的值 )，就拒絕此量測。這是實務上必備的保護機制。

**失效三：時間戳不同步**。不同感測器的資料到達時間不同，若未正確處理時間戳，會把「五十毫秒前的量測」當成「現在的量測」使用。在高速運動時，這会造成顯著的誤差。

**因應之道：**所有感測資料都應帶精確的時間戳，並在融合時做時間對齊 ( 把狀態預測到量測的時刻，或反過來 )。這正是第 8.9.2 節提到 ROS 2 的 TF 系統儲存變換歷史紀錄的原因。

## 24.10 多感測器融合的架構

### 24.10.1 常見的感測器組合

實務的機器人系統通常融合多種感測器，各自貢獻不同的資訊：

表 24-8 常見感測器的互補特性

| 感測器          | 更新率            | 短期精度      | 長期漂移 | 失效情境          |
|--------------|----------------|-----------|------|---------------|
| 輪式里程計        | 高 ( 100+ Hz )  | 高         | 會漂移  | 打滑、抬起         |
| IMU ( 陀螺儀 )  | 極高 ( 1000 Hz ) | 極高        | 會漂移  | 長時間積分         |
| IMU ( 加速度計 ) | 極高             | 低 ( 雜訊大 ) | 不漂移  | 運動加速度干擾       |
| 光達 + 地圖匹配    | 中 ( 10~40 Hz ) | 中高        | 不漂移  | 無特徵環境、動態物體    |
| 視覺標記         | 中 ( 5~30 Hz )  | 高         | 不漂移  | 遮蔽、光線不足       |
| GNSS         | 低 ( 1~10 Hz )  | 低 ( 公尺級 ) | 不漂移  | 室內、隧道、多路徑     |
| 視覺里程計        | 中 ( 10~30 Hz ) | 高         | 會漂移  | 無紋理、快速運動、光線變化 |

**融合的核心邏輯：**用高頻但會漂移的感測器 ( 里程計、IMU ) 提供平滑的短期估計，用低頻但不漂移的感測器 ( 光達、視覺標記、GNSS ) 定期校正。這正是開頭故事的正確解法。

### 24.10.2 鬆耦合與緊耦合

多感測器融合有兩種架構，取捨明確。

**鬆耦合 ( Loosely Coupled )**：每個感測器先各自處理成一個位姿估計，再融合這些位姿。

- **優點**：模組化、易於實作與除錯、單一感測器失效時容易偵測與隔離。
- **缺點**：資訊有損失。例如 GNSS 只收到三顆衛星時無法解出位姿，在鬆耦合下這筆資料就完全浪費了。

**緊耦合 ( Tightly Coupled )**：把所有感測器的**原始量測**直接放入同一個濾波器。

- **優點**：資訊利用最完整、精度最高。上例中三顆衛星的資料仍能提供部分約束。
- **缺點**：實作複雜、狀態維度高、單一感測器的異常會直接污染整個估計。

**實務建議**：從鬆耦合開始（容易除錯），確認系統運作後，若精度不足再考慮緊耦合。多數工業應用鬆耦合已足夠；高精度需求（如自駕車的視覺慣性里程計）才使用緊耦合。

### 24.10.3 ROS 2 中的實務

ROS 2 生態系提供了成熟的融合工具：

- **robot\_localization**：最常用的套件，提供 EKF 與 UKF 節點，能融合任意數量的里程計、IMU 與 GNSS 來源。設定檔中可指定「哪個感測器提供哪些狀態量」——這是鬆耦合的典型實作。
- **AMCL**：粒子濾波器的實作，用於已知地圖中的全域定位（第 25 章會再提及）。
- **TF2**：管理各座標系之間的變換與其時間歷史，是所有融合的基礎設施（第 8.9.2 節）。

一個標準的移動機器人定位架構是：**robot\_localization** 融合里程計與 IMU 產生平滑的 **odom→base\_link** 變換（高頻、會漂移）；**AMCL** 以光達與地圖匹配產生 **map→odom** 變換（低頻、不漂移）。兩者串接即得到不漂移且平滑的定位。這個**兩層架構**是 ROS 導航的標準設計，值得深入理解。

## 24.11 由估測邁向建圖

### 24.11.1 本章的成果

本章建立了機率式估測的完整框架。你現在能夠：以信念分布表示不確定性、理解貝氏濾波「一呼一吸」的節奏、運用協方差矩陣與誤差橢圓描述多維不確定性、實作 Kalman 濾波器並理解增益的自動調節、以 EKF 或 UKF 處理非線性、以粒子濾波器處理多峰分布與全域定位、並診斷濾波器的常見失效。

更重要的是那個核心洞見：**依可信度加權，而非二選一**。這個看似簡單的原則，是所有感測器融合的基礎，也回答了開頭故事中小好的困境。

我們也回收了第 6.3.2 節的伏筆——**互補濾波就是增益固定的 Kalman 濾波器**，兩者在時域與頻域上是同一件事的兩種表述。

### 24.11.2 接下來：如果連地圖都沒有呢

但本章的所有定位方法都有一個共同的前提：**必須有地圖**。粒子濾波器要「用光達與地圖匹配」、視覺標記要「事先知道標記的位置」、地圖匹配更是顧名思義。

問題是——**地圖從哪裡來**？人工測繪一棟建築物既昂貴又緩慢，而且環境會改變。理想的做法是讓機器人自己建立地圖。

但這裡出現了一個看似無解的循環：**要建立地圖，必須知道自己在哪裡（否則不知道把觀測到的東西畫在地圖的哪個位置）；要知道自己在哪裡，又必須有地圖（否則沒有參照）**。這就像雞生蛋、蛋生雞。

**第 25 章 (SLAM)** 將回答這個問題。同步定位與建圖 (Simultaneous Localization and Mapping) 是機器人學最著名的難題之一，而它的解法正是建立在本章之上——把「機器人位姿」與「地圖中所有地標的位置」**放進同一個狀態向量**，一起估計。屆時你會看到 EKF 與粒子濾波器如何被擴展來處理這個高維問題，以及為何現代的方法轉向了「圖最佳化」的觀點。

換句話說：第 24 章讓機器人在已知地圖中知道自己在哪裡，第 25 章讓它在未知環境中同時搞清楚地圖與自己的位置。

## 完整計算例題

### 例題 24-1 一維資訊融合

題目：機器人用兩種方式估計自己的位置。里程計給出  $x_1 = 10.20 \text{ m}$ ，標準差  $\sigma_1 = 0.30 \text{ m}$ ；視覺標記給出  $x_2 = 10.50 \text{ m}$ ，標準差  $\sigma_2 = 0.10 \text{ m}$ 。(a) 求融合後的估計值與標準差。(b) 計算兩者的權重比。(c) 若視覺標記在光線不佳處，標準差劣化為  $0.50 \text{ m}$ ，重新計算。(d) 比較 (a) 與 (c)，說明濾波器的行為。

解 (a)：由 24.4.1 節的公式：

$$\mu = (\sigma_2^2 \mu_1 + \sigma_1^2 \mu_2) / (\sigma_1^2 + \sigma_2^2) = (0.01 \times 10.20 + 0.09 \times 10.50) / (0.09 + 0.01)$$

代入數值。注意權重是用對方的變異數。

$$\mu = (0.1020 + 0.9450) / 0.10 = 1.0470 / 0.10 = 10.47 \text{ m}$$

融合後的估計值。

$$\sigma^2 = (\sigma_1^2 \sigma_2^2) / (\sigma_1^2 + \sigma_2^2) = (0.09 \times 0.01) / 0.10 = 0.009 \Rightarrow \sigma = 0.0949 \text{ m}$$

融合後的標準差約 9.5 公分。

**關鍵觀察：**融合後的標準差 0.0949 m 小於兩者中較小的 0.10 m——這印證了 24.4.2 節的洞見二：融合必定使估計更確定。

解 (b)：權重正比於  $1/\sigma^2$ ：

$$w_1 : w_2 = 1/0.09 : 1/0.01 = 11.11 : 100 = 1 : 9$$

視覺標記的權重是里程計的 9 倍。因此融合結果 10.47 明顯偏向視覺的 10.50。

解 (c)： $\sigma_2 = 0.50 \text{ m}$  時：

$$\mu = (0.25 \times 10.20 + 0.09 \times 10.50) / (0.09 + 0.25) = (2.550 + 0.945) / 0.34 = 3.495 / 0.34 = 10.28 \text{ m}$$

融合後的估計值。

$$\sigma^2 = (0.09 \times 0.25) / 0.34 = 0.0662 \Rightarrow \sigma = 0.257 \text{ m}$$

融合後的標準差。

$$w_1 : w_2 = 1/0.09 : 1/0.25 = 11.11 : 4.0 = 2.78 : 1$$

現在里程計的權重反而較大。

解 (d)：兩種情形的對照極具啟發性：

表 24-9 例題 24-1 的結果對照

| 情形       | 視覺 $\sigma_2$ | 融合結果    | 偏向                  | 融合後 $\sigma$ |
|----------|---------------|---------|---------------------|--------------|
| (a) 光線良好 | 0.10 m        | 10.47 m | 偏向視覺 ( 權重 9:1 )     | 0.095 m      |
| (c) 光線不佳 | 0.50 m        | 10.28 m | 偏向里程計 ( 權重 1:2.78 ) | 0.257 m      |

這正是 Kalman 濾波器的價值：不需要人工判斷「現在該相信誰」，只要正確描述各感測器的不確定性，權重就會自動調整。若採用開頭故事中「直接覆蓋」的作法，兩種情形都會得到



10.50——在光線不佳時將是嚴重的錯誤。

## 例題 24-2 Kalman 濾波器的完整迭代

題目：一維的位置追蹤問題。狀態為位置  $x$ ，運動模型為  $x_t = x_{t-1} + u_t$  ( $u$  為里程計增量)，過程雜訊  $Q = 0.04 \text{ m}^2$ ；量測模型為  $z_t = x_t$ ，量測雜訊  $R = 0.09 \text{ m}^2$ 。初始估計  $\hat{x}_0 = 0$ 、 $P_0 = 0.25 \text{ m}^2$ 。(a) 若第一步的里程計  $u_1 = 1.00 \text{ m}$ ，求預測值與預測協方差。(b) 若量測值  $z_1 = 1.30 \text{ m}$ ，求 Kalman 增益。(c) 求更新後的估計值與協方差。(d) 若接連三步都沒有量測，求協方差的演變。

解 (a)：由方程①② (此處  $A = 1$ 、 $B = 1$ )：

$$\hat{x}_1^- = A \hat{x}_0 + B u_1 = 1 \times 0 + 1 \times 1.00 = 1.00 \text{ m}$$

狀態預測。

$$P_1^- = A P_0 A^T + Q = 1 \times 0.25 \times 1 + 0.04 = 0.29 \text{ m}^2$$

協方差預測。不確定性由 0.25 增加到 0.29——這正是預測步驟的必然結果。

解 (b)：由方程③ ( $H = 1$ )：

$$K_1 = P_1^- H^T (H P_1^- H^T + R)^{-1} = 0.29 / (0.29 + 0.09) = 0.29 / 0.38 = 0.7632$$

Kalman 增益約 0.76。

判讀： $K = 0.76 > 0.5$ ，代表濾波器較信任量測。這合理，因為此時預測的不確定性 (0.29) 大於量測的不確定性 (0.09)。

解 (c)：由方程④⑤：

$$\hat{x}_1 = \hat{x}_1^- + K_1(z_1 - H \hat{x}_1^-) = 1.00 + 0.7632 \times (1.30 - 1.00) = 1.00 + 0.2290 = 1.229 \text{ m}$$

更新後的估計值。

注意：結果 1.229 落在預測值 1.00 與量測值 1.30 之間，且偏向量測——與增益 0.76 的判讀一致。

$$P_1 = (1 - K_1)P_1^- = (1 - 0.7632) \times 0.29 = 0.2368 \times 0.29 = 0.0687 \text{ m}^2$$

更新後的協方差。由 0.29 大幅降到 0.069——量測提供了顯著的資訊。

驗證：融合後的標準差為  $\sqrt{0.0687} = 0.262 \text{ m}$ ，小於預測的  $\sqrt{0.29} = 0.539 \text{ m}$  與量測的

$\sqrt{0.09} = 0.30 \text{ m}$ ——再次印證融合必定更確定。

解 (d)：無量測時只執行預測步驟，協方差逐步累加  $Q$ ：

$$\begin{aligned} P_2 &= 0.0687 + 0.04 = 0.1087 \\ P_3 &= 0.1087 + 0.04 = 0.1487 \\ P_4 &= 0.1487 + 0.04 = 0.1887 \text{ m}^2 \end{aligned}$$

協方差線性成長 (因  $A = 1$ )。

對應的標準差由  $0.262 \text{ m}$  增至  $0.434 \text{ m}$ 。這正是里程計漂移在機率語言中的表述——沒有外部量測時，不確定性持續增加且無法停止。若長時間如此， $P$  會無限增大，代表系統「完全不知道自己在哪裡」。

### 例題 24-3 互補濾波與 Kalman 的等價

題目：某平衡車以互補濾波融合陀螺儀與加速度計，取  $\alpha = 0.98$ ，取樣週期  $5 \text{ ms}$ 。(a) 求對應的等效 Kalman 增益  $K$ 。(b) 若已知加速度計的角度量測雜訊  $R = 4.0 \text{ deg}^2$ ，反推此  $\alpha$  對應的預測協方差  $P^-$ 。(c) 計算互補濾波的等效時間常數。(d) 討論若改用  $\alpha = 0.90$  會如何。

解 (a)：由 24.8.1 節的對應關係  $\alpha = 1 - K$ ：

$$K = 1 - \alpha = 1 - 0.98 = 0.02$$

等效 Kalman 增益。極小的  $K$  代表幾乎完全信任陀螺儀的預測。

解 (b)：由  $K = P^-/(P^- + R)$  反解：

$$0.02 = P^-/(P^- + 4.0) \Rightarrow 0.02(P^- + 4.0) = P^- \Rightarrow 0.08 = 0.98 P^- \Rightarrow P^- = 0.0816 \text{ deg}^2$$

隱含的預測協方差，對應標準差  $0.286 \text{ 度}$ 。

判讀：這代表工程師在設  $\alpha = 0.98$  時，實質上假設了「陀螺儀在一個取樣週期內的預測誤差約  $0.29 \text{ 度}$ 、而加速度計的量測誤差約  $2 \text{ 度}$ 」。這個假設是否合理，取決於實際的感測器——這正是互補濾波「隱藏了假設」的地方。

解 (c)：互補濾波的等效時間常數為：

$$\tau = \alpha \Delta t / (1 - \alpha) = 0.98 \times 0.005 / 0.02 = 0.245 \text{ s}$$

時間常數約  $0.25 \text{ 秒}$ 。

物理意義：陀螺儀的漂移會在約  $0.25 \text{ 秒}$  內被加速度計修正。因此慢於  $0.25 \text{ 秒}$  的漂移會被消

除，快於 0.25 秒的加速度計雜訊會被濾掉——這正是「互補」的頻域意義。

解 (d) :  $\alpha = 0.90$  時：

$$K = 0.10, \quad \tau = 0.90 \times 0.005 / 0.10 = 0.045 \text{ s}$$

增益變大五倍、時間常數縮短為 45 毫秒。

**後果：**更快速地修正陀螺儀漂移（好事），但也更容易受加速度計雜訊影響（壞事）。在平衡車上，這可能導致車體在振動時角度估計抖動，進而使控制器產生高頻的馬達指令。

**實務結論：** $\alpha$  的選擇是「抗漂移」與「抗雜訊」的取捨——與第 16 章 PID 增益的取捨、第 18 章 LQR 的 Q/R 權重是同一類問題。若機器振動大，應提高  $\alpha$ （更信任陀螺儀）；若陀螺儀品質差、漂移快，則應降低  $\alpha$ 。

## 程式範例：Kalman 濾波器與粒子濾波器

以下 Python 程式實作一維與二維的 Kalman 濾波器、擴展 Kalman 濾波器，以及粒子濾波器，並比較它們在不同情境下的表現。

```
import numpy as np

# ===== 一維 Kalman 濾波器 =====
class KalmanFilter1D:
    """最簡單的 Kalman 濾波器 (24.5 節的五條方程式)"""
    def __init__(s, x0=0.0, P0=1.0, Q=0.04, R=0.09):
        s.x, s.P, s.Q, s.R = x0, P0, Q, R

    def predict(s, u=0.0):
        s.x = s.x + u                # ① 狀態預測
        s.P = s.P + s.Q              # ② 協方差預測 (必然增加)
        return s.x, s.P

    def update(s, z):
        K = s.P / (s.P + s.R)        # ③ Kalman 增益
        innov = z - s.x               # ④ 創新
        s.x = s.x + K*innov           # ⑤ 狀態更新
        s.P = (1 - K)*s.P             # ⑥ 協方差更新 (必然減少)
        return s.x, s.P, K, innov

# ===== 例題 24-1：一維資訊融合 =====
print("=== 例題 24-1：一維資訊融合 ===")
def fuse(m1, s1, m2, s2):
    v1, v2 = s1**2, s2**2
```

```

mu = (v2*m1 + v1*m2)/(v1 + v2)
var = v1*v2/(v1 + v2)
return mu, np.sqrt(var), (1/v1)/(1/v2)

print(f"{'情境':<12}{'里程計':>10}{'視覺':>10}{'融合結果':>12}"
      f"{'融合  $\sigma$ ':>10}{'權重比':>12}")
for label, s2 in (("光線良好", 0.10), ("光線普通", 0.25), ("光線不佳", 0.50)):
    mu, sig, ratio = fuse(10.20, 0.30, 10.50, s2)
    print(f"{label:<12}{'10.20':>10}{'10.50':>10}{mu:12.4f}"
          f"{sig:10.4f}{f'1 : {1/ratio:.2f}':>12}")
print("→ 視覺越不準，結果越偏向里程計—權重自動調整")

# ===== 例題 24-2：完整迭代 =====
print("\n=== 例題 24-2：Kalman 濾波器的完整迭代 ===")
kf = KalmanFilter1D(x0=0.0, P0=0.25, Q=0.04, R=0.09)
x_pred, P_pred = kf.predict(u=1.00)
print(f"(a) 預測:  $\hat{x}^- = \{x\_pred:.4f\}$  m,  $P^- = \{P\_pred:.4f\}$  m2 "
      f"(由 0.25 增至  $\{P\_pred:.2f\})$ ")
x_upd, P_upd, K, innov = kf.update(z=1.30)
print(f"(b) Kalman 增益  $K = \{K:.4f\}$  "
      f"({'偏向量測' if K > 0.5 else '偏向預測'})")
print(f"(c) 更新:  $\hat{x} = \{x\_upd:.4f\}$  m,  $P = \{P\_upd:.4f\}$  m2 "
      f"(創新 =  $\{innov:.2f\})$ ")
print(f"     $\sigma$  由  $\{np.sqrt(P\_pred):.3f\}$  降至  $\{np.sqrt(P\_upd):.3f\}$  m")
print("(d) 接連三步無量測:")
for i in range(3):
    _, Pk = kf.predict(u=0.5)
    print(f"    第  $\{i+2\}$  步:  $P = \{Pk:.4f\}$  m2 ( $\sigma = \{np.sqrt(Pk):.3f\}$  m)")
print("    → 無量測時不確定性持續增加，這就是漂移")

# ===== 完整模擬：追蹤一個移動目標 =====
print("\n=== 模擬：追蹤等速運動的目標（比較三種策略）===")
np.random.seed(42)
T, dt = 200, 0.05
true_v = 1.0
true_x = np.cumsum(np.full(T, true_v*dt))
odom_bias = 1.03 # 里程計有 3% 的比例誤差
odom = np.cumsum(np.full(T, true_v*dt*odom_bias)
                 + np.random.randn(T)*0.005)
meas = true_x + np.random.randn(T)*0.30 # 視覺量測， $\sigma=0.3$  m
meas_avail = np.arange(T) % 10 == 0 # 每 10 步才有一次量測

# 策略一：只用里程計
err_odom = np.abs(odom - true_x)
# 策略二：直接覆蓋（開頭故事的錯誤作法）
est_ovr, jumps = [], []
x = 0.0
for k in range(T):
    x += true_v*dt*odom_bias + np.random.randn()*0.005

```

```

if meas_avail[k]:
    jumps.append(abs(meas[k] - x))
    x = meas[k] # 直接覆蓋
    est_ovr.append(x)
err_ovr = np.abs(np.array(est_ovr) - true_x)
# 策略三：Kalman 融合
kf2 = KalmanFilter1D(x0=0.0, P0=0.01, Q=0.0004, R=0.09)
est_kf = []
for k in range(T):
    kf2.predict(u=true_v*dt*odom_bias + np.random.randn()*0.005)
    if meas_avail[k]:
        kf2.update(meas[k])
    est_kf.append(kf2.x)
err_kf = np.abs(np.array(est_kf) - true_x)

print(f"{'策略':<20}{'平均誤差':>12}{'最大誤差':>12}{'跳動':>12}")
print(f"{'只用里程計':<20}{err_odom.mean():10.4f} m{err_odom.max():10.4f} m"
      f"{'無':>12}")
print(f"{'直接覆蓋':<20}{err_ovr.mean():10.4f} m{err_ovr.max():10.4f} m"
      f"{'np.mean(jumps):10.4f} m")
print(f"{'Kalman 融合':<20}{err_kf.mean():10.4f} m{err_kf.max():10.4f} m"
      f"{'平滑':>12}")
print("→ Kalman 兼顧了準確度與平滑性")

# ===== 二維 EKF：差速車定位 =====
print("\n=== 擴展 Kalman 濾波器：差速車定位 ===")
class EKF_DiffDrive:
    """狀態 [x, y,  $\theta$ ] · 量測為到已知地標的距離與方位"""
    def __init__(s, x0, P0, Q, R):
        s.x, s.P, s.Q, s.R = np.array(x0, float), np.array(P0), Q, R

    def predict(s, ds, dth):
        th = s.x[2]
        # 非線性運動模型 (用函數本身)
        s.x = s.x + np.array([ds*np.cos(th + dth/2),
                              ds*np.sin(th + dth/2), dth])
        s.x[2] = np.arctan2(np.sin(s.x[2]), np.cos(s.x[2])) # 角度正規化
        # 雅可比 (用於協方差傳播)
        G = np.array([[1, 0, -ds*np.sin(th + dth/2)],
                      [0, 1, ds*np.cos(th + dth/2)],
                      [0, 0, 1]])
        s.P = G @ s.P @ G.T + s.Q

    def update(s, z, landmark):
        lx, ly = landmark
        dx, dy = lx - s.x[0], ly - s.x[1]
        q = dx*dx + dy*dy
        r = np.sqrt(q)
        z_hat = np.array([r, np.arctan2(dy, dx) - s.x[2]])
        H = np.array([[-dx/r, -dy/r, 0],
                      [dy/q, -dx/q, -1]])
        innov = z - z_hat
        innov[1] = np.arctan2(np.sin(innov[1]), np.cos(innov[1]))

```

```

S = H @ s.P @ H.T + s.R
# 馬氏距離門檻 ( 24.9.3 節 )
d2 = innov @ np.linalg.solve(S, innov)
if d2 > 9.21:                                     # 卡方 99%, 2 自由度
    return False, d2
K = s.P @ H.T @ np.linalg.inv(S)
s.x = s.x + K @ innov
s.P = (np.eye(3) - K @ H) @ s.P
return True, d2

ekf = EKF_DiffDrive([0., 0., 0.], np.diag([0.01, 0.01, 0.01]),
                    np.diag([0.001, 0.001, 0.0005]),
                    np.diag([0.04, 0.01]))
landmark = np.array([5.0, 3.0])
print(f"{'步':>4}{ 'x':>9}{ 'y':>9}{ 'θ(度)':>9}{ 'σx':>9}{ 'σy':>9}{ '量測':>10}")
for k in range(10):
    ekf.predict(0.5, np.deg2rad(5))
    tag = "無"
    if k % 3 == 2:
        dx, dy = landmark - ekf.x[:2]
        z_true = np.array([np.hypot(dx, dy) + np.random.randn()*0.2,
                            np.arctan2(dy, dx) - ekf.x[2]
                            + np.random.randn()*0.1])
        ok, d2 = ekf.update(z_true, landmark)
        tag = f"接受" if ok else f"拒絕"
    print(f"{k:4d}{ekf.x[0]:9.3f}{ekf.x[1]:9.3f}"
          f"{np.rad2deg(ekf.x[2]):9.2f}{np.sqrt(ekf.P[0,0]):9.4f}"
          f"{np.sqrt(ekf.P[1,1]):9.4f}{tag:>10}")
print("> 有量測時 σ 下降 · 無量測時 σ 持續上升")

# ===== 粒子濾波器：全域定位 =====
print("\n=== 粒子濾波器：一維全域定位（走廊有三扇門）===")
doors = [2.0, 5.0, 8.0]                                # 門的位置（地圖）
M = 2000
particles = np.random.uniform(0, 10, M)                # 初始：完全不知道在哪
weights = np.ones(M)/M

def sense_door(x):
    """量測：是否在門旁邊（±0.4 m 內）"""
    return any(abs(x - d) < 0.4 for d in doors)

def measurement_prob(x, saw_door):
    near = any(abs(x - d) < 0.4 for d in doors)
    return 0.9 if near == saw_door else 0.1

true_x = 2.1
moves = [0.0, 3.0, 3.0]                                # 前進序列
print(f"{'步':>4}{ '真實位置':>10}{ '看到門':>9}{ '粒子均值':>11}"
      f"{'粒子標準差':>12}{ '有效粒子':>11}")
for step, mv in enumerate(moves):
    # 預測
    true_x += mv

```

```

particles = particles + mv + np.random.randn(M)*0.15
particles = np.clip(particles, 0, 10)
# 加權
saw = sense_door(true_x)
weights = np.array([measurement_prob(p, saw) for p in particles])
weights /= weights.sum()
n_eff = 1.0/np.sum(weights**2)
print(f"{step:4d}{true_x:10.2f}{'是' if saw else '否':>9}"
      f"{np.average(particles, weights=weights):11.3f}"
      f"{np.sqrt(np.average((particles-np.average(particles,weights=weights))**2,
weights=weights)):12.3f}"
      f"{n_eff:11.0f}")
# 重取樣 (僅在有效粒子數過低時 · 24.7.3 節)
if n_eff < M/2:
    idx = np.random.choice(M, M, p=weights)
    particles = particles[idx]
    weights = np.ones(M)/M
print("> 一開始標準差極大 (不知道在哪) · 逐步收斂到真實位置")
print("> 這是 Kalman 濾波器做不到的——初期是多峰分布")

```

請試著修改：(一) 在 Kalman 模擬中把  $Q$  設為原本的 100 倍，觀察估計是否變得雜訊很大。(二) 在 EKF 中故意讓某一次的量測有 5 公尺的誤差 (模擬離群值)，驗證馬氏距離門檻是否成功拒絕它。(三) 在粒子濾波器中把粒子數降到 50，觀察是否會發生粒子退化而收斂到錯誤的位置。

## 實作：感測器融合的實驗

本章實作以第 23 章的移動機器人為平台，加裝 IMU 後實現感測器融合。

- 感測器雜訊量測：**讓機器人靜止不動，連續記錄 IMU 與里程計的讀值數千筆。計算各軸的均值 (應為零或已知的偏差) 與變異數——這就是  $R$  矩陣的來源。
- 偏差辨識：**特別注意陀螺儀的零漂。靜止十分鐘，觀察積分後的角度如何線性增長，由斜率估算零漂量。這是必須補償的系統性誤差。
- 互補濾波實作：**以第 6.3.2 節的公式融合陀螺儀與加速度計 (或與里程計的航向)。測試不同的  $\alpha$  (0.90、0.95、0.98、0.995)，記錄各自的漂移速率與雜訊水準。
- 互補濾波的驗證：**把機器人放在轉盤上，緩慢旋轉一整圈後回到原位。比較純陀螺儀積分、純加速度計、與互補濾波三者的角度誤差。
- 一維 Kalman 實作：**以里程計為預測、以視覺標記 (或超音波測距到牆面) 為量測，實作一維位置的 Kalman 濾波器。



6. **Q、R 調參實驗**：系統性地改變 Q 與 R（各三個量級），記錄估計的平滑度與追隨速度。製作 3×3 的效果對照表，找出最佳組合。
7. **創新序列診斷**：記錄整個運行過程的創新序列，計算其均值與變異數，並與濾波器預測的 S 比較（24.9.2 節）。判斷 Q、R 是否設定合理。
8. **離群值測試**：故意在某幾個時刻注入錯誤的量測（如把視覺結果加上 2 公尺），驗證馬氏距離門檻能否成功拒絕。
9. **三種策略的對照**：讓機器人沿固定路徑行走 20 公尺，分別以（甲）只用里程計、（乙）直接覆蓋、（丙）Kalman 融合三種方式估計位置。記錄終點誤差與軌跡的平滑度。這是開頭故事的實驗版本。
10. **（進階）EKF 二維定位**：擴充為二維的 EKF，狀態為  $(x, y, \theta)$ ，以到已知地標的距離與方位為量測。繪出估計軌跡與誤差橢圓隨時間的演變。

表 24-10 感測器融合實驗記錄表（前兩列為範例）

| 測試項目       | 設定 | 終點誤差   | 軌跡平滑度 | 備註       |
|------------|----|--------|-------|----------|
| 只用里程計      | —  | 1.82 m | 平滑    | 系統性漂移    |
| 直接覆蓋       | —  | 0.21 m | 明顯跳動  | 轉彎處控制器震盪 |
| （自行填寫其餘測試） |    |        |       |          |

## 國中小也看得懂：誰比較可信就相信誰多一點

假設你和朋友在猜一個西瓜有多重。你說「大概 5 公斤」，朋友說「應該有 7 公斤」。那到底是幾公斤呢？

你可能會說：取中間，6 公斤！這是一個好方法——但還不是最好的。

再想想：如果你昨天才用秤量過類似的西瓜，而你朋友只是隨口猜的，那答案應該比較靠近你的 5 公斤才對。反過來，如果朋友是水果店老闆，那就該相信他多一點。

這就是這一章最重要的道理：不是各取一半，而是「誰比較可信，就相信誰多一點」。

機器人也一樣。它有好幾種方法知道自己在哪裡：

**方法一：數輪子轉了幾圈。** 這個方法反應很快，但走久了會越來越不準（因為輪子會打滑）。

**方法二：看天花板上的標記。** 這個方法很準，但要等看到標記才知道，而且光線暗的時候會看不清楚。

聰明的機器人不會「二選一」，而是**兩個都用**：平常用輪子的資料快速更新，看到標記時就拿來修正。而且它還會判斷「現在這個標記看得清不清楚」——看得清楚就多相信一點，看不清楚就少相信一點。

這個方法有一個名字，叫做「**卡爾曼濾波器**」。它是 1960 年一位工程師發明的，後來被用在**阿波羅太空船**上——就是載人類第一次登陸月球的那艘太空船！沒有它，太空人可能就找不到回家的路了。

## 叫獸提醒與常見錯誤

- 誤區一：用「切換」或「覆蓋」取代融合。這會扔掉資訊並造成估計跳動，正確作法是依可信度加權。
- 誤區二：以為加入一個差的感測器會讓結果變糟。只要誤差模型正確（無偏差），多一個量測必定使估計更確定。
- 誤區三： $Q$  設得太小。這使濾波器過度自信、拒絕接受量測、最終發散且無法恢復；不確定時應寧可設大。
- 誤區四：忽略未建模的偏差。Kalman 假設雜訊均值為零，若感測器有系統性偏差，應把它加入狀態向量一起估計。
- 誤區五：沒有離群值保護。單一個錯誤量測就可能破壞估計，必須加入馬氏距離門檻。
- 誤區六：忽略時間戳。把舊的量測當成當前的使用，在高速運動時會造成顯著誤差。
- 誤區七：對強非線性系統直接用 EKF。線性化誤差可能導致發散，應考慮 UKF 或縮短更新週期。
- 誤區八：用 Kalman 濾波器做全域定位。它只能表示單峰分布，無法處理「可能在好幾個地方」的情形，應改用粒子濾波器。
- 誤區九：粒子濾波器每一步都重取樣。這會加速粒子退化，應只在有效粒子數低於門檻時才重取樣。
- 誤區十：粒子濾波器用於高維狀態。維度詛咒使所需粒子數指數成長，高維問題應改用 Kalman 家族。

- 誤區十一：忘記角度的正規化。角度差必須用  $\text{atan2}(\sin, \cos)$  包裹到  $\pm\pi$ ，否則在跨越  $\pm 180^\circ$  時會產生巨大的假創新。
- 誤區十二：過早使用複雜的方法。應先用互補濾波把系統跑起來確認硬體無誤，再視需求升級為 Kalman。

## 課堂討論

1. 開頭故事中小紓用「直接覆蓋」的方式融合兩種定位。除了跳動之外，請討論這個作法還有哪些隱患？（提示：考慮視覺偶爾誤判的情形）
2. 24.4.2 節的洞見二說「融合必定使估計更確定」，但 24.9.3 節又說「未建模的偏差會系統性拉偏結果」。這兩者矛盾嗎？請說明前者成立的前提條件。
3. Kalman 濾波器與第 18 章的 LQR 在數學上是对偶的（都解 Riccati 方程）。請討論這個對偶在工程直覺上意味著什麼——「估計」與「控制」為何是同一件事的兩面？
4. 粒子濾波器能處理多峰分布，但受維度詛咒限制。請討論：第 25 章的 SLAM 問題（狀態含數百個地標）該用哪一類方法？為什麼？
5. 本書刻意在第 6 章先給互補濾波、第 24 章才揭曉其原理。請討論這種「先用後懂」的教學安排的優缺點，以及它在工程實務上的對應（提示：對照第 2 章的即時學習策略）。

## 章末摘要

- 機率式機器人學的核心轉換：不問「在哪裡」，而問「在各個位置的可能性各有多大」。
- 信念分布的形狀承載資訊：窄峰代表確定、寬峰代表不確定、多峰代表有多個候選、拉長代表某方向不確定。
- 定位問題分三個層次：位置追蹤（初始已知）、全域定位（初始未知）、綁架問題（運行中突然錯位）。
- 貝氏濾波由兩步驟循環：預測（信念擴散、不確定性增加）與更新（信念收攏、不確定性減少）。
- 馬可夫假設使遞迴計算成為可能——當前狀態包含所有必要資訊，歷史可以丟棄。
- 高斯分布因「只需兩參數、線性運算下封閉、相乘仍為高斯」而成為標準選擇。
- 協方差矩陣對應誤差橢圓：特徵向量為主軸方向、特徵值平方根為半徑、非對角元素代表相

關性。

- 一維融合公式：權重與變異數成反比；融合必定使不確定性減小；資訊量  $1/\sigma^2$  是可加的。
- Kalman 濾波器五條方程式：狀態預測、協方差預測、增益計算、狀態更新、協方差更新。
- Kalman 增益  $K = P^-(P^-+R)$  自動調節：預測越不準則越信任量測，反之亦然。
- Kalman 濾波器在線性高斯假設下是最小均方誤差意義的最佳估測器，且與 LQR 互為對偶。
- EKF 以雅可比線性化處理非線性（一階、需推導雅可比）；UKF 以 sigma points 取樣傳遞（二階、不需雅可比）。
- 粒子濾波器以大量樣本表示任意分布，能處理多峰與全域定位，但受維度詛咒限制。
- 粒子退化的三個對策：只在有效粒子數過低時重取樣、注入隨機粒子、使用低變異數重取樣。
- 互補濾波是「增益固定」的 Kalman 濾波器： $\alpha = 1 - K$ ；其名稱來自兩感測器的頻率響應恰好互補。
- 實務三大陷阱：未建模的偏差（應狀態擴增）、離群值（應馬氏距離門檻）、時間戳不同步（應時間對齊）。
- 創新序列的一致性檢驗是診斷濾波器健康的黃金標準；不確定時應寧可把  $Q$  設大。
- 融合的核心邏輯：高頻但漂移的感測器提供短期平滑，低頻但不漂移的感測器定期校正。

## 觀念題與計算題

### 觀念題

1. 說明機率式機器人學的核心觀念轉換，並列舉這個轉換帶來的三個好處。
2. 定義信念分布，並說明五種分布形狀各代表什麼情境。
3. 區分位置追蹤、全域定位與綁架機器人問題，並指出各需要什麼方法。
4. 敘述貝氏濾波的兩個步驟，並說明各自對不確定性的影響。
5. 說明馬可夫假設的內容，以及它為何是遞迴計算的前提。
6. 列舉高斯分布被選用的四個理由，並說明哪兩個對遞迴特別關鍵。
7. 說明協方差矩陣的幾何意義，並解釋非對角元素代表什麼。
8. 推導一維資訊融合的公式，並說明「融合必定更確定」的原因與前提。

9. 寫出 Kalman 濾波器的五條方程式，並說明各項的物理意義。
10. 說明 Kalman 增益如何自動調節，並列出三種極端情形的行為。
11. 比較 EKF 與 UKF 的原理、精度與計算量，並說明各自的適用場合。
12. 說明粒子濾波器的三個步驟，以及重取樣為何是關鍵機制。
13. 什麼是粒子退化？請說明三種對策及其原理。
14. 推導互補濾波與 Kalman 濾波器的等價關係，並說明「互補」一詞的頻域意義。
15. 列舉感測器融合的三个實務陷阱，並說明各自的因應之道。
16. 說明創新序列的一致性檢驗，並列出三種可由此診斷的問題。

## 計算題

1. 兩個獨立的位置估計：GNSS 給出 25.4 m ( $\sigma = 2.0$  m)、視覺里程計給出 24.8 m ( $\sigma = 0.5$  m)。(a) 求融合後的估計值與標準差。(b) 求兩者的權重比。(c) 若 GNSS 進入城市峽谷使  $\sigma$  劣化為 8.0 m，重新計算。(d) 比較兩種情形並說明。
2. 一維 Kalman 濾波器： $Q = 0.09 \text{ m}^2$ 、 $R = 0.16 \text{ m}^2$ 、初始  $\hat{x} = 5.0$  m、 $P = 0.36 \text{ m}^2$ 。(a) 里程計增量  $u = 2.0$  m，求預測值與  $P^-$ 。(b) 量測  $z = 7.5$  m，求 Kalman 增益。(c) 求更新後的估計值與協方差。(d) 若接連五步無量測，求  $P$  的演變並計算對應的標準差。
3. 某互補濾波器取  $\alpha = 0.995$ 、取樣週期 2 ms。(a) 求等效的 Kalman 增益。(b) 求等效時間常數。(c) 若加速度計的角度量測變異數  $R = 9.0 \text{ deg}^2$ ，反推隱含的預測協方差  $P^-$ 。(d) 若機器振動加劇使加速度計雜訊倍增，討論應如何調整  $\alpha$ 。
4. 某 EKF 的創新為  $v = [0.35, -0.12]^T$ ，創新協方差  $S = [[0.04, 0.01], [0.01, 0.09]]$ 。(a) 計算  $S$  的反矩陣。(b) 計算馬氏距離的平方  $d^2$ 。(c) 若門檻取卡方分布 95% (2 自由度為 5.99)，判斷此量測應接受或拒絕。(d) 若改用 99% 門檻 (9.21)，結論是否改變？

## 動手做與專題延伸

### 本章實作：狀態估測模組

延續第 23 章的移動機器人程式庫，加入狀態估測模組。這是第 25、26 章的必要基礎。

- 階段一：實作一維 Kalman 濾波器，並用例題 24-1、24-2 驗證。
- 階段二：實作二維 EKF 的差速車定位，狀態為  $(x, y, \theta)$ 、量測為到已知地標的距離與方位。

加入角度正規化與馬氏距離門檻。

- **階段三**：實作誤差橢圓的繪製，即時顯示協方差矩陣的幾何意義。
- **階段四**：實作一維與二維的粒子濾波器，含有效粒子數判斷與隨機粒子注入。
- **階段五**：實作創新序列的一致性檢驗工具，能自動診斷  $Q$ 、 $R$  是否設定合理。

## 專題延伸

- **誤差橢圓的演化動畫**：模擬機器人沿一條路徑行進，途中經過數個地標。即時繪製協方差橢圓——你會看到橢圓在無量測時持續脹大、且沿行進方向拉長（因航向誤差累積），而在看到地標時瞬間收縮。這是本章所有理論最直觀的視覺化，強烈建議做成動畫。
- **三種濾波器的競賽**：設計一組測試情境（位置追蹤、全域定位、綁架問題、強非線性），對 EKF、UKF 與粒子濾波器各自測試。記錄估計誤差、計算時間與失敗率，製作完整的比較表。你會發現沒有一種方法在所有情境下都最好——這正是工程選型的依據。
- **進階：Q 與 R 的自動調參**。實作一個以創新序列統計為基礎的自適應方法：持續監測創新的實際變異數與濾波器預測的  $S$ ，若兩者持續不符就自動調整  $Q$ 。在一個模擬環境中測試（可故意讓感測器雜訊在中途改變），觀察自適應機制能否追上變化。討論這個方法的穩定性風險——若調整過於激進，濾波器可能震盪甚至發散。

## 參考文獻與延伸閱讀

- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press（機率式機器人學的權威教材，第 2~4、7~8 章直接對應本章）。
- Kalman, R. E., "A New Approach to Linear Filtering and Prediction Problems," ASME J. Basic Engineering, 1960（Kalman 濾波器的原始論文）。
- Julier, S. J. and Uhlmann, J. K., "Unscented Filtering and Nonlinear Estimation," Proceedings of the IEEE, 2004（UKF 的完整說明）。
- Bar-Shalom, Y., Li, X.-R. and Kirubarajan, T., Estimation with Applications to Tracking and Navigation, Wiley（估測理論的工程導向專著，含大量實務細節）。
- Simon, D., Optimal State Estimation: Kalman,  $H_\infty$ , and Nonlinear Approaches, Wiley（涵蓋各類估測方法的完整教材）。
- Doucet, A., de Freitas, N. and Gordon, N. (Eds.), Sequential Monte Carlo Methods in

Practice, Springer ( 粒子濾波器的理論與應用 ) 。

- Dellaert, F. et al., "Monte Carlo Localization for Mobile Robots," ICRA 1999 ( 蒙地卡羅定位的奠基論文 ) 。
- Mahony, R., Hamel, T. and Pflimlin, J.-M., "Nonlinear Complementary Filters on the Special Orthogonal Group," IEEE Trans. Automatic Control, 2008 ( 互補濾波的嚴謹推廣 ) 。
- Bailey, T. and Durrant-Whyte, H., "Simultaneous Localization and Mapping: Part II," IEEE Robotics & Automation Magazine, 2006 ( 銜接第 25 章 ) 。
- ROS 2 robot\_localization 與 AMCL 官方文件：EKF/UKF 融合與蒙地卡羅定位的實務參考。

# 第 25 章

## 同步定位與建圖 ( SLAM )

編著：李世淵 ( 機器人叫獸 )

助教：李天宇、李宇晴

【章首情境圖建議】左半為迴路閉合前後的地圖對照：上圖是只靠里程計建出的地圖——走廊呈弧形彎曲，繞行一圈後最後一段牆與第一段錯開約兩公尺，並以紅色虛線圈出錯位處；下圖是偵測到迴路閉合並最佳化後的同一張地圖——牆面筆直對齊、房間方正。右半為位姿圖的彈簧類比示意：一串珠子 ( 節點 ) 以彈簧 ( 里程計邊 ) 相連成環，環的兩端另接一根粗彈簧 ( 迴路閉合邊 )，箭頭表示系統被拉回平衡的過程。畫面下緣文字：定位需要地圖，建圖需要定位——那就一起解。



## 叫獸開講 「要建圖得先定位，要定位得先有地圖——怎麼辦？」

叫獸把一台裝了光達的機器人放在實驗室門口，然後對全班說：「請你們讓它畫出這層樓的地圖。」

小翰很快動手：「簡單，讓它走一圈，每一步用里程計知道自己在哪，把光達看到的牆畫上去就好。」他寫完程式，機器人繞了一圈回到門口——螢幕上的地圖卻是一團扭曲的線條，走廊變成弧形，最後一段牆與第一段錯開了將近兩公尺。

「為什麼？」叫獸問。小好反應過來：「因為里程計會漂移。走到後面，它以為自己在 A 點，其實已經在 B 點了，所以把牆畫錯了位置。」

「那就先定位準一點再建圖。」小翰說。叫獸點頭：「好，那你怎麼定位？」

「用第 24 章的方法，跟地圖比對……」小翰說到一半停住了。

「地圖從哪來？」叫獸接話，「你要建圖，得先知道自己在哪；你要知道自己在哪，又得先有地圖。」他在白板上畫了一個圈：**定位 → 需要地圖 → 需要定位**。「這就是機器人學最有名的一个循環，叫做 **SLAM**——同步定位與建圖。」

教室安靜了幾秒。小好問：「那……有解嗎？」

「有，而且答案出乎意料地優雅。」叫獸在白板上寫下一行字：**不要分開解，一起解**。「與其先定位再建圖，不如把『我在哪裡』和『地標在哪裡』放進同一個未知數清單，一次全部估計出來。」

他停頓一下，補充了關鍵的一點：「而且這裡有一個很美的機制。當你觀測到一個地標時，你同時得到了兩件事的資訊——**地標在哪裡，以及你自己在哪裡**。它們透過這次觀測綁在一起了。累積夠多的觀測，這些互相牽制的關係就會把整個系統『拉』到一個一致的答案上。」

最後他拿起一條繩子，圍成一個環：「而 SLAM 最戲劇性的時刻，叫做**迴路閉合**。當機器人繞了一大圈，突然認出『這個地方我來過』——這一瞬間，累積了幾百公尺的誤差會被一口氣攤平，整張地圖會**當場扭正**。看過那個畫面的人，都不會忘記。」

## 本章為什麼重要

SLAM 是移動機器人能夠自主運作的關鍵技術。沒有它，機器人只能在事先人工測繪好的環

境中運作——而人工測繪既昂貴又緩慢，且環境一改變就得重來。

它的應用已經滲透到日常生活：掃地機器人建立家中的地圖、AR 眼鏡定位使用者在房間中的位置、自駕車建立高精地圖、無人機在 GPS 訊號不良處導航、倉儲 AGV 在不斷變動的貨架間穿梭。

但 SLAM 的價值不只在應用，更在於它是一個完美的整合範例。它需要：第 22 章的視覺與感知、第 23 章的運動模型、第 24 章的機率估測、第 8 章的座標變換、以及第 18 章的最佳化思維。要把 SLAM 做好，前面二十四章的知識必須全部到位——這也是為什麼它被安排在這個位置。

本章要建立三個層次的理解。**第一，問題的本質**：為何 SLAM 是雞生蛋問題，以及「一起解」為何可行。**第二，三代主流方法**：EKF-SLAM、粒子濾波 SLAM、以及現代的圖最佳化 SLAM，理解它們各自的洞見與限制。**第三，實務的關鍵**：迴路閉合、資料關聯、與長期運行的挑戰。

## 學習目標

- 能陳述 SLAM 問題並說明其「雞生蛋」的本質。
- 能說明為何聯合估計比先後求解更有效。
- 能比較三種地圖表示法及其適用場合。
- 能說明 EKF-SLAM 的狀態向量結構與其  $O(n^2)$  的複雜度來源。
- 能解釋地標之間的相關性為何是 SLAM 收斂的關鍵。
- 能說明 Rao-Blackwell 分解的思想與 FastSLAM 的架構。
- 能說明佔據柵格地圖的更新原理與對數機率的優勢。
- 能說明圖最佳化 SLAM 的節點與邊，並解釋其為何成為現代主流。
- 能說明迴路閉合的作用、偵測方法與誤判的風險。
- 能說明資料關聯問題及其處理方法。
- 能比較視覺 SLAM 與光達 SLAM 的特性與適用場合。
- 能為特定應用選擇合適的 SLAM 方案並說明理由。

## 先備知識

第 24 章的貝氏濾波、Kalman 濾波器與粒子濾波器（本章直接建立在其上）、第 23 章的運

動模型與里程計、第 22 章的視覺與特徵匹配、第 8 章的座標變換、第 18 章的最佳化與 Bellman 原理。

本章是全書整合度最高的章節之一。建議讀法：若對第 24 章的內容有任何不確定，務必先回頭複習——SLAM 的所有方法都是那些工具的延伸。25.6 節的圖最佳化涉及較多數學，可先掌握其幾何直覺（「把彈簧系統拉到平衡」），細節之後再補。

## 核心名詞中英對照

表 25-1 本章核心名詞

| 中文      | 英文                              | 說明               |
|---------|---------------------------------|------------------|
| 同步定位與建圖 | SLAM                            | 同時估計機器人位姿與環境地圖   |
| 完整 SLAM | Full SLAM                       | 估計整條軌跡與地圖        |
| 線上 SLAM | Online SLAM                     | 只估計當前位姿與地圖       |
| 地標      | Landmark                        | 環境中可重複辨識的特徵      |
| 特徵地圖    | Feature Map                     | 以離散地標點表示的地圖      |
| 佔據柵格地圖  | Occupancy Grid Map              | 以格點的佔據機率表示的地圖    |
| 拓撲地圖    | Topological Map                 | 以節點與連結表示的地圖      |
| 資料關聯    | Data Association                | 判斷觀測對應到哪一個已知地標   |
| 迴路閉合    | Loop Closure                    | 辨認出曾經到過的地方       |
| 感知混淆    | Perceptual Aliasing             | 不同地點看起來相同的現象     |
| 圖最佳化    | Graph Optimization / Pose Graph | 以節點與約束邊表示並最佳化的方法 |
| 因子圖     | Factor Graph                    | 以變數與因子表示機率關係的圖   |
| 稀疏性     | Sparsity                        | 矩陣中大量元素為零的性質     |
| 前端      | Front-end                       | 處理感測資料、建立約束的模組   |
| 後端      | Back-end                        | 求解最佳化問題的模組       |
| 對數勝算比   | Log-odds                        | 以對數形式表示機率的方法     |
| 視覺 SLAM | Visual SLAM, vSLAM              | 以相機為主要感測的 SLAM   |
| 視覺慣性里程計 | Visual-Inertial Odometry, VIO   | 融合視覺與 IMU 的里程計   |

## 25.1 SLAM 問題的本質

### 25.1.1 雞生蛋的循環

如開頭故事所述，SLAM 的核心困難是一個循環依賴：

要建圖 → 必須知道自己在哪（否則不知道把觀測畫在地圖的何處）

要定位 → 必須有地圖（否則沒有參照物）

*SLAM 的雞生蛋問題。兩者互為前提。*

這個循環之所以無法用「先做一個、再做另一個」的方式打破，是因為兩者的誤差會互相污染。位姿估計錯了，地圖就畫歪；地圖歪了，下次定位又更錯——誤差會正回饋地放大，這正是小翰第一次嘗試失敗的原因。

### 25.1.2 解法：聯合估計

SLAM 的關鍵洞見是：不要試圖打破循環，而是承認兩者互相依賴，把它們一起估計。

$$p(x_{1:t} \, m \mid z_{1:t} \, u_{1:t})$$

*完整 SLAM 的機率表述。同時估計整條軌跡  $x_{1:t}$  與地圖  $m$ ，條件是所有的觀測  $z$  與控制  $u$ 。*

這個表述有一個變體，只估計當前位姿而非整條軌跡：

$$p(x_t \, m \mid z_{1:t} \, u_{1:t})$$

*線上 SLAM。計算量較小，適合即時運行，但無法回頭修正過去的軌跡。*

兩者的差別在第 25.6 節會顯得極為重要——迴路閉合需要修正過去的軌跡，因此本質上是完整 SLAM 的問題。這正是現代方法由濾波轉向圖最佳化的原因。

### 25.1.3 為何聯合估計可行

聯合估計聽起來只是把問題變大（未知數更多了），為何反而可解？關鍵在於觀測同時約束了兩者。

#### ◎ 一次觀測，兩份資訊

當機器人觀測到「地標 A 在我的右前方 3 公尺處」時，它同時得到了兩件事的資訊：若我知道自己在哪，就知道 A 在哪；若我知道 A 在哪，就知道自己在哪。這一次觀測把「我的位姿」與「A 的位置」綁在一起了——兩者的誤差從此相關。累積夠多這樣的約束，整個系統就會被「拉」到一個唯一一致的解上。這正是 SLAM 能夠收斂的根本原因。

用一個生活化的類比：這就像玩拼圖。單看一片拼圖無法確定它的位置，但當你把許多片互相咬合起來，每一片的位置都被鄰居約束，最終整幅圖只有一種拼法。SLAM 中的「咬合」就是

觀測，而「唯一的拼法」就是最佳解。

#### 25.1.4 SLAM 的三個子問題

實務上，SLAM 系統面對三個層次的問題：

- **狀態估計**：給定觀測與控制，求最可能的軌跡與地圖。這是 25.3 ~ 25.6 節的主題。
- **資料關聯**：這次看到的地標，是之前哪一個？還是全新的？這是 25.8 節的主題，也是實務上最容易出錯的環節。
- **迴路閉合**：機器人是否回到了曾經到過的地方？這是 25.7 節的主題，也是消除長期漂移的唯一手段。

三者的難度並不對稱。狀態估計有成熟的數學工具，反而是資料關聯與迴路閉合才是真正的難點——因為它們涉及「辨識」而非「計算」，而辨識必然有錯誤的可能。

## 25.2 地圖的表示

### 25.2.1 三種主流表示法

在討論演算法之前，必須先決定「地圖長什麼樣」。這個選擇會深刻影響後續的所有設計。

**表示法一：特徵地圖 (Feature Map)**。地圖是一組離散的地標點，每個地標有其座標（可能還有描述子）。例如「牆角在 (3.2, 5.1)」、「柱子在 (7.8, 2.4)」。

- **優點**：資料量小、與 EKF 天然契合（每個地標就是狀態向量中的兩三個元素）、易於做資料關聯。
- **缺點**：需要能穩定偵測的特徵；無法表示「哪裡是空的」（這對路徑規劃很重要）；不規則的環境難以用點表示。

**表示法二：佔據柵格地圖 (Occupancy Grid Map)**。把空間切成規則的格子，每格記錄「被佔據的機率」。這是掃地機器人與 ROS 導航最常用的表示法。

- **優點**：能表示任意形狀、明確區分「佔據 / 空曠 / 未知」三種狀態、直接可用於路徑規劃（第 26 章）。
- **缺點**：記憶體隨面積與解析度平方成長（三維時是立方）、不含語意資訊。

**表示法三：拓撲地圖 (Topological Map)**。只記錄「地點」與「地點之間的連結」，不含

精確的幾何。例如「房間 A 連到走廊 B、走廊 B 連到房間 C」。

- **優點**：極為精簡、適合大範圍導航與任務規劃、接近人類的空間認知方式。
- **缺點**：無法做精確定位與避障，通常需與其他表示法混用。

表 25-2 三種地圖表示法的比較

| 面向        | 特徵地圖              | 佔據柵格地圖                                | 拓撲地圖        |
|-----------|-------------------|---------------------------------------|-------------|
| 表示內容      | 離散地標點             | 每格的佔據機率                               | 地點與連結關係     |
| 記憶體需求     | 小 ( 正比於地標數 )      | 大 ( 正比於面積 $\times$ 解析度 <sup>2</sup> ) | 極小          |
| 能否表示空曠區   | 否                 | 是                                     | 否           |
| 可直接用於路徑規劃 | 否                 | 是                                     | 部分 ( 高層規劃 ) |
| 適用的估測方法   | EKF-SLAM          | 粒子濾波 SLAM                             | 圖 SLAM 的高層  |
| 典型應用      | 早期 SLAM、稀疏視覺 SLAM | 掃地機器人、室內 AGV                          | 大範圍導航、語意地圖  |

## 25.2.2 現代的混合表示

實務系統往往同時維護多種表示。一個典型的架構是：用特徵 ( 或關鍵影格 ) 做定位與最佳化、用佔據柵格做路徑規劃、用拓撲層做高階任務決策。

近年還興起了兩種新的表示：語意地圖 ( 除幾何外還標註「這是門、這是桌子」，為第 32 章的任務規劃提供基礎 ) 與神經隱式表示 ( 如 NeRF，用一個神經網路隱含地表示整個場景的幾何與外觀 )。後者是目前的研究前沿，但尚未在工業上普及。

## 25.3 EKF-SLAM

### 25.3.1 把地圖放進狀態向量

EKF-SLAM 是最早的完整解法，其思想直接體現了 25.1.2 節的「一起估計」：把所有地標的座標，都放進 Kalman 濾波器的狀態向量中。

$$\mathbf{x} = [x_r, y_r, \theta_r, m_{1x}, m_{1y}, m_{2x}, m_{2y}, \dots, m_{Nx}, m_{Ny}]^T$$

*EKF-SLAM 的狀態向量。前三個是機器人位姿，其餘是  $N$  個地標的座標。維度為  $3 + 2N$ 。*

對應的協方差矩陣則是  $(3+2N) \times (3+2N)$  的方陣。它的結構極具啟發性：

$$\Sigma = \begin{bmatrix} \Sigma_{rr} & \Sigma_{rm} \\ \Sigma_{mr}^T & \Sigma_{mm} \end{bmatrix}$$

分塊結構。 $\Sigma_{rr}$  是機器人位姿的不確定性、 $\Sigma_{mm}$  是地標的不確定性、 $\Sigma_{rm}$  是兩者之間的相關性。

### 25.3.2 相關性：SLAM 收斂的關鍵

那個非對角區塊  $\Sigma_{rm}$  是整個 EKF-SLAM 最重要的部分。它記錄了「機器人位姿的誤差」與「地標位置的誤差」之間的**相關性**。

為何相關性如此重要？考慮一個具體情境：

1. 機器人在某處觀測到地標 A，此時它自己的位置有 10 公分的誤差，因此 A 的估計位置也有約 10 公分的誤差——**而且是同方向的誤差**（因為源自同一個位姿誤差）。
2. 機器人移動並累積更多誤差，現在自己的位置誤差變成 30 公分。
3. 此時它再次觀測到 A。由於 A 的位置與機器人當時的位姿相關，這次觀測**同時修正了兩者**——機器人的位置誤差降回接近 10 公分。

若沒有記錄**相關性**（把  $\Sigma_{rm}$  當成零），系統會誤以為 A 的位置是獨立已知的，於是過度自信地修正，導致估計不一致甚至發散。這是 SLAM 早期研究的一個重要教訓。

#### ◎ SLAM 的收斂定理

Dissanayake 等人於 2001 年證明了 EKF-SLAM 的三個重要性質：一、**地圖的不確定性單調遞減**——隨著觀測累積，地標的位置估計只會越來越準。二、**地標之間的相關性會趨於完全相關**——所有地標最終形成一個剛性的「骨架」，它們的相對位置極為精確，只是整體相對於世界座標系仍有不確定性。三、**地圖不確定性的下界，由第一次觀測時機器人的位姿不確定性決定**。這三個性質給出了 SLAM 為何可行的嚴格保證。

### 25.3.3 $O(n^2)$ 的複雜度困境

EKF-SLAM 有一個致命的實務問題：**協方差矩陣的更新是  $O(n^2)$  的複雜度**（ $n$  為地標數）。

原因是：每一次觀測，即使只看到一個地標，**整個協方差矩陣的所有元素都必須更新**——因為所有地標之間都有相關性。這使得：

- **100 個地標**：矩陣有  $203 \times 203 \approx 41000$  個元素，尚可即時處理。
- **1000 個地標**： $2003 \times 2003 \approx 400$  萬個元素，每次更新需要數百毫秒——已經無法即時。
- **10000 個地標**：4 億個元素，完全不可行。

這個限制使 EKF-SLAM 只適用於**小規模、地標稀疏**的環境。它在教學上仍極有價值（結構



清晰、直接體現 SLAM 的本質)，但實務上已被後續方法取代。

此外它還繼承了 EKF 的所有缺點（第 24.6.2 節）：線性化誤差、只能表示單峰分布、且資料關聯錯誤無法修正——一旦把兩個不同的地標誤認為同一個，錯誤會永久留在協方差矩陣中。

## 25.4 粒子濾波 SLAM 與 FastSLAM

### 25.4.1 Rao-Blackwell 分解

FastSLAM 的突破來自一個巧妙的觀察，稱為 **Rao-Blackwell 分解**：

#### ◎ 關鍵洞見

如果已經知道機器人的完整軌跡，那麼各個地標的位置就互相獨立了。因為地標之間的相關性，完全源自於「共用了不確定的機器人位姿」——一旦軌跡確定，這個共同原因就消失了。

這個洞見允許把聯合分布分解為兩部分：

$$p(x_{1:t}, m \mid z, u) = p(x_{1:t} \mid z, u) \cdot \prod_i p(m_i \mid x_{1:t}, z)$$

*Rao-Blackwell 分解。第一項用粒子濾波器處理軌跡、第二項的每個地標用一個獨立的小型 EKF 處理。*

### 25.4.2 FastSLAM 的架構

由上述分解，FastSLAM 的結構為：

- 每個粒子代表一條完整的軌跡假設（而非只是當前位姿）。
- 每個粒子攜帶自己的一整張地圖——由  $N$  個獨立的  $2 \times 2$  小型 EKF 構成（每個地標一個）。
- 權重由觀測與該粒子地圖的吻合程度決定。

這個架構的三個關鍵優勢：

**優勢一：複雜度降為  $O(M \log N)$** （ $M$  為粒子數、 $N$  為地標數）。因為每個地標只需維護  $2 \times 2$  的小矩陣，不再有巨大的協方差矩陣。

**優勢二：能處理資料關聯的不確定性。**不同的粒子可以做出不同的資料關聯決定——若某個粒子關聯錯了，它的地圖會與後續觀測不符，權重下降而被淘汰。錯誤會被自動篩除，這是 EKF-SLAM 做不到的。

**優勢三：能表示多峰分布，**繼承了粒子濾波器處理全域定位的能力。



### 25.4.3 FastSLAM 的限制

FastSLAM 的主要問題是**粒子退化**（第 24.7.3 節）——而在 SLAM 中，這個問題特別嚴重。

原因是：重取樣會淘汰粒子，而**被淘汰的粒子連同它的整條軌跡歷史一起消失**。經過長時間運行後，所有粒子可能都源自同一個祖先，代表「過去的軌跡只剩一種假設」——此時系統已無法修正過去的錯誤。這使 FastSLAM 在**大迴路閉合**時表現不佳。

儘管如此，FastSLAM 的一個變體在實務上極為成功：**GMapping**，它是 ROS 中最廣泛使用的二維光達 SLAM 套件之一，採用佔據柵格地圖而非特徵地圖。它的成功關鍵在於一個改良——用當前的光達掃描來改善提議分布（proposal distribution），使粒子更集中在正確的位置附近，大幅減少所需的粒子數。

表 25-3 EKF-SLAM 與 FastSLAM 的比較

| 面向     | EKF-SLAM        | FastSLAM                 |
|--------|-----------------|--------------------------|
| 核心思想   | 地圖進狀態向量、聯合估計    | Rao-Blackwell 分解、軌跡與地圖分離 |
| 軌跡表示   | 只有當前位姿（線上 SLAM） | 整條軌跡（每個粒子一條）             |
| 地圖表示   | 單一地圖            | 每個粒子一張地圖                 |
| 複雜度    | $O(N^2)$        | $O(M \log N)$            |
| 能否多峰   | 否               | 是                        |
| 資料關聯錯誤 | 無法修正            | 可被篩除                     |
| 主要限制   | 地標數受限（< 幾百）     | 粒子退化、大迴路表現差              |
| 代表實作   | 教學與早期研究         | GMapping（ROS）            |

## 25.5 佔據柵格地圖

### 25.5.1 每一格的機率

佔據柵格地圖把空間切成規則的格子（二維常用 5 公分或 10 公分），每格記錄一個機率值：這格被障礙物佔據的機率。

與特徵地圖相比，它有一個關鍵優勢：**能明確區分三種狀態**——已知佔據（機率接近 1）、已知空曠（接近 0）、未探索（0.5）。第三種狀態對自主探索極為重要，因為機器人需要知道「哪裡還沒去過」。

地圖更新的基本假設是：**各格互相獨立**。這在物理上並不完全正確（相鄰的格子通常屬於同一面牆），但它使計算大幅簡化，且實務效果良好。

$$p(m \mid z_{1:t} x_{1:t}) = \prod_i p(m_i \mid z_{1:t} x_{1:t})$$

獨立性假設。每一格可以單獨更新，不需考慮鄰居。

### 25.5.2 對數勝算比

直接以機率值更新會遇到兩個問題：反覆相乘導致數值下溢、以及每次更新都需要正規化。標準的解法是改用**對數勝算比**（log-odds）：

$$l(m) = \log[ p(m)/(1 - p(m)) ]$$

對數勝算比。機率 0.5 對應  $l = 0$ ；機率趨近 1 時  $l \rightarrow +\infty$ ；趨近 0 時  $l \rightarrow -\infty$ 。

這個轉換的威力在於：**貝氏更新從乘法變成加法**。

$$l_t = l_{t-1} + l_{\text{sensor}} - l_0$$

更新規則。 $l_{\text{sensor}}$  是本次觀測提供的證據、 $l_0$  是先驗（通常為 0）。只需要加減法。

三個實務好處：一、**計算極快**（只有加減）；二、**不會數值下溢**（不會出現  $0.0001 \times 0.0001$  這種問題）；三、**天然實現了「記憶」**——同一格被反覆觀測到是障礙物， $l$  就持續累加，越來越確定。

### 25.5.3 感測器逆模型

那麼  $l_{\text{sensor}}$  該取多少？這由**感測器逆模型**（inverse sensor model）決定。以光達的一條射線為例：

- **射線經過的格子**（在量測距離之前）：這些格子應該是空的，給予負的  $l_{\text{sensor}}$ （如  $-0.4$ ）。
- **射線終點的格子**：這裡有障礙物，給予正的  $l_{\text{sensor}}$ （如  $+0.85$ ）。
- **射線終點之後的格子**：被遮擋，無資訊， $l_{\text{sensor}} = 0$ （不更新）。

這個「沿射線標記空曠、終點標記佔據」的程序稱為**射線追蹤**（ray casting），實作上常用 Bresenham 演算法（原本用於電腦繪圖畫直線）高效完成。

**兩個實務細節**：一、 $l$  應該設定上下限（如  $\pm 5$ ），否則長時間觀測後某格會變得「無限確定」，導致環境改變時無法更新——這對動態環境（有人走動、傢俱移動）極為重要。二、佔據

的證據 ( +0.85 ) 通常大於空曠的證據 ( -0.4 )，因為「打到東西」比「沒打到」更可靠。

圖 25-1 佔據柵格地圖的射線更新

建議配圖：俯視圖。機器人位於左下，一條光達射線射向右上方並在某處打到牆面。射線經過的格子以淺藍色標示 (  $l$  減少，趨向空曠 )、終點的格子以深紅色標示 (  $l$  增加，趨向佔據 )、終點之後的格子以灰色標示 ( 被遮擋，不更新 )。右側附一張對數勝算比的數線圖，標出  $l = 0$  ( 未知， $p = 0.5$  )、 $l = +5$  ( 確定佔據 )、 $l = -5$  ( 確定空曠 )，並以箭頭表示每次觀測如何推移  $l$  值。下方另附三格小圖，顯示同一區域被觀測 1 次、5 次、20 次後地圖的清晰度變化。

## 25.6 圖最佳化 SLAM

### 25.6.1 觀念的轉向

EKF-SLAM 與 FastSLAM 都是濾波的思路：維護一個當前的信念，隨時間遞迴更新，過去的資料用完就丟。這個作法有一個根本的限制——無法回頭修正過去的軌跡。

但迴路閉合恰恰需要這個能力。當機器人繞了一大圈後認出「我回到起點了」，它需要把整條軌跡都調整一遍，才能消除累積的誤差。

因此現代 SLAM 轉向了完全不同的思路：把所有歷史資料都保留下來，當成一個大型的最佳化問題來解。這稱為圖最佳化 SLAM ( Graph-based SLAM )，也稱位姿圖 ( pose graph ) 最佳化。

### 25.6.2 節點與邊

圖最佳化把 SLAM 表示成一個圖：

- **節點 ( Node )**：待估計的變數。通常是機器人在各個時刻的位姿 ( 有時也包含地標的位置 )。
- **邊 ( Edge )**：兩個節點之間的約束，來自某次觀測。每條邊記錄「這兩個節點之間的相對位姿應該是多少」以及「這個約束有多可信」( 資訊矩陣 )。

邊有兩種來源：

**里程計邊**：相鄰時刻之間的約束，來自輪速或 IMU。這些邊構成一條鏈，且必定存在。

**迴路閉合邊**：不相鄰的兩個時刻之間的約束，來自「認出這個地方我來過」。這些邊把鏈接成環，是消除漂移的關鍵。

### 25.6.3 彈簧系統的直觀理解

圖最佳化有一個極為直觀的物理類比：

### ◎ 彈簧系統的類比

把每個節點想成一顆珠子、每條邊想成一根彈簧。彈簧的自然長度就是觀測所預期的相對位姿；彈簧的勁度就是該觀測的可信度（資訊矩陣）。若把珠子按里程計的估計擺放，所有彈簧都會被拉伸或壓縮——系統處於高能量狀態。**最佳化的過程，就是放手讓這個彈簧系統自己找到平衡。**當總彈性位能最小時，就是最一致的軌跡與地圖。

這個類比也解釋了迴路閉合的戲劇性效果：加入一條迴路閉合邊，等於在環的兩端接上一根強力彈簧。系統為了平衡這根新彈簧，會把整個環「拉正」——累積的誤差被均勻地攤到整條軌跡上。這正是開頭故事中「地圖當場扭正」的物理解釋。

## 25.6.4 數學形式

形式化地說，圖最佳化求解下列問題：

$$\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \sum_{(i,j) \in E} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \Omega_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

圖最佳化的目標函數。 $\mathbf{e}_{ij}$  為第  $(i,j)$  條邊的誤差（預期的相對位姿與實際估計的差）、 $\Omega_{ij}$  為資訊矩陣（協方差的反矩陣）。

這是一個非線性最小平方問題，標準解法是 Gauss-Newton 或 Levenberg-Marquardt 迭代——與第 22.3.2 節的相機校正、第 10.7 節的數值逆運動學是同一類方法。

**與第 18 章的關聯：**這也是一個最佳化問題，但性質不同——第 18 章求的是「未來的最佳控制序列」，這裡求的是「過去的最一致解釋」。前者往前看、後者往回看，但數學工具相通。

## 25.6.5 稀疏性：可解的關鍵

圖最佳化的問題規模可能極大——一趟數公里的行程可能有數萬個節點。為何仍然可解？

關鍵在於**稀疏性**：每個節點只與少數幾個節點有邊相連（相鄰時刻加上少數迴路閉合）。這使得問題的**資訊矩陣（Hessian）是稀疏的**——絕大多數元素為零。

利用稀疏性，可以用專門的線性代數方法（如稀疏 Cholesky 分解）高效求解。一個包含數萬個節點的位姿圖，在現代電腦上通常能在數秒內最佳化完成。

**這正是圖最佳化能取代濾波方法的根本原因：**EKF-SLAM 的協方差矩陣是**稠密的**（所有地標互相相關，第 25.3.2 節），因此是  $O(n^2)$ ；而圖最佳化的資訊矩陣是**稀疏的**，因此能處理大得多

的問題。

### ◎ 稀疏與稠密的深層原因

為何濾波是稠密的、圖最佳化是稀疏的？關鍵在於是否邊緣化 (marginalize) 過去的變數。濾波法為了維持固定的狀態維度，會把過去的位姿「積掉」——而這個積分操作會在剩餘的變數之間產生新的相關性，使矩陣變稠密。圖最佳化則保留所有變數不做邊緣化，因此原始的稀疏結構得以保存。用更多的記憶體，換取更低的計算複雜度與回頭修正的能力——這是一個極為划算的交易，也是現代 SLAM 的主流選擇。

## 25.6.6 前端與後端

現代 SLAM 系統普遍採用**前端—後端**的架構分工：

**前端 (Front-end)**：處理感測資料，建立圖的節點與邊。包含特徵擷取、掃描匹配、資料關聯、迴路閉合偵測。這部分**與感測器類型高度相關**（光達與相機的前端完全不同）。

**後端 (Back-end)**：接收前端建立的圖，求解最佳化問題。這部分**與感測器無關**，是純粹的數學問題。

這個分工的價值在於**模組化**：後端可以是通用的（如 g2o、GTSAM、Ceres 等函式庫），前端則針對不同感測器分別開發。這使得研究者能專注在自己的領域，而不需重複實作最佳化器。

## 25.7 迴路閉合

### 25.7.1 為何如此關鍵

迴路閉合 (loop closure) 是 SLAM 中最戲劇性、也最關鍵的機制。它的作用是：**辨認出「這個地方我曾經來過」**，並據此建立一條連接遙遠時刻的約束邊。

沒有迴路閉合，SLAM 就退化為「里程計 + 局部建圖」——誤差仍會無止境地累積。**迴路閉合是消除長期漂移的唯一手段。**

它的效果極為顯著。一個典型的例子：機器人繞行一棟建築物一圈（約 200 公尺），回到起點時里程計的累積誤差可能達 5 ~ 10 公尺。一旦成功偵測到迴路閉合並最佳化，**整條軌跡的誤差會降到數十公分**——改善了一到兩個數量級。

### 25.7.2 偵測方法

如何判斷「我來過這裡」？主要有三類方法。

**方法一：幾何驗證。**若當前的估計位姿與某個歷史位姿接近，就嘗試把兩處的感測資料做匹配（如光達的掃描匹配）。若匹配成功且誤差小，就認定為迴路閉合。

- **優點：**可靠、能直接得到精確的相對位姿。
- **缺點：**依賴當前的位姿估計——若漂移已經很大，機器人根本不知道自己回到了附近，就不會去嘗試匹配。這是一個雞生蛋的小循環。

**方法二：外觀比對 ( Appearance-based )。**完全不依賴位姿，純粹比較「現在看到的景象」與「歷史上看過的景象」是否相似。

最著名的技術是**詞袋模型 ( Bag of Words, BoW )**：把影像的特徵描述子量化成「視覺詞彙」，用詞彙的分布來描述整張影像。比對時只需比較詞彙分布的相似度，速度極快（毫秒級即可搜尋數萬張歷史影像）。

- **優點：**不依賴位姿估計、能從嚴重漂移中恢復、計算高效。
- **缺點：**可能被**感知混淆**（見下節）欺騙、對光線與視角變化敏感。

**方法三：學習式方法。**用深度神經網路直接學習「場景描述子」，使同一地點在不同光線、季節、視角下也能匹配。這是近年的研究重點（如 NetVLAD），對長期運行的機器人特別有價值。

### 25.7.3 感知混淆與誤判的災難

迴路閉合最大的風險是**誤判**。而誤判的主因是**感知混淆 ( perceptual aliasing )**——不同的地點看起來一模一樣。

典型的例子：辦公大樓中每一層的走廊完全相同、旅館的房門排列一致、倉庫中一模一樣的貨架、對稱的建築物。

#### ◎ 一次錯誤的迴路閉合，可能毀掉整張地圖

這是 SLAM 中最嚴重的失效模式。若系統誤認「三樓的走廊」是「二樓的走廊」，就會加入一條錯誤的強約束邊。最佳化器會忠實地執行它的任務——把兩個實際上相隔一層樓的位置「拉」到一起。結果是整張地圖被折疊、扭曲，且**這個錯誤無法自動修復**（因為那條錯誤的邊仍然存在，且它的「彈簧」很強）。相較之下，漏掉一次迴路閉合只是損失一次改善的機會，代價小得多。因此實務上的原則是：**寧可漏判，不可誤判**。

三個標準的防護機制：

- **幾何驗證**：外觀相似只是候選，必須再用幾何匹配確認（如檢查特徵點的空間分布是否一致）。這能過濾掉大部分的感知混淆。
- **時間一致性**：要求連續數個時刻都偵測到同一個迴路，才予以確認。孤立的單次匹配不採信。
- **強健核函數 (Robust Kernel)**：在最佳化的目標函數中，對誤差很大的邊自動降低其權重（如 Huber 或 Cauchy 核）。這使得少數錯誤的邊不會主導整個結果——這是後端的最後一道防線。

## 25.8 資料關聯

### 25.8.1 被低估的難題

**資料關聯 (data association)** 問的是：這次觀測到的特徵，對應到地圖中的哪一個地標？還是一個全新的地標？

這個問題在教科書中常被輕描淡寫（多數推導直接假設「已知關聯」），但在實務上它才是 **SLAM 最容易失敗的環節**。原因是：狀態估計有嚴謹的數學保證，而資料關聯本質上是一個**辨識**問題——必然有出錯的可能，且錯誤會污染後續的所有估計。

### 25.8.2 三種處理方法

**方法一：最近鄰 (Nearest Neighbor)**。把觀測關聯到預測位置最接近的地標。判斷時使用**馬氏距離**（第 24.9.3 節）而非歐氏距離，因為它考慮了不確定性。

關聯到地標  $j$ ，若  $d^2_{ij} = \mathbf{v}^T \mathbf{S}^{-1} \mathbf{v}$  最小且低於門檻

*最近鄰關聯。簡單快速，但在地標密集時容易出錯。*

**方法二：聯合相容性 (Joint Compatibility)**。不逐一關聯，而是考慮**所有觀測的整體一致性**。例如同時看到三個地標時，檢查這三個關聯組合起來是否幾何上合理。

這顯著提升了可靠度——即使個別的關聯有歧義，整體的幾何約束往往能唯一確定正確的組合。代價是計算量增加。

**方法三：多重假設 (Multi-hypothesis)**。不做決定，而是同時保留多種可能的關聯，讓



後續的觀測來判別。FastSLAM 的每個粒子可以做不同的關聯決定（第 25.4.2 節），正是這個思想的實現。

### 25.8.3 實務原則

三個經驗法則：

- **寧可新增，不可誤配。**若不確定觀測是否對應到已知地標，寧可把它當成新地標。多一個冗餘的地標只是浪費一點記憶體；誤配則會產生錯誤的約束，破壞整張地圖。
- **用描述子而非只用位置。**視覺 SLAM 中，特徵的外觀描述子提供了位置之外的判別資訊，能大幅降低誤配率。
- **設定嚴格的門檻。**馬氏距離的門檻應偏保守（如卡方 99% 而非 95%），寧可多產生一些新地標。

## 25.9 視覺 SLAM 與光達 SLAM

### 25.9.1 兩條技術路線

依主要感測器的不同，SLAM 分為兩大陣營，各有其生態系與適用場合。

**光達 SLAM ( LiDAR SLAM )**：以雷射掃描為主要輸入。其核心操作是**掃描匹配**（scan matching）——把當前的掃描與已有的地圖（或前一次掃描）對齊，求出相對位姿。常用的演算法是 ICP（Iterative Closest Point）與其變體 NDT。

**視覺 SLAM ( Visual SLAM )**：以相機為主要輸入。其核心操作是**特徵匹配與三角測量**——在連續影像間追蹤特徵點，由視差恢復三維結構與相機運動（第 22.4.2 節）。

表 25-4 光達 SLAM 與視覺 SLAM 的比較

| 面向     | 光達 SLAM    | 視覺 SLAM       |
|--------|------------|---------------|
| 感測器成本  | 高（數萬至數十萬元） | 低（數百至數千元）     |
| 直接取得深度 | 是          | 否（需三角測量或深度相機） |
| 對光線的依賴 | 無（主動測距）    | 高（黑暗中失效）      |
| 對紋理的依賴 | 無          | 高（白牆上失效）      |
| 資訊豐富度  | 低（只有幾何）    | 高（顏色、紋理、語意）   |
| 迴路閉合能力 | 較弱（幾何易混淆）  | 強（外觀描述子）      |



| 面向   | 光達 SLAM       | 視覺 SLAM                 |
|------|---------------|-------------------------|
| 尺度   | 絕對尺度          | 單目有尺度歧義；雙目 / RGB-D 無此問題 |
| 典型應用 | 室內 AGV、自駕車、測繪 | AR/VR、無人機、消費機器人         |

### 25.9.2 視覺慣性里程計 ( VIO )

純視覺 SLAM 有三個弱點：快速運動時影像模糊、單目有尺度歧義、紋理不足時失效。而 IMU 恰好能補足這三點——它更新率極高 ( 1000 Hz )、能提供絕對尺度 ( 透過重力加速度 )、且完全不依賴環境外觀。

視覺慣性里程計 ( Visual-Inertial Odometry, VIO ) 融合兩者，是目前 AR 眼鏡、消費級無人機與掃地機器人的主流方案。其互補性極為漂亮：

視覺：低頻、不漂移、需要紋理與光線

IMU：高頻、會漂移、完全不依賴環境

兩者的誤差特性恰好互補——這正是第 24.8 節互補濾波思想的高階版本。

VIO 的實作有兩種路線：濾波式 ( 如 MSCKF，用 EKF 融合，計算量小、適合嵌入式 ) 與最佳化式 ( 如 VINS-Mono、OKVIS，用滑動視窗的圖最佳化，精度較高但計算量大 )。近年後者逐漸成為主流，因為行動裝置的運算能力已足夠。

### 25.9.3 多感測器融合的趨勢

現代的高階系統普遍融合多種感測器，取各家之長：

- **光達 + IMU + 輪速**：自駕車與工業 AGV 的標準組合。光達提供精確幾何、IMU 補足高頻運動、輪速提供尺度與冗餘。
- **相機 + IMU**：消費級裝置的主流 ( 成本考量 )，如 AR 眼鏡、掃地機器人。
- **光達 + 相機 + IMU + GNSS**：戶外自駕車的完整配置。GNSS 提供全域參照以消除長期漂移。

融合的原則與第 24.10 節相同：用高頻但漂移的感測器提供平滑的短期估計、用不漂移的感測器定期校正。SLAM 只是把這個原則應用到「地圖」這個特殊的參照上。

## 25.10 實務挑戰與現代發展

### 25.10.1 動態環境

經典的 SLAM 假設環境是靜態的。但真實世界充滿移動的人、車、被搬動的傢俱。

動態物體會造成兩個問題：一、**被誤認為地標**，一旦它移動，地圖就與現實不符；二、**干擾掃描匹配**，使位姿估計偏移。

三種因應策略：

- **偵測並剔除**：用語意分割（第 22.8 節）辨識出人、車等動態類別，直接排除其上的特徵。這是目前最有效的作法。
- **統計剔除**：多次觀測後，若某區域的佔據狀態反覆變化，就標記為不可靠。佔據柵格的 I 值上下限（第 25.5.3 節）正是這個機制的簡單版本。
- **明確建模**：把動態物體也納入估計（追蹤它們的運動）。這是 SLAM 與多目標追蹤的結合，複雜度高但資訊最完整。

### 25.10.2 長期運行與地圖維護

若機器人需要在同一環境中運行數月甚至數年，會遇到「**長期 SLAM**」的挑戰：

**挑戰一：外觀變化**。同一地點在白天與夜晚、夏天與冬天、燈開與燈關時看起來截然不同。這使外觀式的迴路閉合失效。

**挑戰二：結構變化**。傢俱被搬動、貨架被重新排列、施工改變了通道。地圖必須能更新，但又不能因為暫時的變化就把長期正確的部分抹掉。

**挑戰三：地圖膨脹**。持續累積資料會使地圖無限增長。需要機制來**遺忘**——刪除過時的資訊、合併重複的節點。

這些挑戰目前仍是研究前沿。實務上的折衷作法包括：定期重新建圖、維護多張不同時段的地圖、或採用「經驗地圖」（Experience Maps）的架構——為同一地點保留多個不同外觀的版本。

### 25.10.3 學習與 SLAM 的結合

深度學習正在改變 SLAM 的多個環節，但方式與想像的不同——它**主要強化了前端**，而非取代後端。

表 25-5 深度學習在 SLAM 中的角色

| 環節     | 傳統方法           | 學習方法的貢獻                    | 是否已成熟 |
|--------|----------------|----------------------------|-------|
| 特徵擷取   | SIFT、ORB 等手工設計 | SuperPoint 等學習式特徵，對光線變化更穩健 | 接近成熟  |
| 特徵匹配   | 描述子最近鄰         | SuperGlue 等，考慮全域一致性        | 接近成熟  |
| 迴路閉合   | 詞袋模型           | NetVLAD 等場景描述子，跨季節仍可匹配     | 接近成熟  |
| 動態物體剔除 | 統計方法           | 語意分割直接辨識人車                 | 已成熟   |
| 深度估計   | 立體匹配           | 單目深度預測網路                   | 部分成熟  |
| 位姿估計   | 幾何最佳化          | 端到端網路直接回歸位姿                | 尚不成熟  |
| 地圖表示   | 點雲、柵格          | 神經隱式表示 ( NeRF )            | 研究前沿  |

一個重要的觀察：學習方法在「辨識」類的任務上（特徵、匹配、迴路閉合、語意）表現優異，但在「幾何最佳化」上仍不如傳統方法。原因是幾何有嚴謹的數學結構，而神經網路難以保證滿足這些約束。

因此目前的共識是：學習做前端、幾何做後端。這與第 21.9.5 節「學習產生候選、解析驗證品質」的混合式架構、第 17 章「模型處理已知、回授處理未知」的哲學，是同一種思維的不同體現。

## 25.11 由建圖邁向規劃

### 25.11.1 本章的成果

本章解開了第 24 章結尾拋出的雞生蛋難題。你現在能夠：陳述 SLAM 問題並理解聯合估計為何可行、比較三種地圖表示法、說明 EKF-SLAM 的結構與其  $O(n^2)$  困境、理解 Rao-Blackwell 分解與 FastSLAM、實作佔據柵格地圖的對數勝算比更新、理解圖最佳化的彈簧類比與稀疏性優勢、以及處理迴路閉合與資料關聯這兩個實務難點。

最重要的是三個核心洞見：一、一次觀測同時約束位姿與地標——這是 SLAM 可解的根本原因。二、迴路閉合是消除長期漂移的唯一手段，但寧可漏判不可誤判。三、不做邊緣化就能保持稀疏——這是圖最佳化取代濾波法的關鍵。

### 25.11.2 接下來：知道在哪裡，然後呢

經過第 22 ~ 25 章，機器人終於具備了完整的空間認知能力：它能看見環境（第 22 章）、

能移動（第 23 章）、能估計自己的狀態（第 24 章）、並能在未知環境中建立地圖並定位（第 25 章）。

但知道自己在哪裡，並不等於知道該怎麼走。給定一張地圖與一個目標，機器人必須規劃出一條路徑——這條路徑必須避開障礙、符合運動約束（第 23 章的非完整性）、且盡可能地短或快。

第 26 章（路徑規劃與導航）將處理這個問題。我們會學到：位形空間這個把「機器人的形狀」轉化為「一個點」的關鍵抽象、 $A^*$  等圖搜尋演算法、RRT 等取樣式方法（處理高維與非完整約束）、人工勢場與動態視窗法等即時避障方法、以及完整導航系統的分層架構——全域規劃負責大方向、局部規劃負責即時避障。

屆時你會發現，第 25 章的佔據柵格地圖正是路徑規劃最直接的輸入，而第 23 章的運動約束則決定了哪些路徑真正可行。

換句話說：第 25 章讓機器人知道世界長什麼樣、自己在哪裡，第 26 章讓它決定該往哪裡走。

## 完整計算例題

### 例題 25-1 EKF-SLAM 的狀態維度與複雜度

題目：某 EKF-SLAM 系統在二維環境中運行，機器人位姿為  $(x, y, \theta)$ ，每個地標以  $(x, y)$  表示。（a）當地圖中有 50 個地標時，求狀態向量的維度與協方差矩陣的元素數。（b）當地標增至 500 個時，重新計算。（c）若協方差更新的運算量正比於元素數，計算（b）相對於（a）的運算量倍數。（d）若（a）的更新耗時 2 毫秒，估算（b）的耗時並判斷能否以 20 Hz 即時運行。

解（a）：狀態維度為  $3 + 2N$ ：

$$\dim = 3 + 2 \times 50 = 103$$

狀態向量有 103 個元素。

$$\text{協方差元素數} = 103^2 = 10609$$

約一萬個元素。

解（b）：

$$\text{dim} = 3 + 2 \times 500 = 1003 \quad \text{協方差元素數} = 1003^2 = 1006009$$

約一百萬個元素。

解 (c) :

$$\text{倍數} = 1006009/10609 \approx 94.8 \text{ 倍}$$

地標數增加 10 倍，運算量增加約 95 倍——這正是  $O(N^2)$  的體現。

解 (d) :

$$\text{耗時} \approx 2 \text{ ms} \times 94.8 = 189.6 \text{ ms}$$

近 190 毫秒。

20 Hz 即時運行要求每次更新在 50 毫秒內完成。 $189.6 \text{ ms} > 50 \text{ ms}$ ，無法即時運行。

**工程結論：**EKF-SLAM 在此配置下，地標數的上限約為 250 個左右（此時耗時約 50 ms）。這印證了 25.3.3 節的說法——它只適用於小規模環境。若環境需要更多地標，必須改用 FastSLAM ( $O(M \log N)$ ) 或圖最佳化（利用稀疏性）。

## 例題 25-2 佔據柵格的對數勝算比更新

題目：某佔據柵格地圖使用對數勝算比更新。感測器逆模型設定為：偵測到障礙物時  $l_{\text{occ}} = +0.85$ 、偵測為空曠時  $l_{\text{free}} = -0.40$ ，先驗  $l_0 = 0$ （機率 0.5）。（a）某格連續三次被偵測為障礙物，求其  $l$  值與對應的機率。（b）接著兩次被偵測為空曠，重新計算。（c）若  $l$  的上下限設為  $\pm 5.0$ ，求需要連續幾次偵測為障礙物才會達到上限。（d）討論設定上下限的必要性。

解 (a)：三次累加：

$$l = 0 + 3 \times 0.85 = 2.55$$

對數勝算比。

換算回機率（由  $l = \log[p/(1-p)]$  反解）：

$$p = 1/(1 + e^{(-l)}) = 1/(1 + e^{(-2.55)}) = 1/(1 + 0.0781) = 0.9276$$

佔據機率約 92.8%——已相當確定此處有障礙物。

解 (b)：再累加兩次空曠的證據：

$$l = 2.55 + 2 \times (-0.40) = 2.55 - 0.80 = 1.75$$

對數勝算比下降。

$$p = 1/(1 + e^{(-1.75)}) = 1/(1 + 0.1738) = 0.8519$$

機率降至約 85.2%。

**觀察：**兩次「空曠」的證據，只把機率由 92.8% 降到 85.2%——這是因為  $I_{occ}$  ( 0.85 ) 大於  $|I_{free}|$  ( 0.40 )。這個不對稱的設定反映了「打到東西比沒打到更可靠」的物理事實 ( 第 25.5.3 節 )。

解 ( c )：由  $I_0 = 0$  開始，每次 +0.85：

$$n \times 0.85 \geq 5.0 \Rightarrow n \geq 5.88 \Rightarrow n = 6 \text{ 次}$$

需要連續 6 次偵測為障礙物才會達到上限。

此時的機率為  $p = 1/(1 + e^{(-5.0)}) = 0.9933$ ，約 99.3%。

解 ( d )：上下限是必要的，理由有三：

- **允許環境改變：**若沒有上限，某格被觀測一千次為障礙物後， $I$  會達到 850——此時即使障礙物被移走，需要超過兩千次的空曠觀測才能翻轉。這使地圖無法適應動態環境。
- **避免數值溢位：**長時間運行後  $I$  會無限增長。
- **限制單一錯誤的影響：**若感測器暫時故障產生大量錯誤讀值，上下限能限制其破壞程度。

以本例的上限 5.0 而言，一個「確定的障礙物」只需約 13 次空曠觀測 ( $5.0/0.40 = 12.5$ ) 就能被翻轉為空曠——這是一個合理的反應速度。

### 例題 25-3 迴路閉合的效果估算

題目：某機器人沿一條封閉路徑行走 300 公尺後回到起點。里程計的航向誤差率為每公尺 0.05 度 ( 已校正後的水準 )。( a ) 估算回到起點時的累積航向誤差。( b ) 估算終點的位置誤差 ( 假設誤差均勻累積 )。( c ) 若成功偵測到迴路閉合並執行圖最佳化，估算誤差如何分配。( d ) 若迴路閉合誤判 ( 把起點誤認為路徑中點 )，討論後果。

解 ( a )：累積航向誤差：

$$\theta_{\text{error}} = 0.05^\circ/\text{m} \times 300 \text{ m} = 15^\circ$$

回到起點時，機器人的朝向估計偏差達 15 度。

解 ( b )：位置誤差的估算較複雜，但可用一個簡化模型：若航向誤差線性累積，則位置誤

差約為「路徑長度 × 平均航向誤差的正弦」：

$$\text{平均航向誤差} = 15^\circ / 2 = 7.5^\circ$$

$$\text{位置誤差} \approx 300 \times \sin(7.5^\circ) \approx 300 \times 0.1305 = 39.2 \text{ m}$$

理論上限的粗估。實務上因路徑封閉（誤差部分抵消），實際誤差通常較小，典型為路徑長度的 2~5%。

取 3% 估算：

$$\text{實際位置誤差} \approx 300 \times 0.03 = 9 \text{ m}$$

回到起點時，估計位置與真實起點相差約 9 公尺。

解（c）：迴路閉合提供了一個強約束：「終點 = 起點」。圖最佳化會把這 9 公尺的誤差均勻攤到整條軌跡上。

$$\text{最佳化後的平均誤差} \approx 9 \text{ m} / \sqrt{(\text{節點數})} \text{ 或更小}$$

誤差不再集中在終點，而是分散到全程。

實務上的典型結果是：最大誤差由 9 公尺降至 0.2~0.5 公尺，改善約 20~45 倍。這正是 25.7.1 節所說「改善一到兩個數量級」的具體數字。

解（d）：若誤判——把「起點」誤認為「路徑中點」（假設在 150 公尺處），後果是：

- 最佳化器會加入一條錯誤的強約束，強迫「300 公尺處」與「150 公尺處」重合。
- 整條軌跡會被折疊——後半段被拉到前半段上方。
- 地圖上會出現重疊的牆面與虛假的通道，使路徑規劃完全失效。
- 最嚴重的是：這個錯誤無法自動修復。那條錯誤的邊仍在圖中，且其資訊矩陣可能很大（因為外觀匹配的信心很高）。

因應之道（第 25.7.3 節）：一、幾何驗證——檢查兩處的光達掃描是否真的能對齊；二、時間一致性——要求連續數次偵測；三、強健核函數——在最佳化中自動降低異常邊的權重。三者應同時採用，因為單一機制都可能被特殊情況突破。

## 程式範例：佔據柵格建圖與位姿圖最佳化

以下 Python 程式實作佔據柵格地圖的對數勝算比更新與射線追蹤，以及一個簡化的位姿圖最佳化，展示迴路閉合如何「扭正」整條軌跡。

```
import numpy as np
```



```
# ===== 佔據柵格地圖 =====
class OccupancyGrid:
    """對數勝算比表示的佔據柵格地圖 ( 25.5 節 ) """
    def __init__(s, w=200, h=200, res=0.05, l_occ=0.85, l_free=-0.40,
                 l_min=-5.0, l_max=5.0):
        s.w, s.h, s.res = w, h, res
        s.l = np.zeros((h, w))          # 對數勝算比 · 初值 0 ( 未知 )
        s.l_occ, s.l_free = l_occ, l_free
        s.l_min, s.l_max = l_min, l_max

    def world_to_grid(s, x, y):
        return int(x/s.res + s.w/2), int(y/s.res + s.h/2)

    def prob(s):
        """轉換為機率地圖"""
        return 1.0/(1.0 + np.exp(-s.l))

    def bresenham(s, x0, y0, x1, y1):
        """射線追蹤：回傳由 (x0,y0) 到 (x1,y1) 經過的格子"""
        cells = []
        dx, dy = abs(x1-x0), abs(y1-y0)
        sx = 1 if x0 < x1 else -1
        sy = 1 if y0 < y1 else -1
        err = dx - dy
        x, y = x0, y0
        while True:
            cells.append((x, y))
            if x == x1 and y == y1:
                break
            e2 = 2*err
            if e2 > -dy: err -= dy; x += sx
            if e2 < dx: err += dx; y += sy
        return cells

    def update_ray(s, rx, ry, angle, dist, max_range=5.0):
        """以一條光達射線更新地圖 ( 25.5.3 節 ) """
        gx0, gy0 = s.world_to_grid(rx, ry)
        hit = dist < max_range
        d = min(dist, max_range)
        ex, ey = rx + d*np.cos(angle), ry + d*np.sin(angle)
        gx1, gy1 = s.world_to_grid(ex, ey)
        cells = s.bresenham(gx0, gy0, gx1, gy1)
        for i, (cx, cy) in enumerate(cells):
            if not (0 <= cx < s.w and 0 <= cy < s.h):
                continue
            is_end = (i == len(cells)-1)
            delta = s.l_occ if (is_end and hit) else s.l_free
            s.l[cy, cx] = np.clip(s.l[cy, cx] + delta, s.l_min, s.l_max)

    def stats(s):
        p = s.prob()
        return (np.sum(p > 0.7), np.sum(p < 0.3),
                np.sum((p >= 0.3) & (p <= 0.7)))
```

```

# ===== 例題 25-2 驗證 =====

print("=== 例題 25-2：對數勝算比更新 ===")
def l2p(l): return 1.0/(1.0 + np.exp(-l))
l_occ, l_free = 0.85, -0.40
l = 0.0
print(f"{'事件':<20}{'l 值':>10}{'機率':>12}")
print(f"{'初始 (未知)':<20}{l:10.3f}{l2p(l):11.2%}")
for i in range(3):
    l += l_occ
    print(f"{'第 %d 次偵測為障礙' % (i+1):<20}{l:10.3f}{l2p(l):11.2%}")
for i in range(2):
    l += l_free
    print(f"{'第 %d 次偵測為空曠' % (i+1):<20}{l:10.3f}{l2p(l):11.2%}")
n_needed = int(np.ceil(5.0/l_occ))
print(f"\n 達到上限 l=5.0 需連續 {n_needed} 次偵測為障礙 "
      f"(機率 {l2p(5.0):.2%})")
print(f"由上限翻轉為空曠需 {int(np.ceil(10.0/abs(l_free)))} 次空曠觀測")

# ===== 建圖模擬 =====

print("\n=== 佔據柵格建圖模擬 (矩形房間) ===")
grid = OccupancyGrid(w=200, h=200, res=0.05)
# 定義一個 4x3 公尺的房間 (牆面)
def raycast_room(rx, ry, angle, w=2.0, h=1.5):
    """計算射線到矩形房間牆面的距離"""
    dx, dy = np.cos(angle), np.sin(angle)
    ts = []
    for wall, d, p in ((w, dx, rx), (-w, dx, rx), (h, dy, ry), (-h, dy, ry)):
        if abs(d) > 1e-9:
            t = (wall - p)/d
            if t > 0:
                px, py = rx + t*dx, ry + t*dy
                if -w-0.01 <= px <= w+0.01 and -h-0.01 <= py <= h+0.01:
                    ts.append(t)
    return min(ts) if ts else 5.0

np.random.seed(0)
positions = [(0.0, 0.0), (1.0, 0.5), (-1.0, -0.5), (1.2, -0.8), (-1.2, 0.9)]
for step, (rx, ry) in enumerate(positions):
    for angle in np.linspace(-np.pi, np.pi, 180, endpoint=False):
        d = raycast_room(rx, ry, angle) + np.random.randn()*0.01
        grid.update_ray(rx, ry, angle, d)
    occ, free, unk = grid.stats()
    print(f" 掃描 {step+1} 次後：佔據 {occ:5d} 格, "
          f"空曠 {free:5d} 格, 未知 {unk:5d} 格")
print("> 隨著觀測累積，未知區域逐步減少")

# ===== 位姿圖最佳化 (迴路閉合) =====

print("\n=== 位姿圖最佳化：迴路閉合的效果 ===")
class PoseGraph2D:
    """簡化的二維位姿圖 (只估計 x, y，省略角度以簡化)"""
    def __init__(s, n):
        s.n = n

```

```

s.nodes = np.zeros((n, 2))
s.edges = [] # (i, j, dx, dy, weight)

def add_edge(s, i, j, dx, dy, w=1.0):
    s.edges.append((i, j, dx, dy, w))

def optimize(s, iters=100):
    """Gauss-Newton 迭代 (25.6.4 節)"""
    for _ in range(iters):
        H = np.zeros((2*s.n, 2*s.n))
        b = np.zeros(2*s.n)
        # 固定第一個節點 (否則系統有平移自由度)
        H[0:2, 0:2] += np.eye(2)*1e6
        for (i, j, dx, dy, w) in s.edges:
            e = (s.nodes[j] - s.nodes[i]) - np.array([dx, dy])
            A = -np.eye(2)*w
            B = np.eye(2)*w
            H[2*i:2*i+2, 2*i:2*i+2] += A.T @ A
            H[2*i:2*i+2, 2*j:2*j+2] += A.T @ B
            H[2*j:2*j+2, 2*i:2*i+2] += B.T @ A
            H[2*j:2*j+2, 2*j:2*j+2] += B.T @ B
            b[2*i:2*i+2] += A.T @ (w*e)
            b[2*j:2*j+2] += B.T @ (w*e)
        dx_all = np.linalg.solve(H, -b)
        s.nodes += dx_all.reshape(-1, 2)
        if np.max(np.abs(dx_all)) < 1e-9:
            break
    return s.nodes

# 建立一個正方形迴路：20 個節點，每邊 5 步
N = 20
true_poses = []
for i in range(N):
    side, k = i//5, i % 5
    if side == 0: true_poses.append([k*1.0, 0.0])
    elif side == 1: true_poses.append([5.0, k*1.0])
    elif side == 2: true_poses.append([5.0-k*1.0, 5.0])
    else: true_poses.append([0.0, 5.0-k*1.0])
true_poses = np.array(true_poses, float)

# 里程計有系統性偏差 (每步偏 3%)
pg = PoseGraph2D(N)
odom = np.zeros((N, 2))
bias = 1.03
for i in range(N-1):
    d_true = true_poses[i+1] - true_poses[i]
    d_odom = d_true*bias + np.random.randn(2)*0.02
    odom[i+1] = odom[i] + d_odom
    pg.add_edge(i, i+1, d_odom[0], d_odom[1], w=1.0)
pg.nodes = odom.copy()

err_before = np.linalg.norm(odom[-1] - (true_poses[-1] - true_poses[0]))
print(f"最佳化前 (僅里程計):")
print(f"終點估計 = ({odom[-1,0]:6.3f}, {odom[-1,1]:6.3f})")
print(f"真實終點 = ({true_poses[-1,0]:6.3f}, {true_poses[-1,1]:6.3f})")

```

```

print(f" 終點誤差 = {err_before:.3f} m")

# 加入迴路閉合邊：節點 19 應該回到節點 0 附近
d_loop = true_poses[0] - true_poses[N-1]
pg.add_edge(N-1, 0, d_loop[0], d_loop[1], w=5.0) # 高權重
pg.optimize()

errs = np.linalg.norm(pg.nodes - (true_poses - true_poses[0]), axis=1)
errs_before = np.linalg.norm(odom - (true_poses - true_poses[0]), axis=1)
print(f"\n 最佳化後 ( 加入迴路閉合 ) :")
print(f" 終點估計 = ({pg.nodes[-1,0]:6.3f}, {pg.nodes[-1,1]:6.3f})")
print(f" 終點誤差 = {errs[-1]:.3f} m")
print(f" 全程平均誤差: {errs_before.mean():.3f} m → {errs.mean():.3f} m")
print(f" 全程最大誤差: {errs_before.max():.3f} m → {errs.max():.3f} m")
print(f" 改善倍數: {errs_before.max()/max(errs.max(),1e-9):.1f}x")
print(f"→ 誤差被均勻攤到整條軌跡上，這就是「地圖當場扭正」")

# ===== 誤判迴路閉合的災難 =====
print("\n=== 誤判迴路閉合的後果 ( 例題 25-3d ) ===")
pg_bad = PoseGraph2D(N)
pg_bad.nodes = odom.copy()
for i in range(N-1):
    d = odom[i+1] - odom[i]
    pg_bad.add_edge(i, i+1, d[0], d[1], w=1.0)
# 錯誤：把節點 19 誤認為節點 10 ( 路徑中點 )
pg_bad.add_edge(N-1, 10, 0.0, 0.0, w=5.0)
pg_bad.optimize()
errs_bad = np.linalg.norm(pg_bad.nodes - (true_poses - true_poses[0]), axis=1)
print(f" 正確迴路閉合：平均誤差 {errs.mean():.3f} m, "
      f"最大 {errs.max():.3f} m")
print(f" 錯誤迴路閉合：平均誤差 {errs_bad.mean():.3f} m, "
      f"最大 {errs_bad.max():.3f} m")
print(f"→ 誤判使誤差反而增加 {errs_bad.mean()/errs.mean():.1f} 倍——"
      f"軌跡被錯誤地折疊")

```

請試著修改：（一）把迴路閉合邊的權重由 5.0 改為 0.5，觀察最佳化的效果如何減弱。（二）在建圖模擬中加入一個「動態障礙物」（某次掃描時出現、之後消失），觀察 I 值的上下限如何幫助地圖恢復。（三）把里程計的偏差由 3% 增至 10%，觀察迴路閉合的改善倍數如何變化。

## 實作：SLAM 系統的建立與評估

本章實作以 ROS 2 環境為主（可用模擬器或實體機器人）。若無 ROS 環境，前四項仍可用

純 Python 完成。

1. **佔據柵格建圖**：實作對數勝算比的地圖更新與 Bresenham 射線追蹤。用模擬的光達資料（或第 6 章的實體光達）建立一個房間的地圖。
2. **參數敏感度測試**：改變  $l_{occ}$ 、 $l_{free}$  與上下限，觀察地圖的清晰度與對動態變化的反應速度。找出適合你的感測器的參數組合。
3. **掃描匹配實作**：實作簡化的 ICP 演算法——給定兩次光達掃描，求出它們之間的相對位姿。這是光達 SLAM 前端的核心。
4. **位姿圖最佳化**：實作二維的位姿圖最佳化（可參考本章程式），驗證迴路閉合的效果。
5. **ROS 2 SLAM 實測**：使用 `slam_toolbox`（ROS 2 中最常用的二維 SLAM 套件），讓機器人在實際環境中建圖。錄製 `rosvbag` 以便重複實驗。
6. **迴路閉合的觀察**：規劃一條封閉路徑，讓機器人繞行一圈。錄影記錄地圖在迴路閉合瞬間的變化——這是本章最值得親眼看見的現象。
7. **里程計品質的影響**：故意讓里程計的參數失準（如把輪距設錯 5%），重複建圖，比較地圖的品質。這連結了第 23 章的校正工作。
8. **動態環境測試**：在建圖過程中讓人走過機器人前方，觀察地圖上是否留下「鬼影」。測試不同的  $l$  值上下限對此的影響。
9. **感知混淆測試**：在一個有重複結構的環境（如相似的走廊、對稱的房間）中建圖，觀察是否發生錯誤的迴路閉合。若有，分析其成因。
10. **（進階）地圖品質評估**：以人工測量的真實尺寸為基準，量測建出的地圖中數個關鍵距離（如走廊長度、房間寬度），計算誤差百分比。這是評估 SLAM 品質的標準作法。

表 25-6 SLAM 實驗記錄表（前兩列為範例）

| 測試項目       | 設定                           | 地圖品質 | 定位誤差       | 備註         |
|------------|------------------------------|------|------------|------------|
| 基準建圖       | <code>slam_toolbox</code> 預設 | 牆面清晰 | 終點誤差 12 cm | 無迴路閉合      |
| 加入迴路閉合     | 繞行一圈                         | 牆面對齊 | 終點誤差 3 cm  | 閉合瞬間地圖明顯扭正 |
| （自行填寫其餘測試） |                              |      |            |            |

## 國中小也看得懂：蒙著眼睛畫地圖

想像你被蒙住眼睛，帶到一個從來沒去過的房間，然後有人跟你說：「請你畫出這個房間的地圖。」

你會怎麼做？大概是伸手摸索——摸到一面牆，往前走幾步，再摸到另一面牆……慢慢在腦中拼湊出房間的樣子。

但這裡有一個大問題：**你怎麼知道自己走了多遠？** 你可能以為自己走了 5 步，其實只走了 4 步半。走得越多，錯得越多。等你摸完一整圈回到門口，你畫出來的地圖可能是**歪的**——最後一面牆跟第一面牆對不起來！

這就是機器人遇到的難題，它有一個名字叫 **SLAM**（唸作「斯朗姆」）。它同時要做兩件互相依賴的事：

**要畫地圖 → 必須知道自己在哪裡**（不然不知道要把牆畫在哪）

**要知道自己在哪裡 → 必須有地圖**（不然沒有東西可以對照）

聽起來像「先有雞還是先有蛋」對不對？科學家想了很久才找到答案：**不要分開做，一起做！**

而最神奇的一刻叫做「**繞回來了**」。當機器人繞了一大圈，突然認出「咦，這個角落我來過！」——這時候它就知道：「原來我走歪了這麼多。」然後**整張地圖會在一瞬間被扭正**，歪掉的牆全部對齊！

這個畫面真的很酷，如果你有機會操作掃地機器人的建圖功能，可以留意看看——當它繞完一圈回到起點時，螢幕上的地圖會突然「跳」一下變整齊。那就是這一章講的東西！

## 叫獸提醒與常見錯誤

- 誤區一：先定位再建圖，或先建圖再定位。兩者互為前提且誤差會互相污染，必須聯合估計。
- 誤區二：忽略地標之間的相關性。若把  $\Sigma_{rm}$  當成零，系統會過度自信地修正，導致估計不一致甚至發散。
- 誤區三：在大規模環境使用 EKF-SLAM。其  $O(N^2)$  複雜度使地標數超過數百個就無法即時運行。
- 誤區四：佔據柵格不設 I 值的上下限。這會使地圖無法適應環境變化——障礙物被移走後仍

長期顯示為佔據。

- 誤區五：以為濾波法能修正過去的軌跡。線上 SLAM 只維護當前位姿，無法回頭調整——迴路閉合需要完整 SLAM。
- 誤區六：迴路閉合只做外觀比對而不做幾何驗證。感知混淆會導致誤判，而一次誤判可能毀掉整張地圖。
- 誤區七：把「漏判」與「誤判」等同看待。漏判只是損失一次改善機會，誤判則會產生無法自動修復的錯誤約束。
- 誤區八：資料關聯用歐氏距離而非馬氏距離。後者考慮了不確定性，在地標密集時判別力顯著較佳。
- 誤區九：不確定時強行關聯到已知地標。應遵循「寧可新增，不可誤配」的原則。
- 誤區十：忽略動態物體。移動的人或車會被誤認為地標，並在地圖上留下「鬼影」，干擾後續的定位與規劃。
- 誤區十一：以為深度學習能端到端取代 SLAM。學習在辨識類任務（特徵、迴路閉合、語意）上優異，但幾何最佳化仍應交給傳統方法。
- 誤區十二：用純視覺 SLAM 處理快速運動或低紋理環境。應融合 IMU（VIO）以補足視覺的三個弱點。

## 課堂討論

1. 25.1.3 節說「一次觀測同時約束了位姿與地標」。請用拼圖的類比進一步說明：為何累積夠多的約束後，整個系統只有一種一致的解？
2. 圖最佳化保留所有歷史資料而不做邊緣化，因此保持稀疏。請討論這個「用記憶體換計算量」的交易，在什麼硬體條件下會不划算？
3. 25.7.3 節主張「寧可漏判，不可誤判」。請討論這個原則的普遍性——本書中還有哪些場合採用了類似的不對稱風險考量？（提示：第 7 章的失效導向安全、第 20 章的力控制）
4. 深度學習改變了 SLAM 的前端卻未取代後端。請討論這個分工背後的原因，並與第 21.9.5 節的混合式抓取、第 17 章的模型加回授做比較。
5. 長期 SLAM 面臨「地圖必須更新，但不能因暫時的變化就抹掉長期正確的部分」的兩難。請設計一個機制來平衡這兩者，並討論其可能的失效情境。



## 章末摘要

- SLAM 的本質是雞生蛋問題：建圖需要定位、定位需要地圖；解法是把兩者放進同一個估計問題中聯合求解。
- 聯合估計可行的根本原因：一次觀測同時約束了機器人位姿與地標位置，累積足夠的約束後解唯一。
- 完整 SLAM 估計整條軌跡（能修正過去），線上 SLAM 只估計當前位姿（計算量小但無法回頭修正）。
- 三種地圖表示：特徵地圖（精簡、配 EKF）、佔據柵格（能表示空曠、可直接用於規劃）、拓撲地圖（極精簡、用於高階導航）。
- EKF-SLAM 把所有地標放進狀態向量，維度為  $3+2N$ ；非對角區塊  $\Sigma_{rm}$  記錄的相關性是收斂的關鍵。
- EKF-SLAM 的協方差是稠密的，更新複雜度為  $O(N^2)$ ，因此只適用於數百個地標以內的環境。
- SLAM 收斂定理：地圖不確定性單調遞減、地標間趨於完全相關、下界由首次觀測時的位姿不確定性決定。
- Rao-Blackwell 分解：已知完整軌跡時各地標互相獨立；FastSLAM 據此用粒子處理軌跡、用小型 EKF 處理各地標。
- FastSLAM 的複雜度為  $O(M \log N)$ ，且能處理資料關聯的不確定性（錯誤的粒子會被淘汰）。
- 佔據柵格使用對數勝算比  $l = \log[p/(1-p)]$ ，使貝氏更新由乘法變成加法；必須設定上下限以適應環境變化。
- 圖最佳化把 SLAM 表示為節點（位姿）與邊（約束）的圖，最佳化即「讓彈簧系統找到平衡」。
- 圖最佳化因不做邊緣化而保持稀疏，這是它能處理大規模問題並取代濾波法的關鍵。
- 前端負責建立圖（與感測器相關），後端負責求解最佳化（與感測器無關），這個分工使系統模組化。
- 迴路閉合是消除長期漂移的唯一手段，能將誤差改善一到兩個數量級。
- 迴路閉合偵測有三類：幾何驗證（依賴位姿估計）、外觀比對（詞袋模型，不依賴位姿）、

學習式描述子。

- 一次錯誤的迴路閉合可能毀掉整張地圖且無法自動修復；防護機制為幾何驗證、時間一致性與強健核函數。
- 資料關聯的實務原則：寧可新增不可誤配、用描述子輔助判別、門檻設得保守。
- 光達 SLAM 精確但昂貴且資訊單一；視覺 SLAM 便宜且資訊豐富但依賴光線與紋理；VIO 融合視覺與 IMU 補足彼此弱點。

## 觀念題與計算題

### 觀念題

1. 陳述 SLAM 問題並說明其雞生蛋的本質，以及為何無法用「先做一個再做另一個」解決。
2. 說明「一次觀測同時約束位姿與地標」的意義，並解釋這為何是 SLAM 可解的根本原因。
3. 區分完整 SLAM 與線上 SLAM，並說明為何迴路閉合本質上需要前者。
4. 比較三種地圖表示法的優缺點與適用場合。
5. 寫出 EKF-SLAM 的狀態向量結構，並說明協方差矩陣中非對角區塊的意義。
6. 說明 EKF-SLAM 為何是  $O(N^2)$  的複雜度，以及這帶來什麼實務限制。
7. 敘述 SLAM 收斂定理的三個性質。
8. 說明 Rao-Blackwell 分解的核心洞見，以及 FastSLAM 如何據此設計。
9. 說明對數勝算比的定義與三個優點，並解釋為何必須設定上下限。
10. 說明佔據柵格的感測器逆模型，以及為何佔據的證據通常大於空曠的證據。
11. 用彈簧系統的類比說明圖最佳化，並解釋迴路閉合為何能「扭正」整條軌跡。
12. 解釋為何濾波法的矩陣是稠密的而圖最佳化是稀疏的，並說明這個差異的後果。
13. 說明前端與後端的分工，以及這個架構的價值。
14. 比較三類迴路閉合偵測方法的原理與優缺點。
15. 說明感知混淆的意義，以及為何「寧可漏判，不可誤判」。
16. 列舉三種資料關聯的方法，並說明實務上的三個原則。
17. 比較光達 SLAM 與視覺 SLAM，並說明 VIO 為何能補足純視覺的弱點。

## 計算題

1. 某 EKF-SLAM 在三維環境中運行，機器人位姿為 6 維、每個地標為 3 維。(a) 當有 80 個地標時，求狀態維度與協方差元素數。(b) 地標增至 400 個時重新計算。(c) 求運算量的倍數。(d) 若 (a) 耗時 5 ms，判斷 (b) 能否以 10 Hz 運行。
2. 某佔據柵格使用  $l_{occ} = +0.7$ 、 $l_{free} = -0.35$ 、上下限  $\pm 4.0$ 。(a) 某格連續四次偵測為障礙物，求  $l$  值與機率。(b) 接著三次偵測為空曠，重新計算。(c) 求達到上限所需的連續偵測次數。(d) 求由上限翻轉為「未知」( $l = 0$ ) 所需的空曠觀測次數。
3. 某機器人沿封閉路徑行走 500 公尺，里程計航向誤差率為每公尺 0.08 度。(a) 求累積航向誤差。(b) 若實際位置誤差為路徑長度的 4%，求終點誤差。(c) 若迴路閉合後誤差降至 0.3 公尺，求改善倍數。(d) 若路徑長度加倍為 1000 公尺，估算迴路閉合前後的誤差。
4. 某位姿圖有 500 個節點，每個節點為二維位姿 (3 個變數)。里程計邊 499 條，迴路閉合邊 20 條。(a) 求變數總數與資訊矩陣的維度。(b) 若矩陣為稠密，求元素總數。(c) 實際上每個節點平均只與 2.08 個節點相連，估算非零元素的比例。(d) 討論稀疏性帶來的計算量差異。

## 動手做與專題延伸

### 本章實作：SLAM 模組

延續第 23、24 章的程式庫，建立 SLAM 的核心模組。

- **階段一**：實作佔據柵格地圖 (對數勝算比、Bresenham 射線追蹤、上下限)，並用模擬光達建圖。
- **階段二**：實作簡化的 EKF-SLAM (特徵地圖、二維環境)，驗證地標不確定性隨觀測遞減、且地標間逐漸相關。
- **階段三**：實作二維位姿圖最佳化 (Gauss-Newton)，並驗證迴路閉合的效果。
- **階段四**：實作簡化的掃描匹配 (ICP)，作為位姿圖邊的來源。
- **階段五**：整合為完整的 SLAM 系統——前端 (掃描匹配 + 迴路偵測) 加後端 (位姿圖最佳化)。

### 專題延伸

- **迴路閉合的視覺化動畫**：模擬機器人繞行一個大迴路，記錄每一步的軌跡估計與地圖。製作

動畫顯示「里程計逐漸偏離 → 偵測到迴路閉合 → 整條軌跡瞬間扭正」的完整過程。這是本章最具戲劇性的現象，也是理解圖最佳化最直觀的方式。

- **稀疏性的量化實驗**：建立不同規模的位姿圖（100、500、2000、10000 個節點），分別以稠密與稀疏的線性求解器計算，記錄執行時間。繪出「節點數 vs 時間」的對數圖，驗證稠密解法為  $O(n^3)$  而稀疏解法接近  $O(n)$ 。這個實驗能讓抽象的複雜度差異變得具體。
- **進階：感知混淆的實驗與防護**。建立一個有重複結構的模擬環境（如四條一模一樣的走廊）。實作外觀式的迴路閉合偵測，故意不加防護機制，觀察錯誤的迴路閉合如何折疊地圖。接著逐一加入三種防護（幾何驗證、時間一致性、強健核函數），量化每一種對誤判率的降低效果。討論三者的成本與效益，並提出一個綜合的防護策略。

## 參考文獻與延伸閱讀

- Thrun, S., Burgard, W. and Fox, D., Probabilistic Robotics, MIT Press (第 9 ~ 13 章涵蓋佔據柵格、EKF-SLAM、FastSLAM 與圖 SLAM)。
- Durrant-Whyte, H. and Bailey, T., "Simultaneous Localization and Mapping: Part I & II," IEEE Robotics & Automation Magazine, 2006 (SLAM 的經典回顧論文)。
- Cadena, C. et al., "Past, Present, and Future of Simultaneous Localization and Mapping," IEEE Trans. Robotics, 2016 (現代 SLAM 的全面回顧，強烈推薦)。
- Dissanayake, M. W. M. G. et al., "A Solution to the Simultaneous Localization and Map Building Problem," IEEE Trans. Robotics and Automation, 2001 (EKF-SLAM 的收斂性證明)。
- Montemerlo, M. et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI 2002 (FastSLAM 的原始論文)。
- Grisetti, G. et al., "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010 (圖最佳化 SLAM 的優秀教學文章)。
- Mur-Artal, R. and Tardós, J. D., "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras," IEEE Trans. Robotics, 2017 (視覺 SLAM 的代表性系統)。
- Qin, T., Li, P. and Shen, S., "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Trans. Robotics, 2018 (VIO 的代表性工作)。

- Shan, T. and Englot, B., "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping," IROS 2018 ( 光達 SLAM 的高效實作 ) 。
- ROS 2 slam\_toolbox、Cartographer 與 RTAB-Map 官方文件：三套廣泛使用的開源 SLAM 系統，各有不同的設計取舍。

# 第 26 章

## 路徑規劃與導航

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為位形空間的轉換示意：上圖是工作空間——一台長方形機器人推車要繞過一根柱子，同一條路徑在車頭朝左時能過、朝右時卡住；下圖是位形空間——柱子被膨脹了機器人的半徑，機器人縮成一個點，路徑變成點的移動，旁註「在位形空間中，機器人是一個點」。右半為分層導航的對照：一張完整地圖上以藍色粗線標出  $A^*$  規劃的全域路徑，路徑周圍以綠色扇形疊出 DWA 的局部候選軌跡群，其中一條被選中；畫面標註「全域看得遠、局部反應快」。下緣文字：先想清楚大方向，再邊走邊閃。

## 叫獸開講 「六維空間長什麼樣？機器人如何在看不見的空間裡找路？」

叫獸在白板上畫了一張簡單的平面圖：一個房間、中間有一根柱子、機器人在左下角、目標在右上角。

「請小翰上來畫出機器人該走的路線。」小翰很快畫了一條繞過柱子的曲線。「很好。現在，」叫獸把機器人的圖示擦掉，換成一個長方形——一台推著大型推車的機器人，長 1.2 公尺、寬 0.8 公尺。「同樣的房間，重畫一次。」

小翰猶豫了。這次不只要考慮「路徑會不會穿過柱子」，還要考慮「機器人的車身會不會撞到柱子」——而這取決於機器人當時的**朝向**。他畫了幾筆，發現同一條路徑，車頭朝左時能過、朝右時會卡住。

「這就是路徑規劃第一個、也是最重要的觀念。」叫獸說，「你剛才不自覺地在做一件很累的事——**同時追蹤機器人的形狀與位置**。有沒有辦法讓它變簡單？」

小好舉手：「把柱子畫大一點？大到機器人可以當成一個點？」

「非常好。」叫獸在柱子外圍畫了一圈虛線，「這叫做**膨脹**。你把障礙物長胖了機器人的半徑，然後就可以把機器人當成一個**點**來規劃。這個轉換叫做**位形空間**，是整個路徑規劃的地基。」

他接著在白板上另一側寫下三個字：**點、線、面**。「但這裡有一個關鍵問題：機器人不是圓的。你的推車是長方形，轉個方向佔的空間就不一樣——所以真正的位形空間不是二維的平面，而是**三維的**：x、y、還有角度  $\theta$ 。」

小翰皺眉：「三維還好吧？」

「三維還好。但如果是第 9 章那支六軸手臂呢？」叫獸敲了敲白板，「它的位形空間是**六維**的。而規劃演算法必須在這個六維空間裡，找出一條不碰到任何障礙的路。你能想像六維空間長什麼樣嗎？」

教室安靜了。叫獸笑了：「沒有人能。這就是為什麼路徑規劃在 1990 年代之前幾乎是無解的問題——直到有人想出一個聰明的辦法：**既然畫不出來，那就不要畫。用隨機取樣去探索它。**」

最後他在白板上寫下一句話：「今天要學的，是機器人如何在一個**看不見也畫不出來**的空間裡找路。而這個問題有兩層——**先想清楚大方向，再邊走邊閃**。兩者缺一不可。」

## 本章為什麼重要

經過第 22 ~ 25 章，機器人已經能看見環境、能移動、能估計自己的狀態、並能建立地圖。但知道自己在哪裡，不等於知道該怎麼走。

路徑規劃回答的正是這個問題：給定當前位置、目標位置與環境地圖，找出一條可行的路徑。這條路徑必須**避開障礙**、**符合運動約束**（第 23 章的非完整性）、且**盡可能地短或快**。

這個問題的困難程度常被低估。對人類而言「繞過那根柱子」是本能，但對機器人來說，它涉及三個層次的挑戰：一、**如何表示「哪些位置是可行的」**——這需要位形空間的抽象；二、**如何在高維空間中搜尋**——六軸手臂的位形空間是六維的，人類無法想像；三、**如何處理動態與不確定性**——環境會變、感知有誤差、規劃出的路徑未必走得準。

本章也是本書兩條主線的交會處。它同時服務於**移動機器人**（在地圖上找路）與**機械手臂**（在關節空間避開障礙），而兩者用的是同一套理論。此外，本章的搜尋與取樣思想，也將在第 32 章的任務規劃中再次出現。

## 學習目標

- 能定義位形空間，並說明其為何是路徑規劃的核心抽象。
- 能運用障礙物膨脹把機器人簡化為點。
- 能區分路徑規劃與軌跡生成的分工。
- 能實作 Dijkstra 與  $A^*$  演算法並說明兩者的差異。
- 能定義啟發函數的可採納性，並說明其對最佳性的保證。
- 能說明格點式規劃在高維空間的維數詛咒。
- 能說明 PRM 與 RRT 的原理與適用場合。
- 能說明 RRT\* 如何達成漸近最佳性。
- 能說明非完整約束對規劃的影響與 Dubins 路徑的意義。
- 能說明人工勢場法的原理與局部極小值問題。
- 能說明動態視窗法的原理與其如何納入動力學限制。
- 能說明分層導航架構中全域與局部規劃的分工。



## 先備知識

第 25 章的佔據柵格地圖（規劃的直接輸入）、第 23 章的運動約束與非完整性、第 15 章的軌跡生成（規劃的下游）、第 4 章的自由度概念、第 24 章的機率思維。

演算法方面需要：圖的表示、優先佇列、以及基本的搜尋概念。若對資料結構生疏，可先閱讀附錄 D.4。本章的數學不深，但**空間想像力很重要**——建議多畫圖，尤其是位形空間的部分。

## 核心名詞中英對照

表 26-1 本章核心名詞

| 中文        | 英文                                 | 說明                 |
|-----------|------------------------------------|--------------------|
| 位形空間      | Configuration Space, C-space       | 以機器人所有位形為點所構成的空間   |
| 自由空間      | Free Space, C_free                 | 位形空間中不發生碰撞的部分      |
| 障礙物膨脹     | Obstacle Inflation                 | 把障礙物擴大以將機器人簡化為點    |
| 完備性       | Completeness                       | 有解時必定能找到的性質        |
| 機率完備性     | Probabilistic Completeness         | 取樣數趨於無窮時找到解的機率趨於 1 |
| 最佳性       | Optimality                         | 所找到的解是最短或代價最小的     |
| 啟發函數      | Heuristic Function                 | 估計某點到目標的剩餘代價       |
| 可採納       | Admissible                         | 啟發函數不高估真實代價的性質     |
| 一致性       | Consistency                        | 啟發函數滿足三角不等式的性質     |
| 機率路標圖     | Probabilistic Roadmap, PRM         | 先建圖再查詢的取樣式規劃法      |
| 快速探索隨機樹   | Rapidly-exploring Random Tree, RRT | 以隨機取樣長出樹狀結構的規劃法    |
| Dubins 路徑 | Dubins Path                        | 有最小轉彎半徑限制下的最短路徑    |
| 人工勢場      | Artificial Potential Field         | 以吸引與排斥力導引運動的方法     |
| 局部極小值     | Local Minimum                      | 勢場中非目標的低點，會使機器人卡住  |
| 動態視窗法     | Dynamic Window Approach, DWA       | 在速度空間中搜尋的局部規劃法     |
| 全域規劃      | Global Planner                     | 依完整地圖規劃大方向路徑       |
| 局部規劃      | Local Planner                      | 依即時感測避開動態障礙        |
| 代價地圖      | Costmap                            | 含膨脹層與代價值的規劃用地圖     |

## 26.1 位形空間

### 26.1.1 把機器人變成一個點

路徑規劃最重要的抽象是**位形空間** ( configuration space , 簡稱 C-space ) 。它的定義是：

#### ◎ 位形空間的定義

**位形** ( configuration ) 是完整描述機器人所有部位位置所需的最少參數集合。**位形空間**則是所有可能位形所構成的空間——空間中的**每一個點**，代表機器人的**一個完整姿態**。

這個轉換的威力在於：**在位形空間中，整個機器人變成了一個點**。原本「這個形狀複雜的物體會不會撞到障礙物」的困難問題，變成了「這個點會不會落在禁區內」的簡單問題。

幾個具體的例子：

表 26-2 各類機器人的位形空間

| 機器人        | 位形參數                                       | 維度 | 空間的形狀                |
|------------|--------------------------------------------|----|----------------------|
| 平面上的圓形機器人  | (x, y)                                     | 2  | 平面                   |
| 平面上的長方形機器人 | (x, y, $\theta$ )                          | 3  | 平面 $\times$ 圓 ( 環狀 ) |
| 差速驅動車      | (x, y, $\theta$ )                          | 3  | 同上，但有非完整約束           |
| 平面 2R 手臂   | ( $\theta_1, \theta_2$ )                   | 2  | <b>環面 ( 甜甜圈 )</b>    |
| 六軸工業手臂     | ( $\theta_1, \dots, \theta_6$ )            | 6  | 六維環面                 |
| 行動操作機器人    | (x, y, $\theta, \theta_1 \dots \theta_6$ ) | 9  | 九維                   |

**第四列特別值得注意：**平面 2R 手臂的位形空間是一個**環面** ( torus , 甜甜圈的形狀 ) ——因為兩個關節角度都是週期性的 (  $0^\circ$  與  $360^\circ$  是同一個位形 ) 。這說明位形空間未必是平坦的歐氏空間，其拓撲結構可能相當特殊。

### 26.1.2 自由空間與障礙

位形空間可以分成兩部分：

$$C = C_{\text{free}} \cup C_{\text{obs}}$$

$C_{\text{free}}$  ( 自由空間 ) 是不發生碰撞的位形； $C_{\text{obs}}$  是會碰撞的位形。路徑規劃的目標，就是在  $C_{\text{free}}$  中找出一條由起點到終點的連續曲線。

關鍵的困難在於： **$C_{\text{obs}}$  的形狀通常無法解析地寫出來**。即使工作空間中的障礙物是一個簡單的圓，在位形空間中對應的  $C_{\text{obs}}$  可能是一個極為複雜的形狀——尤其當機器人有多個關節時。

這正是為何 26.4 節的取樣式方法如此重要：它們不需要知道  $C_{obs}$  的形狀，只需要能回答「這個點是否在  $C_{obs}$  中」（即碰撞檢測）。這個「不建構、只查詢」的策略，是路徑規劃在高維空間中的突破關鍵。

### 26.1.3 障礙物膨脹

對於平移的機器人，有一個實用的簡化：把障礙物膨脹（inflate）機器人的半徑，然後把機器人當成一個點。

$$C_{obs} = \text{障礙物} \oplus \text{機器人形狀 (Minkowski 和)}$$

數學上這稱為 Minkowski 和。對圓形機器人，就是把障礙物往外擴張一個半徑。

這正是開頭故事中小好的想法。它在實務上極為常用——ROS 導航的代價地圖（costmap）就內建了膨脹層（inflation layer）。

但要注意兩個限制：一、只對圓形機器人完全正確。非圓形機器人的膨脹量會隨朝向改變，若用最大半徑膨脹會過於保守（狹窄通道可能被誤判為不可通行）。二、膨脹會抹除細節。若膨脹半徑大於通道寬度的一半，該通道在膨脹後會完全消失。

實務上的折衷是：用外接圓半徑膨脹做粗略規劃，再用精確的機器人輪廓做碰撞檢查。ROS 的 costmap 正是採用這種「內切圓 + 外接圓」的雙半徑設計。

圖 26-1 位形空間與障礙物膨脹

建議配圖：三格對照。左格為工作空間——一個圓形機器人（半徑  $r$ ）與一個方形障礙物，機器人繞行障礙物的路徑以虛線表示。中格為位形空間——障礙物被膨脹了  $r$ （原障礙以實線、膨脹後以虛線圈出），機器人縮成一個點，同一條路徑變成點的移動。右格為平面  $2R$  手臂的位形空間——橫軸  $\theta_1$ 、縱軸  $\theta_2$ ，工作空間中的一個圓形障礙物在此對應成一片不規則的黑色區域（ $C_{obs}$ ），並標註「四邊相連成環面」。下方文字：在位形空間中，機器人是一個點。

## 26.2 規劃問題的分類與判準

### 26.2.1 路徑、軌跡與導航

必須先釐清三個容易混淆的層次（部分已在第 15 章討論）：

- 路徑規劃（Path Planning）：找出一條幾何曲線，只關心「經過哪些位形」，不含時間。本章的主題。
- 軌跡生成（Trajectory Generation）：為路徑加上時間參數化，決定速度與加速度。第 15

章的主題。

- **導航 ( Navigation )**：完整的系統，包含定位、規劃、執行與即時避障的循環。26.9 節的主題。

三者的關係是**由上而下**：導航系統呼叫路徑規劃取得幾何路徑，再交給軌跡生成加上時間，最後由控制器執行。而執行過程中若發現新的障礙，又會觸發重新規劃——形成一個閉環。

### 26.2.2 演算法的三個判準

評估規劃演算法有三個標準的判準，理解它們的差異極為重要。

**判準一：完備性 ( Completeness )**。若解存在，演算法**保證**能找到；若不存在，能**判定**無解。

- **完備**：格點搜尋（在該解析度下）、Dijkstra、A\*。
- **機率完備**：取樣式方法（PRM、RRT）——取樣數趨於無窮時，找到解的機率趨於 1，但**永遠無法判定無解**。
- **不完備**：人工勢場法——可能卡在局部極小值而找不到存在的解。

**判準二：最佳性 ( Optimality )**。找到的解是否為代價最小的？

- **最佳**：Dijkstra、A\*（在可採納啟發函數下）。
- **漸近最佳**：RRT\*——取樣數增加時，解逐漸收斂到最佳。
- **非最佳**：RRT、人工勢場法——找到的路徑可能繞遠路。

**判準三：計算效率**。實務上往往是決定性的因素——一個最佳但需要一小時的演算法，對即時導航毫無用處。

#### ◎ 三者的取捨

這三個判準通常無法兼得，而**工程上的選擇取決於應用**。倉儲 AGV 在固定地圖上運行，可以離線計算最佳路徑（選 A\*）；機械手臂在雜亂的桌面上抓取，位形空間六維且障礙隨時改變，只能用取樣法快速找出**可行解**（選 RRT）；而移動機器人的即時避障需要毫秒級反應，連可行性都只能局部保證（選 DWA）。**沒有一種演算法適用於所有場合**，理解它們的取捨才是本章的核心。

## 26.3 圖搜尋：Dijkstra 與 A\*

### 26.3.1 把空間變成圖

最直觀的規劃方法是：把位形空間離散化成一張圖，然後用圖搜尋演算法找最短路徑。

離散化的方式有兩種：

**方式一：格點 ( Grid )**。把空間切成規則的格子，每格是一個節點，相鄰的格子之間有邊。

這與第 25 章的佔據柵格地圖天然契合——佔據機率低的格子就是可通行的節點。

**方式二：路標圖 ( Roadmap )**。在自由空間中選取一些代表性的位形作為節點，再連接彼此可見的節點。26.4 節的 PRM 屬於此類。

### 26.3.2 Dijkstra 演算法

Dijkstra 是最經典的最短路徑演算法。它的策略是：永遠先擴展目前已知代價最小的節點。

1. 把起點的代價設為 0，其餘節點設為無窮大。
2. 從尚未處理的節點中，取出代價最小的節點。
3. 更新其所有鄰居的代價（若經由此節點更短，則更新）。
4. 標記此節點為已處理，回到步驟 2，直到目標被處理或無節點可取。

Dijkstra 保證找到最短路徑，但它有一個明顯的低效：它往所有方向均勻地擴展，像水波一樣向外蔓延。即使目標就在正東方，它也會花大量時間探索西方的節點。

### 26.3.3 A\*：加入方向感

A\* 的改進極為簡單卻極為有效：在選擇節點時，不只看「已經走了多遠」，還要估計「還要走多遠」。

$$f(n) = g(n) + h(n)$$

$A^*$  的評價函數。 $g(n)$  是由起點到  $n$  的實際代價、 $h(n)$  是由  $n$  到目標的估計代價（啟發函數）。

演算法的流程與 Dijkstra 相同，只是排序的依據由  $g$  換成  $f$ 。這個小改動使搜尋有了方向感——朝向目標的節點會優先被擴展。

常用的啟發函數（在格點地圖上）：

表 26-3 常見的啟發函數

| 名稱    | 公式                        | 適用的移動方式      | 特性  |
|-------|---------------------------|--------------|-----|
| 曼哈頓距離 | $ \Delta x  +  \Delta y $ | 只能上下左右（4 連通） | 可採納 |

| 名稱            | 公式                                   | 適用的移動方式           | 特性           |
|---------------|--------------------------------------|-------------------|--------------|
| 對角線距離         | $\max( \Delta x ,  \Delta y )$       | 可斜走且代價相同 ( 8 連通 ) | 可採納          |
| 歐氏距離          | $\sqrt{(\Delta x)^2 + (\Delta y)^2}$ | 任意方向              | 可採納但可能低估     |
| 零 ( $h = 0$ ) | 0                                    | 任意                | 退化為 Dijkstra |

### 26.3.4 可採納性：最佳性的保證

$A^*$  保證找到最短路徑的條件是：啟發函數必須是可採納的 ( admissible ) 。

$$h(n) \leq h^*(n) \quad \text{對所有 } n \text{ 成立}$$

可採納性。  $h^*(n)$  是由  $n$  到目標的真實最短代價。即：啟發函數絕不能高估。

為何不能高估？直觀的理解是：若  $h$  高估了某條路徑的剩餘代價， $A^*$  會認為那條路「不划算」而跳過它——但它其實可能是最短的。低估則只會使搜尋多探索一些節點，不影響最終結果的正確性。

還有一個更強的條件稱為一致性 ( consistency )：

$$h(n) \leq c(n, n') + h(n') \quad \text{對任意相鄰的 } n, n' \text{ 成立}$$

一致性 ( 也稱單調性 )。滿足此條件時，每個節點只需處理一次，演算法更高效。一致性蘊含可採納性。

#### ◎ 啟發函數的取舍

$h$  越接近真實值，搜尋越高效——若  $h = h^*$  ( 完美的啟發 )， $A^*$  會直接沿最短路徑走，不浪費任何探索。但完美的  $h$  通常無法取得 ( 若能取得，問題就已經解決了 )。反之， $h = 0$  時  $A^*$  退化為 Dijkstra——保證最佳但效率低。因此  $h$  的設計是一個取舍：在「可採納」的前提下，盡可能接近真實值。實務上還有一種常見的作法：故意讓  $h$  略微高估 ( 乘上一個大於 1 的權重 )，這稱為加權  $A^*$ 。它犧牲最佳性換取速度，找到的路徑保證不超過最佳解的該權重倍——這在即時系統中往往是划算的交易。

### 26.3.5 格點法的維數詛咒

格點法在二維與三維中效果良好，但在高維空間中會遭遇與第 18.4.3 節相同的問題：維數詛咒。

若每一維切成 100 格：

2 維： $10^4$  格

3 維： $10^6$  格

6 維： $10^{12}$  格

9 維： $10^{18}$  格

格點數隨維度指數成長。六軸手臂的位形空間根本無法建表。

這使得格點法只適用於低維問題——典型是移動機器人的二維或三維位形空間。對機械手臂與高自由度系統，必須改用下一節的取樣式方法。

## 26.4 取樣式規劃：PRM 與 RRT

### 26.4.1 核心思想：不建構，只查詢

取樣式方法的突破性洞見是：

#### ◎ 關鍵洞見

我們不需要知道  $C_{obs}$  的完整形狀，只需要能回答「這個特定的位形是否碰撞」。而後者只是一次碰撞檢測——對任何維度的機器人都能在毫秒內完成。既然如此，就不要試圖建構整個空間的表示，而是隨機取樣一些點，用它們拼出一條路。

這個轉換使高維規劃成為可能。它放棄了完備性（改為機率完備）與最佳性，換取的是在任意維度中都能運作的能力。

### 26.4.2 機率路標圖（PRM）

PRM ( Probabilistic Roadmap ) 採用「先建圖、再查詢」的策略，分為兩個階段。

#### 階段一：建構路標圖（離線）

1. 在位形空間中隨機取樣  $N$  個點，丟棄碰撞的點。
2. 對每個保留的點，嘗試連接其鄰近的  $k$  個點。
3. 若兩點之間的直線路徑無碰撞（用局部規劃器檢查），就在圖中加入這條邊。
4. 得到一張涵蓋自由空間的路標圖。

#### 階段二：查詢（線上）

1. 把起點與終點加入路標圖，各自連接到鄰近的節點。
2. 用  $A^*$  或 Dijkstra 在圖上搜尋最短路徑。

PRM 的最大優勢是「多次查詢」（multi-query）：路標圖建好之後，可以用它回答任意多組起訖點的查詢，每次只需一次圖搜尋。這使它非常適合環境固定但任務多變的場合——如工廠中的機械手臂，環境（治具、機台）固定不變，但每天要執行數千次不同的取放。



缺點是：建圖階段耗時、且若環境改變（有人走過、工件移位）就必須重建。

### 26.4.3 快速探索隨機樹 ( RRT )

RRT ( Rapidly-exploring Random Tree ) 採用完全不同的策略：從起點長出一棵樹，逐步探索空間，直到觸及目標。

1. 以起點為樹根。
2. 在位形空間中隨機取樣一個點  $x_{rand}$ 。
3. 找出樹上距離  $x_{rand}$  最近的節點  $x_{near}$ 。
4. 由  $x_{near}$  朝  $x_{rand}$  前進一小步（步長  $\epsilon$ ），得到新節點  $x_{new}$ 。
5. 若  $x_{near}$  到  $x_{new}$  的路徑無碰撞，就把  $x_{new}$  加入樹中。
6. 重複，直到樹觸及目標附近。

RRT 的名稱來自其驚人的探索特性：這棵樹會快速地朝未探索的區域生長。原因是——空曠的區域面積大，隨機取樣落在那裡的機率高，因此樹會優先往那些方向延伸。這是一個極為優雅的自組織性質。

一個關鍵的實作技巧：目標偏置 ( goal bias )。以某個機率（典型 5 ~ 10%）不做隨機取樣，而是直接取目標點作為  $x_{rand}$ 。這使樹在探索的同時也朝目標推進，能大幅加速收斂。

### 26.4.4 RRT\*：漸近最佳性

原始的 RRT 有一個明顯的缺點：找到的路徑通常繞遠路，因為樹的結構是隨機生長的，不會回頭優化。

RRT\* ( RRT-star ) 加入了兩個關鍵改進：

**改進一：選擇最佳父節點。**加入新節點時，不是直接連到最近的節點，而是在其鄰域內尋找使總代價最小的父節點。

**改進二：重新佈線 ( rewiring )。**加入新節點後，檢查其鄰域內的既有節點——若經由新節點能使它們的代價更低，就改變它們的父節點。

這兩個改進使 RRT\* 具備漸近最佳性 ( asymptotic optimality )：

當取樣數  $\rightarrow \infty$  時，解  $\rightarrow$  最佳解



漸近最佳。這是 Karaman 與 Frazzoli 於 2011 年證明的重要結果。

代價是計算量增加（每次都要檢查鄰域），且**收斂較慢**——RRT 可能在一秒內找到一條可行路徑，而 RRT\* 可能需要數十秒才能收斂到接近最佳。

**表 26-4 PRM 與 RRT 家族的比較**

| 面向      | PRM     | RRT       | RRT*      |
|---------|---------|-----------|-----------|
| 策略      | 先建圖、後查詢 | 由起點長樹     | 長樹 + 重新佈線 |
| 查詢類型    | 多次查詢    | 單次查詢      | 單次查詢      |
| 完備性     | 機率完備    | 機率完備      | 機率完備      |
| 最佳性     | 取決於路標密度 | 非最佳       | 漸近最佳      |
| 適合動態環境  | 否（需重建）  | 是         | 是         |
| 處理非完整約束 | 困難      | 容易        | 容易        |
| 計算成本    | 建圖高、查詢低 | 低         | 中高        |
| 典型應用    | 固定環境的手臂 | 動態環境、快速求解 | 需要品質的離線規劃 |

### 26.4.5 為何 RRT 特別適合非完整系統

RRT 有一個 PRM 所沒有的重要優勢：它**天然能處理非完整約束**（第 23.3 節）。

原因在於生長的方式。PRM 需要「連接任意兩點」——但對差速車而言，兩個位形之間未必存在直線路徑（不能側移）。RRT 則不同：它只需要「從現有節點**沿著可行的運動**前進一小步」。

具體作法是：把步驟 4 的「朝  $x_{\text{rand}}$  前進」改為「**在可行的控制輸入中，選擇使結果最接近  $x_{\text{rand}}$  的那一個**」。這樣長出的樹，每一條邊都自動滿足運動約束。

這個版本稱為 **kinodynamic RRT**，是自駕車與無人機路徑規劃的主流方法之一。

## 26.5 非完整約束下的規劃

### 26.5.1 為何不能直接用最短路徑

第 23 章證明了差速車與汽車都受**非完整約束**——它們不能側移。這對路徑規劃有直接的影響：

在位形空間中，兩點之間的直線段未必是可行的路徑。A\* 在格點上找出的路徑，可能包含

「原地橫移」這種物理上做不到的動作；PRM 連接兩個節點時假設的直線路徑，對汽車而言也不存在。

因此非完整規劃需要一個關鍵的替換：把「直線」換成「該機器人真正能走的最短路徑」。

## 26.5.2 Dubins 路徑

對於**只能前進**且有最小轉彎半徑  $R$  的車輛（如飛機、某些 AGV），Dubins 於 1957 年證明了一個優美的結果：

### ◎ Dubins 定理

在只能前進且轉彎半徑不小於  $R$  的限制下，連接任意兩個位姿的**最短路徑**，必定是下列六種組合之一：LSL、LSR、RSL、RSR、RLR、LRL。其中 L 表示左轉最小半徑圓弧、R 表示右轉、S 表示直線。

這個結果的實用價值極高：只需計算六種候選路徑的長度，取最短者即可——不需要任何搜尋或最佳化。計算成本是常數時間。

典型的 Dubins 路徑長這樣：先以最小半徑轉一個彎對準目標方向、直線行駛、再轉一個彎對準目標朝向。這正是飛機航線與 AGV 路徑常見的形狀。

## 26.5.3 Reeds-Shepp 路徑

若車輛**可以倒車**（如汽車），問題變得更複雜但也更靈活。Reeds 與 Shepp 於 1990 年將 Dubins 的結果推廣，證明最短路徑必定屬於 **48 種**基本組合之一（含前進與後退的切換）。

Reeds-Shepp 路徑的實用意義極為重大：它讓路邊停車這類需要來回的動作有了數學上的**最佳解**。自駕車的自動停車功能，其底層就是 Reeds-Shepp 路徑的計算。

兩者的比較：

表 26-5 Dubins 與 Reeds-Shepp 路徑

| 面向    | Dubins           | Reeds-Shepp    |
|-------|------------------|----------------|
| 是否可倒車 | 否                | 是              |
| 基本組合數 | 6 種              | 48 種           |
| 路徑長度  | 較長               | 較短（可倒車繞近路）     |
| 適用車輛  | 飛機、固定翼無人機、單向 AGV | 汽車、自駕車、可倒車 AGV |

| 面向   | Dubins    | Reeds-Shepp |
|------|-----------|-------------|
| 典型應用 | 航線規劃、巡邏路徑 | 自動停車、狹窄空間掉頭 |

## 26.5.4 混合 A\*

實務上最常用的非完整規劃方法是 **Hybrid A\*** (混合 A\*)，它結合了格點搜尋與連續運動。

與標準 A\* 的差別在於：

- **節點是連續的位姿** ( $x, y, \theta$ )，而非格點中心。格點只用來記錄哪些區域已探索過 (每格最多保留一個節點)。
- **擴展時使用可行的運動基元** (motion primitive) ——例如「以最大轉角左轉前進 0.5 公尺」，而非簡單的上下左右。
- **啟發函數同時考慮兩種資訊**：忽略障礙的 Reeds-Shepp 距離 (考慮運動約束)、以及忽略運動約束的柵格距離 (考慮障礙)。取兩者的最大值，兼顧兩方面。

Hybrid A\* 是自駕車路徑規劃的主流方法之一，DARPA 城市挑戰賽的優勝隊伍即採用此法。它的優勢是：產生的路徑天然可行 (不需後處理平滑)、且能處理倒車與狹窄空間。

## 26.6 人工勢場法

### 26.6.1 把規劃變成物理

人工勢場法 (Artificial Potential Field) 採取了一個完全不同的思路：**不做搜尋，而是讓機器人「被吸引」到目標、「被排斥」離開障礙。**

具體作法是定義一個勢場函數，機器人沿其負梯度方向運動：

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q) \quad \text{運動方向：} -\nabla U(q)$$

總勢場為吸引項與排斥項之和。機器人像一顆球，沿著勢場的斜坡滾向最低點。

吸引勢通常取與目標距離的平方 (像彈簧)：

$$U_{\text{att}}(q) = \frac{1}{2} k_{\text{att}} \cdot \|q - q_{\text{goal}}\|^2 \Rightarrow F_{\text{att}} = -k_{\text{att}}(q - q_{\text{goal}})$$

吸引力正比於距離，方向指向目標。

排斥勢只在障礙物附近作用，且距離越近力越大：

$$U_{\text{rep}}(q) = \frac{1}{2} k_{\text{rep}} (1/d - 1/d_0)^2 \quad \text{若 } d < d_0, \text{ 否則為 } 0$$

$d$  為到最近障礙的距離、 $d_0$  為影響範圍。 $d \rightarrow 0$  時排斥力  $\rightarrow \infty$ ，確保不會碰撞。

### 26.6.2 優點：極為簡單且即時

勢場法的三個優點使它在實務上仍有一席之地：

- **計算極快**：只需計算當前位置的梯度，不需要搜尋。可以在微秒級完成，適合高頻的即時控制迴路。
- **天然處理動態障礙**：障礙物移動時，勢場自動改變，機器人立即反應——不需重新規劃。
- **與控制器天然銜接**：輸出直接是一個速度或力的向量，可直接送給控制器（甚至與第 20 章的阻抗控制形式相同）。

### 26.6.3 致命弱點：局部極小值

但勢場法有一個著名的、也是致命的問題：**局部極小值**（local minimum）。

#### ◎ 局部極小值問題

當吸引力與排斥力恰好互相抵消時，機器人會**卡住不動**——即使目標明明可以到達。最典型的情境是 **U 形障礙物**（如一個凹陷的死角）：機器人被吸引進入凹陷，但兩側的牆壁把它推回中央，形成一個穩定的平衡點。機器人就這樣停在那裡，永遠到不了目標。

這使勢場法**不具備完備性**（第 26.2.2 節）——它可能找不到明明存在的解。

三種常見的緩解方法：

- **隨機擾動**：偵測到卡住時，隨機移動一小段距離跳出極小值。簡單但不保證有效。
- **調和勢場（Harmonic Potential）**：使用滿足 Laplace 方程的勢場，數學上可證明**沒有局部極小值**。代價是需要在整個空間上數值求解偏微分方程，計算量大且需要完整地圖。
- **只作為局部規劃器**：這是實務上最常用的作法——用**全域規劃器**（如  $A^*$ ）決定大方向，勢場法只負責局部的即時避障。由於全域路徑已經避開了大的凹陷，局部極小值的風險大幅降低。

第三種作法直接引出了 26.8 節的分層架構。

## 26.7 動態視窗法

### 26.7.1 在速度空間中搜尋

動態視窗法 ( Dynamic Window Approach, DWA ) 是移動機器人最廣泛使用的局部規劃器，也是 ROS 導航的預設選項之一。

它的核心洞見是：**不要在位置空間中規劃路徑，而是直接在速度空間中選擇指令。**

對差速驅動車，控制指令是  $(v, \omega)$ 。DWA 在這個二維的速度空間中搜尋最佳的組合。「動態視窗」指的是**當前可達的速度範圍**，由三個限制共同決定：

- **速度限制**： $|v| \leq v_{\max}$ 、 $|\omega| \leq \omega_{\max}$  ( 馬達能力，第 23.2.4 節 )。
- **加速度限制**：在下一個控制週期  $\Delta t$  內，速度只能改變  $a \cdot \Delta t$ ——這是**動態視窗**一詞的由來。
- **安全限制**：必須能在撞到障礙物前煞停。這給出  $v \leq \sqrt{2 \cdot d \cdot a_{\text{brake}}}$ ，其中  $d$  為到障礙的距離。

$$V_d = \{ (v, \omega) : \text{同時滿足上述三個限制} \}$$

*動態視窗。它是速度空間中的一個小矩形區域，隨當前速度與障礙分布而變。*

### 26.7.2 評分函數

在動態視窗內取樣多組  $(v, \omega)$ ，對每一組**模擬未來一小段時間的軌跡**（用第 23 章的運動模型），再依三個指標評分：

$$G(v, \omega) = \alpha \cdot \text{heading}(v, \omega) + \beta \cdot \text{clearance}(v, \omega) + \gamma \cdot \text{velocity}(v, \omega)$$

*DWA 的評分函數。三項分別代表朝向目標、遠離障礙、行進快速。*

- **heading ( 朝向 )**：模擬軌跡終點的朝向與目標方向的一致程度。鼓勵朝目標前進。
- **clearance ( 淨空 )**：模擬軌跡上到最近障礙物的距離。鼓勵遠離障礙。
- **velocity ( 速度 )**：前進速度的大小。鼓勵快速移動，避免機器人為了安全而停滯不前。

取得分最高的  $(v, \omega)$  作為本週期的控制指令。整個過程在每個控制週期（典型 10 ~ 20 Hz）重複一次。

### 26.7.3 DWA 的優勢與參數

DWA 相對於勢場法的關鍵優勢是：它**明確考慮了動力學限制**。勢場法算出的方向可能是機器人根本無法立即達成的（例如要求瞬間掉頭）；DWA 則只在「下一步真正做得到」的範圍內選擇，因此產生的指令必定可執行。

三個權重的調整直接影響行為：

表 26-6 DWA 權重的影響

| 症狀            | 原因                 | 調整方向               |
|---------------|--------------------|--------------------|
| 機器人繞遠路、不直接朝目標 | $\alpha$ ( 朝向 ) 太小 | 提高 $\alpha$        |
| 太貼近障礙物、有擦撞風險  | $\beta$ ( 淨空 ) 太小  | 提高 $\beta$         |
| 行進緩慢、在原地猶豫    | $\gamma$ ( 速度 ) 太小 | 提高 $\gamma$        |
| 在狹窄通道前停住不敢進入  | $\beta$ 太大         | 降低 $\beta$ 或縮小膨脹半徑 |
| 轉彎時撞到內側       | 模擬時間太短             | 延長軌跡模擬時間           |

**DWA 的限制：**它是**局部**的方法——只看未來一小段時間，因此同樣會遇到局部極小值（如走進死巷）。這再次說明了為何需要全域規劃器配合。

圖 26-2 動態視窗法的速度空間搜尋

建議配圖：左半為速度空間圖——橫軸為線速度  $v$ 、縱軸為角速度  $\omega$ 。畫出三層限制：最外層的矩形為馬達速度上限、中間的小矩形為加速度限制下的可達範圍（動態視窗，以藍色標出）、其中被安全限制排除的區域以紅色斜線標示。視窗內取樣的候選點以小圓點表示，得分最高者以綠色星號標出。右半為對應的工作空間圖——機器人前方有兩個障礙物，數條由不同  $(v, \omega)$  產生的模擬軌跡以灰色曲線畫出，被選中的軌跡以綠色粗線標示，並標註三項評分（朝向、淨空、速度）的示意。

## 26.8 分層導航架構

### 26.8.1 為何需要分層

前面各節的方法各有所長，也各有致命的限制：

- $A^*$  保證最佳，但需要完整地圖且無法即時反應動態障礙（重新規劃需要數十至數百毫秒）。
- DWA 與勢場法反應極快，但只看局部，會卡在死巷。

實務系統的解法是分層：把兩者結合，各司其職。

#### ◎ 分層導航的核心思想

**全域規劃器負責「大方向」：**依完整的地圖，規劃一條由當前位置到目標的路徑。更新頻率低（0.5~2 Hz 或只在必要時），但視野涵蓋整張地圖。**局部規劃器負責「邊走邊閃」：**依即時的感測資料，決定下一個控制週期該用什麼速度。更新頻率高（10~20 Hz），但只看周圍數公尺。**兩者的分工類似人類走路——**你腦中有一個「往車站方向」的大計畫，同時眼睛不斷地避開路上的行人與坑洞。

## 26.8.2 完整的導航堆疊

以 ROS 2 的導航系統 ( Nav2 ) 為例，一個完整的導航堆疊包含五個部分：

1. **定位 ( Localization )**：AMCL 或 SLAM，提供機器人在地圖中的位姿 ( 第 24、25 章 )。
2. **代價地圖 ( Costmap )**：把地圖與即時感測資料融合，產生含膨脹層的規劃用地圖。分為全域 ( 整張地圖 ) 與局部 ( 機器人周圍數公尺 ) 兩張。
3. **全域規劃器 ( Global Planner )**：在全域代價地圖上用 A\* 或 Dijkstra 找出路徑。
4. **局部規劃器 ( Local Planner / Controller )**：DWA、TEB 或 MPPI，依局部代價地圖產生速度指令。
5. **行為樹 ( Behavior Tree )**：協調上述模組，並處理異常情況 ( 見下節 )。

## 26.8.3 代價地圖的分層設計

代價地圖本身也是分層的，這個設計極為實用：

表 26-7 代價地圖的圖層

| 圖層                | 來源           | 作用                        |
|-------------------|--------------|---------------------------|
| 靜態層 ( Static )    | SLAM 建立的地圖   | 已知的牆壁與固定障礙                |
| 障礙層 ( Obstacle )  | 即時的光達 / 深度相機 | 動態障礙、地圖上沒有的東西             |
| 膨脹層 ( Inflation ) | 上述兩層的處理結果    | 把障礙擴大機器人半徑 ( 第 26.1.3 節 ) |
| 其他自訂層             | 如禁行區、社交距離    | 人為設定的規則                   |

各層的結果依序疊加，得到最終的代價值。**這個分層設計的價值在於模組化**——可以獨立地加入或移除某一層，而不影響其他部分。例如在醫院的機器人可以加一層「病房門口減速區」，在工廠可以加一層「堆高機通道禁行區」。

## 26.8.4 復原行為

真實的導航系統必定會遇到規劃失敗的情況：被人包圍、走進死巷、定位漂移導致以為自己在牆裡。因此需要**復原行為** ( recovery behaviors )。

典型的復原序列 ( 由溫和到激烈 )：

1. **清除代價地圖**：清掉障礙層的舊資料，可能是暫時經過的人已經離開了。
2. **原地旋轉**：轉一圈重新掃描環境，更新周圍的障礙資訊。



3. **後退**：往後退一小段，脫離可能的卡住位置。
4. **重新定位**：若懷疑定位錯誤，觸發全域重定位（第 24.1.3 節的綁架問題）。
5. **放棄並求援**：通知操作員或上層系統。

復原行為的設計往往比規劃演算法本身更影響系統的可用性。一個規劃器可能 99% 的情況都能成功，但若剩下的 1% 會讓機器人永久卡住，整個系統就無法無人化運行。這是實務與論文的一個重要差距——論文報告成功率，實務關心失敗時怎麼辦。

## 26.9 機械手臂的路徑規劃

### 26.9.1 與移動機器人的異同

本章至此的討論多以移動機器人為例，但路徑規劃同樣是機械手臂的核心問題——而且更困難。

**相同之處**：位形空間的抽象、碰撞檢測的需求、取樣式方法的適用性，兩者完全相通。

**不同之處**有四點：

表 26-8 移動機器人與機械手臂規劃的比較

| 面向     | 移動機器人         | 機械手臂             |
|--------|---------------|------------------|
| 位形空間維度 | 2~3           | 6~7 (或更高)        |
| 主要方法   | 格點搜尋 (A*、DWA) | 取樣式 (RRT、PRM)    |
| 碰撞檢測對象 | 機器人本體 vs 環境   | 各連桿 vs 環境、以及自我碰撞 |
| 目標的指定  | 位置 (二維點)      | 位姿 (需逆運動學，可能多解)  |
| 非完整約束  | 有 (不能側移)      | 無 (各關節獨立)        |
| 典型規劃時間 | 毫秒~數十毫秒       | 數十毫秒~數秒          |

**第三列特別值得注意**：機械手臂必須檢查**自我碰撞**——手臂的某一節可能撞到另一節，或撞到自己的基座。這使碰撞檢測的成本大幅增加 ( $n$  個連桿需要  $O(n^2)$  次配對檢查)。

### 26.9.2 目標的多重性

另一個機械手臂特有的問題是：**目標通常以末端位姿指定，而非關節角度**。

這帶來一個機會與一個困難。**機會**是：由第 10 章可知，同一個末端位姿可能對應多組關節



角度（六軸手臂典型有八組）。規劃器可以**把所有的逆解都當成候選目標**，只要能到達其中任何一個就算成功——這大幅提高了找到解的機率。

**困難**是：某些逆解可能無法到達（被障礙擋住、或超出關節極限）。因此實務的作法是：先求出所有逆解、過濾掉不可行者、再以剩餘的解作為多目標的規劃問題。

### 26.9.3 路徑平滑

取樣式方法（尤其是 RRT）產生的路徑通常**曲折而不自然**——因為它是由隨機取樣點連接而成的。直接執行會使機器人走出鋸齒狀的動作，既不美觀也浪費時間。

因此規劃後必須做**路徑平滑**（path smoothing）。最常用的方法是**捷徑法**（shortcutting）：

1. 隨機選取路徑上的兩個點。
2. 檢查它們之間的直線是否無碰撞。
3. 若無碰撞，就用這條直線取代原本兩點之間的曲折路段。
4. 重複數百次，路徑會逐漸被拉直。

捷徑法簡單有效，通常能把 RRT 路徑的長度縮短 20~40%。之後再交給第 15 章的軌跡生成加上時間參數化，即可執行。

**完整的流程**是：逆運動學（求目標關節角）→ 路徑規劃（RRT）→ 路徑平滑（捷徑法）→ 軌跡生成（第 15 章）→ 動力學檢驗（第 15.8 節）→ 控制執行（第 17 章）。這條鏈完整地串起了本書第二、三、四篇的內容。

## 26.10 現代發展

### 26.10.1 最佳化式規劃

除了搜尋與取樣，還有第三條路線：**把規劃直接表述為一個最佳化問題**。

代表性的方法是 **CHOMP** 與 **TrajOpt**。它們的作法是：先給一條初始軌跡（可能是直線，即使它會穿過障礙），然後用梯度下降逐步把它「推」出障礙區，同時最小化路徑長度與平滑度。

$$\min_{\xi} f_{\text{smooth}}(\xi) + f_{\text{obstacle}}(\xi)$$

軌跡最佳化的目標函數。第一項鼓勵平滑、第二項懲罰接近障礙。

**優點：**產生的軌跡天然平滑（不需後處理）、能同時考慮多個目標（平滑、避障、力矩限制）、且收斂快。

**缺點：**只能找到**局部**最佳解——若初始軌跡的「這一側」有障礙，它不會考慮繞到「另一側」。因此對複雜的環境，可能需要多個初始猜測。

這與第 18 章的最佳控制、第 25 章的圖最佳化屬於同一類數學工具——**都是用梯度把一個初始猜測推向更好的解。**

## 26.10.2 學習式規劃

深度學習在規劃中的角色，與第 25 章的 SLAM 類似——**主要是加速而非取代**。三個具體的應用方向：

- **學習取樣分布：**與其均勻隨機取樣，不如學習「哪些區域比較可能包含解」。例如在狹窄通道附近多取樣。這能使 RRT 的收斂速度提升數倍。
- **學習啟發函數：**用神經網路預測「由此點到目標的真實剩餘代價」，取代手工設計的歐氏距離。這能使 A\* 的搜尋更有方向感。
- **端到端的策略：**直接由感測資料學習「該往哪走」，跳過顯式的規劃。這是第 31 章強化學習的路線，在複雜或難以建模的環境中有其價值，但缺乏保證。

**目前的共識與 SLAM 相同：**學習擅長「哪裡值得探索」這類需要經驗與直覺的判斷，而傳統方法擅長「這條路是否真的可行」這類需要嚴格保證的驗證。混合式架構仍是主流。

## 26.10.3 多機器人與社會導航

兩個近年的重要方向：

**多機器人規劃：**當數十台 AGV 在同一個倉庫運行時，必須避免互相碰撞與死鎖。這需要**協調**——可以是集中式（一個中央規劃器分配路徑）或分散式（各自規劃並協商）。核心的挑戰是**死鎖偵測與解除**：兩台機器人在狹窄通道相遇時，必須有一方讓路，而這需要一致的規則。

**社會導航（Social Navigation）：**當機器人在人群中移動時，「不碰撞」是不夠的——它還必須符合**人類的社會慣例**。例如：不要突然從人的背後竄出、與人保持適當的社交距離、在走廊上靠右行、在通過門口時禮讓。這些規則難以用傳統的代價函數表達，是目前研究的熱點，也直接關係到機器人能否被社會接受（第 35 章）。

## 26.11 由導航邁向飛行與行走

### 26.11.1 本章的成果

本章完成了自主移動的最後一塊拼圖。你現在能夠：運用位形空間把機器人簡化為一個點、實作 A\* 並理解啟發函數的可採納性、運用 PRM 與 RRT 在高維空間中規劃、處理非完整約束下的路徑 (Dubins 與 Reeds-Shepp)、實作勢場法與 DWA 進行即時避障、並理解分層導航架構中全域與局部規劃的分工。

最重要的是三個核心洞見：一、**位形空間的抽象**——把「複雜形狀的碰撞」轉化為「一個點是否在禁區」。二、**不建構、只查詢**——取樣式方法放棄完整表示，換取高維空間中的可行性。三、**分層的必然性**——全域看得遠但反應慢、局部反應快但看得近，兩者缺一不可。

### 26.11.2 第四篇的整合

第 22 ~ 26 章共同構成了一個完整的自主移動系統：

看見 (第 22 章) → 移動 (第 23 章) → 估測 (第 24 章) → 建圖與定位 (第 25 章) →  
規劃與導航 (第 26 章)

*第四篇的邏輯鏈。每一章都是下一章的必要輸入。*

這五章也共同體現了一個與第三篇截然不同的思維方式。第三篇的世界是**確定的**——給定關節角度，末端位置唯一確定；本篇的世界則充滿**不確定性**——每一個感測讀值都帶著雜訊、每一個估計都帶著誤差、每一次辨識都可能出錯。

因應這個轉變，本篇反覆出現三個原則：一、**量化不確定性而非忽略它** (第 24 章的協方差)；二、**依可信度融合而非二選一** (第 24 章的 Kalman 增益)；三、**寧可保守也不要誤判** (第 25 章的迴路閉合、第 26 章的膨脹半徑)。

### 26.11.3 接下來：離開地面

但第 22 ~ 26 章的討論都隱含一個假設：**機器人在地面上、以輪子移動**。這使問題大幅簡化——只需考慮二維或三維的位形空間、有地面提供穩定的支撐、且運動學相對單純。

**第 27 章 (無人機)** 與 **第 28 章 (腿式機器人)** 將處理兩種截然不同的移動方式，它們共有一個關鍵特性：**欠驅動** (第 23.3.4 節的概念在此重現)。

無人機有六個自由度但只有四個控制輸入（四旋翼）——它無法一邊保持水平一邊橫移，必須先傾斜才能移動。而且它一刻都不能停止控制——馬達一停就會墜落。

腿式機器人在騰空階段完全無法控制質心的軌跡（只能靠落地時的觸地點修正），且必須持續維持平衡。它的位形空間維度高達數十維，且接觸狀態不斷切換。

這兩者將把第三篇的控制理論（尤其是第 18 章的 LQR 與第 19 章的穩定性）與第四篇的感知定位技術完整地結合起來——它們是全書技術的綜合考驗。

## 完整計算例題

### 例題 26-1 A\* 的搜尋與啟發函數

題目：在一個 4 連通的格點地圖上，機器人由 (0,0) 走向 (5,3)，每步代價為 1。（a）計算曼哈頓距離啟發函數在起點的值。（b）若某節點在 (2,1)，計算其 f 值（設  $g = 3$ ）。（c）判斷歐氏距離在此 4 連通情形下是否可採納。（d）若把啟發函數乘上 2（加權 A\*），討論其後果。

解（a）：曼哈頓距離為兩軸差值的絕對值之和：

$$h(0,0) = |5 - 0| + |3 - 0| = 5 + 3 = 8$$

起點的啟發值。由於 4 連通每步只能走一格且代價為 1，這正好等於真實的最短距離（無障礙時）。

解（b）：

$$\begin{aligned} h(2,1) &= |5 - 2| + |3 - 1| = 3 + 2 = 5 \\ f(2,1) &= g + h = 3 + 5 = 8 \end{aligned}$$

評價函數值。

判讀： $g = 3$  代表已走三步，但曼哈頓距離顯示由 (0,0) 到 (2,1) 最少需要 3 步——因此這條路徑沒有繞路。 $f = 8$  等於起點的 h 值，代表這條路徑仍在最短路徑上。

解（c）：歐氏距離為：

$$h_{\text{euclid}}(0,0) = \sqrt{5^2 + 3^2} = \sqrt{34} \approx 5.83$$

歐氏距離。

由於  $5.83 < 8$ （真實最短距離），歐氏距離是可採納的（沒有高估）。但它低估太多——這會使 A\* 探索大量不必要的節點，效率接近 Dijkstra。

結論：在 4 連通格點上應使用曼哈頓距離，它既可採納又緊貼真實值，效率最高。若誤用歐

氏距離，結果仍正確但速度會明顯下降。

解 (d)：加權  $A^*$  ( $h' = 2h$ ) 時：

$$h'(0,0) = 2 \times 8 = 16 > 8 = h^*$$

*高估了真實代價，不再可採納。*

後果是：**不再保證找到最短路徑**，但可以證明找到的路徑長度**不超過最佳解的 2 倍**（一般地，權重為  $w$  時保證不超過  $w$  倍）。

**取捨**：搜尋速度通常能提升數倍到數十倍（因為  $A^*$  會更「急切」地衝向目標，少探索側面的節點）。在即時系統中，「1.2 倍長的路徑但快 10 倍算出來」往往比「最短路徑但來不及」更有價值。實務上常用  $w = 1.1 \sim 1.5$ 。

## 例題 26-2 位形空間與膨脹

題目：某圓形移動機器人半徑  $r = 0.35 \text{ m}$ ，在一個走廊中運行。走廊寬  $1.2 \text{ m}$ ，中間有一根直徑  $0.4 \text{ m}$  的圓柱。（a）求膨脹後的圓柱等效半徑。（b）判斷機器人能否從圓柱的一側通過。（c）若機器人改為長  $0.9 \text{ m}$ 、寬  $0.5 \text{ m}$  的長方形，求其外接圓與內切圓半徑。（d）討論用外接圓膨脹的後果。

解 (a)：圓柱原半徑  $0.2 \text{ m}$ ，膨脹機器人半徑  $0.35 \text{ m}$ ：

$$R_{\text{inflated}} = 0.2 + 0.35 = 0.55 \text{ m}$$

*膨脹後的等效半徑。在位形空間中，機器人的中心不能進入這個範圍。*

解 (b)：走廊寬  $1.2 \text{ m}$ ，圓柱在中央。圓柱兩側各剩：

$$\text{剩餘空間} = (1.2 - 0.4)/2 = 0.4 \text{ m (每側)}$$

*工作空間中每側的實際通道寬度。*

在位形空間中，機器人中心可行走的範圍為：

$$\text{可行寬度} = 0.4 - 0.35 = 0.05 \text{ m}$$

*扣除機器人半徑後，中心只剩 5 公分的可行帶。*

理論上可以通過 ( $0.05 \text{ m} > 0$ )，但**實務上極為危險**——考慮到定位誤差（第 24 章，典型數公分）與控制誤差，5 公分的餘裕遠遠不夠。

**工程判斷：**應視為不可通行，並在代價地圖中把此處標為高代價區，讓規劃器優先選擇其他路徑。實務上建議至少保留機器人半徑 20 ~ 30% 的餘裕（本例約 10 公分）。

解 (c)：長方形  $0.9 \times 0.5 \text{ m}$ ：

$$\text{外接圓半徑} = \sqrt{((0.9/2)^2 + (0.5/2)^2)} = \sqrt{(0.2025 + 0.0625)} = \sqrt{0.265} = 0.515 \text{ m}$$

$$\text{內切圓半徑} = \min(0.9, 0.5)/2 = 0.25 \text{ m}$$

兩個半徑。外接圓是內切圓的兩倍以上。

解 (d)：若用外接圓 (0.515 m) 膨脹，圓柱的等效半徑變為  $0.2 + 0.515 = 0.715 \text{ m}$ 。此時：

$$\text{可行寬度} = 0.4 - 0.515 = -0.115 \text{ m} < 0$$

通道被判定為完全不可通行。

但實際上，若機器人沿走廊方向擺放（長邊平行於走廊），它的等效寬度只有 0.5 m，需要的通道為  $0.5/2 = 0.25 \text{ m}$ ——0.4 m 的通道是綽綽有餘的。

這正是 26.1.3 節提到的限制：用外接圓膨脹過於保守，會誤判可通行的通道。實務的解法是 ROS costmap 的雙半徑設計——用內切圓做快速的粗略判斷（絕對安全的區域）、用外接圓標記絕對危險的區域、兩者之間的區域則用精確的多邊形碰撞檢測逐一判斷。這是「精度」與「速度」的分層取捨。

### 例題 26-3 動態視窗的計算

題目：某差速驅動機器人  $v_{\max} = 0.8 \text{ m/s}$ 、 $\omega_{\max} = 1.5 \text{ rad/s}$ 、線加速度上限  $a_v = 0.5 \text{ m/s}^2$ 、角加速度上限  $a_\omega = 2.0 \text{ rad/s}^2$ 。控制週期  $\Delta t = 0.1 \text{ s}$ 。當前速度  $(v, \omega) = (0.4, 0.3)$ 。(a) 求加速度限制下的動態視窗。(b) 若正前方 1.5 m 處有障礙物，煞車減速度為  $0.5 \text{ m/s}^2$ ，求安全速度上限。(c) 綜合求出最終的動態視窗。(d) 討論若控制週期加長為 0.5 s 的影響。

解 (a)：加速度限制在一個週期內能改變的速度為：

$$\Delta v_{\max} = a_v \times \Delta t = 0.5 \times 0.1 = 0.05 \text{ m/s}$$

$$\Delta \omega_{\max} = a_\omega \times \Delta t = 2.0 \times 0.1 = 0.2 \text{ rad/s}$$

一個週期內的速度變化量。

$$v \in [0.4 - 0.05, 0.4 + 0.05] = [0.35, 0.45] \text{ m/s}$$

$$\omega \in [0.3 - 0.2, 0.3 + 0.2] = [0.10, 0.50] \text{ rad/s}$$

動態視窗。注意這個範圍極小——這正是「動態」的意義。

**觀察：**雖然機器人的最高速度是 0.8 m/s，但在下一個週期它最多只能達到 0.45 m/s。DWA 只在這個小範圍內搜尋，因此產生的指令必定可執行。

解 (b)：安全條件是「能在撞到障礙前煞停」。由運動學：

$$v \leq \sqrt{2 \cdot d \cdot a_{\text{brake}}} = \sqrt{2 \times 1.5 \times 0.5} = \sqrt{1.5} = 1.22 \text{ m/s}$$

安全速度上限。

由於 1.22 m/s > 0.8 m/s (馬達上限)，在此距離下安全限制不起作用——機器人可以用任何速度前進。

若障礙在 0.3 m 處： $v \leq \sqrt{2 \times 0.3 \times 0.5} = \sqrt{0.3} = 0.548 \text{ m/s}$ ——此時安全限制開始生效。

若障礙在 0.1 m 處： $v \leq \sqrt{0.1} = 0.316 \text{ m/s}$ ——限制更嚴格。

解 (c)：綜合三個限制取交集：

$$v \in [0.35, 0.45] \cap [0, 0.8] \cap [0, 1.22] = [0.35, 0.45] \text{ m/s}$$

$$\omega \in [0.10, 0.50] \cap [-1.5, 1.5] = [0.10, 0.50] \text{ rad/s}$$

最終的動態視窗。本例中由加速度限制主導。

DWA 會在這個範圍內取樣 (例如  $5 \times 5 = 25$  組)，對每組模擬軌跡並評分，取最佳者。

解 (d)：若  $\Delta t = 0.5 \text{ s}$ ：

$$\Delta v_{\text{max}} = 0.5 \times 0.5 = 0.25 \text{ m/s} \Rightarrow v \in [0.15, 0.65]$$

$$\Delta \omega_{\text{max}} = 2.0 \times 0.5 = 1.0 \text{ rad/s} \Rightarrow \omega \in [-0.7, 1.3] \text{ (受 } \omega_{\text{max}} \text{ 限制)}$$

動態視窗大幅擴大。

這看似是好事 (選擇變多)，實際上是壞事，理由有三：

- **反應變慢：**兩次決策之間隔了 0.5 秒，機器人在這段時間內是「盲目」的。若有人突然走出來，可能來不及反應。
- **軌跡模擬的誤差變大：**DWA 假設速度在一個週期內固定，週期越長這個假設越不準。
- **與第 19.9.3 節的結論一致：**取樣週期過長會侵蝕穩定裕度。

**實務建議：**DWA 的控制週期應在 50 ~ 200 ms 之間 (5 ~ 20 Hz)。過短則計算來不及，過長則反應遲鈍。

## 程式範例：A\*、RRT 與動態視窗法

以下 Python 程式實作三種代表性的規劃演算法：格點上的 A\*、取樣式的 RRT，以及局部規劃的 DWA，並比較它們的行為。

```
import numpy as np
import heapq, random

# ===== 地圖與碰撞檢測 =====
class GridMap:
    def __init__(s, w=50, h=30):
        s.w, s.h = w, h
        s.grid = np.zeros((h, w), dtype=int)  # 0 空曠, 1 障礙

    def add_rect(s, x0, y0, x1, y1):
        s.grid[y0:y1, x0:x1] = 1

    def free(s, x, y):
        xi, yi = int(round(x)), int(round(y))
        if not (0 <= xi < s.w and 0 <= yi < s.h):
            return False
        return s.grid[yi, xi] == 0

    def inflate(s, r):
        """障礙物膨脹 (26.1.3 節)"""
        out = s.grid.copy()
        ri = int(np.ceil(r))
        ys, xs = np.where(s.grid == 1)
        for y, x in zip(ys, xs):
            y0, y1 = max(0, y-ri), min(s.h, y+ri+1)
            x0, x1 = max(0, x-ri), min(s.w, x+ri+1)
            out[y0:y1, x0:x1] = 1
        m = GridMap(s.w, s.h); m.grid = out
        return m

# ===== A* 演算法 =====
def astar(gmap, start, goal, heuristic="manhattan", weight=1.0):
    """26.3 節的 A*。回傳 (路徑, 展開節點數)"""
    def h(n):
        dx, dy = abs(n[0]-goal[0]), abs(n[1]-goal[1])
        if heuristic == "manhattan": return dx + dy
        if heuristic == "euclidean": return np.hypot(dx, dy)
        return 0.0  # Dijkstra
    open_set = [(h(start)*weight, 0.0, start, None)]
    came, gscore, expanded = {}, {start: 0.0}, 0
    while open_set:
        f, g, cur, parent = heapq.heappop(open_set)
        if cur in came: continue
        came[cur] = parent
        expanded += 1
        if cur == goal:
            path, n = [], cur
            while n is not None:
                path.append(n); n = came[n]
            return path[::-1], expanded
        for dx, dy in ((1,0), (-1,0), (0,1), (0,-1)):
            nb = (cur[0]+dx, cur[1]+dy)
```



```

        if not gmap.free(*nb) or nb in came: continue
        ng = g + 1.0
        if ng < gscore.get(nb, 1e18):
            gscore[nb] = ng
            heapq.heappush(open_set, (ng + weight*h(nb), ng, nb, cur))
    return None, expanded

# ===== RRT 演算法 =====
def rrt(gmap, start, goal, max_iter=5000, step=1.5, goal_bias=0.1,
        goal_tol=2.0, seed=0):
    """26.4.3 節的 RRT。回傳 (路徑, 節點數)"""
    rng = random.Random(seed)
    nodes = [np.array(start, float)]
    parents = [-1]
    for it in range(max_iter):
        # 目標偏置
        if rng.random() < goal_bias:
            x_rand = np.array(goal, float)
        else:
            x_rand = np.array([rng.uniform(0, gmap.w),
                               rng.uniform(0, gmap.h)])

        # 找最近節點
        d = [np.linalg.norm(n - x_rand) for n in nodes]
        i_near = int(np.argmin(d))
        x_near = nodes[i_near]

        # 前進一步
        vec = x_rand - x_near
        norm = np.linalg.norm(vec)
        if norm < 1e-9: continue
        x_new = x_near + vec/norm*min(step, norm)

        # 碰撞檢查 (沿線取樣)
        ok = True
        for t in np.linspace(0, 1, 8):
            p = x_near + t*(x_new - x_near)
            if not gmap.free(p[0], p[1]): ok = False; break
        if not ok: continue
        nodes.append(x_new); parents.append(i_near)
        if np.linalg.norm(x_new - np.array(goal)) < goal_tol:
            path, i = [], len(nodes)-1
            while i != -1:
                path.append(tuple(nodes[i])); i = parents[i]
            return path[::-1], len(nodes)
    return None, len(nodes)

def path_length(path):
    if not path: return float('inf')
    return sum(np.linalg.norm(np.array(path[i+1]) - np.array(path[i]))
               for i in range(len(path)-1))

def shortcut(gmap, path, iters=300, seed=0):
    """26.9.3 節的捷徑法平滑"""
    rng = random.Random(seed)
    p = [np.array(q, float) for q in path]
    for _ in range(iters):
        if len(p) < 3: break
        i = rng.randint(0, len(p)-3)
        j = rng.randint(i+2, len(p)-1)

```

```

    ok = True
    for t in np.linspace(0, 1, 20):
        q = p[i] + t*(p[j] - p[i])
        if not gmap.free(q[0], q[1]): ok = False; break
    if ok: p = p[:i+1] + p[j:]
    return [tuple(q) for q in p]

# ===== 建立測試地圖 =====
gm = GridMap(50, 30)
gm.add_rect(15, 0, 18, 20)           # 上方伸下的牆
gm.add_rect(30, 10, 33, 30)         # 下方伸上的牆
gm.add_rect(40, 5, 44, 12)          # 一個方塊
start, goal = (2, 15), (47, 15)

print("=== A* : 不同啟發函數的比較 ===")
print(f"{'啟發函數':<14}{'權重':>6}{'路徑長度':>10}{'展開節點':>10}{'最佳?':>8}")
for hname, w in (("dijkstra", 1.0), ("euclidean", 1.0),
                 ("manhattan", 1.0), ("manhattan", 1.5),
                 ("manhattan", 3.0)):
    path, exp = astar(gm, start, goal, hname, w)
    opt = "是" if w == 1.0 else f"≤{w:.1f}×"
    print(f"{'hname':<14}{'w':>6.1f}{'len(path)-1':>10d}{'exp':>10d}{'opt':>8}")
print("→ 曼哈頓最有效率；加權 A* 用最佳性換速度")

# ===== RRT 與平滑 =====
print("\n=== RRT : 取樣式規劃 ===")
print(f"{'種子':>6}{'節點數':>9}{'原始長度':>11}{'平滑後':>10}{'縮短':>9}")
for seed in range(5):
    p, n = rrt(gm, start, goal, seed=seed)
    if p is None:
        print(f"{'seed':>6d}{'n':>9d}{'失敗':>11}"); continue
    L0 = path_length(p)
    ps = shortcut(gm, p, seed=seed)
    L1 = path_length(ps)
    print(f"{'seed':>6d}{'n':>9d}{'L0':>11.2f}{'L1':>10.2f}{'(1-L1/L0)*100':>8.1f}%")
print("→ RRT 每次結果不同 (隨機性) ; 捷徑法可縮短 20~40%")

# ===== 目標偏置的影響 =====
print("\n=== 目標偏置對 RRT 的影響 (10 次平均) ===")
print(f"{'偏置率':>8}{'成功率':>9}{'平均節點數':>12}{'平均長度':>11}")
for gb in (0.0, 0.05, 0.1, 0.3, 0.6):
    ns, Ls, ok = [], [], 0
    for sd in range(10):
        p, n = rrt(gm, start, goal, goal_bias=gb, seed=sd)
        if p: ok += 1; ns.append(n); Ls.append(path_length(p))
    if ns:
        print(f"{'gb':>8.2f}{'ok':>10d}{'np.mean(ns)':>12.0f}{'np.mean(Ls)':>11.2f}")
    else:
        print(f"{'gb':>8.2f}{'ok':>10d}{'-' :>12}{'-' :>11}")
print("→ 適度的目標偏置 (5~10%) 能加速收斂；過高則易卡在障礙前")

```

```
# ===== 動態視窗法 (例題 26-3) =====
print("\n=== 例題 26-3：動態視窗的計算 ===")
class DWA:
    def __init__(s, v_max=0.8, w_max=1.5, av=0.5, aw=2.0, dt=0.1):
        s.v_max, s.w_max, s.av, s.aw, s.dt = v_max, w_max, av, aw, dt

    def window(s, v, w, obs_dist=None, a_brake=0.5):
        vs = [max(-s.v_max, v - s.av*s.dt), min(s.v_max, v + s.av*s.dt)]
        ws = [max(-s.w_max, w - s.aw*s.dt), min(s.w_max, w + s.aw*s.dt)]
        if obs_dist is not None:
            v_safe = np.sqrt(2*obs_dist*a_brake)
            vs[1] = min(vs[1], v_safe)
        return vs, ws

dwa = DWA()
vs, ws = dwa.window(0.4, 0.3)
print(f"(a) 加速度限制:  $v \in [{vs[0]:.3f}, {vs[1]:.3f}]$  m/s, "
      f" $w \in [{ws[0]:.3f}, {ws[1]:.3f}]$  rad/s")
print(f"\n(b) 不同障礙距離的安全速度上限:")
for d in (1.5, 0.5, 0.3, 0.1):
    v_safe = np.sqrt(2*d*0.5)
    binding = "生效" if v_safe < 0.8 else "不生效"
    print(f"    障礙 {d:.1f} m  $\rightarrow v \leq {v_safe:.3f}$  m/s ({binding})")
vs2, ws2 = dwa.window(0.4, 0.3, obs_dist=0.3)
print(f"\n(c) 障礙 0.3 m 時的最終視窗: "
      f" $v \in [{vs2[0]:.3f}, {vs2[1]:.3f}]$ ,  $w \in [{ws2[0]:.3f}, {ws2[1]:.3f}]$ ")
print(f"\n(d) 控制週期的影響:")
for dt in (0.05, 0.1, 0.3, 0.5):
    d2 = DWA(dt=dt)
    v2, w2 = d2.window(0.4, 0.3)
    print(f"     $\Delta t={dt:.2f}$ s  $\rightarrow v$  範圍寬度  ${v2[1]-v2[0]:.3f}$  m/s, "
          f" $w$  範圍寬度  ${w2[1]-w2[0]:.3f}$  rad/s")
print("     $\rightarrow$  週期越長視窗越大，但反應越遲鈍 (見 19.9.3 節)")

# ===== 膨脹的影響 (例題 26-2) =====
print("\n=== 例題 26-2：膨脹半徑對可通行性的影響 ===")
gm2 = GridMap(40, 20)
gm2.add_rect(18, 8, 22, 12) # 走廊中央的柱子
print(f"{ '膨脹半徑':>10}{ 'A* 結果':>12}{ '路徑長度':>11}")
for r in (0, 1, 2, 3, 4):
    gi = gm2.inflate(r) if r > 0 else gm2
    p, _ = astar(gi, (2, 10), (37, 10))
    if p:
        print(f"{r:10d}{ '成功':>12}{len(p)-1:11d}")
    else:
        print(f"{r:10d}{ '無解':>12}{ '-':>11}")
print("\n $\rightarrow$  膨脹過大會使原本可通行的路徑消失 (例題 26-2d)")
```

請試著修改：(一) 把 A\* 的權重設為 5.0，觀察路徑長度增加多少、展開節點減少多少。

(二) 把 RRT 的步長由 1.5 改為 0.5 與 4.0，觀察節點數與成功率的變化。(三) 在地圖中加入一個 U 形障礙，比較 A\* (能繞出) 與勢場法 (會卡住) 的差異。

## 實作：導航系統的建立與調校

本章實作以 ROS 2 Nav2 為主 (可用模擬器或實體機器人)。若無 ROS 環境，前四項可用純 Python 完成。

1. **A\* 實作與比較**：在自建的格點地圖上實作 Dijkstra、A\* (曼哈頓)、加權 A\*。記錄三者的路徑長度與展開節點數，驗證表 26-3 的預期。
2. **啟發函數的實驗**：對同一張地圖，測試四種啟發函數 (零、歐氏、曼哈頓、對角線)，繪出「展開節點數 vs 啟發函數」的長條圖。討論為何曼哈頓在 4 連通下最有效率。
3. **RRT 實作**：實作基本的 RRT 與目標偏置，在含狹窄通道的地圖上測試。執行 20 次，統計成功率與路徑長度的變異——親身體會取樣式方法的隨機性。
4. **路徑平滑**：實作捷徑法，量化平滑前後的路徑長度差異。觀察平滑後的路徑是否更接近人類的直覺。
5. **膨脹半徑的調校**：在 Nav2 的代價地圖中，測試不同的 inflation\_radius (0.2、0.35、0.5、0.8 m)。記錄機器人能否通過狹窄通道、以及是否會過於貼近牆面。找出適合你的機器人的值。
6. **DWA 權重調校**：系統性地改變 DWA 的三個權重 (path\_distance\_bias、goal\_distance\_bias、occdist\_scale)，觀察機器人的行為變化。製作表 26-6 的實測版本。
7. **局部極小值測試**：在環境中放置一個 U 形障礙 (如三面圍起的死角)，讓機器人的目標在其後方。觀察：(甲) 只用局部規劃器時是否卡住；(乙) 加入全域規劃器後是否能繞出。這是 26.6.3 節的實驗版本。
8. **動態障礙測試**：在機器人行進途中放入障礙物 (如紙箱)，觀察局部規劃器的反應時間與避讓行為。測試不同的控制週期對反應速度的影響。
9. **復原行為測試**：故意讓機器人陷入困境 (用障礙物包圍)，觀察 Nav2 的復原行為序列是否被觸發、以及是否成功脫困。
10. **(進階) 多目標導航**：規劃一條經過多個中途點的巡邏路徑，觀察系統如何處理連續的導航目標，並記錄總時間與路徑效率。

表 26-9 導航系統調校記錄表（前兩列為範例）

| 參數               | 設定值   | 行為表現     | 問題            | 調整方向      |
|------------------|-------|----------|---------------|-----------|
| inflation_radius | 0.5 m | 與牆保持適當距離 | 無法通過 0.9 m 走道 | 降至 0.35 m |
| occdist_scale    | 0.02  | 貼近障礙物行走  | 多次擦撞警告        | 提高至 0.1   |
| （自行填寫其餘測試）       |       |          |               |           |

## 國中小也看得懂：走迷宮的兩個層次

你有沒有玩過走迷宮的遊戲？從入口走到出口，中間有很多牆擋住。你會怎麼找路？

最笨的方法是**每一條路都試試看**，走到死路就退回來換一條。這樣一定找得到，但很花時間。電腦有一個方法就是這樣，叫做「**Dijkstra**」（唸作「戴克斯特拉」）——它像水一樣往四面八方慢慢流開，直到流到出口。

聰明一點的方法是：**朝著出口的方向優先找**。如果出口在右邊，那就先試右邊的路，不要浪費時間往左找。這個方法叫做「**A 星**」（寫作  $A^*$ ），是現在最常用的找路方法——連你玩的電動遊戲裡，那些會追著你跑的角色，用的也是它！

但機器人找路還有一個麻煩：**它自己有身體**。你在迷宮裡是一個小點，但機器人是一台大車，路太窄它就過不去。

工程師想到一個很聰明的辦法：**把牆壁「加厚」**。假設機器人半徑是 30 公分，那就把每一面牆都往外長胖 30 公分。這樣一來，就可以把機器人當成一個「點」來找路了——只要那個點不碰到加厚後的牆，真正的機器人就一定不會撞到！

最後還有一件事。機器人找路要分成**兩個層次**，就像你去朋友家：

**大方向**：先看地圖決定「搭捷運到某站再走過去」。這個計畫不會常常改。

**邊走邊閃**：路上有人擋住、有水坑、有腳踏車，你會隨時繞開。這個要一直做。

機器人也一樣——腦中有一個大計畫，眼睛不停地看著前方閃避。兩個都要有，缺一個都會出問題！

## 叫獸提醒與常見錯誤

- 誤區一：忘記機器人有體積。必須用位形空間或障礙物膨脹處理，否則規劃出的路徑會使機器人擦撞。
- 誤區二：對非圓形機器人用外接圓膨脹。這過於保守，會誤判可通行的狹窄通道；應採內切圓加精確碰撞檢測的雙層設計。
- 誤區三：膨脹半徑設得過大。可能使原本可通行的通道完全消失（例題 26-2d）。
- 誤區四：使用不可採納的啟發函數卻期待最佳解。h 高估時 A\* 不再保證最短路徑。
- 誤區五：在高維空間使用格點法。六維以上會遭遇維數詛咒，必須改用取樣式方法。
- 誤區六：以為 RRT 找到的是最短路徑。原始 RRT 非最佳，需要 RRT\* 或後處理的路徑平滑。
- 誤區七：期待取樣式方法能判定無解。它們只有機率完備性——找不到解時，無法區分是「無解」還是「取樣不夠」。
- 誤區八：只用勢場法或 DWA 而無全域規劃器。局部方法會卡在 U 形障礙的局部極小值中。
- 誤區九：忽略非完整約束。對汽車式機器人，位形空間中的直線段未必可行，應使用 Dubins、Reeds-Shepp 或 Hybrid A\*。
- 誤區十：DWA 的控制週期設得過長。視窗雖然變大但反應遲鈍，且軌跡模擬的誤差增加。
- 誤區十一：忽略復原行為的設計。規劃器 99% 成功率仍不足以無人化運行——關鍵在於剩下 1% 失敗時怎麼辦。
- 誤區十二：機械手臂規劃時忘記檢查自我碰撞。連桿之間可能互撞，這需要  $O(n^2)$  的配對檢查。

## 課堂討論

1. 26.4.1 節的「不建構、只查詢」洞見使高維規劃成為可能。請討論這個思維在本書其他章節的對應（提示：第 18 章的維數詛咒、第 24 章的粒子濾波器）。
2. 加權 A\* 犧牲最佳性換取速度，保證路徑不超過最佳解的  $w$  倍。請討論在什麼應用中這個交易是划算的？什麼應用中不可接受？
3. 分層導航架構中，全域規劃器與局部規劃器可能給出衝突的建議（如全域說往左、局部說往右）。請設計一個仲裁機制，並討論其可能的失效情境。
4. 26.8.4 節主張「復原行為的設計往往比規劃演算法本身更影響系統可用性」。請討論這個觀察對研究與實務優先順序的啟示。

5. 社會導航要求機器人符合人類的社會慣例（如靠右行、保持社交距離）。請討論這些規則該如何形式化為代價函數，以及會遇到什麼困難。

## 章末摘要

- 位形空間是路徑規劃的核心抽象：空間中每一點代表機器人的一個完整姿態，機器人因而簡化為一個點。
- 障礙物膨脹（Minkowski 和）使平移機器人可視為點；對非圓形機器人應採內切圓與外接圓的雙層設計。
- 演算法的三個判準：完備性（有解必找到）、最佳性（找到最短）、計算效率；三者通常無法兼得。
- 格點搜尋完備且最佳，取樣式方法僅機率完備，勢場法不完備。
- $A^*$  的評價函數  $f = g + h$ ；啟發函數可採納（ $h \leq h^*$ ）時保證找到最短路徑。
- $h$  越接近真實值搜尋越高效； $h = 0$  時退化為 Dijkstra；加權  $A^*$  犧牲最佳性換速度，保證不超過  $w$  倍。
- 格點法在高維遭遇維數詛咒，六維以上不可行。
- 取樣式方法的核心洞見：不建構  $C_{obs}$ ，只需回答「這個位形是否碰撞」。
- PRM 先建圖後查詢，適合固定環境的多次查詢；RRT 由起點長樹，適合動態環境的單次查詢。
- RRT 會快速朝未探索區域生長；目標偏置（5~10%）能顯著加速收斂。
- RRT\* 透過選擇最佳父節點與重新佈線達成漸近最佳性，代價是收斂較慢。
- RRT 天然能處理非完整約束，因為它只需「沿可行運動前進一步」而非「連接任意兩點」。
- Dubins 路徑（僅前進）有 6 種基本組合；Reeds-Shepp（可倒車）有 48 種，是自動停車的基礎。
- Hybrid  $A^*$  結合格點搜尋與連續運動基元，是自駕車規劃的主流方法。
- 人工勢場法計算極快且能處理動態障礙，但會卡在局部極小值，因此只適合作為局部規劃器。
- DWA 在速度空間中搜尋，明確考慮加速度與煞停限制，因此產生的指令必定可執行。
- 分層導航：全域規劃器看得遠但更新慢，局部規劃器反應快但視野小，兩者缺一不可。



- 代價地圖採分層設計（靜態、障礙、膨脹、自訂），復原行為的設計對系統可用性至關重要。

## 觀念題與計算題

### 觀念題

1. 定義位形空間，並說明它為何是路徑規劃的核心抽象。
2. 說明障礙物膨脹的原理與其兩個限制。
3. 為何平面 2R 手臂的位形空間是一個環面？請說明其拓撲意義。
4. 區分完備性、機率完備性與不完備，並各舉一個演算法為例。
5. 說明 A\* 的評價函數，並解釋可採納性為何是最佳性的保證。
6. 為何啟發函數不能高估？低估會有什麼後果？
7. 說明加權 A\* 的原理與其提供的保證。
8. 說明格點法在高維空間的維數詛咒，並計算六維時的格點數。
9. 敘述取樣式方法的核心洞見，並說明它為何使高維規劃成為可能。
10. 比較 PRM 與 RRT 的策略、適用場合與優缺點。
11. 說明 RRT 為何會「快速朝未探索區域生長」，以及目標偏置的作用。
12. 說明 RRT\* 的兩個改進，以及漸近最佳性的意義。
13. 為何 RRT 比 PRM 更適合處理非完整約束？
14. 說明 Dubins 路徑的六種組合，以及 Reeds-Shepp 的推廣。
15. 說明人工勢場法的原理與局部極小值問題，並列舉三種緩解方法。
16. 說明動態視窗的三個限制，以及 DWA 相對於勢場法的關鍵優勢。
17. 說明分層導航架構的分工，以及代價地圖的分層設計。
18. 為何說「復原行為的設計往往比規劃演算法更影響系統可用性」？

### 計算題

1. 在 8 連通格點上（斜走代價  $\sqrt{2}$ ），機器人由 (0,0) 走向 (8,5)。（a）計算對角線距離啟發函數的值。（b）計算曼哈頓距離的值。（c）判斷兩者是否可採納。（d）說明應使用哪一個並解釋理由。



2. 某長方形移動機器人長 1.0 m、寬 0.6 m。(a) 求外接圓與內切圓半徑。(b) 若走廊寬 1.5 m，判斷用外接圓膨脹後是否可通行。(c) 若機器人沿走廊方向擺放，實際需要的最小走廊寬度為何？(d) 討論兩者的差異與實務作法。
3. 某差速機器人  $v_{\max} = 1.0 \text{ m/s}$ 、 $\omega_{\max} = 2.0 \text{ rad/s}$ 、 $a_v = 0.8 \text{ m/s}^2$ 、 $a_\omega = 3.0 \text{ rad/s}^2$ 、 $\Delta t = 0.1 \text{ s}$ 。當前  $(v, \omega) = (0.6, -0.5)$ 。(a) 求加速度限制下的動態視窗。(b) 若前方 0.4 m 有障礙、煞車減速度  $0.8 \text{ m/s}^2$ ，求安全速度上限。(c) 求最終的動態視窗。(d) 若在此視窗中均勻取樣  $5 \times 5$  組，列出  $v$  與  $\omega$  的取樣值。
4. 某六軸手臂的位形空間為六維，每個關節角度範圍  $\pm 180^\circ$ 。(a) 若每維切成 36 格 (每格  $10^\circ$ )，求總格點數。(b) 若每個格點的碰撞檢測需  $0.1 \text{ ms}$ ，求建表所需的總時間。(c) 若改用 RRT 且每次迭代需  $0.5 \text{ ms}$ 、平均需 3000 次迭代，求規劃時間。(d) 比較兩者並說明為何取樣式方法是唯一可行的選擇。

## 動手做與專題延伸

### 本章實作：路徑規劃模組

延續第 23 ~ 25 章的程式庫，完成導航系統的最後一塊。

- 階段一：實作格點地圖與障礙物膨脹，並實作 Dijkstra 與  $A^*$  (支援多種啟發函數)。
- 階段二：實作 RRT 與目標偏置，並加入捷徑法路徑平滑。
- 階段三：實作  $RRT^*$ ，比較其與 RRT 在路徑品質與收斂時間上的差異。
- 階段四：實作人工勢場法與 DWA，並在含 U 形障礙的地圖上比較兩者的局部極小值行為。
- 階段五：整合為分層導航系統——全域用  $A^*$ 、局部用 DWA，並實作路徑追隨與重新規劃的觸發邏輯。

### 專題延伸

- 演算法的視覺化競賽：對同一組測試地圖 (含開闊區、狹窄通道、U 形陷阱、迷宮)，以動畫呈現五種演算法 (Dijkstra、 $A^*$ 、加權  $A^*$ 、RRT、 $RRT^*$ ) 的搜尋過程。記錄各自的展開節點數、路徑長度與執行時間，製作完整的比較表。這個專題能讓抽象的演算法差異變得直觀。
- 位形空間的視覺化：對平面 2R 手臂，在工作空間中放置數個圓形障礙物。掃描所有的  $(\theta_1, \theta_2)$  組合，判斷是否碰撞，繪出位形空間的黑白圖。你會看到工作空間中的簡單圓形障礙，在

位形空間中變成極為複雜的形狀——這是理解「為何不建構  $C_{obs}$ 」最有力的證據。接著在這張圖上執行  $A^*$ ，觀察路徑在兩個空間中的對應關係。

- **進階：非完整約束的規劃比較。**實作三種方法規劃汽車式機器人的路徑：（甲）忽略約束的  $A^*$ （產生不可行路徑）、（乙）Dubins 路徑（只能前進）、（丙）Hybrid  $A^*$ （可倒車）。在一個包含狹窄停車位的場景中測試，比較三者的可行性與路徑長度。特別觀察 Hybrid  $A^*$  如何自動產生「前進、倒車、再前進」的停車動作——這正是例題中路邊停車的自動化版本。

## 參考文獻與延伸閱讀

- LaValle, S. M., Planning Algorithms, Cambridge University Press（線上公開）（路徑規劃的權威教材，涵蓋本章所有主題）。
- Choset, H. et al., Principles of Robot Motion: Theory, Algorithms, and Implementations, MIT Press（規劃演算法的完整教材，含位形空間的詳細處理）。
- Hart, P. E., Nilsson, N. J. and Raphael, B., "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Trans. SSC, 1968（ $A^*$  演算法的原始論文）。
- Kavraki, L. E. et al., "Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces," IEEE Trans. Robotics and Automation, 1996（PRM 的奠基論文）。
- LaValle, S. M. and Kuffner, J. J., "Randomized Kinodynamic Planning," IJRR, 2001（RRT 的原始論文）。
- Karaman, S. and Frazzoli, E., "Sampling-Based Algorithms for Optimal Motion Planning," IJRR, 2011（RRT\* 與漸近最佳性的證明）。
- Khatib, O., "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," IJRR, 1986（人工勢場法的原始論文）。
- Fox, D., Burgard, W. and Thrun, S., "The Dynamic Window Approach to Collision Avoidance," IEEE Robotics & Automation Magazine, 1997（DWA 的原始論文）。
- Dolgov, D. et al., "Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments," IJRR, 2010（Hybrid  $A^*$  在自駕車上的應用）。

- ROS 2 Nav2 官方文件與 OMPL ( Open Motion Planning Library ) ：導航堆疊與規劃演算法函式庫的實務參考。

## 第 27 章

# 無人機與飛行機器人

編著：李世淵（ 機器人叫獸 ）

助教：李天宇、李宇晴

【章首情境圖建議】左半為四旋翼往前飛行的側視圖——機身明顯前傾，推力向量垂直於機身並分解為「向上抵抗重力」與「向前推進」兩個分量，旁註「想往前，必須先低頭」。機身上以正反槳的箭頭標出兩對相反的旋轉方向。右半為串級控制的四層巢狀方框圖，由外而內標註頻率：軌跡規劃 10 Hz、位置控制 50 Hz、姿態控制 500 Hz、角速度控制 1000 Hz，各層以不同深淺的綠色區隔，並在位置與姿態之間標出轉換公式  $\theta_d = a_x/g$ 。下緣文字：欠驅動且本質不穩定——一刻都不能停止控制。

## 叫獸開講 「六個自由度只有四顆馬達，它憑什麼飛得這麼穩？」

叫獸把一台四旋翼無人機放在桌上，接上電池，螺旋槳開始緩緩轉動。「小翰，我要它往右平移一公尺，但機身不能傾斜。做得到嗎？」

小翰想了幾秒：「右邊兩個螺旋槳.....不對，那是轉向。」他試著在腦中組合四顆馬達的轉速，發現無論怎麼配，只要想產生向右的力，機身就必定會傾斜。

「這是第 23 章那台差速車的空中版本。」叫獸說，「它有六個自由度——三個位置、三個姿態——但只有**四顆馬達**。想往右移動，你**必須先向右傾斜**，讓推力有一個水平分量。位置與姿態是綁在一起的，不能獨立控制。」

小好舉手：「所以跟差速車一樣，是非完整約束？」

「不一樣，而且差別很致命。」叫獸把無人機拿起來，「差速車不能側移，但你**放手它就停在那裡**。無人機呢？」他做出鬆手的姿勢，「馬達一停，它就從三十公尺高摔下來。」

他在白板上寫下兩個詞：**欠驅動、本質不穩定**。「差速車是欠驅動但穩定；無人機是欠驅動**而且**不穩定。它一刻都不能停止控制——這是第 18 章那個倒立擺的三維版本，而且是在空中。」

接著他問了一個更尖銳的問題：「那為什麼四旋翼還能飛得這麼穩？我看過的競速無人機可以做出翻滾動作再穩住。」

小翰試著回答：「因為控制器很快？」

「這是一半的答案。真正的關鍵在於**分層**。」叫獸在白板上畫了兩個方框，「姿態控制跑在**500 到 1000 Hz**——它只管『機身該傾斜幾度』；位置控制只跑**50 Hz**——它只管『該往哪個方向去』，然後把答案翻譯成『請傾斜這麼多度』交給下層。」

「為什麼要分開？」叫獸自問自答，「因為**姿態的動態比位置快十倍以上**。姿態是由力矩直接驅動的、慣量小、反應極快；位置則要透過『傾斜 → 產生水平分力 → 加速 → 積分成速度 → 再積分成位置』，中間隔了好幾層。**把快的和慢的分開處理，各自用合適的頻率**——這就是無人機能穩定飛行的祕密。」

最後他把無人機放回桌上：「今天你們會發現，第 18 章的 LQR、第 19 章的 Lyapunov、第 24 章的互補濾波，全部會在這一章派上用場。**這是全書控制理論的第一次綜合考驗。**」

## 本章為什麼重要

無人機是過去二十年機器人領域最成功的商業化案例之一。從航拍、農噴、巡檢、測繪、物流到災難搜救，它已經是一個成熟的產業。而在技術上，它更是一個**極佳的教學載體**——因為它把前面各章的理論逼到了必須全部用上的程度。

它的三個特性使它與前面所有的機器人都不同：

- **欠驅動且本質不穩定**：六個自由度只有四個輸入，且失去控制就會墜落。第 23 章的差速車雖然也欠驅動，但停下來仍是安全的。
- **沒有支撐點**：機械手臂固定在基座上、輪型機器人有地面支撐，而無人機**完全靠自己的推力懸浮**。任何一個環節失效都是災難。
- **動態極快**：姿態的時間常數只有數十毫秒，這要求 500 Hz 以上的控制頻率——比機械手臂的伺服迴路還快。

因此本章不只是「介紹一種機器人」，而是把第三篇的控制理論與第四篇的感知定位技術真正整合起來的第一個完整案例。第 18 章的 LQR、第 19 章的穩定性、第 24 章的狀態估測、第 26 章的路徑規劃，都會在這裡以具體的形式重現。

## 學習目標

- 能說明四旋翼的推力與力矩產生原理，並解釋為何需要正反槳。
- 能寫出四旋翼由馬達轉速到推力與力矩的分配矩陣。
- 能推導四旋翼的簡化動力學模型並辨識其非線性項。
- 能說明四旋翼的欠驅動特性及其對控制的影響。
- 能說明微分平坦性的意義及其在軌跡規劃上的價值。
- 能說明串級控制架構的分層原理與各層的頻率選擇。
- 能設計姿態控制器並說明角速度內迴路的必要性。
- 能說明位置控制如何轉換為姿態指令。
- 能比較四旋翼與固定翼、直升機、VTOL 的特性與適用場合。
- 能說明無人機的狀態估測架構與各感測器的角色。
- 能分析無人機的安全機制與常見失效模式。

## 先備知識

第 13 章的動力學與慣性張量、第 16 ~ 19 章的控制理論（尤其是第 18 章的 LQR 與第 19 章的欠驅動概念）、第 24 章的狀態估測與互補濾波、第 8 章的姿態表示（四元數在此極為重要）、第 6.3 節的 IMU。

本章是全書整合度最高的章節之一。若對第 18、19、24 章有任何不確定，建議先回頭複習——本章會直接使用那些工具而不重新推導。

## 核心名詞中英對照

表 27-1 本章核心名詞

| 中文           | 英文                        | 說明                  |
|--------------|---------------------------|---------------------|
| 四旋翼          | Quadrotor / Quadcopter    | 以四個螺旋槳產生升力的飛行器      |
| 推力係數         | Thrust Coefficient        | 推力與轉速平方的比例常數        |
| 扭矩係數         | Drag/Torque Coefficient   | 反扭矩與轉速平方的比例常數       |
| 分配矩陣         | Mixer / Allocation Matrix | 由推力力矩指令求各馬達轉速的矩陣    |
| 正反槳          | CW/CCW Propellers         | 順逆時針旋轉的螺旋槳配置        |
| 懸停           | Hover                     | 維持位置不變的飛行狀態         |
| 滾轉 / 俯仰 / 偏航 | Roll / Pitch / Yaw        | 繞機體三軸的旋轉            |
| 欠驅動          | Underactuated             | 控制輸入少於自由度           |
| 微分平坦         | Differential Flatness     | 所有狀態與輸入可由少數輸出及其導數表示 |
| 串級控制         | Cascade Control           | 多層巢狀迴路的控制架構         |
| 姿態控制         | Attitude Control          | 控制機體朝向的內迴路          |
| 位置控制         | Position Control          | 控制空間位置的外迴路          |
| 飛行控制器        | Flight Controller, FC     | 執行飛行控制的嵌入式系統        |
| 地面效應         | Ground Effect             | 接近地面時升力增加的現象        |
| 視覺慣性里程計      | VIO                       | 融合視覺與 IMU 的位姿估計     |
| 失效保護         | Failsafe                  | 異常時的自動保護行為          |
| 垂直起降         | VTOL                      | 能垂直起降的固定翼混合構型       |

## 27.1 飛行的原理與構型

### 27.1.1 四旋翼為何成為主流

旋翼飛行器有多種構型，而四旋翼在過去二十年成為絕對的主流。原因不在於它的飛行效率（實際上比直升機差），而在於**機構的極簡**。

傳統直升機需要**變距機構**（swashplate）——透過改變槳葉的攻角來控制升力與方向。這個機構包含數十個精密的連桿與軸承，製造與維護成本極高。

四旋翼則完全不需要：**所有的控制都靠改變四顆馬達的轉速**。沒有任何活動的機械連桿，只有四個固定的螺旋槳。這使它能够被大量、便宜地製造——這正是消費級無人機爆發的關鍵。

#### ◎ 為什麼是四個而不是三個或兩個

**兩個螺旋槳不夠**：無法同時控制推力、滾轉、俯仰與偏航這四個量。**三個也不行**：三個轉速只能控制三個量，且反扭矩無法平衡（機身會持續自轉）。**四個剛好**：四個轉速對應推力、滾轉、俯仰、偏航四個控制量，且能透過正反槳配置抵消反扭矩。**六個或八個**（六旋翼、八旋翼）則提供了**冗餘**——損失一顆馬達仍能飛行，這對載人或高價值任務極為重要。這是一個典型的「剛好夠用」與「冗餘備份」的工程取舍，與第 4 章討論自由度時的思維一致。

### 27.1.2 正反槳與反扭矩

四旋翼有一個容易被忽略但極為關鍵的設計：**相鄰的螺旋槳旋轉方向相反**（對角線上的兩個同向）。

原因來自牛頓第三定律。螺旋槳推動空氣向下時，空氣也對機身施加一個**反扭矩**（drag torque），使機身朝反方向旋轉。若四個螺旋槳都同向轉，反扭矩會累加，機身會持續自轉。

採用正反槳配置後，四個反扭矩兩兩抵消。而**偏航控制正是刻意打破這個平衡**——讓一對槳加速、另一對減速，總推力不變但反扭矩不再平衡，機身就會繞垂直軸旋轉。

$$\text{偏航力矩 } \tau_{\psi} \propto (\omega_1^2 + \omega_3^2) - (\omega_2^2 + \omega_4^2)$$

偏航由「兩組對角槳的轉速差」產生。這是四旋翼最弱的一個控制通道——因為反扭矩遠小於推力。

**實務意義**：偏航的控制權限（authority）遠低於滾轉與俯仰，因此無人機**轉向較慢**，且在強風中偏航容易被吹偏。這是四旋翼的固有限制。

### 27.1.3 各種飛行構型的比較

表 27-2 飛行機器人構型的比較



| 構型       | 續航          | 載重  | 機動性 | 定點懸停 | 機構複雜度 | 典型應用          |
|----------|-------------|-----|-----|------|-------|---------------|
| 四旋翼      | 短 (15~40 分) | 小~中 | 極佳  | 可    | 極低    | 航拍、巡檢、教學      |
| 六 / 八旋翼  | 短           | 中~大 | 佳   | 可    | 低     | 農噴、專業航拍 (有冗餘) |
| 傳統直升機    | 中           | 大   | 佳   | 可    | 極高    | 載人、大載重        |
| 固定翼      | 長 (數小時)     | 中   | 差   | 不可   | 低     | 測繪、長程巡邏       |
| VTOL 混合翼 | 長           | 中   | 中   | 可    | 高     | 長程且需定點起降      |
| 撲翼 / 仿生  | 短           | 極小  | 特殊  | 部分可  | 高     | 研究、微型偵察       |

選型的核心取舍是「續航」與「懸停能力」。固定翼靠機翼產生升力，效率極高（升阻比可達 10 以上），但必須持續前進；旋翼機把全部能量用於對抗重力，效率低但能停在空中。VTOL 試圖兼得，代價是機構複雜與重量增加。

## 27.2 四旋翼的動力學

### 27.2.1 由轉速到推力與力矩

每個螺旋槳產生的推力與反扭矩，都近似正比於轉速的平方：

$$F_i = k_f \cdot \omega_i^2 \quad M_i = k_m \cdot \omega_i^2$$

$k_f$  為推力係數、 $k_m$  為扭矩係數。兩者的比值  $k_m/k_f$  是一個重要的機構參數，決定了偏航的控制權限。

四個螺旋槳的效果合成為四個控制量。設機臂長度為  $L$ （機心到槳心），採用「十」字配置：

$$\begin{bmatrix} F \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & -L \cdot k_f & 0 & L \cdot k_f \\ L \cdot k_f & 0 & -L \cdot k_f & 0 \\ k_m & -k_m & k_m & -k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

分配矩陣 (mixer)。由四個轉速平方，映射到總推力與三軸力矩。

控制器實際輸出的是  $(F, \tau_\phi, \tau_\theta, \tau_\psi)$ ，再透過這個矩陣的**反矩陣**求出各馬達應有的轉速——這一步稱為**控制分配** (control allocation)。

#### ◎ 飽和與分配的優先順序

實務上有一個重要的細節：當控制器要求的推力與力矩超出馬達能力時，該犧牲哪一個？標準的答

案是**優先保證姿態力矩**。原因是：推力不足只會使無人機緩慢下降（還有時間補救），但姿態失控會立即翻覆墜毀。因此飛控的分配演算法會先滿足  $\tau_\phi$ 、 $\tau_\theta$ 、 $\tau_\psi$ ，再把剩餘的餘裕給總推力  $F$ 。這是「先活著，再談去哪裡」的工程原則，與第 20 章「安全優先於精度」的思維一致。

### 27.2.2 簡化的動力學模型

四旋翼的完整動力學相當複雜，但有一個標準的簡化模型，在多數情況下已足夠準確。

**平移動力學**（在世界座標系中）：

$$m \cdot \mathbf{a} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + R \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ m \cdot g \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix}$$

推力永遠沿機體的  $Z$  軸，經旋轉矩陣  $R$  轉換到世界座標系，再減去重力。

展開後可以清楚看見欠驅動的本質：

$$\begin{aligned} m \cdot \ddot{x} &= F \cdot (\cos \phi \sin \theta \cos \psi + \sin \phi \sin \psi) \\ m \cdot \ddot{y} &= F \cdot (\cos \phi \sin \theta \sin \psi - \sin \phi \cos \psi) \\ m \cdot \ddot{z} &= F \cdot \cos \phi \cos \theta - m \cdot g \end{aligned}$$

水平方向的加速度，完全由姿態角決定。想往前飛，就必須俯仰（低頭）。

**轉動動力學**（在機體座標系中）——這正是第 14.1.1 節的尤拉方程：

$$I \cdot \dot{\omega} + \omega \times (I \cdot \omega) = \tau$$

$I$  為慣性張量、 $\omega$  為機體角速度、 $\tau$  為控制力矩。第二項是陀螺項，在高速旋轉時不可忽略。

### 27.2.3 懸停條件與線性化

懸停是最重要的平衡點。此時所有速度與角速度為零、姿態水平：

$$F_{\text{hover}} = m \cdot g \quad \phi = \theta = 0$$

懸停條件。總推力恰好等於重量。

在懸停點附近做線性化（小角度近似： $\sin \phi \approx \phi$ 、 $\cos \phi \approx 1$ ），得到極為簡潔的結果：

$$\ddot{x} \approx g \cdot \theta \quad \ddot{y} \approx -g \cdot \phi \quad \ddot{z} \approx (F - mg)/m$$

線性化的平移動力學。俯仰角  $\theta$  直接對應  $X$  方向的加速度、滾轉角  $\phi$  對應  $Y$  方向（負號）。

這三條式子是整個無人機控制的基礎。它們揭示了一個關鍵的結構：

**位置控制器輸出「期望加速度」→ 除以  $g$  → 得到「期望姿態角」**

這就是外迴路與內迴路的介面。位置控制不直接控制馬達，而是產生姿態指令交給下層。

## 27.3 欠驅動與微分平坦性

### 27.3.1 欠驅動的精確意義

四旋翼有 6 個自由度 ( $x, y, z, \varphi, \theta, \psi$ ) 但只有 4 個控制輸入 ( $F, \tau_\varphi, \tau_\theta, \tau_\psi$ )。這是典型的欠驅動系統。

但哪四個量可以獨立控制？答案是：位置 ( $x, y, z$ ) 加上偏航角  $\psi$ 。

為什麼是這四個？因為滾轉與俯仰已經被位置的需求佔用了——要往某個方向加速，就必須傾斜到對應的角度。 $\varphi$  與  $\theta$  不是自由的變數，而是位置控制的副產品。

|                          |                          |
|--------------------------|--------------------------|
| 可獨立控制： $(x, y, z, \psi)$ | 被決定： $(\varphi, \theta)$ |
|--------------------------|--------------------------|

四旋翼的控制結構。這也解釋了為何開頭故事中「往右平移但機身不傾斜」是做不到的。

### 27.3.2 微分平坦性：一個極為優美的性質

四旋翼有一個令人驚訝的數學性質，稱為微分平坦 (differential flatness)。它由 Mellinger 與 Kumar 於 2011 年提出，徹底改變了無人機的軌跡規劃。

#### ◎ 微分平坦性

若一個系統的所有狀態與所有控制輸入，都能由少數幾個「平坦輸出」(flat outputs) 及其有限階導數代數地表示出來，則稱該系統是微分平坦的。對四旋翼而言，平坦輸出恰好就是 ( $x, y, z, \psi$ )——也就是那四個可獨立控制的量。

這個性質的實用價值極大。它意味著：

|                                                                                    |
|------------------------------------------------------------------------------------|
| 給定一條 $(x(t), y(t), z(t), \psi(t))$ 的軌跡<br>$\Rightarrow$ 所需的姿態角、角速度、推力、力矩，全部可以直接算出來 |
|------------------------------------------------------------------------------------|

不需要任何積分或求解微分方程。

具體的推導鏈是：

1. 由位置軌跡的二階導數 (加速度)，可算出所需的推力向量 (大小與方向)。
2. 推力方向即機體 Z 軸方向；結合期望的偏航角  $\psi$ ，即可完全確定姿態 (旋轉矩陣  $R$ )。
3. 由位置的三階導數 (急跳度 jerk)，可算出角速度。
4. 由位置的四階導數 (snap)，可算出角加速度，進而由尤拉方程得到所需的力矩。

### 27.3.3 最小 snap 軌跡

上述推導揭示了一個重要的結論：力矩正比於位置的四階導數（snap）。

由於力矩由馬達轉速差產生，而轉速差的變化受限於馬達的響應速度，因此最小化 snap 就等於最小化對馬達的要求。這導出了無人機軌跡規劃的標準作法：

$$\min \int ||d^4r/dt^4||^2 dt \quad (\text{最小 snap 軌跡})$$

在通過指定航點的約束下，最小化 snap 的平方積分。解為分段的七次多項式。

這與第 15 章的軌跡生成一脈相承——那裡最小化的是急跳度（三階導數，因為機械手臂的力矩正比於加速度），這裡因為多了一層（推力方向由姿態決定，而姿態又需要力矩來改變），所以要到四階。

最小 snap 軌跡是目前競速無人機與專業航拍的標準方法。它產生的軌跡極為平滑，能讓無人機以極高的速度通過複雜的路徑而不失控。

## 27.4 串級控制架構

### 27.4.1 為何必須分層

開頭故事中叫獸提到的「分層」，是無人機控制的核心架構。它的理論依據是時間尺度分離（time-scale separation）。

比較兩個迴路的動態：

表 27-3 姿態與位置動態的時間尺度

| 面向   | 姿態迴路             | 位置迴路                 |
|------|------------------|----------------------|
| 驅動方式 | 力矩直接驅動角加速度       | 推力 → 傾斜 → 水平分力 → 加速度 |
| 等效慣量 | 小（機體轉動慣量）        | 大（整機質量）              |
| 積分層數 | 2（力矩 → 角速度 → 角度） | 4（含姿態迴路在內）           |
| 時間常數 | 數十毫秒             | 數百毫秒~數秒              |
| 控制頻率 | 500~1000 Hz      | 20~100 Hz            |
| 感測器  | IMU（陀螺儀為主）       | GNSS、氣壓計、視覺、光流       |

時間常數相差十倍以上，這正是分層的依據。當外迴路（位置）發出一個姿態指令時，內迴路（姿態）已經快到可以視為「瞬間達成」——這使得外迴路的設計可以完全忽略內迴路的動態，

大幅簡化問題。

### ◎ 時間尺度分離的普遍性

這個「快的內迴路 + 慢的外迴路」架構，在本書中已經出現多次：第 5 章伺服驅動器的電流迴路（10 kHz）套在速度迴路（1 kHz）內；第 17 章的計算力矩控制把非線性抵消放在內層、線性 PD 放在外層；第 3 章的時間尺度金字塔更是把整個機器人系統依頻率分層。共同的原則是：讓每一層只處理自己時間尺度上的問題，並把下層視為理想的執行器。這是複雜系統設計最重要的技巧之一。

## 27.4.2 四層架構

一個完整的無人機控制系統包含四層，由外而內：

1. 軌跡規劃層（1~10 Hz）：產生位置與偏航的參考軌跡（第 26 章的路徑規劃 + 27.3.3 節的最小 snap）。
2. 位置控制層（20~100 Hz）：比較實際位置與參考軌跡，輸出期望加速度，再依 27.2.3 節的關係轉換為期望姿態角與期望總推力。
3. 姿態控制層（250~500 Hz）：比較實際姿態與期望姿態，輸出期望角速度。
4. 角速度控制層（500~1000 Hz）：比較實際角速度與期望角速度，輸出力矩指令，最後經分配矩陣轉為四個馬達的轉速。

注意姿態控制本身又分成兩層（姿態角與角速度）。這是因為角速度可以由陀螺儀直接量測（高精度、低延遲），而姿態角必須經過融合估計（第 24 章）才能得到，精度與延遲都較差。把最內層建立在最可靠的感測上，是這個設計的關鍵。

圖 27-1 無人機的串級控制架構

建議配圖：由左至右的方塊圖，四個巢狀的控制迴路。最外層「軌跡規劃」（10 Hz）輸出位置參考；「位置控制」（50 Hz）輸出期望加速度，經過一個標註「 $a \rightarrow \text{姿態角}$ 」的轉換方塊（含公式  $\theta_d = a_x/g$ 、 $\varphi_d = -a_y/g$ ）；「姿態控制」（500 Hz）輸出期望角速度；「角速度控制」（1000 Hz）輸出力矩；最後經「分配矩陣」轉為四個馬達轉速。各層的回授來源以不同顏色標註：角速度來自陀螺儀（紅）、姿態來自 IMU 融合（橙）、位置來自 GNSS/VIO（藍）。右下角標註各層的更新頻率與時間常數對照。

## 27.4.3 角速度內迴路的關鍵性

最內層的角速度控制（俗稱 rate loop）是整個系統中最關鍵的一環，理由有三：

理由一：它決定了系統的極限性能。所有上層的指令最終都必須透過它執行。若 rate loop 不夠快或不夠準，上層再好的規劃也無法實現。

**理由二：它是唯一能直接對抗擾動的層級。**陣風打到機身時，最先反應的就是角速度的變化——rate loop 能在數毫秒內偵測並修正，遠快於姿態或位置層。

**理由三：它的調參最為敏感。**競速無人機玩家花最多時間調的就是 rate loop 的 PID 參數。增益太低則反應遲鈍、太高則產生高頻振盪（並發出特有的嗡嗡聲，稱為 motor noise）。

實務上 rate loop 幾乎都採用 PID，並在微分項前加上低通濾波（因為陀螺儀的雜訊在微分後會被嚴重放大——這正是第 16.7 節提到的問題）。

## 27.5 姿態控制

### 27.5.1 為何不能用歐拉角

姿態控制的第一個關鍵決定是：**用什麼表示姿態**。第 8 章已經給出答案——**內部計算必須用四元數或旋轉矩陣，絕不能用歐拉角**。

對無人機而言，這個選擇尤其關鍵，因為：

- **萬向節鎖是真實的威脅：**競速無人機做出翻滾動作時，俯仰角會經過  $\pm 90^\circ$ ——歐拉角在此處會發散（第 8.3.3 節）。
- **大角度誤差的計算會出錯：**若期望姿態與實際姿態相差  $170^\circ$ ，用歐拉角相減可能得到錯誤的旋轉方向（繞遠路）。
- **內插不平滑：**軌跡追隨時需要在姿態之間內插，歐拉角的線性內插會產生扭曲的路徑。

因此標準作法是：**內部用四元數儲存與計算、只在人機介面上轉換為歐拉角顯示**。這正是第 8.5.4 節的結論在實務上的體現。

### 27.5.2 姿態誤差的計算

設當前姿態為四元數  $q$ 、期望姿態為  $q_d$ 。姿態誤差不是相減，而是**相對旋轉**：

$$q_e = q^* \otimes q_d$$

$q^*$  為  $q$  的共軛（即反向旋轉）。 $q_e$  代表「由當前姿態轉到期望姿態所需的旋轉」。

由  $q_e$  取出誤差向量（用於回授控制）：

$$e = 2 \cdot \text{sign}(q_e, w) \cdot [q_e, x, q_e, y, q_e, z]^T$$

取向量部乘以 2（因為四元數用半角，第 8.5.1 節）。 $\text{sign}$  項處理雙重覆蓋——確保永遠選擇較短的旋轉路徑。

那個 sign 函數極為重要。若省略它，當誤差超過  $180^\circ$  時，控制器會選擇繞遠路的方向——無人機會朝錯誤的方向翻轉。這是實務上一個經典的除錯陷阱。

### 27.5.3 串級的姿態控制器

姿態控制採用兩層串級（第 27.4.2 節）：

外層（姿態角，250~500 Hz）：通常只用比例控制（P），輸出期望角速度。

$$\omega_d = K_{p,att} \cdot e_{att}$$

姿態外層。不需要微分項，因為下一層本身就是速度控制。

內層（角速度，500~1000 Hz）：採用完整的 PID。

$$\tau = K_{p,rate} \cdot (\omega_d - \omega) + K_{i,rate} \cdot \int (\omega_d - \omega) dt + K_{d,rate} \cdot d(\omega_d - \omega)/dt$$

角速度內層。這是整個無人機最關鍵的控制器（第 27.4.3 節）。

三個項的作用在無人機上有具體的意義：

表 27-4 角速度 PID 各項在無人機上的作用

| 項     | 作用              | 調太低的症狀      | 調太高的症狀       |
|-------|-----------------|-------------|--------------|
| P（比例） | 主要的響應力道         | 反應遲鈍、追不上指令  | 高頻振盪、馬達嗡嗡聲   |
| I（積分） | 消除穩態誤差（如重心偏移、風） | 持續偏航、無法保持角度 | 積分飽和、大動作後過衝  |
| D（微分） | 阻尼、抑制超越         | 超越後來回擺盪     | 放大陀螺儀雜訊、機身抖動 |

微分項的雜訊問題在無人機上特別嚴重，因為馬達與螺旋槳產生大量的機械振動，直接傳到陀螺儀上。因此實務上必定會在 D 項前加低通濾波器，且截止頻率的調整是飛控調參的重點之一。

### 27.5.4 為何加入前饋

純回授的姿態控制有一個固有的延遲——它必須等誤差出現才能反應。對於快速的軌跡追隨，這個延遲會造成明顯的落後。

因應之道與第 17 章相同：加入前饋。由 27.3.2 節的微分平坦性，期望的角速度與角加速度都可以由軌跡直接算出：



$$\tau = \tau_{ff} + \tau_{fb} \quad , \text{ 其中 } \tau_{ff} = I \cdot \dot{\omega}_d + \omega_d \times (I \cdot \omega_d)$$

前饋項來自尤拉方程 ( 第 14.1.1 節 ) , 直接算出「執行這個期望運動所需的力矩」。

這正是第 17 章「用模型處理已知的部分、用回授處理未知的部分」哲學的又一次體現。在競速無人機與高速航拍中，前饋能顯著改善軌跡追隨的精度。

## 27.6 位置控制

### 27.6.1 由位置誤差到姿態指令

位置控制的核心，是把「該往哪裡去」翻譯成「該傾斜多少」。其流程分三步。

步驟一：計算期望加速度。用 PID ( 或 LQR ) 對位置誤差產生加速度指令：

$$a_d = K_p(r_d - r) + K_d(\dot{r}_d - \dot{r}) + K_i \int (r_d - r) dt + \ddot{r}_d$$

最後一項  $\ddot{r}_d$  是軌跡的前饋加速度——這使控制器能提前反應而非事後追趕。

步驟二：計算所需的推力向量。加速度加上重力補償：

$$F_{vec} = m \cdot (a_d + g \cdot \hat{z})$$

所需的推力向量 ( 在世界座標系中 ) 。注意必須加上抵抗重力的分量。

步驟三：由推力向量反推姿態與推力大小。

$$F = \|F_{vec}\| \quad ( \text{總推力大小} )$$

$$\hat{z}_{body} = F_{vec} / \|F_{vec}\| \quad ( \text{機體 } z \text{ 軸應指的方向} )$$

這是四旋翼控制最核心的一步：推力永遠沿機體  $z$  軸，因此「推力向量的方向」就直接決定了「機體該朝哪個方向」。

有了  $\hat{z}_{body}$  與期望的偏航角  $\psi_d$ ，即可完全確定期望的姿態矩陣  $R_d$  ( 第 8 章的方法：先由  $\psi_d$  定出參考的  $X$  方向，再用外積建構正交基底 )。

### 27.6.2 小角度的簡化形式

在小角度 ( 一般飛行，傾角小於  $20^\circ$  ) 的情況下，上述流程可以簡化為第 27.2.3 節的線性關係：

$$\theta_d = a_x / g \quad ( \text{俯仰角，控制前後} )$$

$$\phi_d = -a_y / g \quad ( \text{滾轉角，控制左右} )$$

$$F = m \cdot (g + a_z) \quad ( \text{總推力，控制上下} )$$



小角度近似。實作極為簡單，是入門級飛控的標準作法。

這個近似的誤差在傾角  $20^\circ$  時約為 2%、 $30^\circ$  時約為 5%。對一般飛行完全足夠，但在高機動飛行（傾角超過  $45^\circ$ ）時必須使用 27.6.1 節的完整向量形式。

### 27.6.3 高度控制的特殊性

高度（Z 軸）的控制與水平方向有兩個重要的差異：

**差異一：它是直接驅動的。**不需要透過姿態——增加總推力就能上升。因此高度控制的頻寬可以比水平方向高得多。

**差異二：重力補償是必須的。**懸停時推力必須恰好等於  $mg$ （第 27.2.3 節）。因此控制律中必定包含一個常數的前饋項：

$$F = m \cdot g + m \cdot a_{z,desired} + \text{PID 修正}$$

重力前饋。若沒有這一項，控制器必須靠積分項慢慢累積出  $mg$  這麼大的推力——起飛時會嚴重下沉，這是第 17.5 節重力補償概念的直接應用。

**電池電壓下降的影響：**隨著電池放電，同樣的 PWM 指令產生的推力會下降。若不補償，無人機會在飛行後期逐漸下沉。專業飛控會量測電池電壓並做即時補償——這是一個典型的「已知擾動用前饋處理」的例子。

### 27.6.4 外部擾動

無人機面對三種主要的外部擾動：

- **風：**最主要的擾動。持續的風會被積分項補償（表現為機身持續傾斜對抗風力），但陣風則會造成瞬間的位置偏移。
- **地面效應：**接近地面（約一個槳徑的高度內）時，螺旋槳下洗氣流被地面反射，使升力增加 5~20%。這使無人機在降落時「浮起來」而難以觸地，也使起飛時的推力估計不準。
- **渦環狀態（Vortex Ring State）：**快速垂直下降時，螺旋槳會落入自己產生的下洗氣流中，導致升力急遽下降甚至失控。因應之道是降落時保持一定的水平速度，避免純垂直的快速下降。這是無人機失事的常見原因之一。

## 27.7 狀態估測

### 27.7.1 需要估測什麼

無人機的控制需要六個量：**位置 (x, y, z)**、**速度**、**姿態**、**角速度**。其中只有角速度可以直接量測（陀螺儀），其餘都必須估測。

這正是第 24 章的完整應用場合。各感測器的角色分工如下：

**表 27-5 無人機的感測器配置**

| 感測器        | 提供的資訊     | 更新率       | 特性          | 失效情境       |
|------------|-----------|-----------|-------------|------------|
| 陀螺儀        | 角速度       | 1~8 kHz   | 極準但會漂移      | 長時間積分      |
| 加速度計       | 重力方向（→姿態） | 1~8 kHz   | 長期準、雜訊大     | 機動飛行時受干擾   |
| 磁力計        | 偏航角（絕對）   | 50~100 Hz | 不漂移         | 受鐵件與馬達電流干擾 |
| 氣壓計        | 相對高度      | 20~50 Hz  | 解析度約 10 cm  | 受氣流與溫度影響   |
| GNSS       | 絕對位置      | 1~10 Hz   | 公尺級精度       | 室內、樹下、峽谷失效 |
| 光流相機       | 水平速度      | 50~100 Hz | 需要紋理與光線     | 白牆、暗處、高空   |
| 測距（雷射/超音波） | 對地高度      | 20~50 Hz  | 精度高         | 有效距離短（數公尺） |
| 視覺 VIO     | 位姿（六自由度）  | 20~50 Hz  | 精度高、不需 GNSS | 快速運動、低紋理   |

### 27.7.2 姿態估測：互補濾波的經典應用

姿態估測是第 24.8 節互補濾波的最經典應用場景。三個感測器的互補關係極為漂亮：

- **陀螺儀**：積分角速度得到姿態。短期極準，但**會漂移**（每分鐘可達數度）。
- **加速度計**：靜止時量到的就是重力方向，可推算滾轉與俯仰。**不漂移**，但飛行中的加速度會污染讀值。
- **磁力計**：量測地磁方向，提供**偏航角**的絕對參考。加速度計無法提供偏航（因為繞垂直軸旋轉不改變重力方向）。

滾轉、俯仰：陀螺儀（高通）+ 加速度計（低通）

偏航：陀螺儀（高通）+ 磁力計（低通）

互補濾波的分工。這正是第 24.8.2 節「頻率響應互補」的具體實現。

**磁力計的校正極為關鍵。**機身上的鐵件、馬達的電流、甚至電池的位置都會產生磁場干擾。因此每台無人機在首次使用前都必須做**羅盤校正**——那個「拿著無人機轉三圈」的儀式，其實是在收集不同姿態下的磁場讀值，以估算並補償硬鐵與軟鐵的干擾。

### 27.7.3 位置估測的三種情境

位置估測的方案完全取決於環境：

**情境一：戶外開闊（GNSS 可用）。**標準方案是 GNSS + IMU 的 EKF 融合。GNSS 提供不漂移的絕對位置（1~10 Hz）、IMU 提供高頻的平滑預測（1 kHz）——這正是第 24.10.1 節的融合邏輯。典型精度可達 1~2 公尺（一般 GNSS）或數公分（RTK）。

**情境二：室內或 GNSS 拒止。**必須改用視覺方案：

- **光流 + 測距：**最輕量的方案。光流相機量測水平速度（透過影像的移動）、測距感測器量測高度。只能估測速度而非絕對位置，因此仍會緩慢漂移，但足以維持懸停穩定。這是消費級無人機室內定高懸停的標準配置。
- **VIO（視覺慣性里程計）：**完整的六自由度位姿估計（第 25.9.2 節）。精度高但計算量大，且仍會漂移（沒有迴路閉合的話）。
- **VSLAM：**加入迴路閉合，可消除長期漂移（第 25 章）。計算量最大。

**情境三：外部定位系統。**在實驗室或表演場地，可用動作捕捉系統（如 OptiTrack）或 UWB 提供公釐級的絕對位置。這使研究者能專注在控制演算法而不受估測誤差干擾——多數無人機特技表演的影片，都是在這種環境下拍攝的。

#### ◎ 為何室內飛行比室外困難

直覺上室內沒有風、空間又小，應該比較容易。但實際上**室內飛行的技術門檻高得多**，關鍵就在於定位。室外有 GNSS 這個「免費的絕對參照」，而室內完全沒有——必須靠視覺自己建立參照，而視覺會受光線、紋理、運動模糊影響。這也解釋了一個常見的現象：同一台無人機在室外能穩穩懸停，進到室內卻會緩慢地飄移——因為它失去了絕對位置的參照，只能靠光流估測速度，而速度的積分必然漂移（第 23.8 節的宿命）。

## 27.8 安全與失效保護

### 27.8.1 為何安全設計特別關鍵

無人機的安全問題與地面機器人有本質的差異：它沒有「安全停止」這個選項。第 7.9 節的緊急停止在無人機上是無效的——切斷動力等於墜落。

因此無人機的安全設計必須建立在不同的原則上：不是「停止」，而是「降級」——在部分能力失效時，仍能維持最低限度的可控性，並安全著陸。

## 27.8.2 常見的失效模式

表 27-6 無人機的失效模式與因應

| 失效      | 症狀        | 後果                        | 因應機制               |
|---------|-----------|---------------------------|--------------------|
| 電池電量不足  | 電壓下降、推力衰減 | 墜落                        | 低電量自動返航、強制降落       |
| GNSS 失鎖 | 位置估計跳動或消失 | 位置失控、飄移                   | 切換至姿態模式、維持高度       |
| 磁力計干擾   | 偏航角錯誤     | 朝錯誤方向飛 ( toilet bowl 現象 ) | 偵測不一致、改用 GPS 航向    |
| 馬達或槳失效  | 推力不平衡     | 翻覆墜落                      | 六旋翼可容錯；四旋翼可犧牲偏航保平衡 |
| 通訊中斷    | 失去遙控訊號    | 失控飛離                      | 自動返航 ( RTH )       |
| IMU 故障  | 姿態估計錯誤    | 立即失控                      | 多 IMU 冗餘與表決        |
| 渦環狀態    | 垂直下降時失去升力 | 急速墜落                      | 降落時保持水平速度          |

## 27.8.3 分級降級的設計

專業飛控採用分級的飛行模式，能力由高到低依序降級：

1. **位置模式 ( Position Hold )**：完整的位置控制，鬆手即懸停。需要 GNSS 或視覺定位。
2. **高度模式 ( Altitude Hold )**：只維持高度，水平方向由操作員控制。GNSS 失效時降級到此。
3. **姿態模式 ( Attitude / Stabilize )**：只穩定姿態，高度與位置全由操作員控制。氣壓計失效時降級到此。
4. **手動模式 ( Acro / Rate )**：只控制角速度，連姿態都不穩定。這是競速飛行的模式，也是最後的保底——只要陀螺儀還在運作就能飛。

這個降級序列的設計邏輯是「依感測器的可靠度排序」：陀螺儀最可靠（純內部感測）、其次是氣壓計、再來是 GNSS 或視覺（依賴外部環境）。每失去一個感測來源，就降到對應的模式，而非直接失控。

這與第 7 章「失效導向安全」的原則一脈相承，但形式不同——地面機器人的安全狀態是「停止」，無人機的安全狀態是「維持最低限度的可控性」。

#### 27.8.4 法規與責任

無人機是少數受到嚴格法規管制的機器人。台灣的《遙控無人機管理規則》（2020 年起施行）規定了：

- **註冊與檢驗**：250 公克以上須註冊，特定重量以上須檢驗。
- **操作證照**：超過 2 公斤或從事特定活動須考取操作證。
- **飛行限制**：最大高度 400 英尺（約 122 公尺）、不得超出目視範圍、禁飛區（機場、政府機關等）。
- **保險**：商業用途須投保責任險。

**工程上的意義**：這些法規直接影響設計。例如「不得超出目視範圍」使長程自主飛行在多數情況下不合法；「400 英尺高度限制」則限制了通訊與測繪的規劃。在開發任何無人機應用之前，必須先確認法規的可行性——這與第 35 章討論的機器人倫理與社會接受度密切相關。

### 27.9 自主飛行與應用

#### 27.9.1 由遙控到自主的三個層次

無人機的自主程度可分為三個層次，對應第 1 章討論的自主性光譜：

**層次一：穩定輔助**。操作員下達速度或姿態指令，飛控負責穩定。這是消費級無人機的基本模式——人決定去哪裡，機器負責不摔。

**層次二：航點自主**。事先規劃一連串 GPS 航點，無人機自動飛完全程。這是測繪、農噴、巡檢的標準模式。路徑是人規劃的，執行是自主的。

**層次三：任務自主**。給定目標（如「檢查這棟建築的所有窗戶」），無人機自行規劃路徑、避障、並決定何時完成。這需要完整的感知—規劃—控制鏈，是目前的研究前沿。

## 27.9.2 典型應用與其技術需求

表 27-7 無人機應用與技術需求

| 應用     | 核心需求       | 關鍵技術               | 難點              |
|--------|------------|--------------------|-----------------|
| 航拍攝影   | 平穩、精確的軌跡   | 最小 snap 軌跡、雲台穩定    | 風中的穩定性          |
| 測繪     | 精確定位、規律覆蓋  | RTK-GNSS、航線規劃      | 影像重疊率與後處理       |
| 農業噴灑   | 低空穩定、大載重   | 地形跟隨、流量控制          | 藥液晃動、下洗氣流       |
| 基礎設施巡檢 | 接近結構物、細節拍攝 | 視覺定位、避障            | GNSS 遮蔽、金屬干擾磁力計 |
| 物流配送   | 長程、精確降落    | VTOL、視覺標記降落        | 法規、天候、安全        |
| 搜救     | 室內外皆可、自主探索 | SLAM、自主探索          | 無 GNSS、環境未知     |
| 競速     | 極限機動       | 高頻 rate loop、低延遲影傳 | 飛行員技術、機體強度      |

## 27.9.3 無人機叢集

多架無人機的協同 (swarm) 是近年的熱點，其技術挑戰有三：

**挑戰一：相對定位。**叢集中的每一架都必須知道其他架的位置。GNSS 的公尺級精度不足以避免碰撞，因此需要 RTK (公分級) 或機間的相對測距 (UWB)。

**挑戰二：通訊與同步。**數十架無人機的狀態需要即時共享，且動作必須同步。這是一個典型的分散式系統問題。

**挑戰三：分散式決策。**集中式控制 (一台電腦指揮全部) 簡單但脆弱 (單點失效)；分散式 (各自決策) 強健但難以保證整體行為。實務上多採混合式——集中規劃、分散執行。

**表演型叢集** (如燈光秀) 與 **任務型叢集** (如搜救) 的差異很大：前者的軌跡完全預先規劃、環境已知、且可用外部定位；後者則需要即時的感知與決策。目前商業化成熟的主要是前者。

## 27.10 由飛行邁向行走

### 27.10.1 本章的成果

本章完成了第一個「全書技術綜合」的案例。你現在能夠：分析四旋翼的推力與力矩產生機制、推導其動力學並辨識欠驅動的本質、理解微分平坦性及其在軌跡規劃上的價值、設計串級的姿態與位置控制器、選擇合適的狀態估測方案、並設計分級降級的安全機制。

更重要的是，你看到了前面各章的理論如何在一個真實系統中**同時**發揮作用：

表 27-8 本章對前面各章的整合

| 技術         | 來源章節      | 在無人機上的角色          |
|------------|-----------|-------------------|
| 尤拉方程與慣性張量  | 第 13、14 章 | 轉動動力學、前饋力矩計算      |
| 軌跡生成與急跳度   | 第 15 章    | 最小 snap 軌跡（推廣到四階） |
| PID 與微分項濾波 | 第 16 章    | 角速度內迴路            |
| 前饋加回授      | 第 17 章    | 軌跡追隨的精度提升         |
| LQR        | 第 18 章    | 姿態與位置控制器的最佳增益     |
| 欠驅動與穩定性    | 第 19 章    | 本質不穩定系統的分析        |
| 四元數與萬向節鎖   | 第 8 章     | 姿態表示與誤差計算         |
| 互補濾波與 EKF  | 第 24 章    | 姿態與位置的狀態估測        |
| VIO 與 SLAM | 第 25 章    | GNSS 拒止環境下的定位     |
| 路徑規劃       | 第 26 章    | 航點規劃與避障           |

### 27.10.2 接下來：更困難的欠驅動

無人機雖然欠驅動且不穩定，但它有一個關鍵的簡化：**它的動態是連續的**。螺旋槳持續轉動、推力連續變化、沒有任何突變。

**第 28 章的腿式機器人**則面對一個更困難的問題：**接觸狀態不斷切換**。腳著地時，機器人是**一個受約束的多體系統**；腳離地時，它是一個自由飛行的物體。每一步都是一次**混合動態**（hybrid dynamics）的切換。

此外，腿式機器人還有三個無人機沒有的困難：

- **自由度極高**：四足機器人 12~18 個自由度、人形機器人可達 30 個以上，遠超四旋翼的 4 個輸入。
- **接觸力的不確定性**：地面的摩擦係數、軟硬程度都未知且會變化（第 14.7 節與第 21.2.3 節的困難再次出現）。
- **騰空階段完全無法控制質心**：一旦雙腳離地，質心的軌跡就由拋物線決定，只能靠落地點的選擇來修正——這是一種特殊的欠驅動。

但兩者也有深刻的共同點：**都是欠驅動、都本質不穩定、都必須持續控制、也都採用分層架構**。理解了無人機，第 28 章會容易許多。



換句話說：第 27 章讓機器人離開地面，第 28 章讓它用腿走回地面——而後者，出乎意料地更難。

## 完整計算例題

### 例題 27-1 四旋翼的推力與分配

題目：某四旋翼質量  $m = 1.2 \text{ kg}$ ，機臂長  $L = 0.25 \text{ m}$ ，推力係數  $k_f = 1.5 \times 10^{-5} \text{ N} \cdot \text{s}^2$ 、扭矩係數  $k_m = 2.4 \times 10^{-7} \text{ N} \cdot \text{m} \cdot \text{s}^2$ 。(a) 求懸停時所需的總推力與每顆馬達的轉速。(b) 若要產生  $0.15 \text{ N} \cdot \text{m}$  的滾轉力矩，求四顆馬達的轉速調整量。(c) 求偏航與滾轉的控制權限比。(d) 若馬達最高轉速為  $1200 \text{ rad/s}$ ，求最大可用的推力與推重比。

解 (a)：懸停時總推力等於重量：

$$F_{\text{hover}} = m \cdot g = 1.2 \times 9.8 = 11.76 \text{ N}$$

總推力。

四顆馬達平均分擔，每顆  $F_1 = 11.76/4 = 2.94 \text{ N}$ 。由  $F = k_f \cdot \omega^2$ ：

$$\omega = \sqrt{(F/k_f)} = \sqrt{(2.94/1.5e-5)} = \sqrt{196000} = 442.7 \text{ rad/s}$$

每顆馬達的懸停轉速 (約 4228 RPM)。

解 (b)：採「十」字配置，滾轉力矩由左右兩顆馬達的推力差產生 (第 27.2.1 節的分配矩陣)：

$$\tau_{\phi} = L \cdot (F_4 - F_2) = 0.15 \text{ N} \cdot \text{m} \Rightarrow F_4 - F_2 = 0.15/0.25 = 0.6 \text{ N}$$

左右馬達的推力差。

為維持總推力不變，一顆增加  $0.3 \text{ N}$ 、另一顆減少  $0.3 \text{ N}$ ：

$$\begin{aligned} F_4 &= 2.94 + 0.3 = 3.24 \text{ N} \Rightarrow \omega_4 = \sqrt{(3.24/1.5e-5)} = 464.8 \text{ rad/s} \\ F_2 &= 2.94 - 0.3 = 2.64 \text{ N} \Rightarrow \omega_2 = \sqrt{(2.64/1.5e-5)} = 419.5 \text{ rad/s} \end{aligned}$$

調整後的轉速。相差約  $45 \text{ rad/s}$  (約 10%)。

解 (c)：比較兩種力矩在相同轉速差下的大小。

滾轉力矩：由推力差乘力臂產生： $\tau_{\phi} = L \cdot \Delta F = L \cdot k_f \cdot \Delta(\omega^2)$ 。

偏航力矩：由反扭矩差產生： $\tau_{\psi} = k_m \cdot \Delta(\omega^2)$ 。



$$\tau_{\varphi}/\tau_{\psi} = (L \cdot k_f)/k_m = (0.25 \times 1.5e-5)/2.4e-7 = 3.75e-6/2.4e-7 = 15.6$$

滾轉的控制權限是偏航的 15.6 倍。

**工程意義：**這解釋了第 27.1.2 節提到的現象——四旋翼的偏航反應明顯較慢、且在強風中容易被吹偏。若要改善，可以加長機臂（增大  $L$ ，但會增加慣量）或選用扭矩係數較大的螺旋槳（但通常效率較差）。

解（d）：最大推力：

$$F_{\max} = 4 \times k_f \times \omega_{\max}^2 = 4 \times 1.5e-5 \times 1200^2 = 4 \times 1.5e-5 \times 1.44e6 = 86.4 \text{ N}$$

四顆馬達全速的總推力。

$$\text{推重比} = F_{\max}/(m \cdot g) = 86.4/11.76 = 7.35$$

推重比約 7.3。

**判讀：**推重比是無人機最重要的性能指標之一。

- 小於 1.5：無法起飛或極為勉強。
- 2 ~ 3：一般航拍機，穩定但機動性有限。
- 4 ~ 6：運動型，能做較大的機動動作。
- 7 以上：競速機，能做極限特技。

本例的 7.35 屬於競速等級。但要注意：高推重比意味著懸停時馬達只用了 13.6% 的推力，這使油門的解析度變差、懸停時較難精確控制——這是競速機不適合航拍的原因之一。

## 例題 27-2 串級控制的頻寬設計

題目：某四旋翼的轉動慣量  $I_{xx} = 0.012 \text{ kg} \cdot \text{m}^2$ 。（a）若要求姿態迴路的自然頻率為 15 rad/s、阻尼比 1.0，求 PD 增益。（b）依時間尺度分離原則，建議位置迴路的頻寬。（c）依第 19.9.3 節的準則，求各層所需的最低控制頻率。（d）討論若姿態頻寬只有 5 rad/s 會有什麼後果。

解（a）：姿態動態近似為  $I \cdot \ddot{\varphi} = \tau$ （忽略陀螺項）。這是一個雙積分器，套用第 16 章的公式：

$$\begin{aligned} K_p &= I \cdot \omega_n^2 = 0.012 \times 15^2 = 0.012 \times 225 = 2.70 \text{ N} \cdot \text{m/rad} \\ K_d &= I \cdot 2\zeta\omega_n = 0.012 \times 2 \times 1.0 \times 15 = 0.36 \text{ N} \cdot \text{m} \cdot \text{s/rad} \end{aligned}$$

姿態迴路的 PD 增益。

解 (b)：時間尺度分離要求外迴路比內迴路慢至少 3~5 倍 (實務上常取 5~10 倍)：

$$\omega_{n,pos} \leq 15/5 = 3 \text{ rad/s}$$

位置迴路的頻寬建議不超過 3 rad/s (約 0.48 Hz)。

這個數字看似很低，但符合實際——無人機的位置響應確實較慢，從發出「往前 1 公尺」的指令到真正到位，通常需要 1~2 秒。

解 (c)：依第 19.9.3 節，取樣頻率應為閉環頻寬的 20~30 倍：

$$\text{姿態迴路：} f \geq 20 \times 15/(2\pi) = 20 \times 2.39 = 47.8 \text{ Hz} \rightarrow \text{建議 } 250 \text{ Hz 以上}$$

$$\text{位置迴路：} f \geq 20 \times 3/(2\pi) = 20 \times 0.48 = 9.6 \text{ Hz} \rightarrow \text{建議 } 50 \text{ Hz}$$

各層的最低控制頻率。

**注意：**實務上姿態迴路遠高於理論最低值 (500~1000 Hz)，原因有二：一、角速度內迴路的頻寬更高 (可達 50 rad/s 以上)；二、需要足夠的取樣率才能有效濾除馬達振動的雜訊。

解 (d)：若姿態頻寬只有 5 rad/s：

**後果一：**位置迴路必須降到 1 rad/s 以下——無人機的反應會極為遲鈍，從指令到位需要 5 秒以上。

**後果二：**抗擾動能力大幅下降。陣風造成的姿態偏差需要更長的時間才能修正，期間位置已經偏離很遠。

**後果三：**無法執行快速軌跡。最小 snap 軌跡若要求快速的姿態變化，控制器根本跟不上。

**改善方向：**提高姿態增益 (但受限於雜訊與結構共振)、降低轉動慣量 (縮短機臂、減輕外圍重量)、或使用響應更快的馬達與電調。**降低慣量往往是最有效的**——這也是競速無人機把電池與電子件都集中在中心的原因。

### 例題 27-3 位置控制與姿態指令的轉換

題目：某四旋翼質量 1.5 kg，當前位置 (2.0, 1.0, 3.0) m、速度 (0.5, 0, 0) m/s。目標位置 (5.0, 3.0, 3.0) m、目標速度為零。位置控制增益  $K_p = 2.0$ 、 $K_d = 3.0$ 。(a) 計算期望加速度。(b) 用小角度近似計算期望的滾轉與俯仰角。(c) 計算所需的總推力。(d) 驗證小角度近似是否適用。

解 (a)：位置誤差與速度誤差：

$$\begin{aligned} \mathbf{e}_p &= (5.0-2.0, 3.0-1.0, 3.0-3.0) = (3.0, 2.0, 0) \text{ m} \\ \mathbf{e}_v &= (0-0.5, 0-0, 0-0) = (-0.5, 0, 0) \text{ m/s} \end{aligned}$$

誤差向量。

$$\begin{aligned} \mathbf{a}_d &= \mathbf{K}_p \cdot \mathbf{e}_p + \mathbf{K}_d \cdot \mathbf{e}_v = 2.0(3.0, 2.0, 0) + 3.0(-0.5, 0, 0) \\ &= (6.0, 4.0, 0) + (-1.5, 0, 0) = (4.5, 4.0, 0) \text{ m/s}^2 \end{aligned}$$

期望加速度。

注意：這個加速度相當大（水平合成約  $6.0 \text{ m/s}^2$ ，約  $0.61g$ ）。實務上會加上飽和限制，例如把水平加速度限制在  $0.5g$  以內——否則會要求過大的傾角。

解（b）：由第 27.6.2 節的小角度近似：

$$\begin{aligned} \theta_d &= a_x/g = 4.5/9.8 = 0.459 \text{ rad} = 26.3^\circ \quad (\text{俯仰, 前傾}) \\ \phi_d &= -a_y/g = -4.0/9.8 = -0.408 \text{ rad} = -23.4^\circ \quad (\text{滾轉}) \end{aligned}$$

期望姿態角。

解（c）：總推力（垂直方向要求加速度為零，即維持高度）：

$$F = m \cdot (g + a_z) = 1.5 \times (9.8 + 0) = 14.7 \text{ N}$$

小角度近似下的總推力。

但這個近似在此處已不夠準確。完整的計算應為：

$$\begin{aligned} \mathbf{F}_{\text{vec}} &= m \cdot (\mathbf{a}_d + g \cdot \hat{\mathbf{z}}) = 1.5 \times (4.5, 4.0, 9.8) = (6.75, 6.0, 14.7) \text{ N} \\ F &= \|\mathbf{F}_{\text{vec}}\| = \sqrt{(6.75)^2 + (6.0)^2 + (14.7)^2} = \sqrt{(45.6 + 36 + 216.1)} = \sqrt{297.7} = 17.25 \text{ N} \end{aligned}$$

完整向量形式的推力，比小角度近似大了 17%。

解（d）：小角度近似的誤差來源是  $\cos$  項。當傾角為  $26.3^\circ$  時：

$$\text{誤差} \approx 1/\cos(26.3^\circ) - 1 = 1/0.897 - 1 = 0.115 = 11.5\%$$

推力誤差達 11.5%。

結論：此情況下小角度近似不適用，必須使用 27.6.1 節的完整向量形式。

實務原則：

- 傾角  $< 15^\circ$ ：小角度近似的誤差  $< 3.5\%$ ，可以接受。
- 傾角  $15^\circ \sim 30^\circ$ ：誤差  $3.5\% \sim 15\%$ ，建議使用完整形式。
- 傾角  $> 30^\circ$ ：必須使用完整形式，否則高度會明顯下沉。

這也解釋了一個常見的現象：入門級無人機在做大角度轉彎時會掉高度——因為它的飛控使用了小角度近似而未補償推力的餘弦損失。

## 程式範例：四旋翼的動力學與串級控制模擬

以下 Python 程式實作四旋翼的動力學模型、分配矩陣、串級控制器，並模擬懸停與軌跡追隨。同時驗證本章三個例題的計算。

```
import numpy as np

# ===== 四旋翼參數與動力學 =====
class Quadrotor:
    def __init__(s, m=1.2, L=0.25, kf=1.5e-5, km=2.4e-7,
                 Ixx=0.012, Iyy=0.012, Izz=0.022, w_max=1200.0):
        s.m, s.L, s.kf, s.km, s.w_max = m, L, kf, km, w_max
        s.I = np.diag([Ixx, Iyy, Izz])
        s.g = 9.8
        # 分配矩陣 ( 27.2.1 節 · 「十」字配置 )
        s.M = np.array([
            [ kf,      kf,      kf,      kf ], # 總推力
            [ 0,      -L*kf,      0,      L*kf ], # 滾轉  $\tau_\phi$ 
            [ L*kf,      0,      -L*kf,      0 ], # 俯仰  $\tau_\theta$ 
            [ km,      -km,      km,      -km ]]) # 偏航  $\tau_\psi$ 
        s.M_inv = np.linalg.inv(s.M)

    def allocate(s, F, tx, ty, tz):
        """由推力矩指令求各馬達轉速 ( 含飽和處理 ) """
        w2 = s.M_inv @ np.array([F, tx, ty, tz])
        w2 = np.clip(w2, 0, s.w_max**2) # 不能為負、不能超速
        return np.sqrt(w2)

    def hover_speed(s):
        F1 = s.m*s.g/4
        return np.sqrt(F1/s.kf)

    def max_thrust(s):
        return 4*s.kf*s.w_max**2

    def thrust_to_weight(s):
        return s.max_thrust()/(s.m*s.g)

# ===== 例題 27-1 =====
print("=== 例題 27-1：推力與分配 ===")
q = Quadrotor()
print(f"(a) 懸停總推力 = {q.m*q.g:.2f} N")
print(f"    每顆馬達推力 = {q.m*q.g/4:.3f} N")
print(f"    懸停轉速 = {q.hover_speed():.1f} rad/s "
      f"({q.hover_speed()*60/(2*np.pi):.0f} RPM)")

w = q.allocate(q.m*q.g, 0.15, 0, 0)
print(f"\n(b) 產生  $\tau_\phi=0.15$  N·m 時的四顆轉速:")
```

```

for i, wi in enumerate(w, 1):
    F_i = q.kf*wi**2
    print(f"    馬達{i}: {wi:7.1f} rad/s (推力 {F_i:.3f} N)")
print(f"    轉速差 = {w.max()-w.min():.1f} rad/s")

ratio = (q.L*q.kf)/q.km
print(f"\n(c) 滾轉/偏航控制權限比 = (L·kf)/km = {ratio:.1f}")
print(f"    → 偏航是最弱的控制通道")
print(f"\n(d) 最大推力 = {q.max_thrust():.1f} N")
print(f"    推重比 = {q.thrust_to_weight():.2f} "
      f"(懸停時油門 {100/q.thrust_to_weight():.1f}%)")

# ===== 例題 27-2：串級頻寬設計 =====
print("\n=== 例題 27-2：串級控制的頻寬設計 ===")
Ixx, wn_att, zeta = 0.012, 15.0, 1.0
Kp_att = Ixx*wn_att**2
Kd_att = Ixx*2*zeta*wn_att
print(f"(a) 姿態 PD: Kp = {Kp_att:.3f} N·m/rad, "
      f"Kd = {Kd_att:.3f} N·m·s/rad")
print(f"(b) 位置頻寬建議 ≤ {wn_att/5:.1f} rad/s "
      f"({wn_att/5/(2*np.pi):.2f} Hz)")
print(f"(c) 最低控制頻率:")
for name, wn in (("姿態", wn_att), ("位置", wn_att/5)):
    f_min = 20*wn/(2*np.pi)
    print(f"    {name}迴路: ≥ {f_min:.1f} Hz")
print(f"    → 實務上姿態用 500~1000 Hz、位置用 50 Hz")

# ===== 例題 27-3：位置到姿態的轉換 =====
print("\n=== 例題 27-3：位置控制與姿態指令 ===")
m3 = 1.5
pos = np.array([2.0, 1.0, 3.0]); vel = np.array([0.5, 0.0, 0.0])
pos_d = np.array([5.0, 3.0, 3.0]); vel_d = np.zeros(3)
Kp, Kd = 2.0, 3.0

a_d = Kp*(pos_d - pos) + Kd*(vel_d - vel)
print(f"(a) 期望加速度 = ({a_d[0]:.2f}, {a_d[1]:.2f}, {a_d[2]:.2f}) m/s²")
print(f"    水平合成 = {np.hypot(a_d[0], a_d[1]):.2f} m/s² "
      f"({np.hypot(a_d[0], a_d[1])/9.8:.2f} g)")

# 小角度近似
th_d = a_d[0]/9.8; ph_d = -a_d[1]/9.8
F_small = m3*(9.8 + a_d[2])
print(f"\n(b) 小角度近似: θ_d = {np.rad2deg(th_d):.1f}°, "
      f"φ_d = {np.rad2deg(ph_d):.1f}°")
print(f"(c) 小角度推力 F = {F_small:.2f} N")

# 完整向量形式

```

```

F_vec = m3*(a_d + np.array([0, 0, 9.8]))
F_full = np.linalg.norm(F_vec)
tilt = np.arccos(F_vec[2]/F_full)
print(f"    完整形式 F = {F_full:.2f} N "
      f"(比近似大 {(F_full/F_small-1)*100:.1f}%)")
print(f"\n(d) 實際傾角 = {np.rad2deg(tilt):.1f}°")
print(f"    近似誤差 = {(1/np.cos(tilt)-1)*100:.1f}% → 應使用完整形式")

print("\n 傾角與近似誤差的關係:")
print(f"{'傾角':>8}{ '推力誤差':>12} 建議")
for deg in (5, 10, 15, 20, 30, 45):
    err = (1/np.cos(np.deg2rad(deg)) - 1)*100
    rec = "近似可用" if err < 3.5 else ("建議完整" if err < 15 else "必須完整")
    print(f"{deg:7d}°{err:11.1f}% {rec}")

# ===== 串級控制模擬 =====
print("\n=== 串級控制模擬：懸停點的階躍響應 ===")
class CascadeController:
    """27.4~27.6 節的四層串級控制"""
    def __init__(s, quad):
        s.q = quad
        s.Kp_pos, s.Kd_pos = 2.0, 3.0          # 位置層
        s.Kp_att = 12.0                       # 姿態層 (輸出角速度)
        s.Kp_rate = np.array([0.30, 0.30, 0.15]) # 角速度層
        s.Kd_rate = np.array([0.008, 0.008, 0.004])
        s.prev_e_rate = np.zeros(3)

    def update(s, pos, vel, rpy, omega, pos_d, yaw_d, dt):
        # ---- 位置層 (外迴路·慢) ----
        a_d = s.Kp_pos*(pos_d - pos) + s.Kd_pos*(-vel)
        a_d[:2] = np.clip(a_d[:2], -0.5*s.q.g, 0.5*s.q.g) # 限制傾角
        # ---- 轉換為姿態指令 (完整向量形式·27.6.1 節) ----
        F_vec = s.q.m*(a_d + np.array([0, 0, s.q.g]))
        F = np.linalg.norm(F_vec)
        zb = F_vec/F                                # 期望的機體 Z 軸
        # 由 zb 與 yaw_d 建構期望姿態 (簡化：轉為 roll/pitch)
        th_d = np.arctan2(zb[0]*np.cos(yaw_d) + zb[1]*np.sin(yaw_d), zb[2])
        ph_d = np.arcsin(np.clip(zb[0]*np.sin(yaw_d) - zb[1]*np.cos(yaw_d),
                                -1, 1))
        rpy_d = np.array([ph_d, th_d, yaw_d])
        # ---- 姿態層 (中迴路) ----
        e_att = rpy_d - rpy
        e_att[2] = np.arctan2(np.sin(e_att[2]), np.cos(e_att[2])) # 角度包裹
        omega_d = s.Kp_att*e_att
        # ---- 角速度層 (內迴路·快) ----
        e_rate = omega_d - omega
        d_e = (e_rate - s.prev_e_rate)/dt
        s.prev_e_rate = e_rate
        tau = s.Kp_rate*e_rate + s.Kd_rate*d_e
        return F, tau, rpy_d

```

```

def simulate(pos0, pos_d, T=6.0, dt=0.001):
    quad = Quadrotor()
    ctrl = CascadeController(quad)
    pos = np.array(pos0, float); vel = np.zeros(3)
    rpy = np.zeros(3); omega = np.zeros(3)
    log = []
    for k in range(int(T/dt)):
        F, tau, rpy_d = ctrl.update(pos, vel, rpy, omega,
                                    np.array(pos_d), 0.0, dt)

        # 分配並飽和 ( 27.2.1 節的知識框：優先保證力矩 )
        w = quad.allocate(F, tau[0], tau[1], tau[2])
        actual = quad.M @ (w**2)
        F_a, tau_a = actual[0], actual[1:]
        # 動力學積分
        ph, th, ps = rpy
        R_z = np.array([
            np.cos(ph)*np.sin(th)*np.cos(ps) + np.sin(ph)*np.sin(ps),
            np.cos(ph)*np.sin(th)*np.sin(ps) - np.sin(ph)*np.cos(ps),
            np.cos(ph)*np.cos(th)])
        acc = F_a*R_z/quad.m - np.array([0, 0, quad.g])
        vel = vel + acc*dt; pos = pos + vel*dt
        omega_dot = np.linalg.solve(
            quad.I, tau_a - np.cross(omega, quad.I @ omega))
        omega = omega + omega_dot*dt; rpy = rpy + omega*dt
        if k % int(0.5/dt) == 0:
            log.append((k*dt, pos.copy(), np.rad2deg(rpy).copy()))
    return log, pos

print(f"{'時間':>7}{ 'x':>8}{ 'y':>8}{ 'z':>8}"
      f"{'roll':>9}{ 'pitch':>9}")
log, final = simulate([0, 0, 2.0], [2.0, 1.0, 3.0])
for t, p, r in log[:9]:
    print(f"{'t':6.1f}{ 'p[0]:8.3f}{ 'p[1]:8.3f}{ 'p[2]:8.3f}"
          f"{'r[0]:9.2f}{ 'r[1]:9.2f}")
err = np.linalg.norm(final - np.array([2.0, 1.0, 3.0]))
print(f"最終位置誤差 = {err*1000:.1f} mm")
print("> 注意 pitch 先增大 ( 前傾產生水平力 ) 再回到零 ( 到位後回正 ) ")

# ===== 馬達失效的影響 =====
print("\n=== 馬達推力衰減對懸停的影響 ===")
print(f"{'推力衰減':>10}{ '可用推重比':>14}{ '懸停油門':>12} 狀態")
for decay in (0.0, 0.2, 0.5, 0.7, 0.87):
    q2 = Quadrotor(kf=1.5e-5*(1-decay))
    twr = q2.thrust_to_weight()
    throttle = 100/twr if twr > 1 else float('inf')
    st = "正常" if twr > 2 else ("勉強" if twr > 1.2 else
                               "臨界" if twr > 1 else "***無法起飛**")
    tstr = f"{'throttle':.0f}%" if twr > 1 else "-"
    print(f"{'decay*100':9.0f}{ 'twr':14.2f}{ 'tstr':>12} {st}")
print("> 電池電量下降會使 kf 等效降低，這是低電量必須返航的原因")

```

請試著修改：(一) 把位置控制的水平加速度限制由 0.5g 改為 0.2g，觀察響應變慢但傾角

變小。(二)把角速度層的  $K_p$  加倍，觀察是否出現振盪。(三)在動力學中加入一個持續的水平風力(如 0.5 N)，觀察無人機如何自動傾斜對抗，以及穩態位置誤差有多大。

## 實作：無人機的組裝、調校與飛行

本章實作以小型四旋翼為對象。**安全提醒：螺旋槳極為危險，測試時必須拆下螺旋槳或在防護網內進行；戶外飛行必須遵守法規並選擇空曠場地。**

- 參數量測：**量測機身質量、機臂長度。用第 13 章的方法估算轉動慣量(可用三線擺法，或由 CAD 估算)。
- 推力測試：**把單一馬達固定在電子秤上，記錄不同 PWM 指令下的推力與轉速。繪出「轉速 vs 推力」曲線，用二次擬合求出推力係數  $k_f$ ，驗證是否符合  $F = k_f \cdot \omega^2$ 。
- 推重比計算：**由測得的最大推力與機身質量計算推重比，判斷屬於哪個等級(例題 27-1d)。
- 分配矩陣驗證：**在拆下螺旋槳的狀態下，用飛控的除錯介面觀察——當手動傾斜機身時，四顆馬達的輸出如何變化。驗證是否符合分配矩陣的預期。
- IMU 校正：**執行加速度計與磁力計的校正程序。**特別記錄磁力計校正前後的差異——**把手機的指南針 App 靠近馬達，觀察讀值如何被干擾(第 27.7.2 節)。
- 姿態估測驗證：**把無人機固定在雲台或轉盤上，緩慢旋轉到已知角度，比較飛控輸出的姿態與實際角度。測試互補濾波的參數  $\alpha$  對響應速度與雜訊的影響。
- 角速度 PID 調校：**這是最關鍵的一步。從保守的增益開始，逐步提高 P 直到出現高頻振盪(會聽到馬達的嗡嗡聲)，再退回 70%。接著調 D 抑制超越、最後加入少量 I。**每次只改一個參數。**
- 姿態迴路測試：**在安全的防護下(或用測試架)，驗證姿態的階躍響應——手動傾斜後放開，觀察回正的速度與是否超越。
- 懸停測試：**在空曠場地進行首次懸停。觀察是否有持續的漂移(可能是重心偏移或 IMU 未校正)、以及油門懸停點是否符合推重比的預期。
- 位置控制測試：**切換到位置模式，測試定點懸停的精度(用地面標記量測漂移範圍)。比較有無 GNSS 時的差異。
- 失效保護測試：**在安全高度下，測試各項保護機制——關閉遙控器(應觸發返航)、模擬



低電量（應強制降落）。這一步極為重要，必須在實際任務前完成。

12. （進階）軌跡追隨：規劃一個簡單的方形或圓形航線，比較實際飛行軌跡與規劃軌跡的誤差。測試不同速度下的追隨精度。

表 27-9 無人機調校記錄表（前兩列為範例）

| 項目            | 設定值   | 觀察       | 問題     | 調整            |
|---------------|-------|----------|--------|---------------|
| Rate P (roll) | 0.15  | 反應遲鈍、追不上 | 增益不足   | 提高至 0.25      |
| Rate D (roll) | 0.010 | 機身高頻抖動   | 放大陀螺雜訊 | 降至 0.006 並加濾波 |
| （自行填寫其餘項目）    |       |          |        |               |

## 國中小也看得懂：無人機往前飛要先低頭

你有沒有想過，四旋翼無人機只有**四個螺旋槳**，為什麼可以前後左右上下自由飛？

先想一個問題：如果四個螺旋槳**一起加速**，會發生什麼事？答案是**往上飛**。一起減速就往下降。這很簡單。

但如果要**往前飛**呢？這裡有一個很有趣的祕密：**無人機必須先低頭！**

你可以這樣想像：螺旋槳的推力永遠是「垂直於機身」往上推的。如果無人機是平的，推力就完全往上，它只會上下動。但如果它稍微**前傾**，推力就會分成兩份——一份往上（撐住重量）、一份往前（推它前進）。

所以無人機往前飛的時候，其實是**歪著身體**在飛的！下次看到無人機飛過去，仔細觀察它的姿勢，你會發現它真的是斜的。

那怎麼讓它低頭呢？很簡單：**讓後面兩個螺旋槳轉快一點、前面兩個慢一點**。後面被推得比較高，前面就低下去了。

最後告訴你一件很重要的事：無人機和地上跑的機器人有一個**天大的差別**——

**地上的機器人如果程式當掉了，它就停在原地，沒事。但無人機如果程式當掉了，它會從天上掉下來！**

所以無人機的電腦必須**一秒鐘計算一千次**，不停地問自己「我現在歪了嗎？要怎麼調整？」一刻都不能休息。這就是為什麼工程師說：無人機是「**必須一直控制才能活著**」的機器。

## 叫獸提醒與常見錯誤

- 誤區一：以為無人機能「平移而不傾斜」。四旋翼欠驅動，水平加速度必須由傾斜產生，位置與姿態不能獨立控制。
- 誤區二：混淆欠驅動與不穩定。差速車欠驅動但停下來仍安全；無人機欠驅動且本質不穩定，失控即墜落。
- 誤區三：內部用歐拉角表示姿態。大角度機動時會遇到萬向節鎖，必須改用四元數或旋轉矩陣。
- 誤區四：計算姿態誤差時忽略四元數的雙重覆蓋。缺少 sign 項會使無人機朝錯誤方向翻轉。
- 誤區五：不做時間尺度分離。姿態與位置迴路的頻寬應相差 3~5 倍以上，否則兩層會互相干擾而振盪。
- 誤區六：角速度迴路的微分項未加濾波。馬達振動會被微分放大，造成機身高頻抖動與馬達過熱。
- 誤區七：大傾角時仍用小角度近似。傾角超過  $30^\circ$  時推力誤差達 15%，會造成明顯的高度下沉。
- 誤區八：高度控制缺少重力前饋。控制器必須靠積分項累積出  $mg$ ，起飛時會嚴重下沉。
- 誤區九：忽略電池電壓下降的影響。同樣的 PWM 在低電量時推力較小，會使飛行後期逐漸下沉。
- 誤區十：磁力計未校正或安裝位置不當。靠近馬達或鐵件會使偏航角錯誤，導致「馬桶效應」（繞圈飛行）。
- 誤區十一：垂直快速下降。可能進入渦環狀態而急遽失去升力，降落時應保持水平速度。
- 誤區十二：把緊急停止設計為切斷動力。這在無人機上等於墜落；正確的作法是分級降級而非停止。

## 課堂討論

1. 27.4 節的時間尺度分離在本書中已多次出現（第 5 章的伺服迴路、第 17 章的計算力矩控制、第 3 章的時間金字塔）。請歸納這個設計原則的共同結構，並討論它在非機器人領域的對應。
2. 微分平坦性使「給定位置軌跡即可算出所有狀態與輸入」。請討論這個性質為何如此珍貴，

以及它對規劃與控制的分工有何影響。

3. 無人機的安全哲學是「降級」而非「停止」。請比較這與第 7 章「失效導向安全」的異同，並討論還有哪些系統適用「降級」的思維。
4. 例題 27-1 顯示偏航的控制權限只有滾轉的  $1/15.6$ 。請討論這個限制的三個實務後果，並提出至少兩種機構層面的改善方向。
5. 27.7.3 節指出「室內飛行比室外困難」，關鍵在於缺少 GNSS 這個絕對參照。請討論：這個「絕對參照」的概念在本書其他章節如何出現？（提示：第 23 章的里程計漂移、第 25 章的迴路閉合）

## 章末摘要

- 四旋翼因機構極簡（無變距機構、只靠四顆馬達轉速）而成為主流；六旋翼以上則提供馬達失效的冗餘。
- 正反槳配置使反扭矩兩兩抵消；偏航控制則刻意打破此平衡，因此是最弱的控制通道。
- 推力與反扭矩皆正比於轉速平方；分配矩陣把  $(F, \tau_\phi, \tau_\theta, \tau_\psi)$  映射到四顆馬達轉速。
- 控制分配飽和時應優先保證姿態力矩——「先活著，再談去哪裡」。
- 推力永遠沿機體 Z 軸，因此水平加速度完全由姿態決定：這是欠驅動的本質。
- 懸停點線性化後： $\ddot{x} \approx g \cdot \theta$ 、 $\ddot{y} \approx -g \cdot \phi$ ，這正是外迴路與內迴路的介面。
- 四旋翼有 6 自由度但只有 4 個輸入；可獨立控制的是  $(x, y, z, \psi)$ ，滾轉與俯仰被位置需求佔用。
- 四旋翼具微分平坦性，平坦輸出為  $(x, y, z, \psi)$ ；所有狀態與輸入可由其導數代數求出。
- 力矩正比於位置的四階導數（snap），因此軌跡規劃採最小 snap，解為分段七次多項式。
- 串級控制的依據是時間尺度分離：姿態時間常數數十毫秒、位置數百毫秒以上，相差十倍以上。
- 四層架構：軌跡規劃（10 Hz）→ 位置控制（50 Hz）→ 姿態控制（500 Hz）→ 角速度控制（1000 Hz）。
- 角速度內迴路最為關鍵：決定極限性能、唯一能直接對抗擾動、且調參最敏感。
- 姿態必須用四元數表示；姿態誤差為相對旋轉  $q_e = q^* \otimes q_d$ ，並須以 sign 項處理雙重覆蓋。

- 位置控制的三步驟：算期望加速度 → 求推力向量 → 由向量方向反推姿態與推力大小。
- 小角度近似在傾角  $15^\circ$  內誤差小於 3.5%，超過  $30^\circ$  則必須使用完整向量形式。
- 姿態估測是互補濾波的經典應用：陀螺儀高通、加速度計與磁力計低通。
- 位置估測依環境分三種方案：GNSS+IMU（戶外）、光流或 VIO（室內）、外部定位（實驗室）。
- 無人機沒有「安全停止」選項，其安全哲學是分級降級：位置→高度→姿態→手動模式。
- 常見失效包含電量不足、GNSS 失鎖、磁力計干擾、通訊中斷與渦環狀態，各有對應的保護機制。

## 觀念題與計算題

### 觀念題

1. 說明四旋翼為何以四個螺旋槳為最少配置，以及六旋翼以上的優勢。
2. 解釋正反槳配置的原理，以及偏航力矩如何產生。
3. 寫出分配矩陣的形式，並說明控制分配飽和時的優先順序原則。
4. 由平移動力學說明四旋翼的欠驅動本質，並解釋為何「往右平移不能不傾斜」。
5. 寫出懸停點附近的線性化關係，並說明它如何構成串級控制的介面。
6. 說明四旋翼可獨立控制哪四個量，以及為何滾轉與俯仰不是自由的。
7. 定義微分平坦性，並說明四旋翼的平坦輸出為何。
8. 說明為何四旋翼的軌跡規劃採用最小 snap 而非最小 jerk。
9. 說明時間尺度分離的原理，以及它為何是串級控制的依據。
10. 列出無人機控制的四層架構及其頻率，並說明角速度內迴路為何最關鍵。
11. 說明為何姿態內部計算必須用四元數，並寫出姿態誤差的計算方式。
12. 說明位置控制轉換為姿態指令的三個步驟。
13. 說明小角度近似的適用範圍，以及超出範圍時的後果。
14. 說明姿態估測中三個感測器的互補關係，以及磁力計校正的必要性。
15. 比較無人機在戶外、室內與實驗室三種環境下的位置估測方案。
16. 說明為何無人機不能採用「切斷動力」的緊急停止，以及分級降級的設計邏輯。

## 計算題

1. 某四旋翼  $m = 0.9 \text{ kg}$ 、 $L = 0.20 \text{ m}$ 、 $k_f = 1.2\text{e-}5 \text{ N}\cdot\text{s}^2$ 、 $k_m = 1.8\text{e-}7 \text{ N}\cdot\text{m}\cdot\text{s}^2$ 、馬達最高轉速  $1500 \text{ rad/s}$ 。(a) 求懸停時每顆馬達的轉速。(b) 求最大推力與推重比。(c) 求滾轉與偏航的控制權限比。(d) 若要產生  $0.1 \text{ N}\cdot\text{m}$  的俯仰力矩，求前後馬達的推力差。
2. 某四旋翼  $I_{xx} = 0.008 \text{ kg}\cdot\text{m}^2$ 。(a) 若要求姿態自然頻率  $20 \text{ rad/s}$ 、阻尼比  $0.8$ ，求 PD 增益。(b) 建議位置迴路的頻寬上限。(c) 求兩層所需的最低控制頻率。(d) 若把  $I_{xx}$  減半（縮短機臂），重新計算 (a) 並討論其影響。
3. 某四旋翼  $m = 1.8 \text{ kg}$ ，當前位置  $(0, 0, 5) \text{ m}$ 、速度  $(1.0, 0.5, 0) \text{ m/s}$ ，目標位置  $(4, 3, 6) \text{ m}$ 、目標速度為零。 $K_p = 1.5$ 、 $K_d = 2.5$ 。(a) 求期望加速度。(b) 求完整向量形式的推力大小。(c) 求實際傾角。(d) 判斷小角度近似的誤差，並說明是否適用。
4. 某四旋翼的推重比為  $3.0$ 。(a) 求懸停時的油門百分比。(b) 若電池電量下降使推力衰減  $25\%$ ，求新的推重比與懸停油門。(c) 求推力衰減到多少百分比時無法起飛。(d) 若要保留「推重比不低於  $1.5$ 」的安全餘裕，求可容許的最大推力衰減。

## 動手做與專題延伸

### 本章實作：四旋翼模擬器

建立一個完整的四旋翼模擬環境，這是測試控制演算法最安全也最有效率的方式。

- 階段一：實作四旋翼的動力學模型與分配矩陣，驗證懸停平衡與例題 27-1。
- 階段二：實作角速度與姿態的串級控制器，測試姿態的階躍響應。
- 階段三：實作位置控制與姿態指令的轉換（含完整向量形式），測試定點懸停。
- 階段四：實作最小 snap 軌跡生成，並驗證微分平坦性——由軌跡直接算出所需的姿態與推力。
- 階段五：加入感測器雜訊與互補濾波，模擬完整的估測—控制迴路。

### 專題延伸

- 時間尺度分離的驗證：系統性地改變姿態迴路與位置迴路的頻寬比（ $1:1$ 、 $2:1$ 、 $5:1$ 、 $10:1$ ），記錄系統的響應與穩定性。你會看到當比值太小時，兩層會互相干擾而產生振盪——這是理解串級控制原理最直接的實驗。繪出「頻寬比 vs 超越量」的曲線。
- 微分平坦性的視覺化：給定一條三維的最小 snap 軌跡，用平坦性關係直接計算出全程所需

的姿態角、角速度與推力。把姿態以小座標軸的形式沿軌跡繪出，你會看到無人機如何在轉彎時自動傾斜、在加速時前傾——這張圖能讓抽象的平坦性變得直觀。

- **進階：馬達失效的容錯控制。**模擬一顆馬達完全失效的情況，觀察四旋翼如何失控翻覆。接著實作一個容錯策略——**犧牲偏航控制**（讓機身持續自轉），只用剩餘三顆馬達維持推力與滾轉俯仰的平衡。驗證這個策略能否使無人機維持高度並安全降落。這個題目結合了控制分配、欠驅動分析與安全設計，是本章的綜合考驗。

## 參考文獻與延伸閱讀

- Mellinger, D. and Kumar, V., "Minimum Snap Trajectory Generation and Control for Quadrotors," ICRA 2011 (微分平坦性與最小 snap 軌跡的奠基論文)。
- Mahony, R., Kumar, V. and Corke, P., "Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor," IEEE Robotics & Automation Magazine, 2012 (四旋翼建模與控制的優秀綜述)。
- Lee, T., Leok, M. and McClamroch, N. H., "Geometric Tracking Control of a Quadrotor UAV on SE(3)," CDC 2010 (幾何控制方法，避免歐拉角的奇異性)。
- Beard, R. W. and McLain, T. W., Small Unmanned Aircraft: Theory and Practice, Princeton University Press (無人機的完整教材，含固定翼)。
- Quan, Q., Introduction to Multicopter Design and Control, Springer (多旋翼設計與控制的系統性教材)。
- Mahony, R., Hamel, T. and Pflimlin, J.-M., "Nonlinear Complementary Filters on the Special Orthogonal Group," IEEE Trans. Automatic Control, 2008 (姿態估測的互補濾波理論)。
- Faessler, M., Franchi, A. and Scaramuzza, D., "Differential Flatness of Quadrotor Dynamics Subject to Rotor Drag," RA-L, 2018 (考慮空氣阻力的平坦性推廣)。
- Loianno, G. et al., "Estimation, Control, and Planning for Aggressive Flight with a Small Quadrotor," RA-L, 2017 (高機動飛行的完整系統)。
- PX4 與 ArduPilot 開源飛控專案文件：兩套最主流的開源飛控，含完整的控制架構與調參指南。
- 交通部民用航空局《遙控無人機管理規則》與相關公告：台灣無人機的法規依據。



## 第 28 章

# 腿式機器人與運動控制

編著：李世淵（機器人叫獸）

助教：李天宇、李宇晴

【章首情境圖建議】左半為降維抽象的對照：上方是一台三十自由度的人形機器人，下方以粗箭頭指向一根簡單的倒立擺（質點、無質量桿、單一接觸點），標註「 $LIPM: 30+ DOF \rightarrow 3$  個狀態量」。中央畫出關鍵幾何——質心高度  $z$ 、質心水平位置  $x$ 、 $ZMP$  位置，以及捕捉點  $x + \dot{x}/\omega$  的位置（以紅色標出，並畫出一隻正要踩上去的腳）。右半為四足小跑步態的四格連續動作，每格下方以色塊時序圖標出四隻腳的支撐相（實心）與擺動相（空心），並以虛線標出當下的支撐多邊形。下緣文字：透過選擇落腳點，反覆消除運動的發散分量。



## 叫獸開講 「走路的本質，是一連串被打斷的跌倒。」

叫獸請全班站起來：「請你們往前走三步，然後告訴我——在走路的過程中，有沒有哪一個瞬間，你其實是正在跌倒的？」

學生們慢慢走著，小矧先開口：「腳抬起來的時候……身體是往前倒的，只是下一隻腳及時撐住了。」

「完全正確。」叫獸點頭，「人類走路的本質，是一連串被打斷的跌倒。你不是先站穩再移動，而是先讓自己失去平衡，再用下一步接住自己。」

他在白板上畫了兩個字：**靜態**、**動態**。「有一種走路方式叫靜態步行——重心永遠保持在支撐腳的正上方，任何一刻停下來都不會倒。這就是早期人形機器人那種『慢慢挪、慢慢挪』的走法。安全，但慢得可怕，而且能量效率極差。」

「人類不是這樣走的。我們用的是**動態步行**——大部分時間重心根本不在支撐面內，我們靠的是『下一步會接住我』這個預測。」

小翰舉手：「那不是很危險嗎？萬一沒接到？」

「就跌倒啊。」叫獸笑了，「這就是為什麼腿式機器人比輪型機器人難這麼多。輪子隨時可以停下來，腿不行——你一旦開始動態行走，就進入了一個必須持續正確預測的狀態。」

接著他問了一個更關鍵的問題：「而且腿有一個輪子沒有的麻煩。當腳踩在地上時，機器人是一個**被約束的系統**——腳不能穿透地面、不能滑動。但當腳抬起來時，那些約束**全部消失**了。每走一步，系統的方程式就要換一次。」

他在白板上寫下一個詞：**混合動態**。「第 27 章的無人機雖然難，但它的動態是**連續**的——螺旋槳一直在轉，方程式從頭到尾都一樣。腿式機器人不是，它每一步都在切換。這是完全不同層級的困難。」

最後他拿出一張紙，畫了一根倒立的桿子，底下是一個點。「但有一個好消息。**這麼複雜的系統，在規劃層面上可以簡化成一根倒立擺**——第 18 章那個東西。三十個自由度、每步切換的接觸狀態，最後歸結成一個三維的倒立擺模型。今天你們會看到，**簡化模型是如何讓不可能變成可能的**。」

## 本章為什麼重要

腿式機器人是機器人學中最困難、也最能展現技術整合度的領域。過去十年，波士頓動力的 Atlas 與 Spot、宇樹的四足機器人、以及各家人形機器人的快速進展，使它從實驗室走向了實用。

**為什麼要用腿？** 輪子在平整地面上效率遠高於腿——這是無可爭議的。但地球上**只有約一半的陸地表面是輪型載具能到達的**。樓梯、碎石、瓦礫、沙地、森林、以及所有為人類設計的建築內部——這些環境對輪子是障礙，對腿卻是可以跨越的。

在技術上，腿式機器人是本書所有內容的**最終整合考驗**。它需要：第 13、14 章的動力學（自由度極高）、第 15 章的軌跡生成（步態規劃）、第 17~19 章的控制與穩定性（本質不穩定）、第 20 章的力控制（接觸力分配）、第 24 章的狀態估測（沒有絕對參照）、第 26 章的規劃（地形上的落腳點）、以及第 31 章的學習（近年最重要的突破）。

本章也將揭示一個貫穿全書的方法論頂點：**用簡化模型處理複雜系統**。一台三十自由度的人形機器人，在規劃層面可以簡化為一根三維倒立擺——這個抽象使原本不可解的問題變得可解。理解這個思維，比記住任何一條公式都重要。

## 學習目標

- 能說明腿式機器人相對於輪型的優勢與代價。
- 能說明混合動態的意義，以及接觸切換帶來的困難。
- 能區分靜態穩定與動態穩定，並說明各自的判準。
- 能定義支撐多邊形與零力矩點（ZMP），並說明其物理意義。
- 能說明線性倒立擺模型的推導與其在規劃上的價值。
- 能說明捕捉點的概念及其對落腳點選擇的指導。
- 能描述四足與雙足的常見步態並比較其特性。
- 能說明步態的相位、佔空比與其對穩定性的影響。
- 能說明腿式機器人控制的分層架構。
- 能說明全身控制的最佳化形式與接觸力分配。
- 能說明腿式機器人狀態估測的特殊困難與解法。

- 能說明強化學習在腿式運動上的突破與其與模型式方法的關係。

## 先備知識

第 13、14 章的動力學（尤其是自由度與慣性）、第 18 章的 LQR 與倒立擺、第 19 章的穩定性與欠驅動、第 20 章的力控制與接觸、第 24 章的狀態估測、第 27 章的欠驅動系統與分層控制。

本章是全書整合度最高的一章。強烈建議先確認第 18、19、20、27 章已充分理解——本章會直接引用那些結論而不重新推導。若時間有限，至少要掌握 28.4 節的線性倒立擺模型與 28.7 節的分層架構。

## 核心名詞中英對照

表 28-1 本章核心名詞

| 中文      | 英文                                   | 說明             |
|---------|--------------------------------------|----------------|
| 混合動態    | Hybrid Dynamics                      | 連續動態與離散切換交替的系統 |
| 支撐多邊形   | Support Polygon                      | 所有觸地點所張的凸包     |
| 靜態穩定    | Static Stability                     | 重心投影落在支撐多邊形內   |
| 動態穩定    | Dynamic Stability                    | 含慣性效應下仍不傾倒     |
| 零力矩點    | Zero Moment Point, ZMP               | 地面反作用力合力矩為零的點  |
| 壓力中心    | Center of Pressure, CoP              | 地面壓力分布的等效作用點   |
| 線性倒立擺模型 | Linear Inverted Pendulum Model, LIPM | 把質心運動簡化為倒立擺的模型 |
| 捕捉點     | Capture Point                        | 踩下去就能停止的落腳位置   |
| 發散分量    | Divergent Component of Motion, DCM   | 捕捉點的推廣形式       |
| 步態      | Gait                                 | 各腿抬放的時序模式      |
| 支撐相     | Stance Phase                         | 腳著地承重的階段       |
| 擺動相     | Swing Phase                          | 腳離地前擺的階段       |
| 佔空比     | Duty Factor                          | 支撐相佔整個步態週期的比例  |
| 落腳點規劃   | Footstep Planning                    | 決定每一步該踩在哪裡     |
| 全身控制    | Whole-Body Control, WBC              | 統一協調所有關節的控制架構  |
| 接觸力分配   | Contact Force Distribution           | 決定各腳承受多少力      |
| 摩擦錐     | Friction Cone                        | 不打滑的接觸力方向範圍    |

| 中文   | 英文                      | 說明            |
|------|-------------------------|---------------|
| 被動行走 | Passive Dynamic Walking | 不需驅動即可下坡行走的機構 |

## 28.1 為何要用腿

### 28.1.1 輪子的極限

在平整的地面上，輪子的效率遠高於腿。一台輪型機器人的**運輸成本**（cost of transport，單位距離每單位重量所需的能量）可以低到 0.1 以下，而人類行走約為 0.2、多數腿式機器人則在 0.5~2 之間。

但輪子有三個無法克服的限制：

- **必須連續的支撐面**：輪子無法跨越大於其半徑的縫隙或台階。一台輪徑 10 公分的機器人，連 5 公分的門檻都過不去。
- **無法選擇接觸點**：輪子走過的每一點都必須是可通行的。腿則可以**跳過**不可踩的區域——這在碎石、瓦礫、濕滑的地面上是決定性的差異。
- **對地面破壞**：履帶與輪子在轉向時會刮傷地面（第 23.6.1 節），這在草坪、農地與室內是問題。

#### ◎ 為何自然界沒有輪子

生物演化出了飛行、游泳、爬行、行走，卻沒有任何大型生物使用輪子移動。原因有二：一、輪子需要連續旋轉的關節，而這在生物的血管與神經供應上幾乎不可能（必須有一個完全分離、自由旋轉的部件）；二、輪子在自然地形上根本不佔優勢。自然界的地表充滿不規則的起伏，而腿能夠選擇落腳點——這個能力在崎嶇地形上的價值，遠超過輪子在平地上的效率優勢。這也解釋了為何在災難現場、森林、與為人類設計的建築中，腿式機器人有其不可取代性。

### 28.1.2 腿的代價

但腿的代價極為高昂，這解釋了為何工業界至今仍以輪型為主：

表 28-2 輪型與腿式機器人的比較

| 面向   | 輪型 | 腿式          |
|------|----|-------------|
| 平地效率 | 極高 | 低（3~10 倍耗能） |
| 崎嶇地形 | 受限 | 佳           |

| 面向    | 輪型         | 腿式          |
|-------|------------|-------------|
| 跨越障礙  | 小於輪半徑      | 可達腿長的一半以上   |
| 自由度   | 2~4        | 12~30 以上    |
| 控制複雜度 | 低          | 極高          |
| 穩定性   | 天然穩定       | 本質不穩定       |
| 成本    | 低          | 高 ( 多倍 )    |
| 可靠度   | 高          | 低 ( 故障點多 )  |
| 典型應用  | 工廠、倉儲、平整環境 | 巡檢、災難、樓梯、戶外 |

### 28.1.3 被動行走：一個重要的啟發

1990 年 McGeer 展示了一個令人震驚的裝置：一組完全沒有馬達、沒有感測器、沒有控制器的機械腿，放在一個緩坡上輕輕一推，它就能穩定地走下去——步態自然、甚至比許多有動力的機器人更像人。

這稱為**被動動態行走** ( passive dynamic walking )。它的能量完全來自重力位能，控制則完全由機構的動力學自然產生。

它的啟示極為深刻：行走的穩定性，有很大一部分可以由**機構本身的動力學**提供，而非全靠控制器。這與第 21 章「柔順性取代感測與計算」、第 20 章「一個倒角勝過複雜控制」的思維完全一致——在把問題丟給控制器之前，先問能不能用機構解決。

後續的研究由此發展出「**極簡驅動行走**」 ( minimally actuated walking ) ——只在必要處加入少量驅動，補充能量並提供轉向與適應能力，其餘交給被動動力學。這類機器人的能量效率能接近人類水準，遠優於全主動控制的機器人。

## 28.2 混合動態

### 28.2.1 每一步都在換方程式

如開頭故事所述，腿式機器人最根本的困難是**混合動態** ( hybrid dynamics ) ——系統的方程式會隨接觸狀態而改變。

以雙足行走為例，一個步態週期至少包含四種不同的動態模式：

表 28-3 雙足行走的動態模式

| 階段      | 接觸狀態  | 系統性質    | 約束數   |
|---------|-------|---------|-------|
| 雙腳支撐    | 兩腳皆著地 | 過約束（閉環） | 多（12） |
| 單腳支撐（左） | 左腳著地  | 受約束     | 中（6）  |
| 單腳支撐（右） | 右腳著地  | 受約束     | 中（6）  |
| 騰空（跑步時） | 雙腳離地  | 自由飛行    | 0     |

每一次狀態切換，都伴隨兩件事：

一、約束方程改變。著地的腳提供了約束（不能穿透、不能滑動），這使系統的有效自由度減少。抬起時約束消失。

二、碰撞（impact）。腳落地的瞬間是一次碰撞——速度會不連續地改變。這在數學上是一個離散的狀態跳變：

$$q^+ = q^- \quad (\text{位置連續}) \quad q^+ = \Delta(q) \cdot q^- \quad (\text{速度跳變})$$

碰撞映射。 $\Delta$  為碰撞矩陣，由動量守恆推導。部分動能會在碰撞中損失——這是腿式行走能量效率低的主因之一。

## 28.2.2 為何這使問題困難得多

混合動態帶來三個層次的困難：

**困難一：無法用單一模型。**第 13~19 章的所有工具都假設一個固定的動力學方程。混合系統則需要為每個模式各建一套，並處理切換。

**困難二：切換時機也是決策變數。**「什麼時候該抬腳」本身就是一個要規劃的量。這使問題成為混合整數最佳化——比連續最佳化困難得多。

**困難三：穩定性理論不直接適用。**第 19 章的 Lyapunov 方法假設連續動態。對混合系統，必須使用龐加萊映射（Poincaré map）——分析「每走一步後，狀態如何映射到下一步」，若這個映射有穩定的固定點，則週期步態是穩定的。

### ◎ 龐加萊映射：把「一步」當成一次迭代

分析週期運動有一個優雅的技巧：不去追蹤連續的軌跡，而是只看每一步結束時的狀態。例如在雙足行走中，每次左腳觸地的瞬間記錄一次狀態，得到一個離散序列  $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots$ 。若這個序列收斂到一個固定點，就代表機器人能穩定地重複同樣的步態。這把一個複雜的連續混合系統，化簡

為一個離散映射的穩定性問題——而後者可以用第 19.9 節的離散系統判準（特徵值落在單位圓內）分析。這是降維思維的又一個範例。

## 28.3 平衡的判準

### 28.3.1 支撐多邊形與靜態穩定

最基本的平衡概念是**支撐多邊形**（support polygon）——所有觸地點所張成的凸包。

靜態穩定的判準極為簡單：

**重心（CoM）在地面的投影，落在支撐多邊形內部**

*靜態穩定判準。若成立，機器人在任何時刻停下來都不會倒。*

這個判準的價值在於它的保守與可靠。早期的人形機器人（如 Honda P2、ASIMO 的早期版本）與多數四足機器人的慢速步態，都採用靜態穩定。

但它的代價也極為明顯：

- **速度受限**：重心必須始終保持在支撐面內，這使步態必須緩慢。
- **能量效率差**：為了維持重心位置，關節必須持續施力，無法利用擺動的自然動力學。
- **四足必須至少三腳著地**：因為兩點只能張出一條線段，無法形成有面積的多邊形。這使爬行步態（crawl gait）成為唯一選擇。

### 28.3.2 動態穩定與 ZMP

要走得快，就必須放棄靜態穩定。但這時需要一個新的判準——**零力矩點**（Zero Moment Point, ZMP）。

#### ◎ ZMP 的定義

**零力矩點**是地面上的一個點，在該點處，地面反作用力所產生的**水平方向力矩為零**。等價地說，它是地面壓力分布的**等效作用點**（在腳未傾倒時，ZMP 與壓力中心 CoP 重合）。

ZMP 的判準是：

**ZMP 落在支撐多邊形內部 ⇔ 腳不會繞邊緣翻轉**

*動態穩定判準。注意這裡是 ZMP 而非重心投影——兩者在有加速度時會分離。*

為何 ZMP 比重心投影更適用？因為它把慣性效應納入了考量。ZMP 的位置可由下式計算



(簡化的線性倒立擺情形)：

$$x_{zmp} = x_{com} - (z_{com}/g) \cdot \ddot{x}_{com}$$

*ZMP 與重心的關係。第二項是慣性的修正——重心加速度越大，ZMP 偏離重心投影越遠。*

這條式子有極為重要的物理意義：**當重心向前加速時，ZMP 會往後移**。這正是人類前傾起步時的物理——重心已經超出腳掌前緣，但因為向前加速，ZMP 仍在腳掌內，所以不會摔倒。

### 28.3.3 ZMP 的限制

ZMP 判準雖然實用，但有三個重要的限制：

**限制一：只適用於平坦地面且腳為面接觸。**若在碎石上單點著地，或在斜坡上，ZMP 的定義需要推廣（有 FRI、CWC 等推廣形式）。

**限制二：無法處理騰空階段。**跑步或跳躍時雙腳離地，沒有支撐多邊形，ZMP 判準完全失效。

**限制三：它是必要而非充分條件。**ZMP 在支撐面內只保證「腳不會翻轉」，不保證「機器人不會跌倒」——例如上半身可能因為其他原因失去平衡。

因此現代的方法逐漸轉向 28.5 節的**捕捉點**與更一般的接觸扳手錐（CWC）判準。但 ZMP 因其直觀與計算簡單，至今仍是雙足行走規劃的主力工具。

## 28.4 線性倒立擺模型

### 28.4.1 把三十個自由度變成三個

這是本章最重要的觀念，也是腿式機器人得以實用的關鍵。

一台人形機器人有三十個以上的自由度，其完整動力學方程極為龐大。要在毫秒級的控制迴路中規劃步態，直接使用完整模型是不可能的。

**線性倒立擺模型**（Linear Inverted Pendulum Model, LIPM）提供了一個極簡但足夠好的抽象：

#### ◎ LIPM 的三個假設

**假設一：**全身的质量集中在質心（CoM）——忽略各連桿的分布，把機器人視為一個點質量。假

**設二：**質心維持在固定高度（ $z_{com} = \text{常數}$ ）——這使垂直方向的動態消失。**假設三：**不產生繞



**質心的角動量**——忽略手臂擺動等效應。在這三個假設下，機器人簡化為「一根長度可變、頂端有質點的倒立擺」。

在這些假設下，質心的水平動態變得極為簡潔：

$$\ddot{x} = (g/z_{com}) \cdot (x - x_{zmp})$$

*LIPM 方程。這是一個線性方程！質心的加速度正比於「質心與 ZMP 的水平距離」。*

定義一個關鍵的參數：

$$\omega = \sqrt{g/z_{com}}$$

*自然頻率。它是 LIPM 的唯一參數，決定了系統的時間尺度。以  $z_{com} = 0.8\text{ m}$  為例， $\omega = \sqrt{9.8/0.8} = 3.5\text{ rad/s}$ ，對應時間常數 0.29 秒。*

### 28.4.2 解與其物理意義

LIPM 方程  $\ddot{x} = \omega^2(x - x_{zmp})$  的解（設  $x_{zmp}$  為常數）包含兩個模態：

$$x(t) = A \cdot e^{\omega t} + B \cdot e^{-\omega t} + x_{zmp}$$

*一個發散模態 ( $e^{\omega t}$ ) 與一個收斂模態 ( $e^{-\omega t}$ )。*

那個**發散**的模態正是問題的核心——它代表倒立擺會越倒越快。這是第 18.9 節倒立擺不穩定性的重現，只是現在有了明確的時間常數  $1/\omega$ 。

控制的本質，就是消除發散模態。而唯一的手段是**移動 ZMP**——也就是換腳、選擇下一個落腳點。這給出了腿式行走最核心的洞見：

**行走 = 透過選擇落腳點，反覆消除質心運動的發散分量**

*這一句話概括了整個腿式運動控制的本質。*

### 28.4.3 LIPM 為何如此有效

LIPM 的三個假設看似粗暴，但實務效果極佳，原因有三：

**理由一：質心高度確實近似固定。**人類行走時質心的垂直起伏只有約 5 公分（相對於 0.9 公尺的高度，變化不到 6%）。

**理由二：質量確實集中。**人形機器人的軀幹通常佔總質量的 50% 以上，且腿部的質量分布可以用等效方式併入。

**理由三：規劃與控制的分工。**LIPM 只用於**規劃層**（決定質心軌跡與落腳點），實際執行時

仍使用完整的動力學模型（28.7 節的全身控制）。簡化模型負責「該往哪走」，完整模型負責「怎麼走到」——這正是第 27 章分層思維的延續。

圖 28-1 線性倒立擺模型

建議配圖：左半為對照圖——上方是一台複雜的人形機器人（標註「30+ 自由度」），下方箭頭指向一根簡單的倒立擺（一個質點、一根無質量的桿、一個接觸點），標註「LIPM：3 個狀態量」。中央標出關鍵幾何：質心高度  $z_{com}$ （虛線水平面表示「維持固定高度」）、質心水平位置  $x$ 、ZMP 位置  $x_{zmp}$ ，並以箭頭標示兩者的水平距離。右半為 LIPM 的相平面圖（橫軸  $x$ 、縱軸  $\dot{x}$ ），畫出發散模態（沿  $\dot{x} = \omega x$  這條線遠離）與收斂模態（沿  $\dot{x} = -\omega x$  這條線趨近），並標出捕捉點的位置。下緣公式： $\ddot{x} = \omega^2(x - x_{zmp})$ ， $\omega = \sqrt{g/z}$ 。

## 28.5 捕捉點

### 28.5.1 踩在哪裡才能停下來

由 28.4.2 節的結論，控制的核心是消除發散模態。這引出一個極為實用的問題：如果我現在想停下來，該把腳踩在哪裡？

答案就是捕捉點（Capture Point，也稱瞬時捕捉點 ICP）：

$$x_{cp} = x + \dot{x}/\omega$$

捕捉點。它是「當前質心位置」加上「速度除以自然頻率」。

這個定義的意義極為直觀：若把 ZMP（也就是腳）放在捕捉點上，質心會恰好在無窮遠的時間後停在該點，不會超過也不會回頭。

驗證：把  $x_{zmp} = x_{cp}$  代入 LIPM 方程並解之，可以證明發散模態的係數恰好為零——只剩下收斂模態。

#### ◎ 捕捉點的直觀理解

想像你被人從背後推了一把，身體開始往前倒。你會下意識地跨出一步——而那一步該跨多遠？太近會撐不住繼續倒，太遠會被自己絆住而往後彈。捕捉點就是那個「剛剛好」的位置。而公式  $x + \dot{x}/\omega$  說得很清楚：你倒得越快（ $\dot{x}$  越大），就要跨得越遠。這正是人類的本能反應，而它有一個精確的數學形式。

### 28.5.2 捕捉點的三個用途

用途一：判斷是否還能站住。若捕捉點落在可達的落腳範圍內，機器人就能透過一步（或數步）恢復平衡；若落在範圍外，則必定會跌倒——此時應立即執行保護動作（第 28.9 節）。

用途二：決定落腳點。這是最直接的應用。要停下來就踩在捕捉點上；要繼續前進，就踩在

捕捉點稍前方（留下一些殘餘速度）；要加速，就踩在捕捉點後方。

踩在捕捉點上 → 停止

踩在捕捉點前方 → 減速

踩在捕捉點後方 → 加速

*落腳點與速度控制的關係。這是腿式運動最核心的控制律之一。*

**用途三：量化穩定裕度。**捕捉點到支撐多邊形邊界的距離，就是一個直觀的穩定裕度指標——距離越大，能承受的擾動越大。

### 28.5.3 N 步捕捉

上述討論是「一步捕捉」（1-step capturable）。更一般地，可以定義 **N 步捕捉區域**——需要 N 步才能停下來的初始狀態集合。

這個概念在實務上極為有用：一台機器人被推了一把之後，控制器可以判斷「我需要跨兩步才能穩住」，並據此規劃一連串的落腳點。波士頓動力 Atlas 那些被推後踉蹌數步再站穩的著名影片，其底層邏輯正是這個。

### 28.5.4 DCM：捕捉點的推廣

捕捉點的一個重要推廣稱為**發散運動分量**（Divergent Component of Motion, DCM）。它的定義形式相同，但被視為一個**獨立的狀態變數**來控制：

$$\xi = x + \dot{x}/\omega \quad (\text{DCM}) \quad \xi = \omega(\xi - x_{\text{zmp}})$$

*DCM 的動態。它只有一個發散模態——這使控制問題大幅簡化。*

這個轉換的巧妙之處在於：原本的二階系統（含一個發散、一個收斂模態）被**分解**為兩個一階系統——DCM（發散，需要控制）與質心（收斂，會自動跟隨）。

**控制 DCM  $\Rightarrow$  質心自動穩定**

*只需處理發散的部分。這是一個典型的「分而治之」——把不穩定的部分隔離出來單獨處理。*

DCM 是目前雙足行走規劃的主流方法，被廣泛用於各國的人形機器人研究（如 DLR、IHMC、以及多數 DARPA 機器人挑戰賽的隊伍）。

## 28.6 步態

## 28.6.1 步態的描述

步態 ( gait ) 是各條腿抬起與放下的時序模式。描述步態需要三個要素：

- **相位 ( phase )**：各腿在週期中的相對時間偏移。例如四足的對角小跑，是「左前與右後」同相、「右前與左後」相差半個週期。
- **佔空比 ( duty factor )**：支撐相佔整個週期的比例。佔空比大於 0.5 代表隨時至少有一半的腿著地（較穩定但較慢）。
- **步長與步頻**：決定行進速度。速度 = 步長 × 步頻。

## 28.6.2 四足的常見步態

表 28-4 四足機器人的常見步態

| 步態        | 腿的時序     | 佔空比     | 同時著地        | 速度 | 穩定性         |
|-----------|----------|---------|-------------|----|-------------|
| 爬行 Crawl  | 一次抬一腳    | 0.75    | 3 腳         | 最慢 | 靜態穩定        |
| 小跑 Trot   | 對角兩腳同時   | 0.5     | 2 腳 ( 對角 )  | 中  | 動態穩定        |
| 溜蹄 Pace   | 同側兩腳同時   | 0.5     | 2 腳 ( 同側 )  | 中  | 動態 ( 側向不穩 ) |
| 跳躍 Bound  | 前兩腳、後兩腳  | 0.4~0.5 | 2 腳 ( 前或後 ) | 快  | 動態          |
| 飛奔 Gallop | 四腳依序、含騰空 | < 0.4   | 1~2 腳或騰空    | 最快 | 高度動態        |

爬行是唯一的靜態穩定步態——因為它保證隨時有三腳著地，能形成有面積的支撐多邊形。其餘步態都需要動態平衡。

小跑 ( trot ) 是實務上最常用的步態，原因有三：對角的兩腳形成的支撐線通過質心附近（側向較平衡）、機構的對稱性使控制簡單、且速度與穩定性的取舍適中。多數四足機器人的預設步態都是小跑。

自然界的啟示：馬會依速度自動切換步態——慢走用 walk、中速用 trot、快速用 gallop。切換的時機由能量效率決定：每種步態在特定的速度範圍內最省能。這給了機器人設計一個重要的參考——不應只用一種步態，而應依速度切換。

## 28.6.3 雙足的步態

雙足的步態較單純，主要分兩類：

行走 ( walking )：佔空比大於 0.5，因此有**雙腳支撐期**（兩腳同時著地）。這個重疊期提

供了穩定的過渡，也是能量交換的時機。

**跑步 (running)**：佔空比小於 0.5，因此有**騰空期**（雙腳皆離地）。此時機器人是一個自由飛行的物體——完全無法控制質心軌跡，只能為下一次落地做準備。

行走：無騰空期，有雙腳支撐期

跑步：有騰空期，無雙腳支撐期

兩者的定義差異。這也是競走比賽的規則依據——「必須隨時有一腳著地」。

**騰空期**是雙足跑步最困難的部分。在這段時間內：質心沿拋物線運動、無法施加任何外力、只能調整腿的姿態為落地做準備。而落地的瞬間又是一次碰撞，必須精確控制才不會失衡。

## 28.6.4 彈簧負載倒立擺 (SLIP)

對於跑步與跳躍，LIPM 不再適用（因為質心高度會大幅變化）。此時的標準簡化模型是 **SLIP** (Spring-Loaded Inverted Pendulum) ——一個質點加上一根**彈簧腿**。

SLIP 模型的價值在於它捕捉了跑步的本質：**能量在動能、位能與彈性位能之間循環**。著地時腿被壓縮（動能轉為彈性位能）、蹬離時彈回（彈性位能轉回動能）。

這也解釋了為何許多腿式機器人採用**串聯彈性驅動器**（SEA，第 5.9 節）——彈簧不只是為了安全與力控制，更能**儲存並回收能量**，大幅提升跑步的效率。這是「機構參與控制」的又一個範例（呼應 28.1.3 節的被動行走）。

# 28.7 分層控制架構

## 28.7.1 三層架構

腿式機器人的控制採用與第 27 章無人機類似、但更複雜的分層架構。典型分為三層：

表 28-5 腿式機器人的控制分層

| 層級            | 頻率          | 使用的模型            | 輸出         | 任務          |
|---------------|-------------|------------------|------------|-------------|
| 步態與落腳點規劃      | 1~10 Hz     | LIPM / SLIP (簡化) | 落腳點序列、質心軌跡 | 決定往哪走、踩哪裡   |
| 質心軌跡最佳化 (MPC) | 20~100 Hz   | LIPM 或單剛體        | 質心軌跡、接觸力   | 在未來數步內最佳化   |
| 全身控制 (WBC)    | 200~1000 Hz | 完整動力學            | 各關節力矩      | 把上層指令轉為關節力矩 |

這個分層的核心邏輯是「模型複雜度隨頻率遞增而遞減」——最上層用最簡化的模型看得最遠、最下層用最完整的模型算得最細。這正是第 27.4 節時間尺度分離思想的推廣。

### 28.7.2 中層：模型預測控制

中層的 MPC (第 17.8.4 節) 是近十年腿式機器人最重要的突破之一。它的作法是：

1. 以 LIPM 或單剛體模型，預測未來 0.5~1.5 秒 (涵蓋 2~4 步) 的運動。
2. 求解一個最佳化問題：在滿足接觸約束 (摩擦錐、力的上下限) 與 ZMP 約束的前提下，最小化「軌跡追隨誤差」與「接觸力大小」。
3. 只執行最佳解的第一步。
4. 下一個週期以新的狀態重新求解 (滾動時域)。

MPC 的關鍵優勢正是第 18.8.2 節提到的：它能明確處理約束。摩擦錐、力的極限、ZMP 必須在支撐面內——這些都是硬約束，用 LQR 或 PID 都無法直接表達，但 MPC 可以。

MIT Cheetah 3 是這個方法的代表性成果——它用凸最佳化的 MPC 實現了四足機器人的盲走上樓、後空翻與高速奔跑。

### 28.7.3 底層：全身控制

最底層的全身控制 (Whole-Body Control, WBC) 負責把上層的指令，轉換為每一個關節的力矩。它面對的問題是：

已知：期望的質心加速度、期望的腳部軌跡、接觸的狀態

求：n 個關節力矩 + 各接觸點的力

這是一個帶約束的最佳化問題。

典型的表述形式是一個二次規劃 (QP)：

$$\min \|\text{任務誤差}\|^2 + \|\text{力矩}\|^2$$

s.t. 動力學方程、接觸約束、摩擦錐、力矩極限

WBC 的 QP 表述。每個控制週期求解一次 (1 kHz 時只有 1 毫秒)。

它與第 12.8 節分層逆運動學、第 17.9 節動態一致的多任務控制是同一個譜系——都是在處理「多個任務、有限的自由度」的問題。差別在於腿式機器人多了接觸約束這個關鍵的複雜性。

### 28.7.4 接觸力分配與摩擦錐

WBC 中最特殊的部分是**接觸力分配**——決定每隻腳該承受多少力。這個問題有三個約束：

- **單向性**：腳只能**推**地面，不能拉。因此法向力必須為正 ( $f_n \geq 0$ ) ——這與第 21.3.3 節抓取的「手指只能推」完全相同。
- **摩擦錐**：切向力不能超過  $\mu \cdot f_n$ ，否則會打滑（第 21.2.2 節）。
- **合力要求**：所有接觸力的合成，必須等於支撐機器人運動所需的力與力矩。

當有多隻腳著地時，這個問題是**冗餘的**（無限多組解），因此可以最佳化——例如讓各腳的受力盡量平均、或讓接觸力盡量遠離摩擦錐邊界（增加抗滑裕度）。

#### ◎ 第 21 章的抓取與第 28 章的行走是同一個問題

這是一個值得停下來欣賞的觀察：**四足機器人站在地上，與機械手抓住一個物體，在數學上是同一個問題**。四隻腳就是四根手指、地面就是被「抓住」的物體、摩擦錐與力封閉的分析完全相同。差別只在於：抓取時是手指抓物體、行走時是「地面抓住機器人」（或者說，機器人用腳抓住地球）。因此第 21 章的力封閉、摩擦錐、正線性跨度等所有工具，都可以直接應用到腿式機器人的接觸力分析上。這種跨領域的結構相似性，正是理論的力量所在。

## 28.8 狀態估測的特殊困難

### 28.8.1 沒有絕對參照

腿式機器人的狀態估測，比輪型機器人困難得多。核心問題是：**它沒有可靠的里程計**。

輪型機器人可以數輪子的轉動（第 23.7 節），雖然會漂移但短期精準。腿式機器人呢？

- **腳會打滑**：碎石、濕滑地面上的滑動量無法預測。
- **腳會變形**：橡膠腳墊受力會壓縮數公釐。
- **接觸時機不確定**：什麼時候真正著地？觸地感測器有延遲與雜訊。
- **騰空時完全無資訊**：跑步時雙腳離地，此時只能靠 IMU 積分。

### 28.8.2 腿式里程計

儘管困難，仍可以建立一個**腿式里程計**（leg odometry）。其原理是：

**若腳 A 著地且不滑動，則「腳 A 在世界座標系中的位置」保持不變**

接觸點作為暫時的固定參照。由關節編碼器可算出「軀幹相對於腳」的位置（正向運動學），因而反推出軀幹



的移動。

這個方法的三個要點：

- **必須偵測接觸**：用力感測器、接觸開關、或由關節力矩推算（第 11.6.3 節）。誤判接觸是最主要的誤差來源。
- **必須偵測打滑**：若多隻腳同時著地，可以交叉比對——若各腳算出的軀幹速度不一致，代表有腳在滑。
- **必須融合 IMU**：腿式里程計的更新率受步頻限制（數 Hz），必須用 IMU 填補中間的高頻資訊。

### 28.8.3 標準的估測架構

現代腿式機器人的狀態估測，典型採用三層融合：

表 28-6 腿式機器人的狀態估測

| 來源           | 提供的資訊         | 更新率         | 特性        |
|--------------|---------------|-------------|-----------|
| IMU          | 姿態、角速度、加速度    | 500~1000 Hz | 高頻、會漂移    |
| 關節編碼器 + 接觸偵測 | 軀幹相對速度（腿式里程計） | 步頻限制        | 會因打滑而錯誤   |
| 視覺 / 光達 SLAM | 絕對位姿          | 10~30 Hz    | 不漂移但慢     |
| 外部定位（實驗室）    | 絕對位姿          | 100+ Hz     | 精確但僅限特定場地 |

融合的邏輯與第 24.10.1 節相同：IMU 提供高頻預測、腿式里程計提供速度修正、SLAM 提供不漂移的絕對參照。三者以 EKF 或因子圖融合。

一個實務上的關鍵細節：腿式機器人的高度估計特別困難。因為 IMU 的垂直加速度必須扣除重力，而任何姿態估計的微小誤差都會造成巨大的垂直積分誤差。因此高度通常主要依賴「腳的接觸點高度 + 腿的正向運動學」，而非 IMU 積分。

## 28.9 強化學習的突破

### 28.9.1 一個典範轉移

2019 年前後，腿式機器人領域發生了一次典範轉移。ETH Zurich 的 ANYmal 團隊展示了一件事：用強化學習訓練出的控制器，在崎嶇地形上的表現超越了數十年來精心設計的模式方法。



這個結果之所以震撼，是因為模型式方法（28.4 ~ 28.7 節）建立在紮實的物理與最佳化理論之上，而強化學習只是「在模擬中試錯數百萬次」。但後者在**真實崎嶇地形**上的強健性確實更高。

為什麼？三個原因：

- **模型式方法依賴簡化模型**。LIPM 假設質心高度固定、忽略角動量——這些假設在崎嶇地形上會失效。學習式方法則直接從完整的模擬動態中學習，不受簡化的限制。
- **接觸的不確定性難以建模**。地面的軟硬、摩擦係數、是否會滑動——這些在模型中只能用粗略的假設。學習式方法則透過**領域隨機化**（第 14.7.3 節）在訓練中經歷各種可能性，學到對這些變異不敏感的策略。
- **反應式行為難以設計**。踩到石頭滑了一下該怎麼辦？模型式方法需要明確地寫出每種情況的處置；學習則能自動發展出這類「本能反應」。

## 28.9.2 訓練的流程

典型的腿式運動強化學習流程為：

1. **在模擬中建立環境**：用物理引擎（第 14.8 節）模擬機器人與地形。地形通常是程序化生成的（隨機的台階、斜坡、碎石）。
2. **領域隨機化**：每次訓練隨機改變物理參數——質量（ $\pm 20\%$ ）、摩擦係數（0.3~1.2）、馬達力矩上限、感測延遲、地形起伏。**這是 Sim-to-Real 成功的關鍵**。
3. **設計獎勵函數**：鼓勵前進速度、懲罰能耗、懲罰關節極限、懲罰身體傾倒、鼓勵平順的動作。這是最需要工程直覺的部分。
4. **大規模平行訓練**：在 GPU 上同時模擬數千個環境（如 Isaac Gym），數小時內完成相當於數年的真實經驗。
5. **部署到實機**：訓練好的策略是一個神經網路，推論極快（微秒級），可直接在機器人上執行。

## 28.9.3 教師—學生架構

有一個極為巧妙的技巧值得特別介紹，稱為**教師—學生**（teacher-student）訓練：

### ◎ 特權資訊與知識蒸餾

**問題**：在模擬中，我們知道所有的「特權資訊」——精確的地形高度、摩擦係數、機器人的真實狀

態。但在真實機器人上，這些都無法量測。**解法**：先訓練一個「教師」策略，讓它使用所有特權資訊，學到最佳的行為；再訓練一個「學生」策略，它只能看到真實可得的感測資料（IMU、關節編碼器），但被要求**模仿教師的動作**。這樣學生就學會了「如何從有限的感測中推斷出隱藏的狀態」，並做出教師等級的決策。這個技巧使 ANYmal 能在完全看不見地形的情況下（純本體感覺）走過碎石與樓梯。

## 28.9.4 模型與學習的分工

經過數年的發展，目前的共識並非「學習取代模型」，而是**兩者各有所長**：

表 28-7 模型式與學習式方法的比較

| 面向        | 模型式 (MPC + WBC) | 學習式 (RL)     |
|-----------|-----------------|--------------|
| 可解釋性      | 高 (每一步都有物理意義)   | 低 (黑盒子)      |
| 保證        | 有 (約束明確滿足)      | 無 (只有統計上的表現) |
| 崎嶇地形的強健性  | 中               | 高            |
| 調整新行為     | 需重新設計           | 需重新訓練 (數小時)  |
| 處理未建模效應   | 差               | 佳            |
| 計算量 (部署時) | 中 (需即時最佳化)      | 低 (一次前向傳播)   |
| 開發成本      | 高 (需專家知識)       | 中 (需大量計算資源)  |

**混合式架構**因而成為趨勢：用學習處理「反應式的低層行為」（如踩空時的即時修正），用模型式方法處理「需要保證的高層規劃」（如避障、任務約束）。這與第 21.9.5 節的抓取、第 25.10.3 節的 SLAM 是同一種思維——**學習做直覺，模型做保證**。

## 28.10 跌倒與安全

### 28.10.1 跌倒是不可避免的

與無人機類似（第 27.8 節），腿式機器人也沒有「安全停止」這個選項——但情況又不同：無人機停止就墜落，腿式機器人停止則會傾倒。

因此設計哲學必須是：**承認跌倒會發生，並讓它盡可能無害**。三個層次的處理：

**層次一：避免跌倒**。持續監測捕捉點（第 28.5.2 節）——若捕捉點仍在可達範圍內，就執行恢復步伐；若已超出，進入層次二。

**層次二：受控跌倒。**既然無法避免，就**主動選擇怎麼倒**。具體策略包括：

- **選擇倒下的方向：**避開人員、避開陡坡或樓梯。
- **降低質心：**屈膝下蹲，減少落下的高度與衝擊能量。
- **用手臂緩衝：**人形機器人可用手臂先觸地分散衝擊（與人類的本能相同）。
- **保護關鍵部位：**轉動身體使頭部（含感測器）朝上，用背部或側面著地。

**層次三：跌倒後的復原。**多數現代腿式機器人具備**自主站起**的能力。這是一個獨立的控制問題——需要規劃一連串的姿態轉換，且過程中接觸狀態不斷改變。

## 28.10.2 與人共處的安全

腿式機器人（尤其是人形）在與人共處時有特殊的安全考量：

- **質量與速度：**一台 80 公斤的人形機器人以 1.5 m/s 移動時，動能達 90 焦耳——足以造成嚴重傷害。這遠超過第 20 章協作機器人的安全標準。
- **跌倒的方向不可控：**不像固定式手臂能限制工作範圍，腿式機器人可能往任何方向倒。
- **踩到人的風險：**腳的接觸壓力極高（小面積承受全身重量）。

因應之道包括：限制運行速度、設定人員接近時的減速區、加裝軟性外殼、以及最重要的一——在有人的環境中，優先採用保守的靜態穩定步態。

## 28.11 第四篇的總結

### 28.11.1 本章的成果

本章完成了全書技術的最高整合。你現在能夠：說明腿式機器人的優勢與代價、理解混合動態的困難、區分靜態與動態穩定並運用 ZMP 判準、推導並運用線性倒立擺模型、以捕捉點決定落腳點、比較各種步態的特性、理解三層控制架構與全身控制的最佳化形式、處理腿式狀態估測的特殊困難、並理解強化學習帶來的突破。

最重要的是那個貫穿全章的方法論：**用簡化模型處理複雜系統**。三十個自由度的人形機器人，在規劃層面簡化為一根三維倒立擺——這個抽象使原本不可解的問題變得可解。而底層仍用完整模型執行，兩者透過分層架構銜接。

### 28.11.2 第四篇的整合回顧

第 22 ~ 28 章共同構成了一個完整的自主移動機器人系統：

看見 ( 22 ) → 移動 ( 23 ) → 估測 ( 24 ) → 建圖定位 ( 25 ) → 規劃導航 ( 26 ) → 飛行  
( 27 ) → 行走 ( 28 )

第四篇的邏輯鏈。

這七章與第三篇形成了鮮明的對比。**第三篇的世界是確定的**——給定關節角度，末端位置唯一確定；**第四篇的世界充滿不確定性**——每一個感測讀值都帶著雜訊、每一次辨識都可能出錯、每一步都可能打滑。

因應這個轉變，第四篇反覆出現三個原則：

- 量化不確定性而非忽略它 ( 第 24 章的協方差、第 28 章的捕捉點裕度 ) 。
- 依可信度融合而非二選一 ( 第 24 章的 Kalman 增益、第 28 章的多來源估測 ) 。
- 用簡化模型處理複雜系統 ( 第 26 章的位形空間、第 27 章的小角度近似、第 28 章的 LIPM ) 。

### 28.11.3 接下來：從「會動」到「懂得做什麼」

經過二十八章，機器人已經具備了完整的**物理能力**：它能感知環境、能精確地運動、能與環境安全地互動、能在未知空間中導航、能飛行也能行走。

但它仍然缺少一件關鍵的東西——**理解與決策**。目前為止的所有能力，都建立在「人類明確地告訴它該做什麼」之上：軌跡是人規劃的、任務是人指定的、行為是人設計的。

**第五篇「智慧機器人與具身人工智慧」( 第 29 ~ 33 章 )**將處理這個轉變。我們會探討：機器人如何理解自然語言的指令、如何從示範中學習新技能、如何用強化學習自主發展行為、如何用大型模型進行任務層級的推理、以及「具身智慧」這個近年最重要的概念——**智慧不只存在於運算中，也存在於身體與環境的互動裡**。

屆時你會發現，前四篇建立的所有能力，都將成為第五篇的**執行基礎**。一個大型語言模型可以理解「幫我把桌上的杯子拿到廚房」，但要真正做到，需要第 22 章的視覺辨識、第 21 章的抓取、第 26 章的導航、以及第 17 章的控制——**智慧與身體，缺一不可**。

換句話說：前四篇讓機器人**做得到**，第五篇讓它**知道該做什麼**。

## 完整計算例題

### 例題 28-1 線性倒立擺與捕捉點

題目：某人形機器人質心高度  $z_{\text{com}} = 0.85 \text{ m}$ 。(a) 求 LIPM 的自然頻率  $\omega$  與時間常數。(b) 若當前質心位置  $x = 0.05 \text{ m}$  (相對於支撐腳)、速度  $\dot{x} = 0.4 \text{ m/s}$ ，求捕捉點位置。(c) 若腳掌長  $0.25 \text{ m}$  (前緣在  $+0.15 \text{ m}$ 、後緣在  $-0.10 \text{ m}$ )，判斷是否能靠當前這隻腳穩住。(d) 若機器人的最大跨步距離為  $0.6 \text{ m}$ ，判斷能否一步捕捉。

解 (a)：由第 28.4.1 節：

$$\omega = \sqrt{g/z_{\text{com}}} = \sqrt{9.8/0.85} = \sqrt{11.53} = 3.396 \text{ rad/s}$$

自然頻率。

$$\text{時間常數 } \tau = 1/\omega = 1/3.396 = 0.294 \text{ 秒}$$

這是質心發散的特徵時間——約 0.3 秒內誤差會放大  $e$  倍 (2.72 倍)。這解釋了為何腿式機器人的反應必須極快。

解 (b)：由第 28.5.1 節的捕捉點公式：

$$x_{\text{cp}} = x + \dot{x}/\omega = 0.05 + 0.4/3.396 = 0.05 + 0.1178 = 0.168 \text{ m}$$

捕捉點在質心前方約 12 公分處。

解 (c)：腳掌的可用範圍是  $-0.10 \text{ m}$  到  $+0.15 \text{ m}$ 。捕捉點在  $0.168 \text{ m}$ ：

$$0.168 \text{ m} > 0.15 \text{ m} \text{ (腳掌前緣)}$$

捕捉點超出腳掌範圍，無法靠當前這隻腳穩住。

**物理意義：**即使把 ZMP 移到腳尖 (最大的可能)，仍不足以消除發散——機器人必定會繼續往前倒。此時必須跨步。

這正是人類的經驗：當你被推得很急時，光靠腳踝的調整 (把重心壓向腳尖) 是不夠的，必須跨出一步。

解 (d)：需要跨到  $0.168 \text{ m}$  處，而最大跨步為  $0.6 \text{ m}$ ：

$$0.168 \text{ m} < 0.6 \text{ m} \Rightarrow \text{可以一步捕捉 } \checkmark$$

只需跨出約 17 公分即可穩住。

進一步分析：若速度更快，例如  $\dot{x} = 2.5 \text{ m/s}$ ：

$$x_{cp} = 0.05 + 2.5/3.396 = 0.05 + 0.736 = 0.786 \text{ m} > 0.6 \text{ m}$$

超出單步能力，需要兩步以上才能停下（ $N$  步捕捉，第 28.5.3 節）。

**工程意義：**這個計算給出了一個明確的**速度上限**——超過某個速度後，機器人就無法在一步內停止。這是步態規劃時必須考慮的安全約束。

## 例題 28-2 ZMP 與動態穩定

題目：某雙足機器人質量  $m = 60 \text{ kg}$ 、質心高度  $0.9 \text{ m}$ 。腳掌支撐區域由  $-0.08 \text{ m}$  到  $+0.14 \text{ m}$ （以踝關節為原點）。（a）若質心位置在  $+0.05 \text{ m}$  且靜止，判斷是否靜態穩定。（b）若質心以  $2.0 \text{ m/s}^2$  向前加速，求 ZMP 位置並判斷穩定性。（c）求不使 ZMP 超出前緣的最大向前加速度。（d）由（c）的結果，估算機器人能達到的最大水平推力。

解（a）：靜止時 ZMP 與質心投影重合。

$$x_{zmp} = x_{com} = 0.05 \text{ m}$$

在  $-0.08 \sim +0.14$  的範圍內。

判定：靜態穩定 ✓。機器人可以無限期地保持這個姿勢。

解（b）：由第 28.3.2 節的公式：

$$x_{zmp} = x_{com} - (z_{com}/g) \cdot \ddot{x}_{com} = 0.05 - (0.9/9.8) \times 2.0 = 0.05 - 0.1837 = -0.134 \text{ m}$$

ZMP 位置。

判定：不穩定 ✗—— $-0.134 \text{ m}$  超出了後緣  $-0.08 \text{ m}$ 。

**物理意義：**向前加速時 ZMP 往後移（注意公式中的負號）。若加速太猛，ZMP 會退出腳掌後緣，機器人會向後翻倒——這正是快速起步時「往後仰」的感覺。人類的因應之道是前傾身體（把  $x_{com}$  往前移），使 ZMP 回到腳掌內。

解（c）：要求  $x_{zmp} \geq -0.08 \text{ m}$ ：

$$0.05 - (0.9/9.8) \cdot \ddot{x} \geq -0.08 \Rightarrow 0.0918 \cdot \ddot{x} \leq 0.13 \Rightarrow \ddot{x} \leq 1.416 \text{ m/s}^2$$

最大向前加速度約  $1.42 \text{ m/s}^2$ （約  $0.14g$ ）。

若把質心前移到 0.10 m ( 前傾 ) :

$$\ddot{x}_{\max} = (0.10 + 0.08)/0.0918 = 1.961 \text{ m/s}^2$$

加速能力提升了 38%。這定量地說明了為何起跑時要前傾。

解 ( d ) : 由牛頓第二定律 :

$$F_{\max} = m \cdot \ddot{x}_{\max} = 60 \times 1.416 = 85 \text{ N}$$

最大水平推力約 85 牛頓 ( 前傾後可達 118 N ) 。

**工程判讀：**這個數字看似不大，但它揭示了一個重要的限制——**腿式機器人的水平出力受限於腳掌尺寸，而非馬達功率**。要提高推力，可以：加大腳掌 ( 但影響行走靈活度 )、前傾身體 ( 本例的作法 )、或**跨步支撐** ( 把後腳往後蹬，形成更大的支撐多邊形 )——這正是人類推重物時的姿勢。

### 例題 28-3 步態參數與速度

題目：某四足機器人腿長 0.35 m，採用小跑 ( trot ) 步態。( a ) 若步頻為 2.5 Hz、步長 0.25 m，求行進速度。( b ) 求該步態的佔空比與單腳的支撐時間。( c ) 估算此速度下的 Froude 數並判斷步態是否合理。( d ) 若要提升到 2.0 m/s，討論可行的策略。

解 ( a ) : 行進速度為步長乘以步頻。注意小跑步態每個週期兩腳交替，故一個週期前進一個步長：

$$v = \text{步長} \times \text{步頻} = 0.25 \times 2.5 = 0.625 \text{ m/s}$$

行進速度約 0.63 m/s。

解 ( b ) : 小跑的佔空比為 0.5 ( 第 28.6.2 節 ) ——每隻腳有一半的時間著地。

$$\text{週期 } T = 1/2.5 = 0.4 \text{ 秒}$$

$$\text{單腳支撐時間} = 0.5 \times 0.4 = 0.2 \text{ 秒}$$

$$\text{單腳擺動時間} = 0.2 \text{ 秒}$$

時序分配。

**工程意義：**0.2 秒的擺動時間，決定了腿部馬達必須有多快——腿要在 0.2 秒內抬起、前擺 0.25 公尺、再放下。這是選擇馬達與減速比的直接依據 ( 第 5 章 )。

解 ( c ) : **Froude 數**是步態研究中的重要無量綱參數，定義為：



$$Fr = v^2/(g \cdot L)$$

$L$  為特徵長度（通常取腿長或髖關節高度）。它衡量「慣性效應相對於重力效應」的大小。

$$Fr = 0.625^2/(9.8 \times 0.35) = 0.3906/3.43 = 0.114$$

*Froude 數約 0.11。*

生物學的經驗值：

- $Fr < 0.5$ ：行走（walk）較有效率——包含四足的爬行與小跑。
- $Fr \approx 0.5$ ：步態轉換的臨界點——動物在此速度附近會由 walk 換成 trot 或 run。
- $Fr > 1.0$ ：必須使用含騰空期的步態（gallop、run）。

本例  $Fr = 0.11$  遠低於 0.5，小跑步態完全合理（甚至可以說偏保守，還有加速的餘裕）。

解（d）：要達到 2.0 m/s：

$$Fr = 2.0^2/(9.8 \times 0.35) = 4.0/3.43 = 1.166 > 1.0$$

*超過 1.0，小跑步態已不適用，必須改用含騰空期的步態（bound 或 gallop）。*

三種可行的策略：

- **提高步頻**： $v = 0.25 \times f = 2.0 \Rightarrow f = 8 \text{ Hz}$ 。但擺動時間縮短為 0.0625 秒——這對馬達的要求極高（腿要在 62 毫秒內完成前擺），且慣性力大幅增加。
- **加大步長**： $v = L \times 2.5 = 2.0 \Rightarrow L = 0.8 \text{ m}$ 。但步長受限於腿長（0.35 m），最多約 0.5 m（腿張開的極限）——不足以達成目標。
- **改變步態**：改用 bound 或 gallop，利用騰空期延長每一步的前進距離。這是唯一可行的方案，也是自然界動物的選擇。

**結論**：從 0.63 m/s 提升到 2.0 m/s，不是單純調參數就能達成的——必須改變步態的本質。這與汽車換檔的道理相同，也印證了第 28.6.2 節「應依速度切換步態」的建議。

## 程式範例：LIPM、捕捉點與步態模擬

以下 Python 程式實作線性倒立擺模型、捕捉點計算、ZMP 分析與步態時序，並驗證本章三個例題。

```
import numpy as np
```



```
# ===== 線性倒立擺模型 =====
class LIPM:
    """線性倒立擺 ( 28.4 節 ) """
    def __init__(s, z_com=0.85, g=9.8):
        s.z, s.g = z_com, g
        s.w = np.sqrt(g/z_com)          # 自然頻率  $\omega$ 

    def capture_point(s, x, xd):
        """捕捉點 ( 28.5.1 節 ) """
        return x + xd/s.w

    def dcm(s, x, xd):
        """DCM 與捕捉點同義 ( 28.5.4 節 ) """
        return x + xd/s.w

    def step(s, x, xd, x_zmp, dt):
        """LIPM 動態積分： $\ddot{x} = \omega^2(x - x_{zmp})$ """
        xdd = s.w**2 * (x - x_zmp)
        xd_new = xd + xdd*dt
        x_new = x + xd_new*dt
        return x_new, xd_new

    def zmp_from_com(s, x, xdd):
        """由質心加速度反推 ZMP ( 28.3.2 節 ) """
        return x - (s.z/s.g)*xdd

    def max_accel(s, x_com, zmp_limit):
        """在 ZMP 限制下的最大加速度"""
        return (x_com - zmp_limit)*s.g/s.z

# ===== 例題 28-1 =====
print("=== 例題 28-1：LIPM 與捕捉點 ===")
lip = LIPM(z_com=0.85)
print(f"(a)  $\omega = \sqrt{g/z} = \{{lip.w:.3f}\}$  rad/s")
print(f"    時間常數  $\tau = 1/\omega = \{{1/lip.w:.3f}\}$  秒")
print(f"    → 約 0.3 秒內誤差放大 e 倍，反應必須極快")

x, xd = 0.05, 0.4
cp = lip.capture_point(x, xd)
print(f"\n(b) x={x} m,  $\dot{x}=\{{xd}\}$  m/s → 捕捉點 = {cp:.4f} m")

foot_back, foot_front = -0.10, 0.15
ok_ankle = foot_back <= cp <= foot_front
print(f"\n(c) 腳掌範圍 [{foot_back}, {foot_front}] m")
print(f"    捕捉點 {cp:.3f} m {'在' if ok_ankle else '超出'}腳掌範圍"
      f" → {'可靠腳踝穩住' if ok_ankle else '**必須跨步**'}")

max_step = 0.6
```

```

print(f"\n(d) 最大跨步 {max_step} m:")
for v in (0.4, 1.0, 2.0, 2.5, 3.0):
    cp_v = lip.capture_point(0.05, v)
    n_step = "1 步" if cp_v <= max_step else "***需 2 步以上**"
    print(f"     $\dot{x}=\{v:.1f\}$  m/s  $\rightarrow$  捕捉點  $\{cp_v:.3f\}$  m {n_step}")

# ===== 例題 28-2 : ZMP 分析 =====
print("\n=== 例題 28-2 : ZMP 與動態穩定 ===")
lip2 = LIPM(z_com=0.9)
m_body = 60.0
back, front = -0.08, 0.14
x_com = 0.05

print(f"(a) 靜止時 ZMP = x_com = {x_com} m  $\rightarrow$  ")
    f"{'穩定 ✓' if back <= x_com <= front else '不穩定 X'}")

acc = 2.0
zmp = lip2.zmp_from_com(x_com, acc)
print(f"(b) 加速 {acc} m/s2  $\rightarrow$  ZMP = {zmp:.4f} m  $\rightarrow$  ")
    f"{'穩定 ✓' if back <= zmp <= front else '***不穩定 X ( 向後翻倒 ) ***'}")

print(f"\n(c) 最大向前加速度 ( ZMP 不超出後緣 {back} m ) :")
for xc in (0.05, 0.08, 0.10, 0.12):
    a_max = lip2.max_accel(xc, back)
    F_max = m_body*a_max
    print(f"    質心在 {xc:.2f} m  $\rightarrow$  a_max = {a_max:.3f} m/s2"
          f"    ({a_max/9.8:.3f} g), 推力 = {F_max:.1f} N")
print("     $\rightarrow$  前傾 ( 質心前移 ) 能顯著提升加速能力")

# ===== ZMP 隨加速度的變化 =====
print("\n=== ZMP 隨加速度的變化 ( 質心固定在 0.05 m ) ===")
print(f"{'加速度':>10}{ 'ZMP':>10}{ '狀態':>12}")
for a in (-1.0, 0.0, 0.5, 1.0, 1.4, 2.0):
    z = lip2.zmp_from_com(0.05, a)
    st = "穩定" if back <= z <= front else ("超出後緣" if z < back
                                           else "超出前緣")
    print(f"{a:9.1f}2{z:10.4f}{st:>12}")
print("> 向前加速使 ZMP 後移 ; 減速 ( 負加速度 ) 則使 ZMP 前移")

# ===== 例題 28-3 : 步態參數 =====
print("\n=== 例題 28-3 : 步態參數與 Froude 數 ===")
leg_len = 0.35
def froude(v, L=leg_len, g=9.8):
    return v**2/(g*L)

stride, freq = 0.25, 2.5
v = stride*freq

```

```

print(f"(a) 速度 = {stride} × {freq} = {v:.3f} m/s")
T = 1/freq
print(f"(b) 週期 = {T:.3f} s, 佔空比 0.5")
print(f"    支撐時間 = {T*0.5:.3f} s, 擺動時間 = {T*0.5:.3f} s")
print(f"(c) Froude 數 = {froude(v):.4f}")

print(f"\n{'速度':>9}{ 'Froude':>10}{ '建議步態':>16}")
for vv in (0.3, 0.625, 1.0, 1.5, 2.0, 3.0):
    fr = froude(vv)
    if fr < 0.3:    gait = "爬行/慢走"
    elif fr < 0.5: gait = "小跑 trot"
    elif fr < 1.0: gait = "快跑 bound"
    else:          gait = "**飛奔 gallop**"
    print(f"{vv:8.2f}m/s{fr:10.3f}{gait:>16}")
print("→ Fr ≈ 0.5 是步態轉換點 · Fr > 1.0 必須含騰空期")

print(f"\n(d) 要達 2.0 m/s 的三種策略:")
print(f"    提高步頻: f = 2.0/{stride} = {2.0/stride:.1f} Hz "
      f"(擺動時間僅 {0.5/(2.0/stride)*1000:.0f} ms – 馬達要求極高)")
print(f"    加大步長: L = 2.0/{freq} = {2.0/freq:.2f} m "
      f"(超過腿長 {leg_len} m – **不可行**)")
print(f"    改變步態: Fr = {froude(2.0):.2f} > 1.0 – **唯一可行**")

# ===== 捕捉點控制的行走模擬 =====
print("\n=== 以捕捉點決定落腳點的行走模擬 ===")
def walk_sim(v_target=0.5, n_steps=8, T_step=0.4, z=0.85):
    """每步結束時，依捕捉點決定下一個落腳點 ( 28.5.2 節 ) """
    lp = LIPM(z_com=z)
    x, xd = 0.0, 0.0 # 相對於支撐腳
    x_foot = 0.0 # 支撐腳的絕對位置
    dt = 0.001
    print(f"{'步':>3}{ '落腳點':>10}{ '步末 x':>10}{ '步末 ẋ':>10}"
          f"{'捕捉點':>10}")
    for k in range(n_steps):
        # 依捕捉點加上目標速度的偏移，決定下一落腳點
        cp = lp.capture_point(x, xd)
        offset = v_target/lp.w * np.tanh(1.0) # 踩在捕捉點後方以維持速度
        x_next_rel = cp - offset
        # 在這一步內以當前腳為 ZMP 積分
        for _ in range(int(T_step/dt)):
            x, xd = lp.step(x, xd, 0.0, dt)
        cp_end = lp.capture_point(x, xd)
        print(f"{k:3d}{x_foot + x_next_rel:10.4f}{x:10.4f}"
              f"{xd:10.4f}{cp_end:10.4f}")
        # 換腳：座標原點移到新的支撐腳
        x_foot += x_next_rel
        x -= x_next_rel
    return x_foot

```

```

final = walk_sim()
print(f"共前進 {final:.3f} m")
print("> 每步的落腳點都由捕捉點決定，這是腿式行走的核心控制律")

# ===== 步態時序表 =====
print("\n=== 四足步態的相位時序 (0=著地起點, 相位 $\in$ [0,1)) ===")
gaits = {
    "爬行 Crawl": ([0.0, 0.5, 0.25, 0.75], 0.75),
    "小跑 Trot": ([0.0, 0.5, 0.5, 0.0], 0.5),
    "溜蹄 Pace": ([0.0, 0.5, 0.0, 0.5], 0.5),
    "跳躍 Bound": ([0.0, 0.0, 0.5, 0.5], 0.45),
}
print(f"{'步態':<14}{'LF':>7}{'RF':>7}{'LH':>7}{'RH':>7}"
      f"{'佔空比':>9}{'同時著地':>10}")
for name, (ph, duty) in gaits.items():
    # 計算相位 0 時有幾隻腳著地
    n_contact = sum(1 for p in ph if (0.0 - p) % 1.0 < duty)
    print(f"{'name':<14}{'ph[0]':7.2f}{'ph[1]':7.2f}{'ph[2]':7.2f}"
          f"{'ph[3]':7.2f}{'duty':9.2f}{'n_contact':9d} 腳")
print("> 只有爬行能保證 3 腳著地 (靜態穩定)")

```

請試著修改：(一) 把質心高度由 0.85 改為 1.2 公尺 (較高的機器人)，觀察  $\omega$  變小、時間常數變長——這代表反應可以慢一些，但捕捉點會離得更遠。(二) 在 ZMP 分析中把腳掌長度加倍，觀察最大加速度提升多少。(三) 在行走模擬中改變  $v_{\text{target}}$ ，觀察落腳點如何自動調整。

## 實作：腿式機器人的模擬與實驗

本章實作以模擬為主 (實體腿式機器人成本高且危險)。若有四足平台 (如市售的教育型四足機器人)，後段項目可實機驗證。

- LIPM 模擬**：實作線性倒立擺模型，驗證其發散特性——由一個小的初始偏移出發，觀察質心如何指數地遠離。量測實際的發散時間常數並與  $1/\omega$  比較。
- 捕捉點驗證**：在 LIPM 模擬中，把 ZMP 設在捕捉點上，驗證質心確實會漸近地停止。再故意設在捕捉點前後各 5 公分處，觀察質心會往回彈或繼續前衝。
- ZMP 與腳掌限制**：實作 ZMP 計算，掃描不同的質心位置與加速度組合，繪出「可行區域」的圖。驗證例題 28-2 的結論。

4. **步態產生器**：實作四足的步態時序產生器，支援爬行、小跑、跳躍三種步態。以動畫顯示各腿的抬放時序與支撐多邊形的變化。
5. **靜態穩定判定**：對每個時刻計算支撐多邊形與重心投影，判定是否靜態穩定。驗證只有爬行步態能全程保持靜態穩定。
6. **簡化的行走控制**：整合 LIPM、捕捉點與步態產生器，實作一個能在平地上行走的簡化控制器。測試不同的目標速度。
7. **擾動恢復測試**：在行走過程中注入一個突然的速度擾動（模擬被推），觀察捕捉點如何改變、以及控制器如何調整落腳點恢復平衡。找出「能恢復」與「必定跌倒」的臨界擾動大小。
8. **（若有實機）步態參數調校**：在實體四足機器人上測試不同的步長、步頻與佔空比，記錄行進速度、穩定性與耗電量。繪出「速度 vs 能耗」曲線，找出最省能的參數組合。
9. **（若有實機）地形適應測試**：在平地、斜坡、階梯與碎石上分別測試，記錄各步態的成功率。觀察機器人在何種地形上開始失效。
10. **（進階）ROS 2 + Gazebo 模擬**：使用開源的四足機器人模型（如 Champ 或 Unitree 的開源專案），在 Gazebo 中建立完整的模擬環境，實作並測試自己的步態控制器。

表 28-8 腿式機器人實驗記錄表（前兩列為範例）

| 測試項目       | 設定                      | 結果               | 觀察              | 改善方向   |
|------------|-------------------------|------------------|-----------------|--------|
| LIPM 發散驗證  | $z=0.85, x_0=0.01$<br>m | 0.29 s 後放大 2.7 倍 | 與 $1/\omega$ 相符 | —      |
| 擾動恢復       | 小跑, 推力 20 N             | 一步恢復             | 捕捉點在跨步範圍內       | 測試更大擾動 |
| （自行填寫其餘測試） |                         |                  |                 |        |

## 國中小也看得懂：走路就是不斷地往前倒

先做一個實驗：站起來，然後往前走三步。走的時候仔細感覺一下——當你抬起一隻腳的那一瞬間，你的身體是不是正在往前倒？

是的！走路其實就是「不斷地往前倒，然後用下一隻腳接住自己」。你不是先站穩再移動，而是先讓自己失去平衡，再趕快跨一步接住。是不是很神奇？

再試一個：請朋友從背後輕輕推你一下。你會發現自己**自動跨出一步**——而且推得越用力，你跨得越遠。你的身體會自己算出「該跨多遠才接得住」。

科學家幫這個位置取了一個名字，叫做「**捕捉點**」。它有一個公式：**你倒得越快，捕捉點就離得越遠**。機器人也是用這個公式決定要把腳踩在哪裡的！

那為什麼機器人走路這麼難呢？有兩個原因：

**第一，機器人要一直算。**你走路不用想，但機器人每一秒要計算好幾百次「我現在倒到哪裡了？下一步該踩哪裡？」只要算錯一次就會摔倒。

**第二，腳踩地和抬起來完全不一樣。**腳踩在地上時，機器人是被地面「撐著」的；腳抬起來的瞬間，那個支撐就消失了。所以機器人每走一步，計算的方式就要換一次——這比一直用同一個方式難很多。

最後告訴你一件很酷的事：科學家發現，**一台有三十個關節的機器人，在規劃走路的時候可以簡化成「一根倒立的棍子」**！就像你只要知道自己往哪邊倒、倒得多快，就知道該跨多遠一樣。把複雜的東西變簡單，這就是科學家最厲害的本事。

## 叫獸提醒與常見錯誤

- 誤區一：以為腿式機器人只是「多幾個自由度的手臂」。混合動態（接觸切換）是本質上不同的困難，第 13~19 章的工具無法直接套用。
- 誤區二：用重心投影判斷動態穩定。有加速度時必須用 ZMP，兩者會分離（第 28.3.2 節）。
- 誤區三：以為 ZMP 在支撐面內就一定不會跌倒。它只保證腳不翻轉，是必要而非充分條件。
- 誤區四：在騰空期使用 ZMP 判準。此時沒有支撐多邊形，該判準完全失效。
- 誤區五：忽略 LIPM 的假設限制。質心高度大幅變化（跑步、跳躍）時應改用 SLIP 模型。
- 誤區六：把捕捉點當成「重心該去的地方」。它是**腳該踩的地方**——兩者意義完全不同。
- 誤區七：只用一種步態。應依速度切換（Froude 數約 0.5 與 1.0 是轉換點），這也是自然界動物的作法。
- 誤區八：忽略腳部打滑對狀態估測的破壞。腿式里程計的誤差主要來自誤判接觸與未偵測的

打滑。

- 誤區九：接觸力分配時忘記單向性與摩擦錐。腳只能推不能拉，且切向力受  $\mu \cdot f_n$  限制——這與第 21 章的抓取完全相同。
- 誤區十：以為強化學習能完全取代模型式方法。學習擅長反應式的低層行為，模型式方法提供保證與可解釋性，兩者互補。
- 誤區十一：把「避免跌倒」當成唯一目標。跌倒不可避免，應設計受控跌倒與自主站起的能力。
- 誤區十二：低估腿式機器人與人共處的風險。其質量與速度遠超協作機器人的安全標準，且跌倒方向不可控。

## 課堂討論

1. 28.1.3 節的被動行走顯示「穩定性可以由機構本身提供」。請討論這與第 21 章的欠驅動手、第 20 章的 RCC 有何共通之處，並歸納出一個設計原則。
2. 28.7.4 節指出「四足站立與機械手抓取在數學上是同一個問題」。請進一步討論：第 21 章的哪些工具可以直接應用到腿式機器人？哪些需要修改？
3. 強化學習在崎嶇地形上超越了模型式方法，但缺乏保證。請討論：在什麼應用中這個取舍是可接受的？在什麼應用中不可接受？並設計一個混合式架構。
4. 例題 28-2 顯示腿式機器人的水平出力受限於腳掌尺寸而非馬達功率。請討論這對機構設計的三個啟示，並比較人類與四足動物的解法差異。
5. 本書反覆出現「用簡化模型處理複雜系統」的思維（第 26 章的位形空間、第 27 章的小角度近似、第 28 章的 LIPM）。請歸納這個方法論的共同結構，並討論它可能失效的情況。

## 章末摘要

- 腿式機器人的優勢在於能選擇落腳點、跨越障礙、適應崎嶇地形；代價是效率低、自由度高、控制複雜且本質不穩定。
- 被動動態行走證明穩定性可由機構本身提供，這與「用機構取代控制」的思維一脈相承。
- 混合動態是腿式機器人的根本困難：接觸狀態切換使方程式改變，且落地時有速度的離散跳變。

- 分析週期步態可用龐加萊映射——只看每步結束的狀態，把連續混合系統化為離散映射。
- 靜態穩定要求重心投影在支撐多邊形內，任何時刻停下都不會倒；代價是速度慢、效率差。
- ZMP 是地面反力合力矩為零的點；動態穩定要求 ZMP 在支撐多邊形內，這保證腳不會翻轉。
- ZMP 與重心的關係  $x_{zmp} = x_{com} - (z/g) \cdot \ddot{x}_{com}$ ：向前加速時 ZMP 後移。
- 線性倒立擺模型 (LIPM) 以三個假設把 30 自由度簡化為一個線性方程  $\ddot{x} = \omega^2(x - x_{zmp})$ ， $\omega = \sqrt{g/z}$ 。
- LIPM 的解含一個發散模態與一個收斂模態；控制的本質是透過選擇落腳點消除發散分量。
- 捕捉點  $x_{cp} = x + \dot{x}/\omega$  是「踩下去就能停止」的位置；倒得越快就要跨得越遠。
- 捕捉點的三個用途：判斷能否站住、決定落腳點、量化穩定裕度；踩在其前方減速、後方加速。
- DCM 把二階系統分解為一個發散與一個收斂的一階系統，控制 DCM 即可使質心自動穩定。
- 四足步態中只有爬行是靜態穩定的；小跑因對稱性與速度平衡而最常用；速度提高時應切換步態。
- 行走與跑步的差別在於有無騰空期；跑步需改用含彈簧的 SLIP 模型，串聯彈性驅動器可回收能量。
- 控制採三層架構：步態規劃（簡化模型、低頻）、MPC（中等模型、中頻）、全身控制（完整模型、高頻）。
- 全身控制是一個二次規劃問題，需同時滿足動力學、接觸單向性、摩擦錐與力矩極限。
- 四足站立與多指抓取在數學上是同一個問題——摩擦錐與力封閉的分析完全相通。
- 腿式狀態估測缺乏可靠里程計；腿式里程計以「著地腳不動」為參照，需偵測接觸與打滑並融合 IMU。
- 強化學習在崎嶇地形上表現優於模型式方法，關鍵在領域隨機化與教師—學生訓練；兩者應互補而非取代。
- 跌倒不可避免，設計應包含受控跌倒（選方向、降質心、用手緩衝）與自主站起的能力。

## 觀念題與計算題



## 觀念題

1. 說明腿式機器人相對於輪型的三個優勢與三個代價。
2. 說明被動動態行走的原理，以及它對機器人設計的啟示。
3. 什麼是混合動態？它為腿式機器人帶來哪三個層次的困難？
4. 說明龐加萊映射如何用於分析週期步態的穩定性。
5. 區分靜態穩定與動態穩定，並說明各自的判準與代價。
6. 定義 ZMP，並說明它與重心投影在有加速度時為何會分離。
7. 寫出 ZMP 與質心加速度的關係，並解釋「向前加速時 ZMP 後移」的物理意義。
8. 列舉 ZMP 判準的三個限制。
9. 說明 LIPM 的三個假設，並解釋為何這些簡化在實務上仍然有效。
10. 由 LIPM 的解說明「控制的本質是消除發散模態」。
11. 定義捕捉點並說明其物理意義；列舉其三個用途。
12. 說明 DCM 的定義與其「分而治之」的巧妙之處。
13. 比較四足的五種步態，並說明為何只有爬行是靜態穩定的。
14. 說明行走與跑步的定義差異，以及為何跑步需改用 SLIP 模型。
15. 說明腿式機器人三層控制架構的分工與各層使用的模型。
16. 說明全身控制的最佳化形式，以及接觸力分配的三個約束。
17. 說明「四足站立與多指抓取是同一個問題」的意義。
18. 說明腿式里程計的原理與其三個誤差來源。
19. 說明強化學習在腿式運動上突破的三個原因，以及教師—學生架構的巧妙之處。

## 計算題

1. 某人形機器人質心高度 1.0 m。(a) 求  $\omega$  與時間常數。(b) 若  $x = 0.03$  m、 $\dot{x} = 0.6$  m/s，求捕捉點。(c) 若腳掌範圍為  $-0.09 \sim +0.16$  m，判斷能否靠腳踝穩住。(d) 若最大跨步 0.7 m，求能一步捕捉的最大速度。
2. 某雙足機器人  $m = 75$  kg、質心高度 0.95 m、腳掌範圍  $-0.07 \sim +0.15$  m。(a) 質心在  $+0.06$  m 且以  $1.5$  m/s<sup>2</sup> 加速時，求 ZMP 並判斷穩定性。(b) 求在此質心位置下的最大向

前加速度。(c) 求對應的最大水平推力。(d) 若質心前移到  $+0.12\text{ m}$ ，重新計算 (b) (c) 並計算改善百分比。

3. 某四足機器人腿長  $0.4\text{ m}$ ，採小跑步態、步長  $0.3\text{ m}$ 、步頻  $3.0\text{ Hz}$ 。(a) 求速度與 Froude 數。(b) 求週期、支撐時間與擺動時間。(c) 判斷此步態是否合理。(d) 求該機器人使用小跑步態的理論最高速度 (取  $Fr = 1.0$  為上限)。
4. 某 LIPM 系統  $z_{\text{com}} = 0.8\text{ m}$ 。初始  $x = 0.02\text{ m}$ 、 $\dot{x} = 0$ 。(a) 求  $\omega$ 。(b) 若 ZMP 固定在 0，求 0.3 秒後的質心位置 (提示：用雙曲函數解)。(c) 計算此時的捕捉點。(d) 若要在 0.3 秒時停止，ZMP 應設在何處？

## 動手做與專題延伸

### 本章實作：腿式運動模擬器

建立一個腿式機器人的模擬與控制環境，這是本書技術的最終整合。

- 階段一：實作 LIPM 與捕捉點計算，驗證發散特性與捕捉點的物理意義。
- 階段二：實作 ZMP 計算與支撐多邊形判定，建立穩定性的視覺化工具。
- 階段三：實作四足步態產生器 (支援多種步態) 與相位时序的動畫顯示。
- 階段四：整合為簡化的行走控制器——以捕捉點決定落腳點、以 LIPM 規劃質心軌跡。
- 階段五：加入擾動並實作恢復策略，測試 N 步捕捉的能力。

### 專題延伸

- 捕捉點的視覺化動畫：模擬一個 LIPM 系統，即時繪製質心位置、速度與捕捉點的變化。加入互動功能——讓使用者用滑鼠「推」機器人一下，觀察捕捉點如何瞬間跳遠，以及控制器如何選擇落腳點恢復。這是理解本章核心概念最直觀的工具。
- 步態的能耗分析：對同一台四足機器人模型，實作三種步態 (爬行、小跑、跳躍)，在多個速度下模擬並估算能耗 (可用關節力矩乘角速度的積分近似)。繪出「速度 vs 能耗」的曲線族，找出各步態的最佳速度範圍與轉換點。與 28.6.2 節提到的生物學結論比較——你應該會看到與 Froude 數相符的轉換點。
- 進階：ZMP 可行區域的三維視覺化。對一台雙足機器人，以「質心位置」「質心加速度」「腳掌尺寸」為三個維度，掃描所有組合並判定 ZMP 是否在支撐面內。把可行區域繪成三維的體積圖。接著討論：這個圖如何指導機構設計 (腳掌該多大) 與控制設計 (加速度該如

何限制)？並延伸思考——為何人類的腳掌是這個比例？用你的圖來驗證演化的選擇是否接近最佳。

## 參考文獻與延伸閱讀

- Kajita, S. et al., Introduction to Humanoid Robotics, Springer (人形機器人的標準教材，LIPM 與 ZMP 的完整處理)。
- Wieber, P.-B., Tedrake, R. and Kuindersma, S., "Modeling and Control of Legged Robots," in Springer Handbook of Robotics, 2nd ed. (腿式機器人的權威綜述)。
- Vukobratović, M. and Borovac, B., "Zero-Moment Point — Thirty Five Years of Its Life," Int. J. Humanoid Robotics, 2004 (ZMP 概念的回顧，由其提出者撰寫)。
- Pratt, J. et al., "Capture Point: A Step toward Humanoid Push Recovery," Humanoids 2006 (捕捉點的原始論文)。
- Englsberger, J., Ott, C. and Albu-Schäffer, A., "Three-Dimensional Bipedal Walking Control Based on Divergent Component of Motion," IEEE Trans. Robotics, 2015 (DCM 方法)。
- McGeer, T., "Passive Dynamic Walking," IJRR, 1990 (被動動態行走的原始論文)。
- Raibert, M. H., Legged Robots That Balance, MIT Press, 1986 (腿式機器人的開創性著作，SLIP 模型與跳躍控制)。
- Di Carlo, J. et al., "Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control," IROS 2018 (四足 MPC 的代表性工作)。
- Hwangbo, J. et al., "Learning Agile and Dynamic Motor Skills for Legged Robots," Science Robotics, 2019 (強化學習在 ANYmal 上的突破)。
- Lee, J. et al., "Learning Quadrupedal Locomotion over Challenging Terrain," Science Robotics, 2020 (教師—學生架構與盲走崎嶇地形)。